

Теория Информации

Ярмолик Вячеслав Николаевич

2017

Contents

Introduction: Data security. Fundamental concepts of cryptography.

Transposition and Substitution ciphers: Simple transposition. Product cipher. Simple substitution cipher. Caesar cipher. Vigenere cipher.

Mono and Poly alphabetic substitution cipher: Playfair cipher.

Rotor machines. The Enigma: a unique rotor machine.

Data Encryption Standard (DES): History of the DES. DES algorithms. Weak and semi weak keys. Advanced DES versions. IDEA. Blowfish.

Advanced Encryption Standard (AES): Rijndael Algorithm.

Linear Feedback Shift Register: Pseudorandom key generation by LFSR. M-sequences.

Stream cipher: Synchronous stream ciphers. Self-synchronizing cipher.

Contents *cont.*

Number theory: Prime numbers. Euler's function. Euler's theorem. Congruence.

Public Key Cipher: Principles of the public key cipher. One-way function. Deffie and Hellman algorithm.

RSA Cipher: Rivest, Shamir and Adleman public key cipher. Practical aspects.

Cryptographic Keys Management: Keys generation, distribution and authentication of Public Keys

Communication Security: Transport Layer Security (TLS) and its predecessor, Secure Sockets Layer (SSL).

Authentication Protocols: Password-authentication key agreement protocols. Password Authentication Protocol (PAP)

Contents *cont.*

Digital signature: Main definition. Digital signature based on Symmetric cryptosystem.

Hash Functions: Message authentication codes (MAC). MD5. SHA-1.

Digital Signature algorithms: RSA based digital signature. Digital Signature Standard (DSS). *ElGamal* signature scheme.

Digital Signature algorithms modifications: Blind signature. Group signature. Proxy signature.

Elliptic curve cryptography: Elliptic curve cryptosystem (ECC). Elliptic curve *Diffie-Hellman* algorithm. Elliptic curve *Menezes-Qu-Vanstone* cryptosystem.

Elliptic Curve Digital Signature Algorithm: ECDSA algorithm.

Quantum Cryptography: Quantum Key Distribution. BB84, B92, Entanglement-Based quantum key distribution.

Contents Cont.

Physical Cryptography: Physical unclonable function (PUF). Arbiter PUF. Ring oscillator PUF. SRAM based PUF.

Steganography: Textual steganography. Graphical steganography. LSB, BPCS, ABCDE and PCT steganography.

Watermarking and Fingerprinting: Patchwork method. Copyright Protection Watermarking for copy protection

Software protection: Software watermarking, obfuscation, and tamper-proofing. Software dongle. Electronic keys.

E-Commerce security: E-commerce security standards. SET protocol.

Internet Banking security: Online Banking Security. Password and PIN security:

1. Романец, Ю.В. Защита информации в компьютерных системах и сетях / Ю.В. Романец, П.А. Тимофеев, В.Ф. Шаньгин. – М.: Радио и связь, 1999. – 328 с.

2. Харин, Ю.С. Математические и компьютерные основы криптологии / Ю.С. Харин, В.И. Берник, Г.В. Матвеев, С.В. Агиевич. – Минск : Новое Знание, 2003. – 382 с.

3. Шнайер, Б. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си / Б. Шнаейр. – М. : ТРИУМФ, 2002. – 816 с.

4. Ярмолик, В.Н. Элементы теории информации. Практикум для студентов специальности “Программное обеспечение информационных технологий” / В.Н. Ярмолик, А.П. Занкович, С.С. Портянко. – Минск : БГУИР, 2007. – 40 с.

5. Ярмолик, В.Н. Криптография, стеганография и охрана авторского права / В.Н. Ярмолик, С.С. Портянко, С.В. Ярмолик. – Минск : Издательский центр БГУ, 2007. – с.

6. Грибунин, В.Г. Цифровая стеганография / В.Г. Грибунин, И. Н. Оков, И. В. Туринцев. – М. : СОЛОН-Пресс, 2002. – 272 с.

Introduction

Cryptography is the science and study of secret writing.

A **cipher** is a secret method of writing, whereby **plaintext** (or **cleartext**) is transformed into **ciphertext** (**cryptogram**).

Encipherment (**encryption**) is the process of transforming plaintext into ciphertext.

Decipherment (**decryption**) is the reverse process of transforming ciphertext into plaintext. Both encipherment and decipherment are controlled by a cryptographic **key** or **keys**.

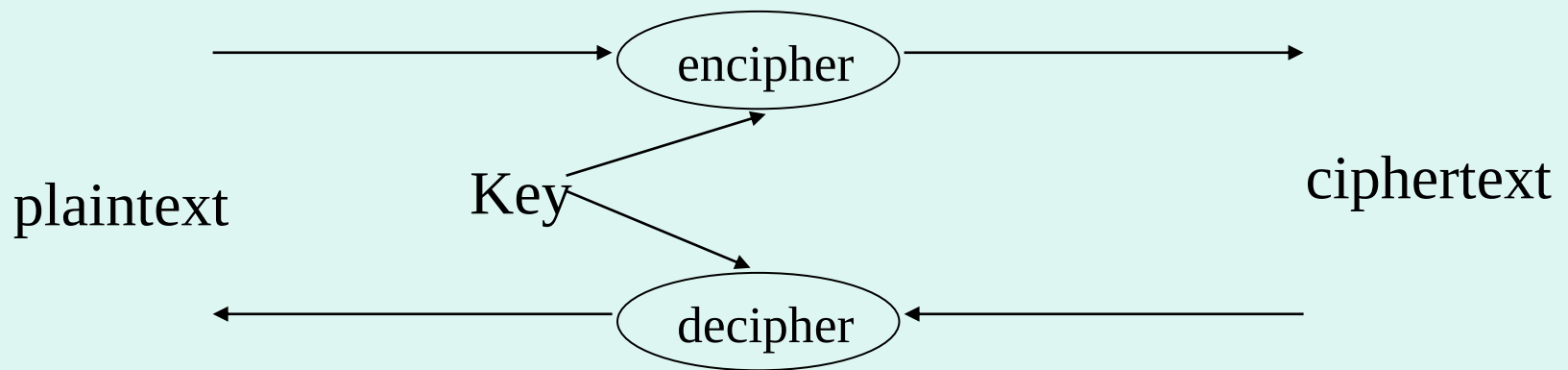


Fig.1.1. Secret writing

Introduction

Transposition ciphers

- There are two basic types of ciphers *transpositions* and *substitutions*.
- *Transposition ciphers* rearrange bits or characters.
- The following simple example of the “*rail-fence*” cipher illustrate this method.

C	R	Y	P	T	O	G	R	A	P	H	Y
				⇓							
C				T				A			
	R		P		O		R		P		Y
		Y				G				H	
				⇓							
C	T	A	R	P	O	R	P	Y	Y	G	H

Fig.1.2. Rail-fence transposition cipher

Introduction

Substitutions ciphers

Substitution ciphers replace bits, characters, or blocks of characters with substitutes.

A simplest type of substitution cipher shifts each letter in the English alphabet forward by ***k*** positions cyclically (shifts past Z cycle back to A). ***k*** is the key to the cipher. This type of cipher is often called a *Caesar* cipher.

C R Y P T O G R A P H Y



F U B S W R J U D S K B

Fig.1.3. Caesar's substitution cipher

ABCDEFGHIJKLMNOPQRSTUVWXYZ

Introduction

Data Security

There are two principle objectives: ***secrecy*** (or ***privacy***), to prevent the unauthorized disclosure of data; and ***authenticity*** or ***integrity***), to prevent the unauthorized modification of data.

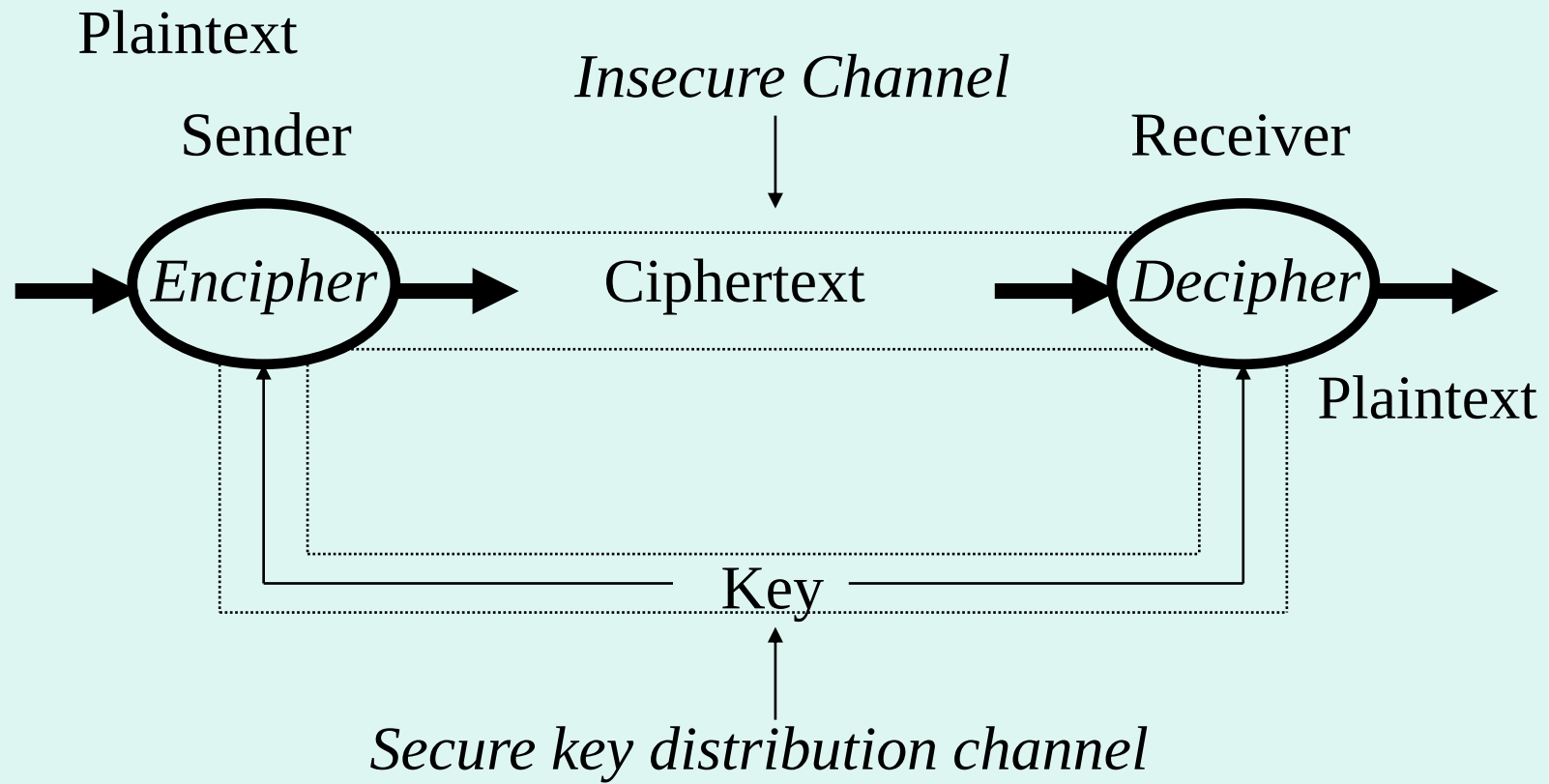


Fig.1.4. Classical information channel

Introduction

Cryptographic Systems

A *cryptographic system* (or *cryptosystem* for short) has five components:

1. A *plaintext message space*, M .
2. A *cipher message space*, C .
3. A *key space*, k .
4. A family of *enciphering transform.*, $E_k: M \rightarrow C$.
5. A family of *deciphering transform.*, $D_k: C \rightarrow M$.

Cryptosystems General Requirements

1. The system must be easy to use.
2. The enciphering and deciphering transformations must be efficient for all keys.
3. The security of the system should depend only on the secrecy of the keys and not on the secrecy of the algorithms ***E*** and ***D***.

Introduction

Requirement for secrecy and authenticity

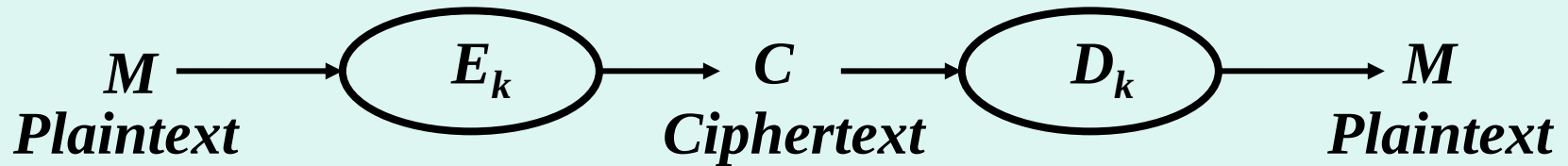


Fig.1.5. Cryptographic System.

Secrecy Requirements

1. It should be computationally infeasible for a cryptanalyst to systematically determine the deciphering transformation D_k from intercepted ciphertext C , even if the corresponding plaintext M is known.
2. It should be computationally infeasible for a cryptanalyst to systematically determine plaintext M from intercepted ciphertext C .

Authenticity Requirements

1. It should be computationally infeasible for a cryptanalyst to systematically determine the enciphering transformation E_k given C even if the corresponding plaintext M is known.
2. It should be computationally infeasible for a cryptanalyst to systematically find ciphertext C' such that $D_k(C')$ is valid plaintext in the set M .

Introduction

Simmons Cryptosystems Classifications

Simmons classifies cryptosystems as ***symmetric (one-key)*** and ***asymmetric (two-key)***.

In ***symmetric*** or ***one-key*** cryptosystems the enciphering and deciphering key are the same (or easily determined from each other).

This means the transformations E_k and D_k are also easily derived from each other. Until recently, all cryptosystems were one-key systems only. There are also usually referred to as ***conventional*** (or ***classical***) systems.

One-key systems provide an excellent way of enciphering user's private files. Each user A has private transformations E_k and D_k for enciphering and deciphering files.

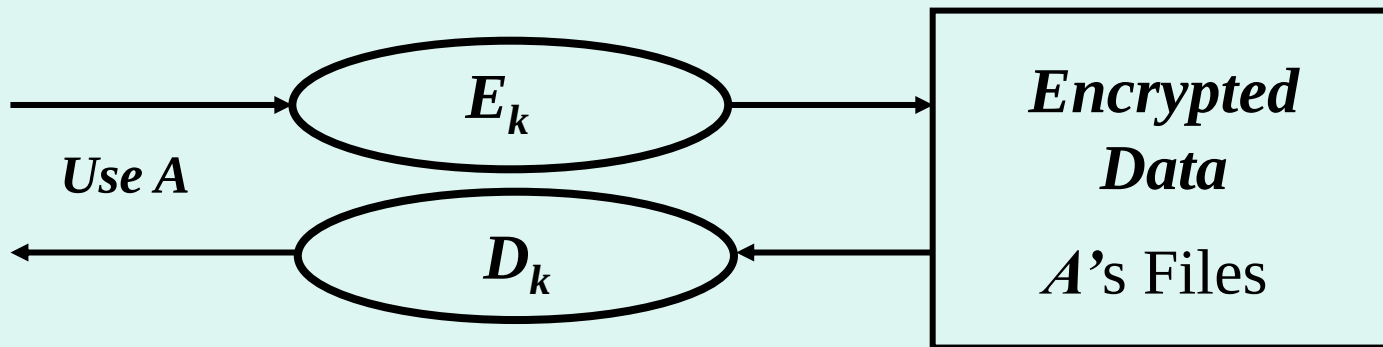


Fig.1.6. Single-key encryption of private files

Introduction

Public Key Cryptosystems

In a public-key system, each user A has a ***public transformation*** E_A , which may be registered with a public directory, and a ***private transformation*** D_A , which is known only to that user.

The private transformation D_A is described by a private key, and the public transformation E_A by a public key derived from the private key by one-way transformation. It must be computationally infeasible to determine D_A from E_A (or even to find a transformation equivalent to D_A).

In a public-key system, secrecy and authenticity are provided by the separate transformations. Suppose user A wishes to send a message M to another user B . If A knows B 's public transformation E_B , A can transmit M to B in secrecy by sending the ciphertext $C = E_B(M)$. On receipt, B deciphers C using B 's private transformation, getting

$$D_B(C) = D_B(E_B(M)) = M.$$

Introduction

Public Key Cryptosystems

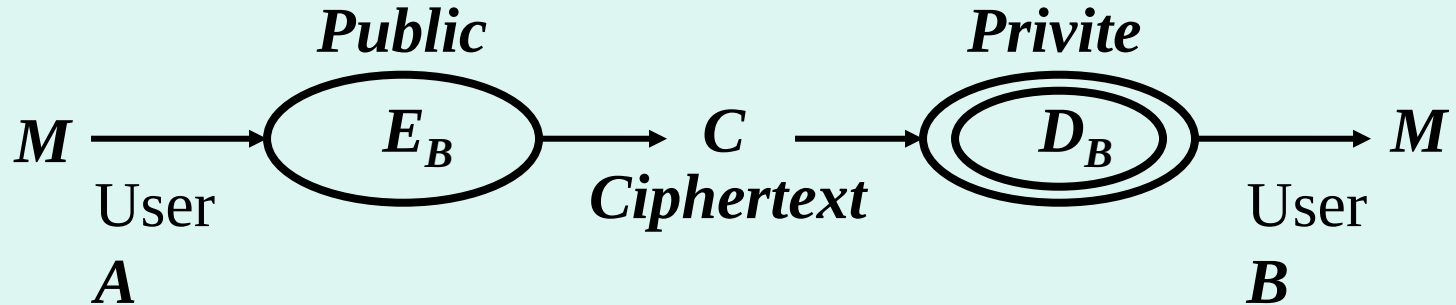


Fig.1.7. Secrecy in public-key system

For authenticity, M must be transformed by A's own private transformation D_A . Ignoring secrecy for the moment, A sends $C = D_A(M)$ to B. On receipt, B uses A's public transformation E_A to compute

$$E_A(C) = E_A(D_A(M)) = M$$

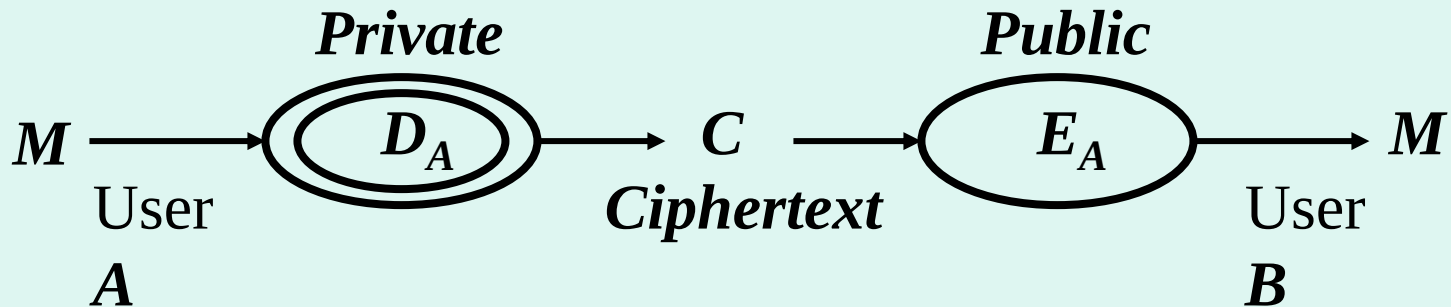


Fig.1.8. Authenticity in public-key system

Introduction

Public Key Cryptosystems

To achieve both secrecy and authenticity, the sender and receiver must each apply two sets of transformations. Sender A generates a ciphertext $C = E_B(D_A(M))$, and B recovers M according to

$$E_A(D_B(C)) = E_A(D_B(E_B(D_A(M)))) = E_A(D_A(M)) = M.$$

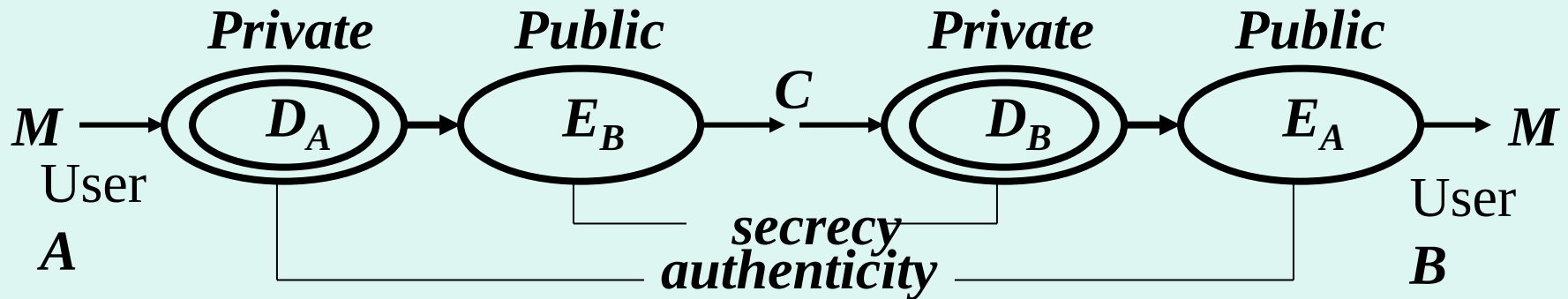


Fig.1.9. Secrecy and Authenticity in public-key system

Теория Информации

В.Н. Ярмолик

Лекция 2

2017

Encryption Algorithms

Transposition Cipher

A lot of codes are simple in design. Just by changing the order of words, letters, or the way read them can turn the message into secret code. The simplest examples of the Transposition Ciphers are:

Lumping Words (Format)

Get rid of spaces and returns to lump words together. Use upper case to make the code harder to read and decode.

Ciphertext: THISISHARDCODEFORMANYPEOPLE

Message: THIS IS HARD CODE FOR MANY PEOPLE

Character Bloks

Block letter of a message by 2,3 or more characters.

Ciphertext: TH IS IS HA RD CO DE FO RM AN YP EO PL E

Message: THIS IS HARD CODE FOR MANY PEOPLE

Backwards English

Writing words, sentences, or entire message backwards can be very confusing!

Ciphertext: SIHT SI DRAH EDOC ROF YNAM ELPOEP

Message: THIS IS HARD CODE FOR MANY PEOPLE

Reading SIHT backwards yields the answer THIS.

Encryption Algorithms

Transposition Cipher

Transposition cipher rearrange characters according to some scheme. This rearrangement was classically done with the aid of some type of geometric figure. Enciphering procedure consists of two steps as shown next:



First, the plaintext was written into the figure according to some “Write-in” path, then the ciphertext was taken off the figure according to some “Take-off” path.

Example 2.1. Suppose that the plaintext CRYPTOGRAPHY is written into 3 rows by 4 columns matrix by rows as follows:

<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>
C	R	Y	P
T	O	G	R
A	P	H	Y

If the columns are taken off in the order 3-1-4-2, the resulting ciphertext is YGHCTAPRYROP.

Encryption Algorithms

Transposition Cipher

Many transposition ciphers permute the characters of the plaintext with a ***fixed period*** d . The key for the cipher is given by the pair $K=(d,f)$, where d is the number of characters within the block and f is function of permutation. Thus, a plaintext message

$$M=m_1, \dots, m_d m_{d+1}, \dots, m_{2d}, \dots$$

is enciphered as

$$E_K(M)=m_{f(1)}, \dots, m_{f(d)} m_{f(d+1)}, \dots, m_{f(2d)}, \dots$$

Decipherment uses the inverse permutation.

Example. Suppose that $d=4$ and the function of permutation f is

i	1	2	3	4
$f(i)$	3	1	4	2

thus, the plaintext is divide into the blocks with 4 bits each, then for every block first plaintext character is moved to the second position, the second character to the fourth position and so forth.

M=CRYP TOGR APHY

Ek(M)=YCPR GTRO HAYP

Encryption Algorithms

Transposition Cipher

Like columnar transposition, periodic permutation cipher can be viewed as transpositions of the columns of a matrix in which the plaintext is written in by rows.

Example. Suppose that $d=12$ and the geometric figure is matrix 4 column by 3 rows and function of columns permutation f has a form

$$\begin{array}{ccccc} i & 1 & 2 & 3 & 4 \\ f(i) & 3 & 1 & 4 & 2 \end{array}$$

M = HERE IS A SECRET MESSAGE ENCIPHERED BY
TRANSPOSITION $\Rightarrow$

H	E	R	E
I	S	A	S
E	C	R	E

T	M	E	S
S	A	G	E
E	N	C	I

P	H	E	R
E	D	B	Y
T	R	A	N

S	P	O	S
I	T	I	O
N			

$E_k(M)$ =RAR HIE ESE ESC EGC TSE SEI MAN EBA PET RYN
HDR OI SIN SO PT

Encryption Algorithms

Transposition Cipher

Using a key word or phrase, such as, for example CONVENIENCE, assign a number to each letter in the word using this rule: the numbers are assigned starting with 1, and they are assigned first by *alphabetical order*, and second, where the same letter appears twice, by *position* in the word.

Example. Thus: the plaintext message M=HERE IS A SECRET MESSAGE ENCIPHERED BY TRANSPOSITION

C	O	N	V	E	N	I	E	N	C	E
1	10	7	11	3	8	6	4	9	2	5
H	E	R	E	I	S	A	S	E	C	R
E	T	M	E	S	S	A	G	E	E	N
C	I	P	H	E	R	E	D	B	Y	T
R	A	N	S	P	O	S	I	T	I	O
N										

Produces next ciphertext C=HECRN CEYI ISEP SGDI RNT
AAES RMPN SSRO EEBT ETIA EEHS

Encryption Algorithms

Transposition Cipher

Example. Another method of transposition is a Variant of column transposition that produces a different cipher:

<i>C</i>	<i>O</i>	<i>N</i>	<i>V</i>	<i>E</i>	<i>N</i>	<i>I</i>	<i>E</i>	<i>N</i>	<i>C</i>	<i>E</i>
1	10	7	11	3	8	6	4	9	2	5
H										
E	R	E	I	S	A	S	E	C	R	
E	T	M	E	S						
S	A	G	E	E	N	C	I			
P	H	E	R	E	D	B	Y	T	R	A
N	S	P	O	S	I	T				
I	O	N								

C=HEESPNI RR SSEES EIY A SCBT EMGEPN ANDI CT
RTAHSO IEERO

This method has the advantage of dividing the text being transposed in a more irregular fashion than ordinary columnar transposition.

Encryption Algorithms

Transposition Cipher

Another interesting form of transposition is the "turning grille", used by Germany during the First World War.

A square grille, divided into a grid of squares, one-quarter of which are punched with holes, is placed over a sheet of paper. The message is written on the paper through the holes, and then the grille is rotated by 90 degrees, and then the message continues to be written, as the grille is turned through all four possible positions.

The trick to designing such a grille is to divide the grille into quarters, numbering the squares in each quarter so that as the grille is rotated, corresponding squares have the same number. Then, choose one square with each number for punching.

Example. Example of a turning grille and its use:

Grid
numbering:

1	2	3	4	1
4	5	6	5	2
3	6	7	6	3
2	5	6	5	4
1	4	3	2	1

Layout:

<u>1</u>	2	<u>3</u>	4	1
4	5	6	5	<u>2</u>
3	6	<u>7</u>	<u>6</u>	3
2	<u>5</u>	6	5	4
1	<u>4</u>	3	2	1

Encryption Algorithms

Transposition Cipher

Example. Example of a turning grille and its use for message:
M=HERE IS A SECRET MESSAGE WRITE

First Round

<u>1</u>	2	<u>3</u>	4	1
4	5	6	5	<u>2</u>
3	6	<u>7</u>	<u>6</u>	3
2	<u>5</u>	6	5	4
1	<u>4</u>	3	2	1

H		E		
				R
		E	I	
	S			
	A			

Second Round

1	2	3	4	<u>1</u>
<u>4</u>	<u>5</u>	6	5	2
3	6	*	6	<u>3</u>
2	5	<u>6</u>	5	4
1	4	3	<u>2</u>	1

H		E		S
E	C			R
		E	I	R
	S	E		
	A		T	

Encryption Algorithms

Transposition Cipher

Example. M=HERE IS A SECRET MESSAGE WRITE

Third Round

1	2	3	<u>4</u>	1
4	5	6	<u>5</u>	2
3	<u>6</u>	*	6	3
<u>2</u>	5	6	5	4
1	4	<u>3</u>	2	<u>1</u>

H		E	M	S
E	C		E	R
	S	E	I	R
S	S	E		
	A	A	T	G

Fourth Round

1	<u>2</u>	3	4	1
4	5	<u>6</u>	5	2
<u>3</u>	6	*	6	3
2	5	6	<u>5</u>	<u>4</u>
<u>1</u>	4	3	2	1

H	E	E	M	S
E	C	W	E	R
R	S	E	I	R
S	S	E	I	T
E	A	A	T	G

C=HEEMS ECWER RSEIR SSEIR SSEIT EAATG

Encryption Algorithms

Simple Substitution Ciphers

Simple substitution cipher replace each character of plaintext with a corresponding character of ciphertext; a single one-to-one mapping from plaintext to ciphertext characters is used.

A **simple substitution** cipher replace each character of an ordered plaintext alphabet, denoted as A , with the corresponding character of an ordered cipher alphabet, denoted of C . Suppose A is an n -character alphabet $\{a_0, a_1, \dots, a_n\}$, then C is an n -character alphabet $\{f(a_0), f(a_1), \dots, f(a_n)\}$, where $f: A \rightarrow C$ is a one-to-one mapping of each character of A to the corresponding character of C . The key K to the cipher is given by C or, equivalently, by the function f .

To encipher, a plaintext message $M = m_1 m_2 \dots$ is written as the ciphertext message $E_K(M) = f(m_1) f(m_2) \dots$

Example 2.7. Suppose that f maps English alphabet $A = \{a_0, a_1, \dots, a_n\}$ into the cipher alphabet C as:

$A: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z$

$C: Y A R M O L I K B C D E F G H J N P Q S T U V W X Z$

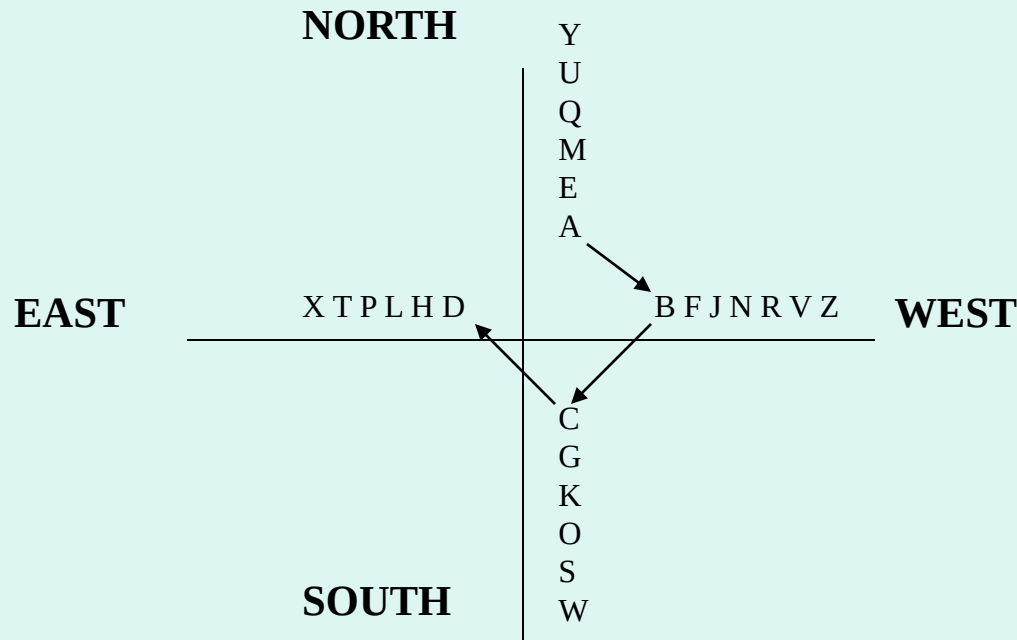
then the plaintext CRYPTOGRAPHY is enciphered as:
 $E_K(M) = R P X J S H I P Y J K X$

Encryption Algorithms

Simple Substitution Ciphers

Compass Cipher-A Method for Alphabet Substitution

There are lots of ways to create a compass cipher key.



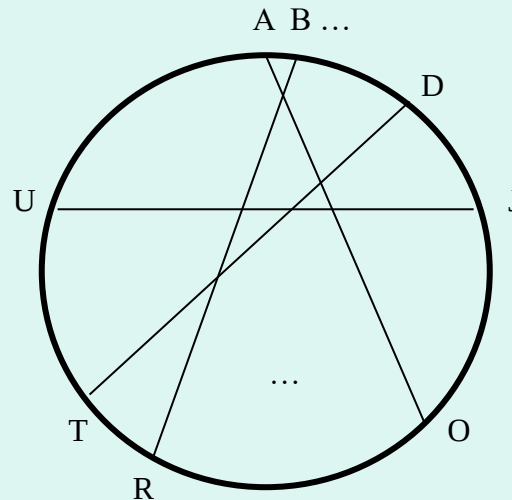
The arrows show the direction used to fill in the compass lines. Once you have all the letters written into your compass write out the letters, from outside in and then combine all of the letters into a long line. **Copass Key:** YUQMEAZVRNJFBWSOKGCXTPLHD

Encryption Algorithms

Simple Substitution Ciphers

Letter Spokes Cipher Clock

It's another way of making up a random alphabet substitution wheel. The letter spokes cipher clock, developed by Angie Wimer, resemble the spokes of a wheel. According to his idea you have to simply take each letter in the wheel and line it up with another letter somewhere else in the wheel.



See if you can decipher the encrypted message “ROT” by making the use of the Letter Spokes clock below the answer is “BAD”.

Encryption Algorithms

Simple Substitution Ciphers

Alphabet & Word Correlated Ciphers.

Sometimes keywords are used to help encipher a message. By writing out a sentence(s) that contains all the letters of the alphabet you can correlate plain alphabet letters within a word found in the keyword phase to make the cipherring a bit more difficult to decipher.

The keyword phrase “***The quick brown fox jumps over the lazy dog***” contains all the letters of the alphabet. Even though some letters appear more than once it shall serve as a keyword phrase because it still contains all the letters of the alphabet. To get started, write out your keyword phrase and number each word:

THE	QUICK	BROWN	FOX	JUMPS	OVER	THE	LAZY	DOG
1	2	3	4	5	6	7	8	9

Plaintext: ***Johnson is a spy***

Ciphertext:

J	O	H	N	S	O	N	I	S	...
51	33	12	35	55	33	35	23	55	

Encryption Algorithms

Simple Substitution Ciphers

Cipher based on **shifted alphabet** (“direct standard alphabets”) shift the letters of the alphabet to the right by k positions, modulo the size of the alphabet according to the equation

$$c = (a + k) \bmod n,$$

where n is the size of the alphabet and a denotes letter and its position in A .

For the English alphabet

0-A, 1-B, 2-C, 3-D, 4-E, 5-F, 6-G, 7-H, 8-I, 9-J, 10-K, 11-L, 12-M, 13-N, 14-O, 15-P, 16-Q, 17-R, 18-S, 19-T, 20-U, 21-V, 22-W, 23-X, 24-Y, 25-Z.

Under the Caesar cipher, our plaintext CRYPTOGRAPHY is enciphered by

$$f(a) = (a + 3) \bmod 26,$$

then $E_K(M) = \text{FUBSWRJUDSKB}$.

There are more complex transformation of the plaintext alphabet. Ciphers based on multiplications (“decimations”) are shown below

$$f(a) = ka \bmod n,$$

where k and n are relatively prime so that the letter of the alphabet produce a complete set of residues. For the plaintext CRYPTOGRAPHY and $k=3$ we will get $f(a) = 3a \bmod 26$, and $E_K(M) = \text{GZUTFQSZATVU}$.

Encryption Algorithms

Simple Substitution Ciphers

Addition (shifting) and multiplication can be combined to give an ***affine transformation***

$$f(a) = (ak_1 + k_0) \bmod n,$$

where k_1 and n are relatively prime.

Higher-order transformations are obtained with ***polynomial transformations*** of degree t :

$$f(a) = (a^t k_t + a^{t-1} k_{t-1} + \dots + ak_1 + k_0) \bmod n,$$

Some substitution cipher use nonstandard ciphertext alphabet. The ***Churchyard cipher*** shown below was engraved on a tombstone in Trinity Churchyard, New York, in 1794.

A*	B*	C*
D*	E*	F*
G*	H*	I-J*

K**	L**	M**
N**	O**	P**
Q**	R**	S**

T	U	V
W	X	Y
Z		

What is the plaintext corresponding to the next ciphertext?

**	*	**	*	**	*	*	**
		*	*	*		*	

Encryption Algorithms

Single-Letter Frequency Analysis

Under the Caesar cipher $c = (a+3) \bmod 26$ with the key $k=3$ the plaintext

M = REMEMBER

is enciphered as

C = UHPHPEHU

0-A, 1-B, 2-C, 3-D, 4-E, 5-F, 6-G, 7-H, 8-I, 9-J, 10-K, 11-L, 12-M, 13-N, 14-O, 15-P, 16-Q, 17-R, 18-S, 19-T, 20-U, 21-V, 22-W, 23-X, 24-Y, 25-Z.

$$c(H) = 7; a(E) = 4. \quad 7 = (4 + k) \bmod 26. \quad k = 3. \quad k^* = 26 - 3 = 23.$$

$$a = (c + k^*) \bmod n. \quad a = (c + 23) \bmod 26.$$

$$a = (c(U) + 23) \bmod 26 = (20 + 23) \bmod 26 = 17.$$

$$a = (c(P) + 23) \bmod 26 = (15 + 23) \bmod 26 = 12.$$

$$a = (c(E) + 23) \bmod 26 = (4 + 23) \bmod 26 = 1.$$

Encryption Algorithms

Single-Letter Frequency Analysis

Simple substitution cipher are generally easy to break in a ciphertext-only attack using single-letter frequency distributions. The letters of English alphabet can be divided into subsets of **high**, **medium**, **low**, and **rare** frequency as shown below:

high:	E T A O N I R S H
medium:	D L U C M
low:	P F Y W G B V
rare:	J K Q X Z

By comparing the letter frequencies in a given ciphertext with the expected frequencies, a cryptanalyst can match the ciphertext letter with the plaintext letters. Diagram and trigram distributions are also helpful.

Ciphers based on shifted alphabets are usually easy to solve, because each ciphertext letter is a constant distance from its corresponding plaintext letter. Cipher based on affine transformations

$$f(a)=(ak_1+k_0) \bmod n,$$

are somewhat trickier. To determine the values of k_1 and k_0 the system of two linear equations have to be solved. So, for the two pairs of $(f(a),a)$: $(10,4)$, $(19,9)$ we have two equations

$$10=(4k_1+k_0) \bmod 26,$$

$$19=(9k_1+k_0) \bmod 26,$$

then subtracting (1) from (2) we get $9=5k_1 \bmod 26$. The last equation can be solved to determine $k_1=7$. Substituting in (1) value of k_1 gives $(28+k_0) \bmod 26=10$. 18

Encryption Algorithms

Homophonic Substitution cipher

A homophonic substitution cipher maps each character a of the plaintext alphabet into a set of ciphertext elements $f(a)$ called homophones. A plaintext message $M=m_1m_2\dots$ is enciphered as $C=c_1c_2\dots$, where each c_i is picked at random from the set of homophones $f(m_i)$.

Example. Let the English letters are enciphered as integers between 00 and 99, where the number of integers assigned to a letter is proportional to the relative frequency of the letter, and no integer is assigned to more than one letter. A possible assignment of integers to the letters in the message CRYPTOGRAPHY are show below (integer assignment for the remaining letters of the alphabet are not given)

letter	Homophones
A	23, 25, 97, 95, 81, 33, 12, 11
C	44, 77, 34, 51
G	87, 41
H	59, 90, 00, 26
O	66, 02, 15, 22, 09, 83, 54
P	04, 58
R	38, 07, 94, 30, 56, 67
T	55, 71, 72, 80, 01, 12, 29, 50, 68
Y	88

One possible encipherment of the message is:

$C = 77\ 07\ 88\ 58\ 72\ 54\ 41\ 30\ 97\ 04\ 00\ 88$

Encryption Algorithms

Homophonic Substitution cipher

1. Single letter frequency distributions.

A	0.0804	H	0.0549	O	0.0760	V	0.0099
B	0.0154	I	0.0726	P	0.0200	W	0.0192
C	0.0306	J	0.0016	Q	0.0011	X	0.0019
D	0.0399	K	0.0067	R	0.0612	Y	0.0173
E	0.1251	L	0.0414	S	0.0654	Z	0.0009
F	0.0230	M	0.0253	T	0.0925		
G	0.0196	N	0.0709	U	0.0271		

Encryption Algorithms

Beale Ciphers

The ***Beale cipher*** is an example of a homophonic substitution cipher. The key is the Declaration of Independence. The words of the Declaration are numbered consecutively as shown below

Declaration of Independence (first 100 words)

01 When, in the course of human events, it becomes necessary
11 for one people to dissolve the political bands which have
21 connected them with another, and to assume among the Powers
31 of the earth the separate and equal station to which
41 the Laws of Nature and of Nature's God entitle them,
51 a decent respect to the opinions of mankind requires that
61 they should declare the causes which impel them to the
71 separation. We hold these truths to be self-evident; that
81 all men are created equal, that they are endowed by
91 their Creator with certain unalienable rights; that among

Beale enciphered each letter in the plaintext message by substituting the number of some word which started with that letter. The letter W, for example, was enciphered with the number 1, 19, 40, 66, 72, As an example let consider the word PLAIN. The result of enciphering are shown below:

$M =$	P	L	A	I	N
$C =$	13	42	81	08	44

Encryption Algorithms

Polygram Substitution Cipher

All of the preceding substitution cipher encipher a single letter of plaintext at a time. By enciphering larger blocks of letters, ***polygram substitution ciphers*** make cryptanalysis harder by destroying the significance of single-letter frequencies.

The Playfair cipher is a diagram substitution cipher named after the English scientist Lyon Playfair; the cipher was actually invented in 1854 by Playfair's friend, Charles Wheatstone, and was used the British during World War I. The key of this cipher is given by a 5 by 5 matrix of 25 letters (J was not used), such as the one shown below.

Key for Playfair cipher

Y	A	R	M
L	I	K	B
D	E	F	G
V	N	P	Q
T	U	W	X

Playfair cipher enciphering rule

Each pair of plaintext letters m_1m_2 is enciphering according to the following rules:

1. If m_1 and m_2 are in the same row, then c_1 and c_2 are the two characters to the right of m_1 and m_2 , respectively, where the first column is considered to be to the right of the last column.
2. If m_1 and m_2 are in the same column, then c_1 and c_2 are the two characters below m_1 and m_2 , respectively, where the first row is considered to be below last row.

3. If m_1 and m_2 are in different rows and columns, then c_1 and c_2 are the other two corner of the rectangle having m_1 and m_2 , as corners, where c_1 is m_1 's row and c_2 is in m_2 's row.

4. If $m_1 = m_2$, a null letter (e.g., X) is inserted into the plaintext between m_1 and m_2 to eliminate the double.

5. If the plaintext has an odd number of characters, a null letter is appended to the end of the plaintext.

2. Encryption Algorithms

2.6. Polygram Substitution Cipher

Playfair has inspired some related **bigraphic** ciphers that, on the one hand, improve security by involving multiple, unrelated alphabets, but on the other hand, are simpler in that they use fewer rules than Playfair.

In the Four-Square cipher, two squares are used to find the two plaintext letters, and two others are used to find the two ciphertext letters:

M	W	X	Y	M	E	P	T	O	L
R	J	E	K	I	C	V	I	Y	Z
U	V	<u>H</u>	P	S	R	M	A	G	B
A	L	B	Z	N	F	W	J	H	S
G	C	O	F	T	U	N	D	X	K
J	T	B	U	E	V	I	M	A	G
Z	H	N	D	X	S	<u>W</u>	P	O	H
L	A	F	R	G	U	T	Z	K	E
P	M	I	Y	C	N	R	D	X	Y
V	S	K	W	O	B	J	L	C	F

Thus, the diagram $m_1m_2=HW$ becomes $c_1c_2=MN$.

Encryption Algorithms

Bifid Cipher

The first of the three rules for Playfair encipherment changes one two-letter group, or digraph, to another by exchanging column co-ordinates. This suggests using row and column co-ordinates in a more general fashion. Let's take the 5 by 5 square above, but number the rows and the columns, like this:

	1	2	3	4	5
1	T	X	V	H	R
2	L	K	M	U	P
3	N	Z	O	Q	E
4	C	G	W	Y	A
5	F	B	S	D	I

Then, another method of encipherment would be as follows: Divide a message into groups of letters of a fixed length, say five letters, and write the row and then the column co-ordinate of each letter beneath it, like this:

T	H	I	S	I	S	M	Y	S	E	C	R	E	T	M
1	1	5	5	5	5	2	4	5	3	4	1	3	1	2
1	4	5	3	5	3	3	4	3	5	1	5	5	1	3

and then, going across within each group, read the numbers in order, and turn them, in pairs, into letters: that is, read 11555 14535 52453 33435... and turn them into the letters corresponding to 11, 55, 51, 45, 35, 52, and so on.

E	S	S	A	G	E
3	5	5	4	4	3
5	3	3	5	2	5

11	55	51	45	35	52	45	33	34	35	41	31	21	55	13	35	54	45	33	52	35
T	I	F	A	E	B	A	O	J	E	C	N	L	I	V	E	D	A	O	B	E

Encryption Algorithms

Trifid Cipher

This is the ***Bifid cipher*** of Delastelle, and the general principle of this form of cipher is called ***seriation***. This is one of the most secure pencil-and-paper ciphers that is still used by hobbyists as a puzzle. It isn't hard to make this kind of cipher just a little bit more complicated, and thereby obtain one that is genuinely secure. It belongs to the class of cipher methods known as ***fractionation***, where letters are divided into smaller pieces, or "fractions". Just as two symbols from 1 to 5 give 25 letters, three symbols from 1 to 3 give 27 letters; and five binary bits provide a 32-character alphabet.

The ***Trifid***, also due to Delastelle, is the analogous cipher using a 27-letter alphabet represented by three symbols from 1 to 3:

W 111	N 211	C 311	A 112	E 212	X 312	K 113	Q 213	I 313
M 121	O 221	T 321	& 122	V 222	J 322	B 123	R 223	F 323
Z 131	L 231	U 331	Y 132	P 232	G 332	H 133	S 233	D 333

to encipher a message by seriation like this:

T	H	I	S	I	S	M		Y	S	E	C	R	E	T		M	E	S	S	A	G	E
3	1	3	2	3	2	1		1	2	2	3	2	2	3		1	2	2	2	1	3	2
2	3	1	3	1	3	2		3	3	1	1	2	1	2		2	1	3	3	1	3	1
1	3	3	3	3	3	1		2	3	2	1	3	2	1		1	2	3	3	2	2	2

which again is read off horizontally after being written in vertically, yielding a cipher message like this: 313-I, 232-P, 123-B, 131-Z, 321-T, 333-D, 331-U, 122-&, 322-J, 333-D, 112-A 122-&, 321-T, 321-T, 122-&, 213-Q, 221-O, 331-U, 311-C, 233-S, 222-V.

Encryption Algorithms

The Straddling Checkerboard

Some ciphers actually used by Soviet spies used a square like this:

	9	8	2	7	0	1	6	4	3	5
	A	T		O	N	E		S	I	R
2	B	C	D	F	G	H	J	K	L	M
6	P	Q	U	V	W	X	Y	Z	.	/

Eight of the most common letters are translated to a single digit. The two digits not used in this way begin two-digit combinations that stand for the remaining letters. This is an example of a ***variable length code*** with the ***prefix property***. When it is possible to tell, from the digits one has already seen of a symbol, whether or not one needs to include the next digit in the symbol, then spaces between the digits of a symbol are not needed, and this is what is known as the prefix property.

Of course, the second digit of a two-digit combination could also have stood, by itself, for another letter; but because when you start from the beginning and move forwards, there is no chance of confusion, this is a workable and usable system. Thus, the message SENDMONEY would become 4 1 0 22 25 7 0 1 66, or, rather, 41022 25701 66 because spaces to show where the letters begin are not needed; the first digit representing a letter determines if its substitute is one or two digits long. More complicated codes that work this way, using only the two binary digits 0 and 1, are used as a form of data compression. The most famous variable-length prefix-property binary codes are the Huffman codes;

Теория Информации

В.Н. Ярмолик

Лекция 3

2017

Encryption Algorithms

The Straddling Checkerboard

Some ciphers actually used by Soviet spies used a square like this:

	9	8	2	7	0	1	6	4	3	5
	A	T		O	N	E		S	I	R
2	B	C	D	F	G	H	J	K	L	M
6	P	Q	U	V	W	X	Y	Z	.	/

Eight of the most common letters are translated to a single digit. The two digits not used in this way begin two-digit combinations that stand for the remaining letters. This is an example of a ***variable length code*** with the ***prefix property***. When it is possible to tell, from the digits one has already seen of a symbol, whether or not one needs to include the next digit in the symbol, then spaces between the digits of a symbol are not needed, and this is what is known as the prefix property.

Encryption Algorithms

The Straddling Checkerboard

Of course, the second digit of a two-digit combination could also have stood, by itself, for another letter; but because when you start from the beginning and move forwards, there is no chance of confusion, this is a workable and usable system. Thus, the message SENDMONEY would become 4 1 0 22 25 7 0 1 66, or, rather, 41022 25701 66 because spaces to show where the letters begin are not needed; the first digit representing a letter determines if its substitute is one or two digits long. More complicated codes that work this way, using only the two binary digits 0 and 1, are used as a form of data compression. The most famous variable-length prefix-property binary codes are the Huffman codes;

Encryption Algorithms

The VIC Cipher

The VIC cipher is an intricate cipher issued by the Soviet Union to at least one of its spies. It is of interest because it seems highly secure, despite being a pencil-and-paper cipher. It was the cipher in which a message was written which was found on a piece of microfilm inside a hollowed-out nickel by a newspaper boy in 1953.

The VIC cipher, which we will demonstrate here adapted to the sending of English-language messages, begins with an involved procedure to produce ten pseudorandom digits. The agent must have memorized:

1. Six digits (which were in the form of a date);
2. The first 20 letters of a key phrase (which was the beginning of a popular song);
3. Five random digits for use as a message indicator.

Let the date be July 4, 1776, to give the digits 741776. (Actually, the Russians used their customary form of dates, with the month second.) And let the random indicator group be 77651.

Encryption Algorithms

The VIC Cipher

1. The first step is to perform digit by digit subtraction (without carries) of the first five digits of the date from the indicator group:

7	7	6	5	1
7	4	1	7	7
--	--	--	--	--
0	3	5	8	4

2. The second step is to take the 20-letter keyphrase, and turn it into 20 digits by dividing it into two halves, and within each half, assigning 1 to the letter earliest in the alphabet, and so on, treating 0 as the last number, and assigning digits in order to identical letters. Thus, if our keyphrase is "I dream of Jeannie with t", that step proceeds:

I	D	R	E	A	M	O	F	J	E		A	N	N	I	E	W	I	T	H	T
6	2	0	3	1	8	9	5	7	4		1	6	7	4	2	0	5	8	3	9

Encryption Algorithms

The VIC Cipher

3. The result of the first step is then expanded to ten digits through a process called ***chain addition***. Starting with a group of a certain number of digits (in this case five: {0 3 5 8 4}, and later we will do the same thing with a group of ten digits), add the first two digits $0+3=3$ in the group together, take only the last digit 3 of the result and append it to the end of the group, then ignore the first digit, and repeat the process $3+5=8$, $5+8=3$, $8+4=2$, $4+3=7$.

4. The 10 digit result 0 3 5 8 4 3 8 3 2 7 is then added, digit by digit, ignoring carries, to the first 10 digits produced from the keyphrase to produce a ten-digit result, as follows:

$$\begin{array}{r} 6\ 2\ 0\ 3\ 1\ 8\ 9\ 5\ 7\ 4\ + \\ 0\ 3\ 5\ 8\ 4\ 3\ 8\ 3\ 2\ 7\ = \\ \hline 6\ 5\ 5\ 1\ 5\ 1\ 7\ 8\ 9\ 1. \end{array}$$

Encryption Algorithms

The VIC Cipher

5. And these **Ten-digits result** are then encoded according to the following algorithm based on **Second half** of keyphrase.

Index	1 2 3 4 5 6 7 8 9 0
Second half	1 6 7 4 2 0 5 8 3 9
Ten-digits result	6 5 5 1 5 1 7 8 9 1
Result	<u>0 2 2 1 2 1 5 8 3 1</u>

6. This ten digit number is used by chain addition to generate 50 pseudorandom digits for use in encipherment: 0 2 2 1 2 1 5 8 3 1 * 2 4
 3 3 3 6 3 1 4 3 * 6 7 6 6 9 9 4 5 7 9 * 3 3 2 5 8 3 9 2 6 2 * 6 5 7 3 1 2 1
 8 8 8 * 1 2 0 4 3 3 9 6 6 9

7. The last row (10 digits) of these digits (which will still be used again) is used like the letters in a keyword for transposition to produce a permutation of the digits 1 through 9 (with 0 last again): 1 2
 0 4 3 3 9 6 6 9 $\Rightarrow$ 1 2 0 5 3 4 8 6 7 9 and those digits are used as the top row of numbers for a straddling checkerboard:

Encryption Algorithms

The VIC Cipher

	1	2	0	5	3	4	8	6	7	9
	A	T		O	N	E		S	I	R
0	B	C	D	F	G	H	J	K	L	M
8	P	Q	U	V	W	X	Y	Z	.	/

Encryption Algorithms

Porta Table

Giovanni Baptista della Porta developed the Porta Table cipher method in 1565. The table uses a keyword and the table below to encipher message.

AB	A	B	C	D	E	F	G	H	I	J	K	L	M
	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
CD	A	B	C	D	E	F	G	H	I	J	K	L	M
	Z	N	O	P	Q	R	S	T	U	V	W	X	Y
EF	A	B	C	D	E	F	G	H	I	J	K	L	M
	Y	Z	N	O	P	Q	R	S	T	U	V	W	X
GH	A	B	C	D	E	F	G	H	I	J	K	L	M
	X	Y	Z	N	O	P	Q	R	S	T	U	V	W
IJ	A	B	C	D	E	F	G	H	I	J	K	L	M
	W	X	Y	Z	N	O	P	Q	R	S	T	U	V
KL	A	B	C	D	E	F	G	H	I	J	K	L	M
	V	W	X	Y	Z	N	O	P	Q	R	S	T	U
MN	A	B	C	D	E	F	G	H	I	J	K	L	M
	U	V	W	X	Y	Z	N	O	P	Q	R	S	T
OP	A	B	C	D	E	F	G	H	I	J	K	L	M
	T	U	V	W	X	Y	Z	N	O	P	Q	R	S
QR	A	B	C	D	E	F	G	H	I	J	K	L	M
	S	T	U	V	W	X	Y	Z	N	O	P	Q	R

Encryption Algorithms

Porta Table

ST	A	B	C	D	E	F	G	H	I	J	K	L	M
	R	S	T	U	V	W	X	Y	Z	N	O	P	Q
UV	A	B	C	D	E	F	G	H	I	J	K	L	M
	Q	R	S	T	U	V	W	X	Y	Z	N	O	P
WX	A	B	C	D	E	F	G	H	I	J	K	L	M
	P	Q	R	S	T	U	V	W	X	Y	Z	N	O
YZ	A	B	C	D	E	F	G	H	I	J	K	L	M
	O	P	Q	R	S	T	U	V	W	X	Y	Z	N

To begin, write out your plain message and write out the keyword above it, as shown below:

Keyword: JACKET

Message: LOOK UNDER THE COUCH

J	A	C	K		E	T	J	A	C		K	E	T		J	A	C	K	E
L	O	O	K		U	N	D	E	R		T	H	E		C	O	U	C	H

Encryption Algorithms

Porta Table

The next step is to use the Porta table to create the enciphered message. Use the letters from the keyword (JACKET in the example above) to locate the correct line to use in the Porta table. In the example above “J” is the first keyword letter. Thus, locate “J” on the left hand side of the Porta table. Once you locate the “J”, the 5th set of letters in the Porta table, you use the letter from the plain message to find the enciphered letter above or below it. In this example the value for “L” in the “J” set is “U”.

Ciphertext: UBCS JJZRF LSV YBIXS

Encryption Algorithms

Vigenere Cipher

	ABCDEFGHIJKLMNOPQRSTUVWXYZ
A	ABCDEFGHIJKLMNOPQRSTUVWXYZ
B	BCDEFGHIJKLMNOPQRSTUVWXYZA
C	CDEFGHIJKLMNOPQRSTUVWXYZAB
D	DEFGHIJKLMNOPQRSTUVWXYZABC
E	EFGHIJKLMNOPQRSTUVWXYZABCD
F	FGHIJKLMNOPQRSTUVWXYZABCDE
G	GHIJKLMNOPQRSTUVWXYZABCDEF
H	HIJKLMNOPQRSTUVWXYZABCDEFG
I	IJKLMNOPQRSTUVWXYZABCDEFGH
J	JKLMNOPQRSTUVWXYZABCDEFGHI
K	KLMNOPQRSTUVWXYZABCDEFGHIJ
L	LMNOPQRSTUVWXYZABCDEFGHIJK
M	MNOPQRSTUVWXYZABCDEFGHIJKL
N	NOPQRSTUVWXYZABCDEFGHIJKLM
O	OPQRSTUVWXYZABCDEFGHIJKLMN
P	PQRSTUVWXYZABCDEFGHIJKLMNO
Q	QRSTUVWXYZABCDEFGHIJKLMNOP
R	RSTUVWXYZABCDEFGHIJKLMNOPQ
S	STUVWXYZABCDEFGHIJKLMNOPQR
T	TUVWXYZABCDEFGHIJKLMNOPQRS
U	UVWXYZABCDEFGHIJKLMNOPQRST
V	VWXYZABCDEFGHIJKLMNOPQRSTU
W	WXYZABCDEFGHIJKLMNOPQRSTUV
X	XYZABCDEFGHIJKLMNOPQRSTUVW
Y	YZABCDEFGHIJKLMNOPQRSTUVWX
Z	ZABCDEFGHIJKLMNOPQRSTUVWXY

Example The encipher of the word
CRYPTOGRAPHY under the
key RAND is shown below

$M = \text{CRYP TOGR APHY}$

$K = \text{BAND BAND BAND}$

$E_K(M) = \text{DRLS UOTU BPUB}$

In this example, the first letter of each four-letter group is shifted (mod 26) by 1, the second by 0, the third by 13, and the fourth by 3.

Encryption Algorithms

Vigenere Cipher

The message "Wish you were here" can be encrypted by the three possible methods, using SIAMESE as the keyword:

Straight keyword:

Message:	W	I	S	H	Y	O	U	W	E	R	E	H	E	R	E
Key:	S	I	A	M	E	S	E	S	I	A	M	E	S	E	S
Cipher:	O	Q	S	T	C	G	Y	O	M	R	Q	L	W	V	W

Progressive key:

Message:	W	I	S	H	Y	O	U	W	E	R	E	H	E	R	E
Key:	S	I	A	M	E	S	E	T	J	B	N	F	T	F	U
Cipher:	O	Q	S	T	C	G	Y	P	N	S	R	M	X	W	Y

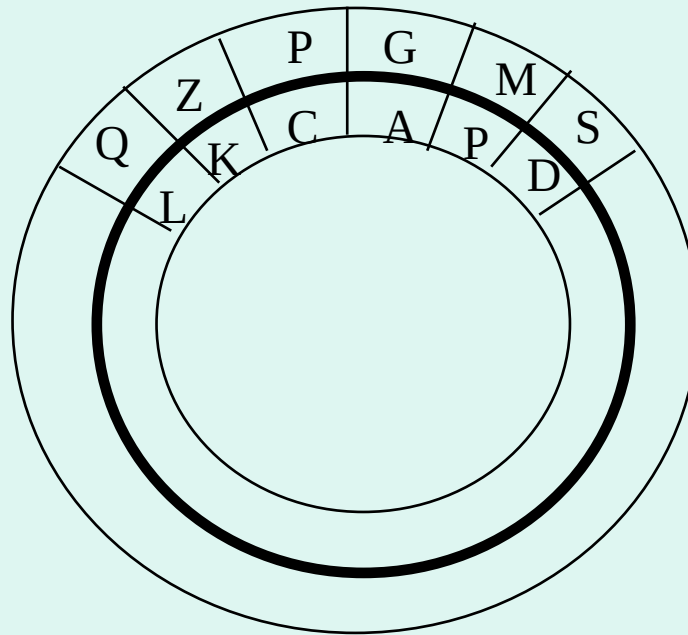
Autokey:

Message:	W	I	S	H	Y	O	U	W	E	R	E	H	E	R	E
Key:	S	I	A	M	E	S	E	W	I	S	H	Y	O	U	W
Cipher:	O	Q	S	T	C	G	Y	S	M	J	L	F	S	L	A

Encryption Algorithms

Polyalphabetic Substitution Ciphers

Simple Substitution cipher use a single mapping from plaintext to ciphertext letters, the single-letter frequency distribution of the plaintext letters is preserved in the ciphertext. Homophonic substitution conceal this distribution by defining multiple ciphertext elements for each plaintext letter. ***Polyalphabetic substitution ciphers*** conceal it by using multiple substitutions.



Encryption Algorithms

Polyalphabetic Substitution Ciphers

In 1568, Alberti published a manuscript describing a cipher disk that defined multiple substitutions. The disk defined n (where n is as an example the number of English letters) possible substitutions from the plaintext letters in the outer ring to the ciphertext letters in the inner ring, depending on the position of the disks. Alberti's important insight was his realization that the substitution could be changed during encipherment by turning the inner disk.

Encryption Algorithms

Rotor's Machines Bases

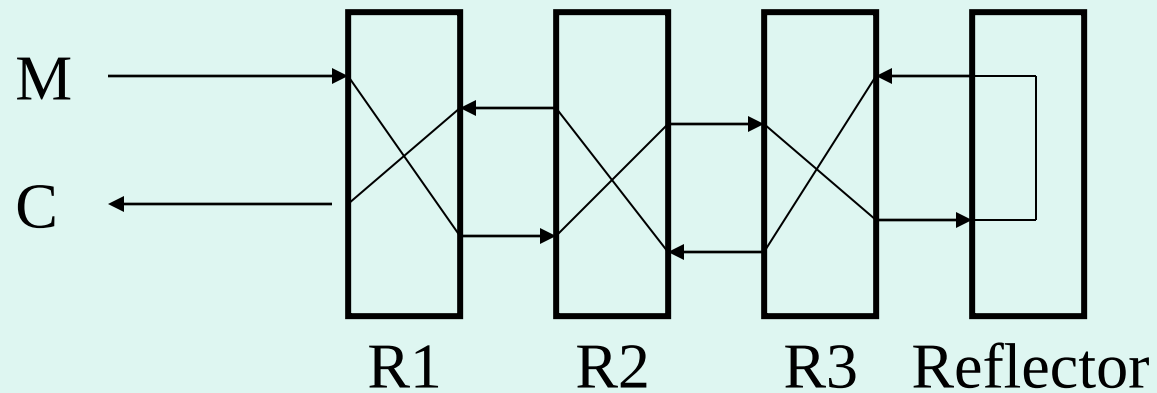
A rotor is a small disk of insulating material, with perhaps 26 equally-spaced electrical contacts in a circle on each side. The contacts on one side are connected to the contacts on the other side in a scrambled order.

In Hebern machines, the contacts on the rotors were simply flat circles of metal; the machine had ball contacts on springs to make contact with them. This allowed the rotors to be put in upside down, for more possible keys. The Enigma, on the other hand, was built more cheaply; the rotors had plain metal contacts on one side, and spring contacts on the other. This almost halved the number of contacts needed, provided you didn't decide to use a new set of rotors.

A rotor provided a changing scrambled alphabet, by (you guessed it!) **rotating**. A rotor with 26 contacts on each side, each corresponding to a letter of the alphabet, that changed E to M before rotating would now change D to L (or F to N, depending on the direction in which it rotated), while E could become any other letter, depending on the way the different wire went that was now brought into position to encipher it.

Encryption Algorithms

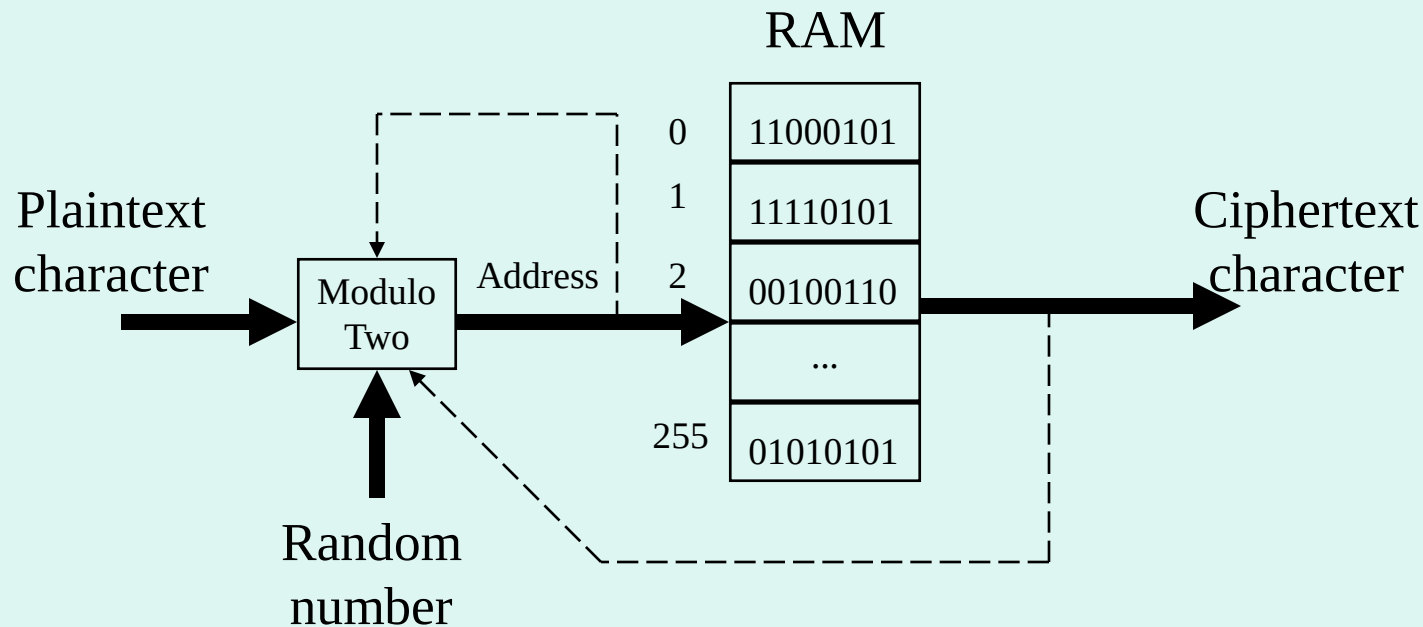
Rotor's Machine



Encryption Algorithms

Modern Rotor Machine

Modern Rotor Machine is advanced version of the electro-mechanical rotor machines have been used during the second World War. The main element of this methods is electronic rotor implemented based on the Random Access Memory (RAM). As an example let us consider electronic rotor for the case of eight bit input data string.



Plaintext character	Random number	Address	Ciphertext character
10101100	10101101	10101100+ +10101101 =00000001	11110101

Теория Информации

В.Н. Ярмолик

Лекция 4

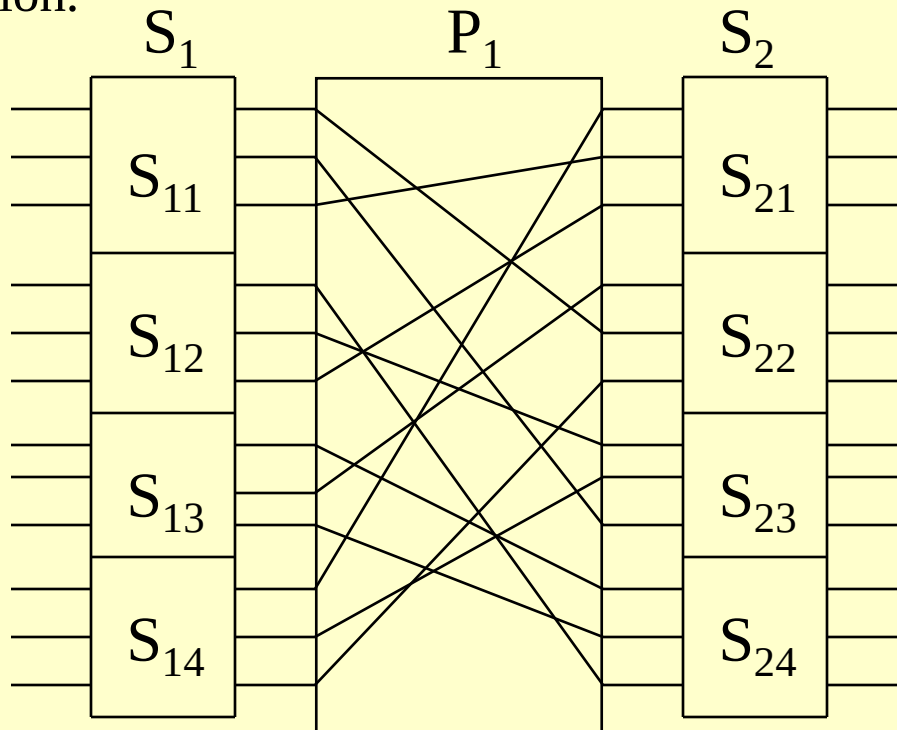
2017

Data Encryption Standard (DES)

Substitution-Permutation Cipher

Shannon proposed composing different kinds of functions to create “mixing transformations”, which randomly distribute the meaningful messages uniformly over the set of all possible ciphertext messages. Mixing transformations could be created, for example, by applying a transformation followed by an alternating sequence of substitutions and simple linear operations.

This approach is embodied in the LUCIFER cipher, designed at IBM by Feistl. LUCIFER uses a transformation that alternately applies substitutions and transposition.



The substitutions S_i are broken into 4 smaller substitutions $S_{i1}, \dots, S_{i4}$ each operating on a 3-bit subblock to reduce the complexity of the circuits.

Because the permutation P_i shuffle all 12-bit of plaintext can conceivable affect each bit of ciphertext.

Data Encryption Standard (DES)

The Data Encryption Standard (DES)

In 1977 the National Bureau of Standards announced a **Data Encryption Standard** to be used in unclassified U.S. Government applications. The encryption algorithm was developed at IBM, and was the outgrowth of LUCIFER.

DES enciphers 64-bit blocks of data with a 56-bit key. There is disagreement over whether a 56-bit key is sufficiently strong; we shall discuss controversy after describing the algorithm.

The algorithm - which is used both to encipher and to decipher - is summarized in next Figure. An input block T is first transposed under an initial permutation IP , $T_0 = IP(T)$. After it has passed through 16 iterations of a function f , it is transposed under the inverse permutation IP^{-1} to give the final result. The permutations IP and IP^{-1} are given in Table 4.1 and Table 4.2, respectively. These tables read left-to-right, top-to-bottom. For example IP transpose $T = t_1 t_2, \dots, t_{64}$ into $T_0 = t_{58} t_{50}, \dots, t_7$. All tables are fixed.

Between the initial and final transpositions, the algorithm performs 16 iterations of a function f that combines substitution and transposition. Let T_i denote the result of the i th iteration, and let L_i and R_i denote the left and right halves of T_i , respectively; that is $T_i = L_i R_i$, where

$$L_i = t_1, \dots, t_{32}, R_i = t_{33}, \dots, t_{64}.$$

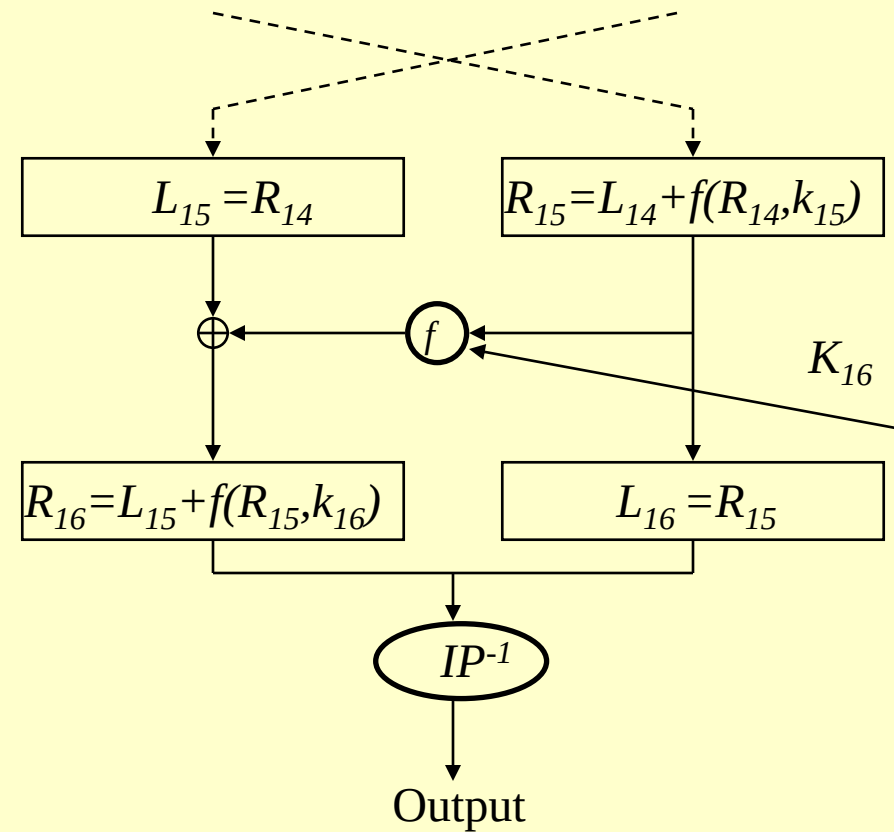
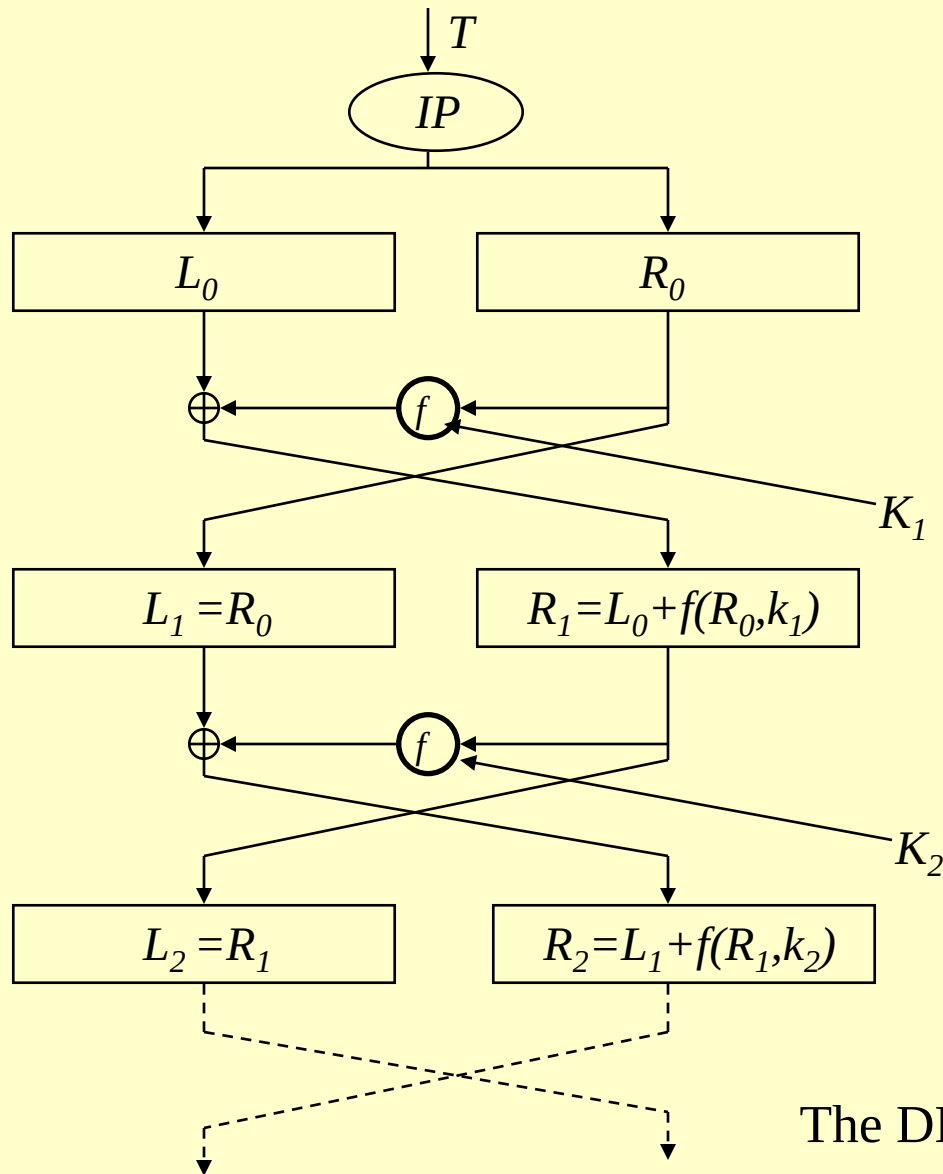
Then

$$L_i = R_{i-1}, R_i = L_{i-1} + f(R_{i-1}, K_i),$$

where “+” is the exclusive-or operation and K_i is a 48-bit key.

Data Encryption Standard (DES)

The Data Encryption Standard (DES)



The DES block diagram

Data Encryption Standard (DES)

The Data Encryption Standard (DES)

Initial Permutation IP

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Final Permutation IP^{-1}

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Bit-selected table E

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

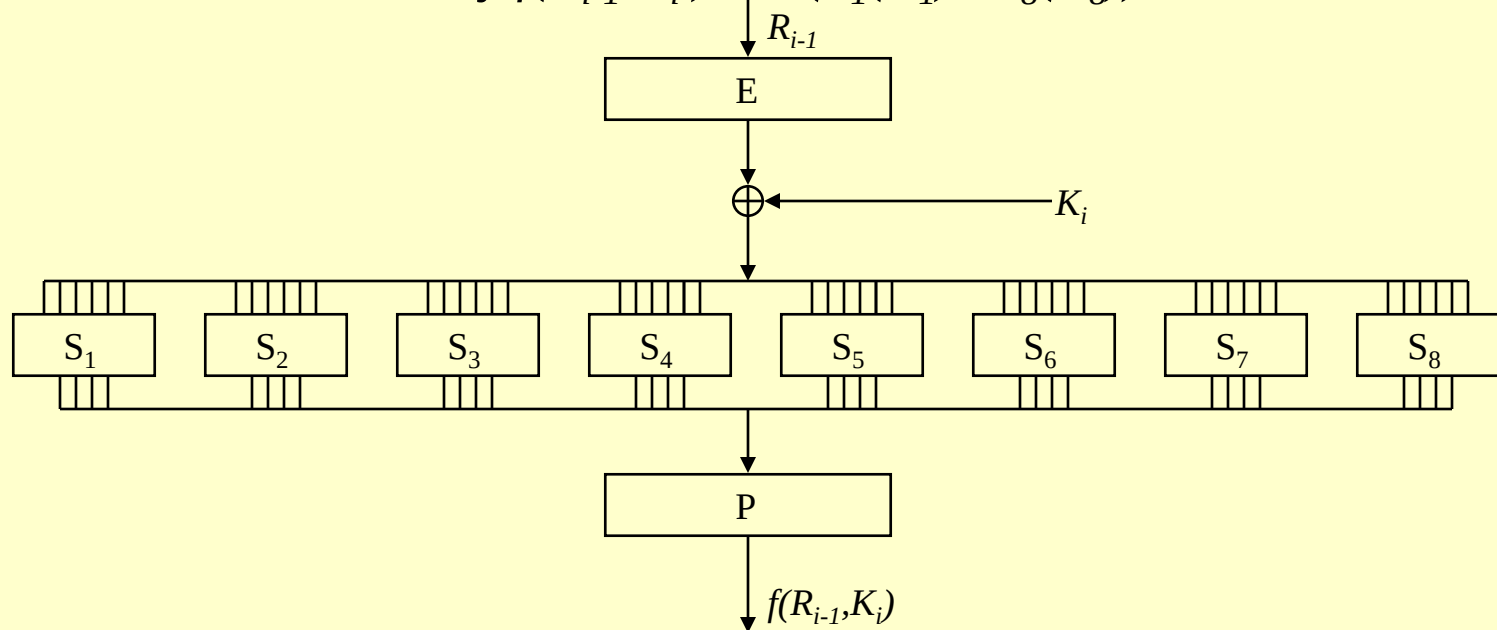
Permutation P

16	7	20	21
29	12	28	17
1	15	23	26
5	12	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

Data Encryption Standard (DES)

DES: The function f

The next *Figure* shows the function $f(R_{i-1}, K_i)$. First R_{i-1} is expanded to a 48-bit block $E(R_{i-1})$ using the bit-selection table E shown in Table 4.3. This table is used in the permutation tables, exempt that bits of R_{i-1} are selected more than once; thus given $R_{i-1} = r_1 r_2 \dots r_{32}$, $E(R_{i-1}) = r_{32} r_1 r_2 \dots r_{32} r_1$. Next, the exclusive-or $E(R_{i-1}) \oplus K_i$ is calculated and the result broken into eight 6-bit blocks $B_1 B_2, \dots, B_8$, where $E(R_{i-1}) \oplus K_i = B_1 B_2, \dots, B_8$. Each 6-bit block B_i is then used as input to selection (substitution) function (S-box) S_j , which returns a 4-bit block $S_j(B_j)$. These blocks are concatenated together, and the resulting 32-bit block is transposed by permutation P shown in Table 4.4. Thus, the block returned by $f(R_{i-1}, K_i)$ is $P(S_1(B_1) \dots S_8(B_8))$.



The function $f(R_{i-1}, K_i)$.

Data Encryption Standard (DES)

DES: S-boxes

Each S-box S_j maps a 6-bit block $B_j = b_1b_2b_3b_4b_5b_6$ into a 4-bit block as defined in next Table.

Selection functions (S-boxes)

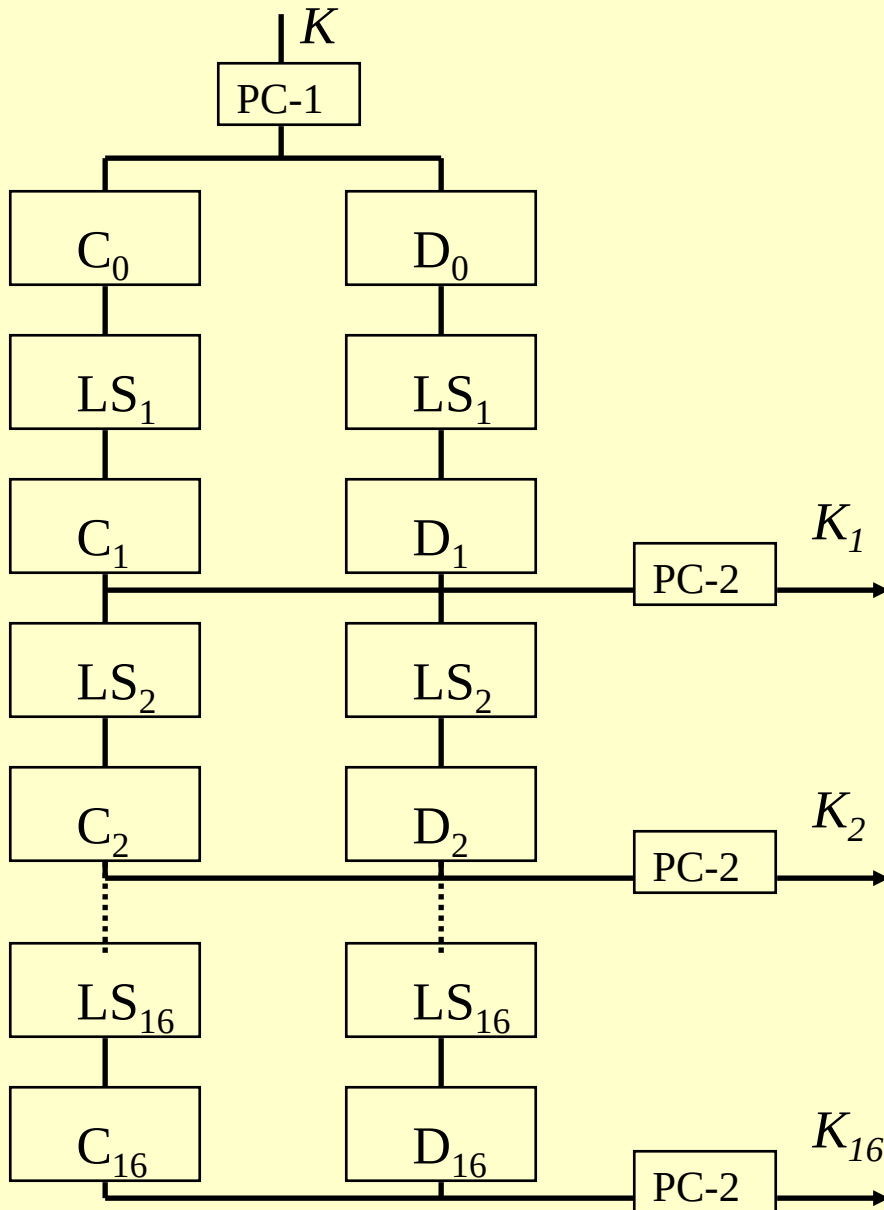
Row	Column															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
...	...															...

This is done as follows: The integer corresponding b_1b_6 selects a row in the table, while the integer corresponding $b_2b_3b_4b_5$ selects a column. The value of $S_j(B_j)$ is then the 4-bit representation of the integer in that row and column.

Example. If $B=010100$, then returns the value in row 0, column 10; this is 6, which is represented as 0110.

4. Data Encryption Standard (DES)

4.5. Key calculation



Key permutation PC-1

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

Key permutation PC-2

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

Key schedule calculation.

Data Encryption Standard (DES)

Key calculation

Each iteration i uses a different 48-bit key K_i derived from the initial key K . Figure 4.3. illustrates how this is done. K is input as a 64-bit block, with 8 parity bits in positions 8, 16,...,64. The permutation PC-1 (permuted choice 1) discards the parity bits and transposes the remaining 56 bits as shown in previous *Table*. The result PC-1(K) is then split into two halves C and D of 28 bits each. The blocks C and D are then successively shifted left to derive each key k_i . Letting C_i and D_i denotes the value of C and D used to derive K_i , we have

$$C_i = LS_j(C_{i-1}), D_i = LS_j(D_{i-1}),$$

where LS_i is a left circular shift by the number of positions shown in next Table and C_0 D_0 are the initial values of C and D . Key k_i is then given by

$$K_j = PC-2(C_i, D_i).$$

Key schedule of left shifts LS.

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Nmuber Shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Data Encryption Standard (DES)

Deciphering and Implementation

Deciphering. Deciphering is performed using the same algorithm, except that K_{16} is used in the first iteration, K_{15} in the second, and so on, with K_1 used in the 16th iteration. This is because the final permutation IP^{-1} is the inverse of the initial permutation IP , and

$$R_{i-1} = L_i, L_{i-1} = R_i + f(L_i, K_i).$$

Note that whereas the order of keys is reversed, the algorithm itself is not.

Implementation. DES has been implemented both in software and in hardware. Hardware implementations achieve encryption rates of several million bps (bits per second).

Two major DES's weaknesses.

1. **Key size:** 56 bits may not provide adequate security.
2. **S-boxes:** The S-boxes may have hidden trapdoors.

Data Encryption Standard (DES)

DES: Weak and Semiweak Keys

It must be realized that the cryptographic strength, would be reduced if the internal keys at each round were the same. For this reason, the condition $K(1)=K(2)=\dots K(16)$ should be avoided. There is, however, a set of **weak keys** within DES which satisfy the above condition. This occurs whenever the bits in register C are all ones or zeros, and the bits in register D are all ones or zeros. In this case, $C1$ and $D1$ are, respectively,

$$k_{49}=k_{41}=k_{33}=\dots=k_{57}=0 \text{ or } 1$$

and

$$k_{55}=k_{47}=k_{39}=\dots=k_{63}=0 \text{ or } 1.$$

Thus there are four weak keys altogether and they are represented by the following (parity-adjusted) external keys:

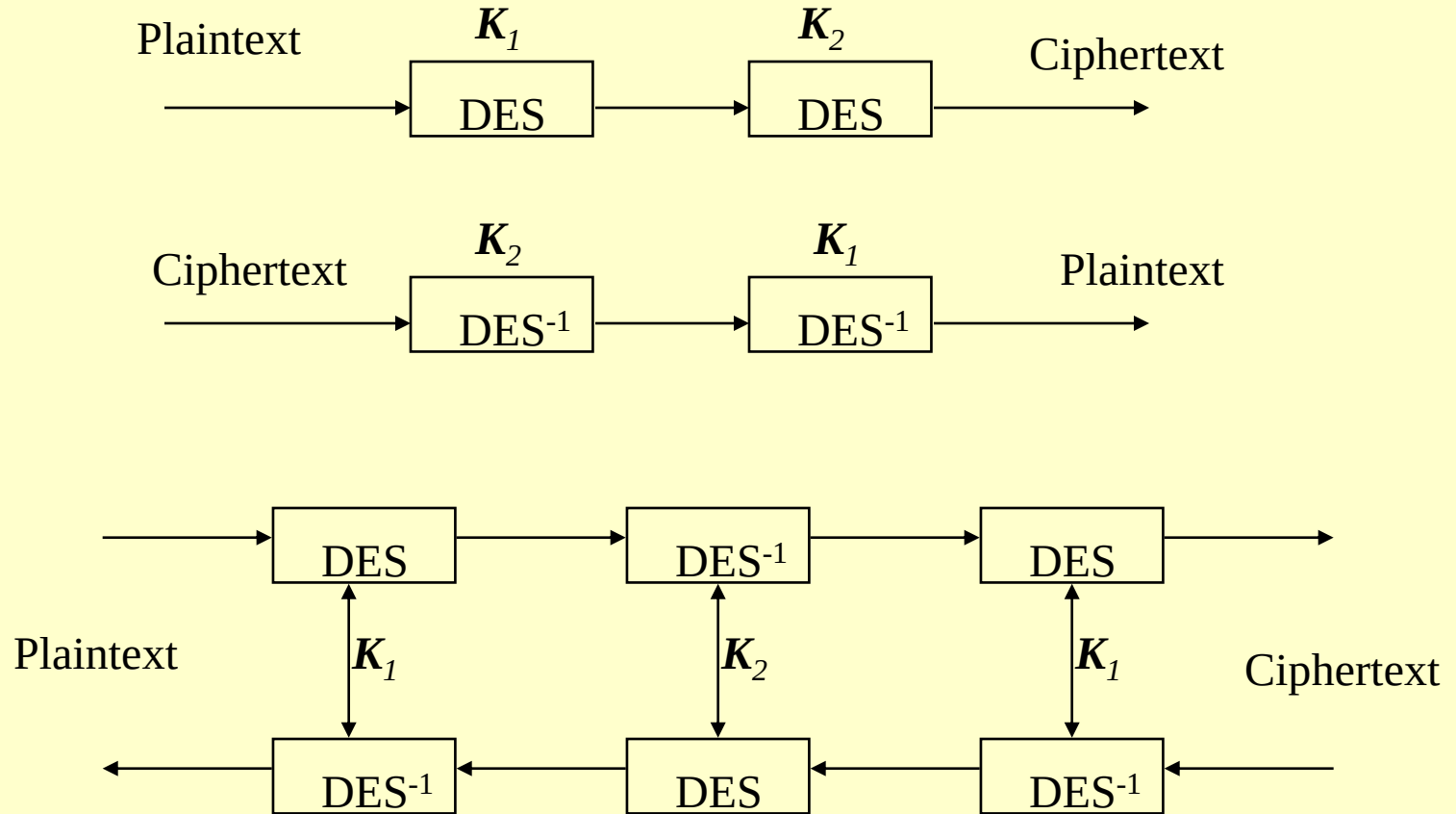
01 01 01 01 01 01 01 01
1F 1F 1F 1F 1F 1F 1F 1F
E0 E0 E0 E0 E0 E0 E0 E0
FE FE FE FE FE FE FE FE

There is another set of keys defined as **semiweak**. These have the property that only two different internal keys are produced, each occurring eight times. A semiweak key occurs whenever

1. Register C or D contains bit pattern 0101...0101 or 1010...1010.
2. The other register (D or C) contains bit pattern 0000...0000, 1111...1111, 0101...0101, 1010...1010.

Data Encryption Standard (DES)

DES: Implementation



Multiple Encipherment with DES.

Теория Информации

В.Н. Ярмолик

Лекция 5

2017

International Data Encryption Algorithm (IDEA)

Shannon proposed composing different kinds of functions to create “mixing transformations”, which randomly distribute the meaningful messages uniformly over the set of all possible ciphertext messages. Mixing transformations could be created, for example, by applying a transformation followed by simple linear operations.

IDEA belongs to the same class of conventional symmetric ciphers. The basic principles of IDEA have been published in 1990 by **J.Massey**, as an alternative to the Data encryption standard (DES).

As the DES algorithm IDEA is a block cipher with the same size of block equals to 64 bits. The length of key is 128 bits. To increase the quality of enciphering the following requirements have been satisfied within the proposed algorithm.

1. **The length of the data block is 64.** There are two basic requirements for the data block length. The first one is the complexity of the algorithm, which requires reduce the data block. The second is the statistical dependences between the data blocks, what can be reduced with increasing the length of data block. As have been proven the appropriate length of data block is 64.

2. **The length of the key.** The length of the key should be enough to eliminate so called direct attack by trying to apply all possible values of keys. The key with size 128 bits looks as a good solution for the recent applications.

3. **Confusion.** Dependence of the ciphertext from the plaintext and key should be complex and not obvious.

4. **Diffusion.** Each bit an open text, as well as each bit of the key should be taken into account to get each bit of the ciphertext.

International Data Encryption Algorithm (IDEA)

To achieve an appropriate level of the **confusion** the mixed application of the following operation on 16 bits data block have been used.

1. Bit wise XOR for two 16 bits operands, what have been denoted as $\oplus$.
2. Addition of the two integer 16 bits operands modulo 2^{16} , denoted as $\boxplus$.
3. Multiplication of the two integer number by modulo $2^{16}+1$, where integer number should be without the signs. There is one exception for the all zero code, which have to be regarded as 2^{16} . This operation denoted as $\odot$.

For this operation next properties are true:

$$a \odot (b \boxplus c) \neq (a \odot b) \boxplus (a \odot c),$$

$$a \boxplus (b \oplus c) \neq (a \boxplus b) \oplus c.$$

All proposed operation can be easily implemented and have a low complexity.

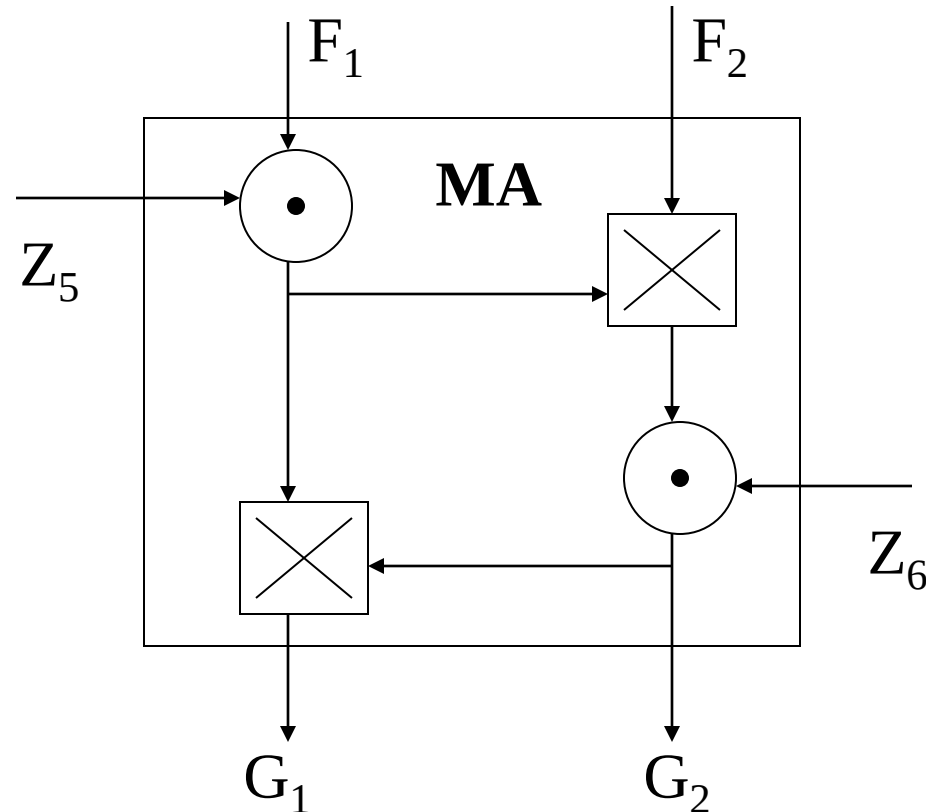
Proposed operation are shown in the following table for the case when the length of the operands equal to 2 bits.

International Data Encryption Algorithm (IDEA)

X	Y	$X \boxtimes Y$	$X \odot Y$	$X \oplus Y$
0 00	0 00	0 00	1 01	0 00
0 00	1 01	1 01	0 00	1 01
0 00	2 10	2 10	3 11	2 10
0 00	3 11	3 11	2 10	3 11
1 01	0 00	1 01	0 00	1 01
1 01	1 01	2 10	1 01	0 00
1 01	2 10	3 11	2 10	3 11
1 01	3 11	0 00	3 11	2 10
2 10	0 00	2 10	3 11	2 10
2 10	1 01	3 11	2 10	3 11
2 10	2 10	0 00	0 00	0 00
2 10	3 11	1 01	1 01	1 01
3 11	0 00	3 11	2 10	3 11
3 11	1 01	0 00	3 11	2 10
3 11	2 10	1 01	1 01	1 01
3 11	3 11	2 10	0 00	0 00

International Data Encryption Algorithm (IDEA)

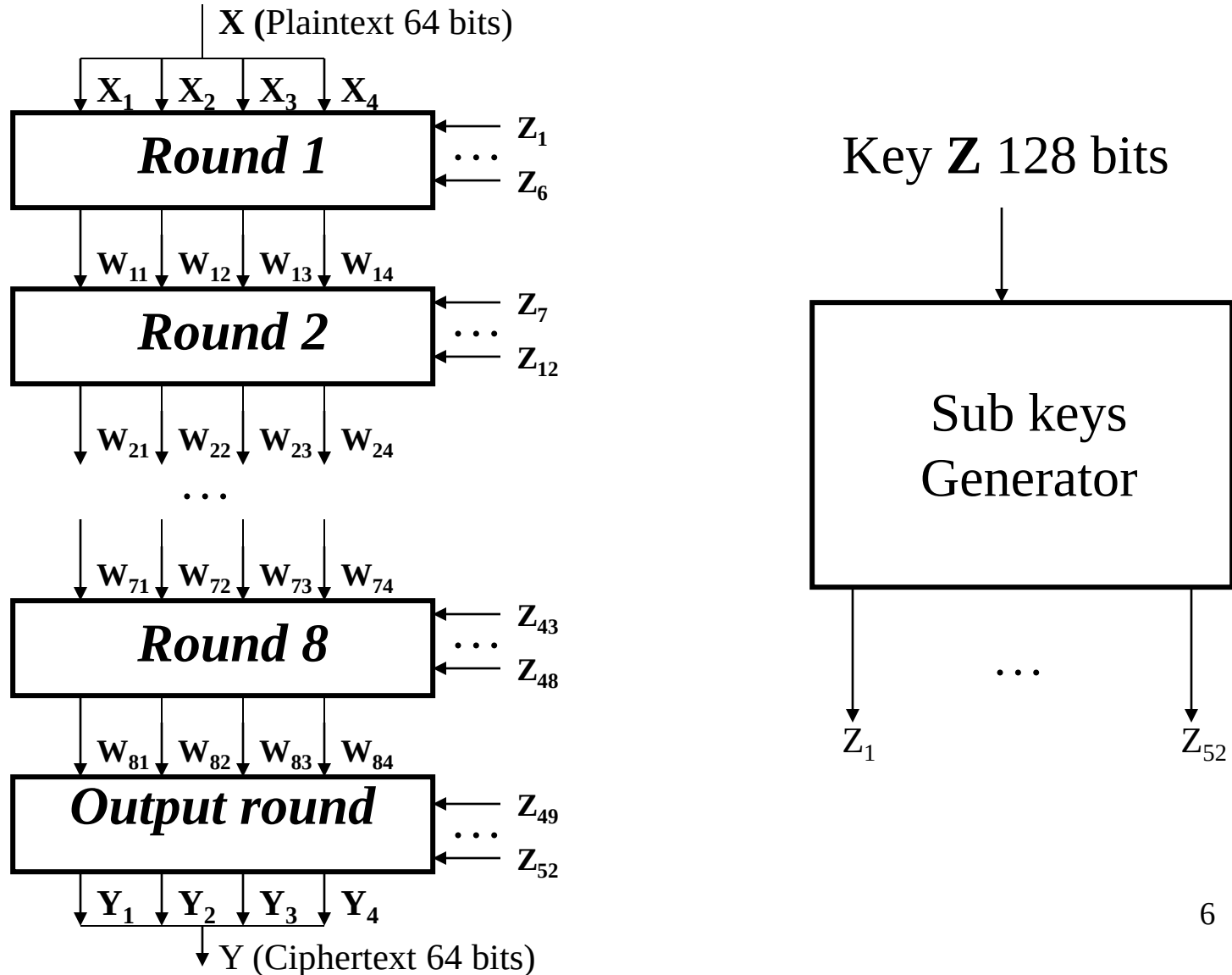
To achieve an appropriate level of the **diffusion** the main building block have been design as multiplicative-additive (**MA**) structure:



Here F_1 and F_2 are 16 bits values obtained from the plaintext; Z_5 and Z_6 are 16 bits sub keys.

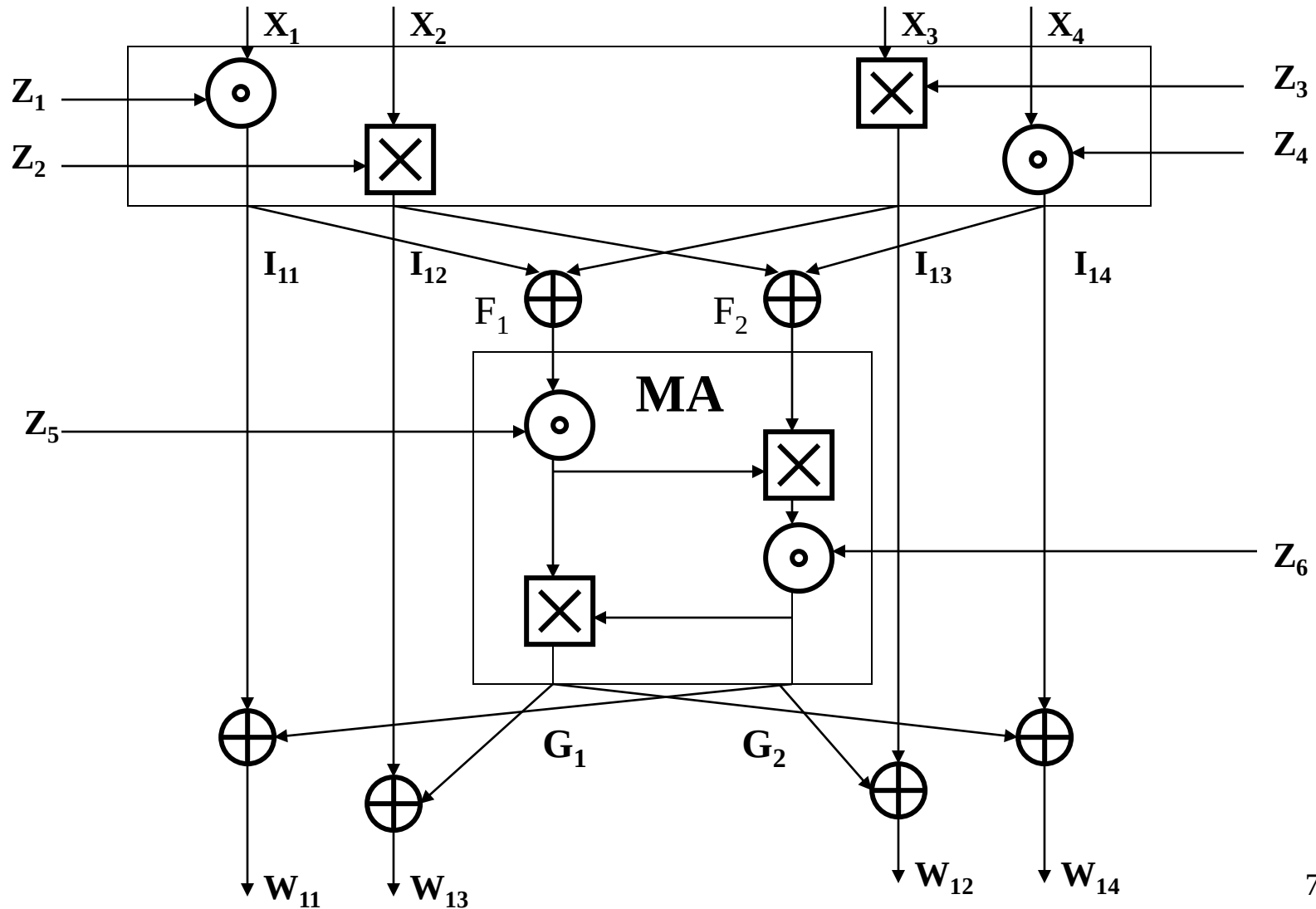
International Data Encryption Algorithm (IDEA)

Enciphering



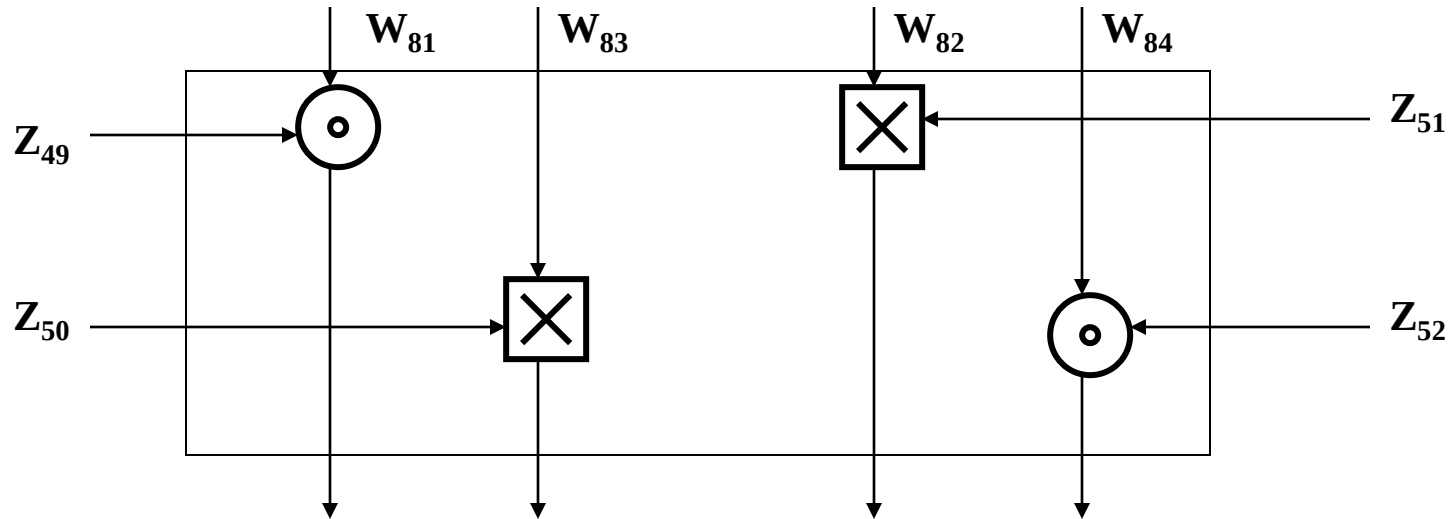
International Data Encryption Algorithm (IDEA)

One round for ciphering (first)



International Data Encryption Algorithm (IDEA)

Output round for ciphering



Sub keys calculations: The first eight sub keys $Z_1, Z_2, \dots, Z_8$ are the consecutive parts of the cryptographic key K . Then K is cyclic shifted on 25 bits to the left and the next eight sub keys are generated as the copy of consecutive parts of resulted vector. This procedure have to be repeated until all 52 sub keys are generated.

If the K can be denoted as $Z[1 \dots 128]$ then the first sub keys for all eight rounds are $Z_1 = Z[1 \dots 16]$, $Z_7 = Z[94 \dots 112]$, $Z_{13} = Z[90 \dots 105]$, $Z_{19} = Z[83 \dots 98]$, $Z_{25} = Z[76 \dots 91]$, $Z_{31} = Z[44 \dots 59]$, $Z_{37} = Z[34 \dots 52]$, $Z_{43} = Z[30 \dots 45]$.

International Data Encryption Algorithm (IDEA)

Sub keys for ciphering

Round	Notation	Equivalent
#1	$Z_1Z_2Z_3Z_4Z_5Z_6$	$Z[1\dots 96]$
#2	$Z_7Z_8Z_9Z_{10}Z_{11}Z_{12}$	$Z[97\dots 128; 26\dots 89]$
#3	$Z_{13}Z_{14}Z_{15}Z_{16}Z_{17}Z_{18}$	$Z[90\dots 128; 1\dots 25; 51\dots 82]$
#4	$Z_{19}Z_{20}Z_{21}Z_{22}Z_{23}Z_{24}$	$Z[83\dots 128; 1\dots 50]$
#5	$Z_{25}Z_{26}Z_{27}Z_{28}Z_{29}Z_{30}$	$Z[76\dots 128; 1\dots 43]$
#6	$Z_{31}Z_{32}Z_{33}Z_{34}Z_{35}Z_{36}$	$Z[44\dots 75; 101\dots 128; 1\dots 36]$
#7	$Z_{37}Z_{38}Z_{39}Z_{40}Z_{41}Z_{42}$	$Z[37\dots 100; 126\dots 128; 1\dots 29]$
#8	$Z_{43}Z_{44}Z_{45}Z_{46}Z_{47}Z_{48}$	$Z[30\dots 125]$
Output round	$Z_{49}Z_{50}Z_{51}Z_{52}$	$Z[23\dots 86]$

International Data Encryption Algorithm (IDEA)

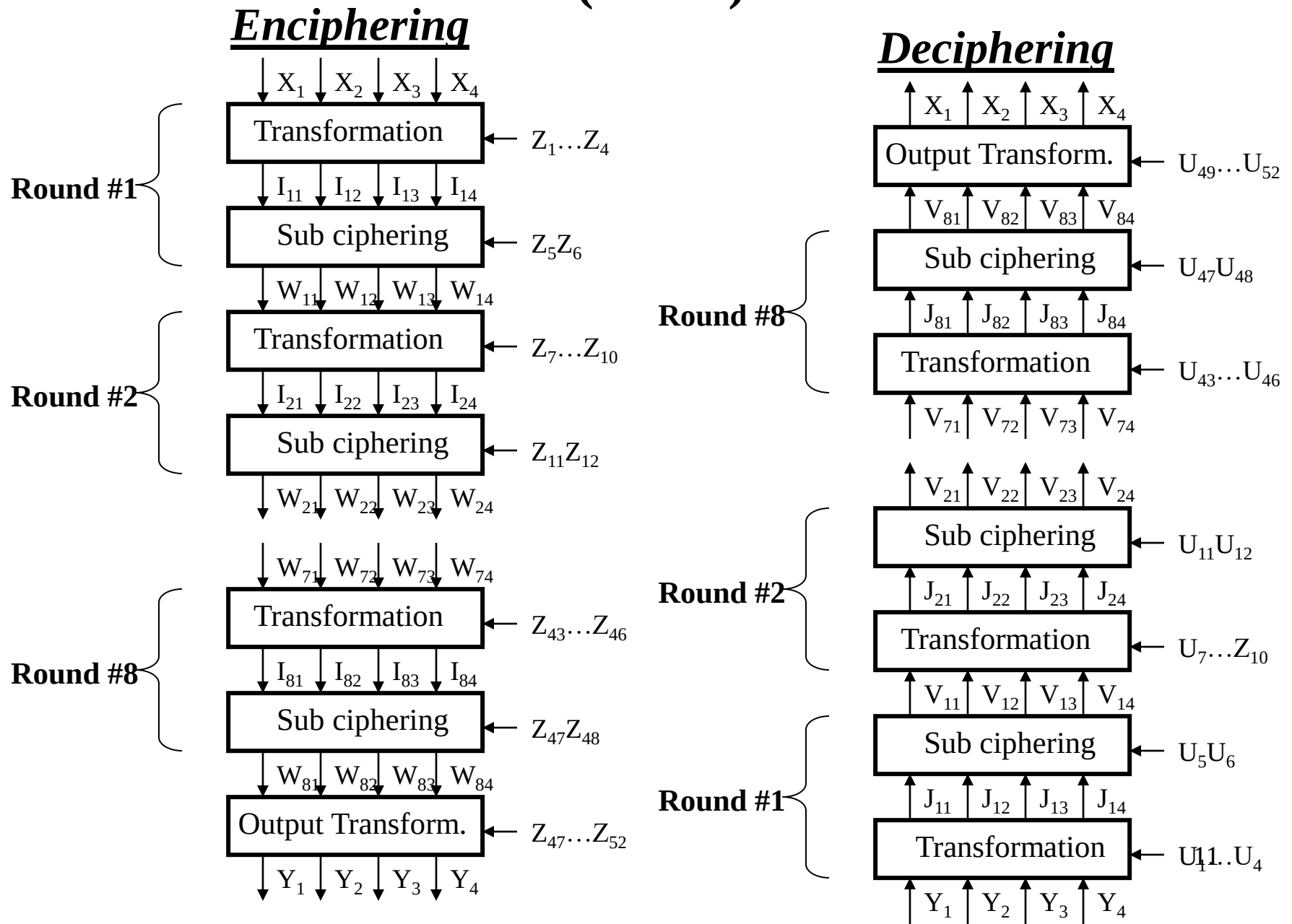
Sub keys for deciphering

Round	Notation	Equivalent
#1	$U_1 U_2 U_3 U_4 U_5 U_6$	$Z_{49}^{-1}, -Z_{50}, -Z_{51}, Z_{52}^{-1}, Z_{47}, Z_{48}$
#2	$U_7 U_8 U_9 U_{10} U_{11} U_{12}$	$Z_{43}^{-1}, -Z_{45}, -Z_{44}, Z_{46}^{-1}, Z_{41}, Z_{42}$
#3	$U_{13} U_{14} U_{15} U_{16} U_{17} U_{18}$	$Z_{37}^{-1}, -Z_{39}, -Z_{38}, Z_{40}^{-1}, Z_{35}, Z_{36}$
#4	$U_{19} U_{20} U_{21} U_{22} U_{23} U_{24}$	$Z_{31}^{-1}, -Z_{33}, -Z_{32}, Z_{34}^{-1}, Z_{29}, Z_{30}$
#5	$U_{25} U_{26} U_{27} U_{28} U_{29} U_{30}$	$Z_{25}^{-1}, -Z_{27}, -Z_{26}, Z_{28}^{-1}, Z_{23}, Z_{24}$
#6	$U_{31} U_{32} U_{33} U_{34} U_{35} U_{36}$	$Z_{19}^{-1}, -Z_{21}, -Z_{20}, Z_{22}^{-1}, Z_{17}, Z_{18}$
#7	$U_{37} U_{38} U_{39} U_{40} U_{41} U_{42}$	$Z_{13}^{-1}, -Z_{15}, -Z_{14}, Z_{16}^{-1}, Z_{11}, Z_{12}$
#8	$U_{43} U_{44} U_{45} U_{46} U_{47} U_{48}$	$Z_7^{-1}, -Z_9, -Z_8, Z_{10}^{-1}, Z_5, Z_6$
Output round	$U_{49} U_{50} U_{51} U_{52}$	$Z_1^{-1}, -Z_2, -Z_3, Z_4^{-1}$

$$Z_j^{-1} \odot Z_j = 1 \bmod (2^{16} + 1);$$

$$-Z_j \boxtimes Z_j = 0 \bmod 2^{16}.$$

International Data Encryption Algorithm (IDEA)



International Data Encryption Algorithm (IDEA)

Let us consider the last boxes on the Enciphering and Deciphering procedures:

For Enciphering $Y_1 = W_{81} \odot Z_{49}$, $Y_2 = W_{83} \boxtimes Z_{50}$, $Y_3 = W_{82} \boxtimes Z_{51}$, $Y_4 = W_{84} \odot Z_{52}$,

For Deciphering $J_{11} = Y_1 \odot U_1$, $J_{12} = Y_2 \boxtimes U_2$, $J_{13} = Y_3 \boxtimes U_3$, $J_{14} = Y_4 \odot U_4$, after substitution of the real value of the sub keys we'll get

$$J_{11} = Y_1 \odot Z_{49}^{-1} = W_{81} \odot Z_{49} \odot Z_{49}^{-1} = W_{81}$$

$$J_{12} = Y_2 \boxtimes \cdot Z_{50} = W_{83} \boxtimes Z_{50} \boxtimes \cdot Z_{50} = W_{83}$$

$$J_{13} = Y_3 \boxtimes \cdot Z_{51} = W_{82} \boxtimes Z_{51} \boxtimes \cdot Z_{51} = W_{82}$$

$$J_{14} = Y_4 \odot Z_{52}^{-1} = W_{84} \odot Z_{52} \odot Z_{52}^{-1} = W_{84}$$

So, the result of the first stage of deciphering is the same as the values before the last stage of enciphering. Now let's consider the next boxes according to the last slide.

$$W_{81} = I_{81} \oplus MA_R(I_{81} \oplus I_{83}, I_{82} \oplus I_{84}),$$

$$W_{82} = I_{81} \oplus MA_R(I_{81} \oplus I_{83}, I_{82} \oplus I_{84}),$$

$$W_{83} = I_{81} \oplus MA_L(I_{81} \oplus I_{83}, I_{82} \oplus I_{84}),$$

$$W_{84} = I_{81} \oplus MA_L(I_{81} \oplus I_{83}, I_{82} \oplus I_{84}),$$

Here $MA_R(A,B)$ means the right output value for MA, as well as $MA_L(A,B)$ – left value

International Data Encryption Algorithm (IDEA)

The next box according to deciphering procedure allow to calculate the value of $V_{11}, V_{12}, V_{13}, V_{14}$, Then

$$\begin{aligned} V_{11} &= J_{11} \oplus MA_R(J_{11} \oplus J_{13}, J_{12} \oplus J_{14}) = \\ &= W_{81} \oplus MA_R(W_{81} \oplus W_{83}, W_{82} \oplus W_{84}) = \\ &= I_{81} \oplus MA_R(I_{81} \oplus I_{83}, I_{82} \oplus I_{84}) \oplus MA_R[(I_{81} \oplus MA_R(I_{81} \oplus I_{83}, I_{82} \oplus I_{84})) \oplus I_{83} \oplus MA_R(I_{81} \oplus I_{83}, I_{82} \oplus I_{84}), \\ &\quad I_{82} \oplus MA_L(I_{81} \oplus I_{83}, I_{82} \oplus I_{84})) \oplus I_{84} \oplus MA_L(I_{81} \oplus I_{83}, I_{82} \oplus I_{84})] = \\ &= I_{81} \oplus MA_R(I_{81} \oplus I_{83}, I_{82} \oplus I_{84}) \oplus MA_R(I_{81} \oplus I_{83}, I_{82} \oplus I_{84}) = \\ &= I_{81} \end{aligned}$$

The same way it is easy to show that $V_{12} = I_{12}$, $V_{13} = I_{13}$ and $V_{14} = I_{14}$

Теория Информации

В.Н. Ярмолик

Лекция 6

2017

Symmetric Block Cipher

BLOWFISH

BLOWFISH belongs to the same class of conventional symmetric ciphers. The basic principles of **BLOWFISH** have been published in 1994 by **Bruce Schneier**, as an alternative to the Data encryption standard (DES) to satisfy the next requirements.

1. Performance. The ciphering and deciphering procedures require minimal CPU time. BLOWFISH ciphers 64 bit data block based on the 32 bit microprocessor during the 18 cycle per byte.

2. Memory space. BLOWFISH requires computer memory space less than 5 Kbytes.

3. Simplicity. The structure of BLOWFISH very simple for implementation, as well as for cryptographic strength evaluation.

4. The length of the key. The length of the key should be enough to eliminate so called direct attack by trying to apply all possible values of keys. At the same time should allow to high rate of enciphering and deciphering procedure. That is why BLOWFISH offers variable key length. It takes value in between 32 and 448 bits.

5. The length of the data block is 64.

Symmetric Block Cipher

BLOWFISH

Sub keys and S-matrixes calculation

Key. BLOWFISH allows to use the key with the length from one 32 bit word K_1 up to the fourteen 32 bit words

$$K_1, K_2, \dots, K_j, 1 \leq j \leq 14.$$

Sub Keys. Sub keys store in the P -array as

$$P_1, P_2, \dots, P_{18}.$$

S-matrixes. There are four S -matrixes. Each matrix consists of 256 32-bit words

$$S_{1,0}, S_{1,1}, \dots, S_{1,255}$$

$$S_{2,0}, S_{2,1}, \dots, S_{2,255}$$

$$S_{3,0}, S_{3,1}, \dots, S_{3,255}$$

$$S_{4,0}, S_{4,1}, \dots, S_{4,255}$$

Symmetric Block Cipher

BLOWFISH

Sub keys and S-matrixes calculation Algorithm

1. For ***P***-array and ***S***-matrixes initialization the number π is used.

$\pi = 3,14159\ 26535\ 89793\ 23846\ 26433\ 83279\ 50288\ 41971\ 69399\ 3751$
 $0\ 58209\ 74944\ 59230\ 78164\ 06286\ 20899\ 86280\ 34825\ 34211\ 70679\ 82$
 $148\ 08651\ 32823\ 06647\ 09384\ 46095\ 50582\ 23172\ 53594\ 08128\ 48111$
 $74502\ 84102\ 70193\ 85211\ 05559\ 64462\ 29489\ 54930\ 38196...$

Such a way that the ***P***₁ equals to the first 32 bits of π the next 32 bits assigns to ***P***₂ and so on. First of all initialization concerns the ***P***-array, then four ***S***-matrixes. For the hexadecimal notation there are
P₁=243F6A88, ***P***₂=85A308D3,..., ***S***_{4,254}=57FD3E3, ***S***_{4,255}=3AC372E6.

2. Bit wise XOR operation is perform between ***P***-array and ***K***-array, repeating the value of key words if necessary. For example, for the maximal length key ***P***₁=***P***₁⊕***K***₁,
P₂=***P***₂⊕***K***₂,...,***P***₁₄=***P***₁₄⊕***K***₁₄,***P***₁₅=***P***₁₅⊕***K***₁,...,***P***₁₈=***P***₁₈⊕***K***₄.

Symmetric Block Cipher

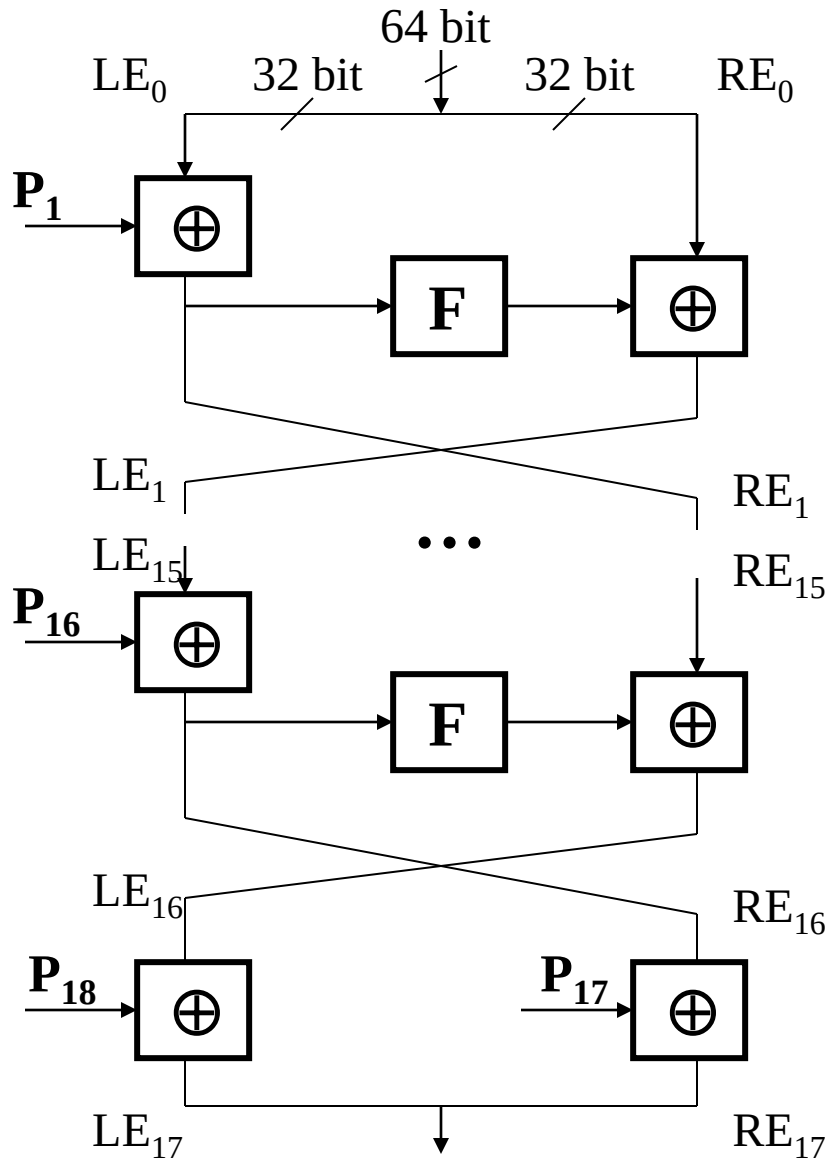
BLOWFISH

Sub keys and S-matrixes calculation Algorithm

3. Based on the current value of $\mathbf{P}$ - and $\mathbf{K}$ -arrays 64 bits all zero data block is enciphering, then the 64 for ciphertext is used as a new values for $\mathbf{P}_1$ and $\mathbf{P}_2$, as well as plaintext data block for the next iteration. New value of $\mathbf{P}$ - and $\mathbf{K}$ -arrays is used for following enciphering iteration to get new values for $\mathbf{P}_3$ and $\mathbf{P}_4$. This procedure can be presented as $\mathbf{P}_1, \mathbf{P}_2 = E_{P,S}[0]$, $\mathbf{P}_3, \mathbf{P}_4 = E_{P,S}[\mathbf{P}_1 \parallel \mathbf{P}_2]$, $\mathbf{P}_5, \mathbf{P}_6 = E_{P,S}[\mathbf{P}_3 \parallel \mathbf{P}_4], \dots, \mathbf{P}_{17}, \mathbf{P}_{18} = E_{P,S}[\mathbf{P}_{15} \parallel \mathbf{P}_{16}]$, $\mathbf{S}_{1,0}, \mathbf{S}_{1,1} = E_{P,S}[\mathbf{P}_{17} \parallel \mathbf{P}_{18}]$, $\mathbf{S}_{1,2}, \mathbf{S}_{1,3} = E_{P,S}[\mathbf{S}_{1,0} \parallel \mathbf{S}_{1,1}], \dots, \mathbf{S}_{4,254}, \mathbf{S}_{4,255} = E_{P,S}[\mathbf{S}_{4,252} \parallel \mathbf{S}_{4,253}]$.

Symmetric Block Cipher

BLOWFISH



Enciphering

For $i=1$ *to* 16 *do*

$$RE_i = LE_{i-1} \oplus P_i;$$

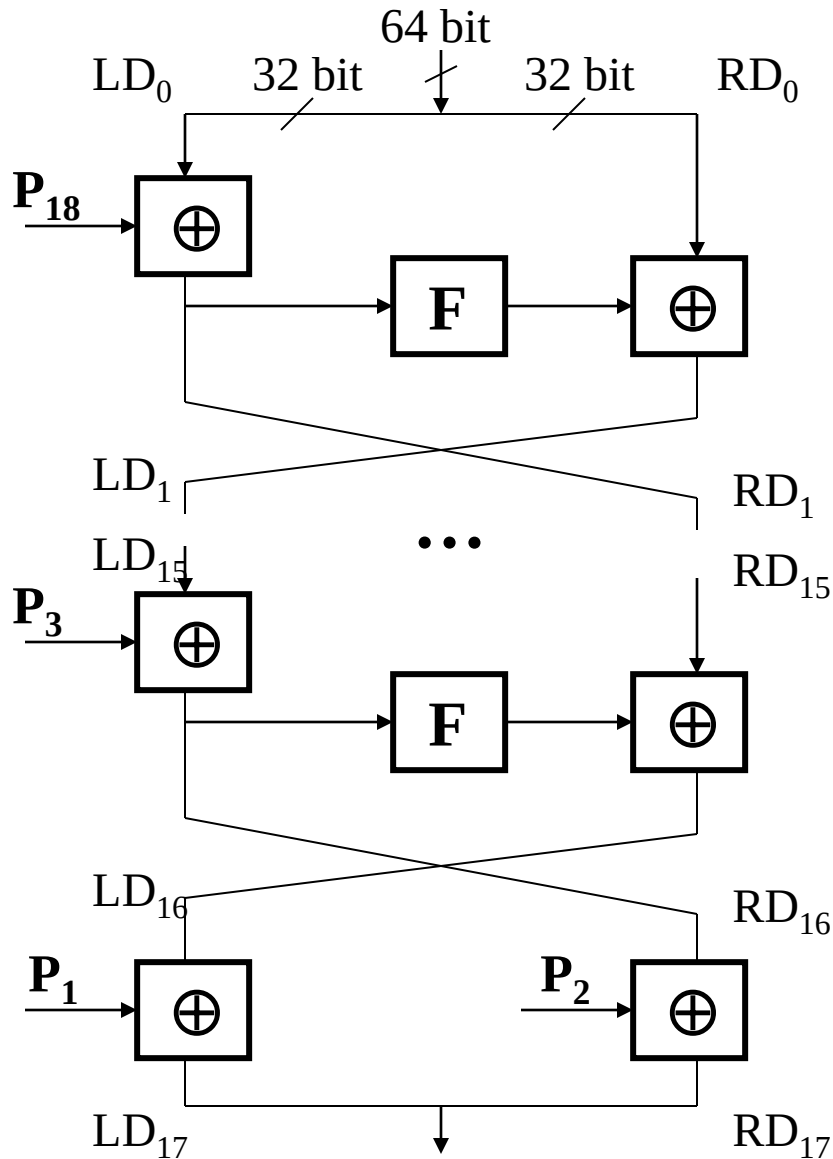
$$LE_i = F[RE_i] \oplus RE_{i-1};$$

$$LE_{17} = RE_{16} \oplus P_{18};$$

$$RE_{17} = LE_{16} \oplus P_{17};$$

Symmetric Block Cipher

BLOWFISH



Deciphering

For i=1 *to* 16 *do*

$$RD_i = LD_{i-1} \oplus P_{19-i};$$

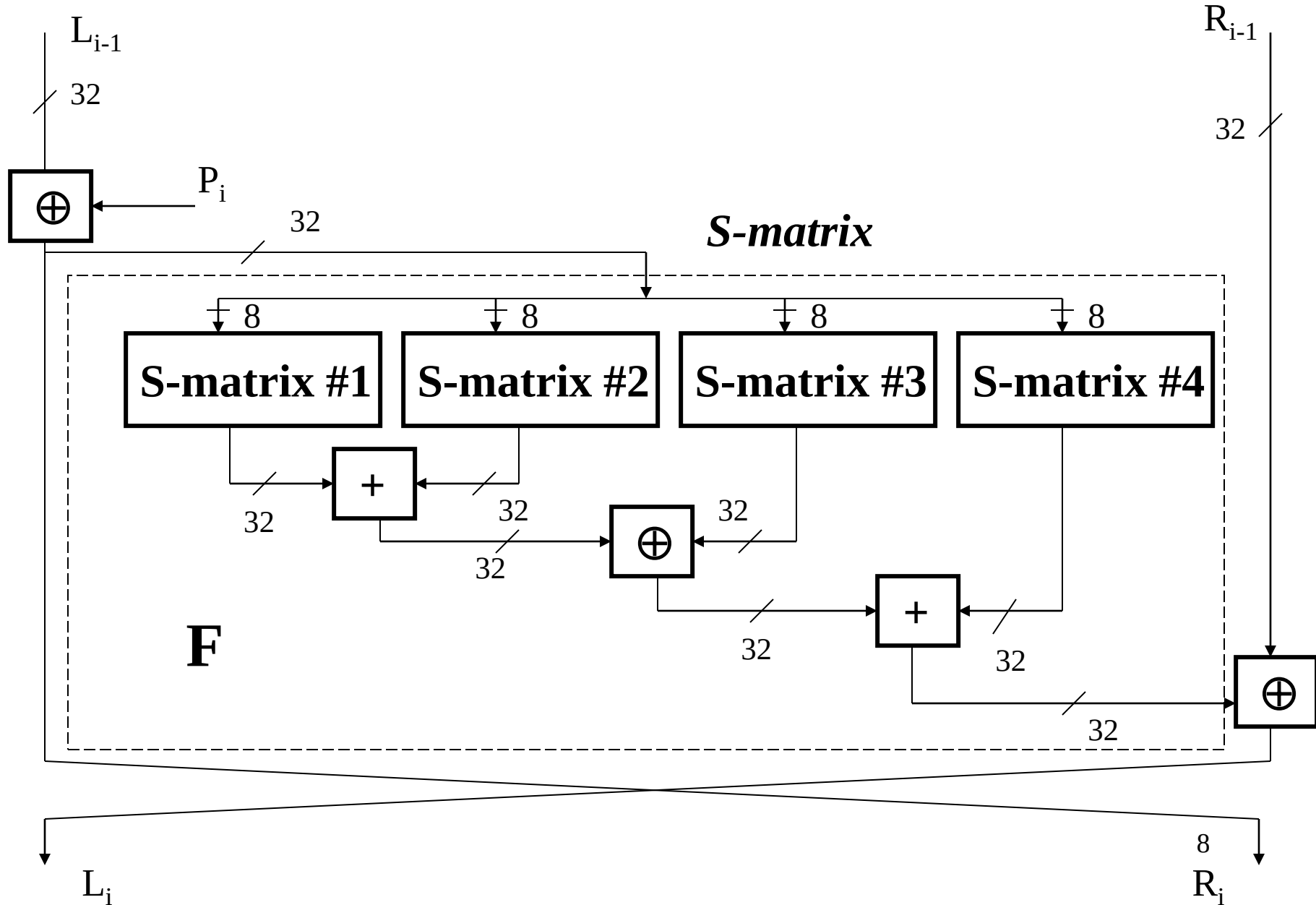
$$LD_i = F[RD_i] \oplus RD_{i-1};$$

$$LD_{17} = RD_{16} \oplus P_1;$$

$$RD_{17} = LD_{16} \oplus P_2;$$

Symmetric Block Cipher

BLOWFISH



Symmetric Block Cipher

BLOWFISH

There are two basic operation for BLOWFISH implementation:

+ addition by modulo 2^{16} ;

$\oplus$ bit wise Exclusive Or operation.

All iteration during ciphering and deciphering includes complex function F , as well as substitutions defined by S -matrixes.

First of all 32 bits input value is divided into four pieces a, b, c, d , 8 bits each. According to the 8 bits code and S -matrix, new 32 value is generated as the output code. Four S -matrixes are generated operands for F function. General expression for this function is:

$$F(a, b, c, d) = (S_{1,a} + S_{2,b}) \oplus S_{3,c} + S_{4,d}$$

Symmetric Block Cipher

BLOWFISH

BLOWFISH has next advantages:

1. Compare with DES algorithm *S*-matrixes for BLOWFISH are depending on the key.
2. During each iteration there are data transformation for both part of the data block – left and right.
3. The enciphering and deciphering rate for BLOWFISH is the highest compare with classical symmetric algorithms.

Algorithm	Number of cycles for iteration	Number of iteration	Number of cycles for data block enciphering
BLOWFISH	9	16	18
DES	18	16	45
IDEA	50	8	50
Triple DES	18	48	108

Теория Информации

В.Н. Ярмолик

Лекция 7

2017

Mathematical Backgrounds.

Information Theory

In 1949, Shannon provides a theoretical foundations for cryptography based on his fundamental work on information theory. He measured the theoretical secrecy of a cipher by the uncertainty about the plaintext given the received ciphertext. If, no matter how much ciphertext is intercepted, nothing can be learned about the plaintext, the cipher achieves *perfect secrecy*.

Entropy and Equivocation

Information theory measures the amount of information in a message by the average number of bits needed to encoded all possible messages in an optimal encoding. The Sex field in a database, for example, contains only one bit of information because it can be encoded with one bit (Male can be represented by “0”, Female by “1”). If the field is represented by an ASCII character encoding of the character strings “MALE” and “FEMALE”, it will take up more space, but will not contain any more information.

Mathematical Backgrounds.

Information Theory

The amount of information in a message is formally measured by the entropy of the message. The entropy is a function of the probability distribution over the set of all possible messages. Let $X_1, \dots, X_n$ be n possible messages occurring with probabilities $p(X_1), \dots, p(X_n)$, the sum of this probabilities $p(X_i)$, $i=1, \dots, n$ equals to one. The entropy of a given message is defined by the weighted average:

$$H(X) = -\sum_i^n p(X_i) \log_2 p(X_i).$$

As the sum taken over all messages X :

$$H(X) = -\sum_X p(X) \log_2 p(X) = \sum_X p(X) \log_2 [1/p(X)].$$

Mathematical Backgrounds.

Information theory in Examples

Intuitively, each term $\log_2 [1/p(X)]$ in last expression represents the number of bits needed to encode message X in an optimal encoding that is, one which minimizes the expected number of bits transmitted over the channel. The weighted average $H(X)$ gives the expected number of bits in optimally encoded messages.

Because $1/p(X)$ decrease as $p(X)$ increase, an optimal encoding uses short codes for frequently occurring messages at the expense of using longer ones for infrequently messages. This principle is applied in ***Morse code***, where the most frequently used letters are assigned the shortest codes.

“Huffman Code” are optimal codes assigned to characters, words, machine instructions, or phases. Single – character ***Huffman*** code are frequently used to compact large files. COMPACT program on UNIX reduced its storage requirements by 38%, which is typical for text files.

Mathematical Backgrounds.

Information theory in Examples

Example. Suppose there are two possibilities: *Male* and *Female*, both equally likely; thus $p(\text{Male})=p(\text{Female})=1/2$. Then

$$\begin{aligned} H(X) &= p(\text{Male})\log_2(1/p(\text{Male})) + p(\text{Female})\log_2(1/p(\text{Female})) = \\ &= (1/2)(\log_2 2) + (1/2)(\log_2 2) = 1, \end{aligned}$$

what confirming our earlier observation that there is 1 bit of information in the *Sex* field of a database.

The following example illustrate the application of entropy to determine the information content of a message.

Example. Let $n=3$, and let the 3 messages be the letters *A*, *B*, and *C*, where $p(A)=1/2$ and $p(B)=p(C)=1/4$. Then

$$\log_2(1/p(A)) = \log_2 2 = 1;$$

$$\log_2(1/p(B)) = \log_2(1/p(C)) = \log_2 4 = 2;$$

what confirming our earlier observation, that for frequently occurring message the minimal number of bits is needed for optimal encoding.

Mathematical Backgrounds.

Information theory in Examples

Example. Let $n=3$, and let the 3 messages be the letter A, B , and C , where $p(A)=1/2$, $p(B)=p(C)=1/4$. Then

$$H(X)=(1/2)\log_2 2 + 2(1/4)\log_2 4 = 0.5 + 1.0 = 1.5.$$

An optimal encoding assigns a 1-bit code to A and 2-bit codes to B and C . For example, A can be encoded with the bit 0, while B and C can be encoded with two bits each, 10 and 11. Using this encoding, the 8-letter sequence $ABCAABAC$ is encoded as the 12-bit sequence 010110010011 as shown next:

A	B	C	A	A	B	A	C
0	10	11	0	0	10	0	11

The average number of bits per letter is $12/8=1.5$.

Mathematical Backgrounds

Information theory in Examples

For a given language, consider the set of all messages N character long. The ***rate of the language for messages of length N*** is defined by

$$r = H(X)/N,$$

That is, the average number of bits of information in each character.

The simplest solution to determine the rate of language (***absolute rate R***) based on the assumption that all letters have the same probability of occurring within the all possible messages, as well as all possible sequences of characters are equally likely. If there are L characters in the language, then the absolute rate is given by

$$R = \log_2 L,$$

For English language this probability is equal to $L = 1/26$, then $R = \log_2 L = \log_2 26 = 4,7 \text{ bit/letter}$.

The absolute rate of the language is defined to be the maximum number of bits of information that could be encoded in each character.

The actual rate of English is thus considerably less than its absolute rate. The reason is that English, like all natural languages, is highly redundant. For example, the phrase “occurring frequently” could be reduced by 58% to “crng frg” without loss of information.

Mathematical Backgrounds

Information theory in Examples

1. Single letter frequency distributions.

A	0.0804	H	0.0549	O	0.0760	V	0.0099
B	0.0154	I	0.0726	P	0.0200	W	0.0192
C	0.0306	J	0.0016	Q	0.0011	X	0.0019
D	0.0399	K	0.0067	R	0.0612	Y	0.0173
E	0.1251	L	0.0414	S	0.0654	Z	0.0009
F	0.0230	M	0.0253	T	0.0925		
G	0.0196	N	0.0709	U	0.0271		

Then $r = H(1\text{-grams})/1 = 4.15$.

2. Digrams frequency distributions. Certain digrams (pair of letters) such as *TH* and *EN* occur much more frequently than others. Some digrams (e.g., *OZ*) never occur in meaningful messages (acronyms are an exception). Then $r = H(2\text{-grams})/2 = 3.62$.

3. Trigrams frequency distributions. The proportion of meaningful sequences decreases when trigrams are considered (e.g. *BB* is meaningful but *BBB* is not). Such as *THE* and *ING* occur much more frequently than others. Then $r = H(3\text{-grams})/3 = 3.22$.

Mathematical Backgrounds

Information theory in Examples

The rate of a language (entropy per character) is determined by estimating the entropy of N -grams for increasing values of N . As N increases, the entropy per character decreases because there are fewer choices and certain choices are much more likely. For $N \rightarrow \infty, r = 1 \div 1.5$.

The redundancy of a language with rate r and absolute rate R is defined by $D = R - r$. For $R = 4.7$ and rate $r = 1$, $D = 3.7$, whence the ratio D/R shows English to be about 79% redundant; for $r = 1.5$, $D = 3.2$, implying a redundancy of 68%.

Mathematical Backgrounds

Perfect Secrecy

Shannon studied the information theoretic properties of cryptographic systems in terms of three classes of information:

1. Plaintext messages M occurring with prior probabilities $p(M)$, where $\sum_M p(M) = 1$.
2. Ciphertext messages C occurring with prior probabilities $p(C)$, where $\sum_C p(C) = 1$.
3. Keys K occurring with prior probabilities $p(K)$, where $\sum_K p(K) = 1$.

Let $p_c(M)$ be the probability that message M was sent given that C was received (thus C is the encryption of message M). ***Perfect secrecy*** is defined by the condition.

$$p_c(M) = p(M)$$

That is, intercepting the ciphertext gives a cryptanalyst no additional information.

Mathematical Backgrounds

Perfect Secrecy

A necessary and sufficient condition for perfect secrecy is that for every C ,

$$p_M(C) = p(C) \text{ for all } M,$$

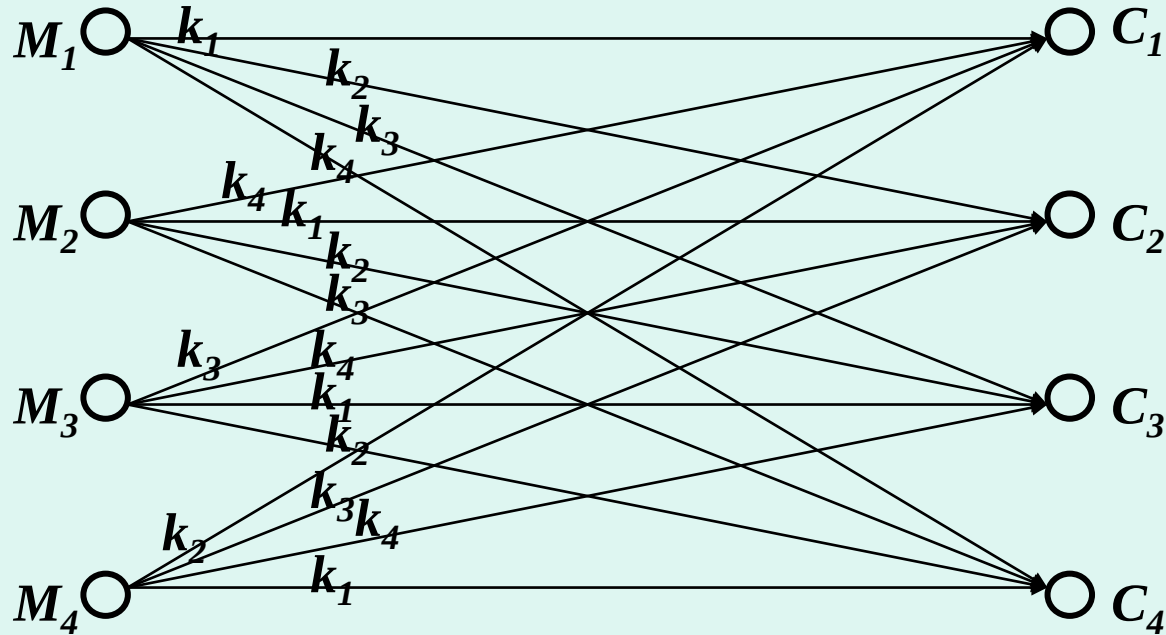
This means the probability of receiving a particular ciphertext C given that M was sent (enciphered under the same key) is the same as the probability of receiving C given that some other message M' was sent (enciphered under a different key).

Perfect secrecy is possible using completely random keys at least as long as the messages they encipher.

Mathematical Backgrounds

Perfect Secrecy

Next figure illustrates a perfect secrecy system with four messages, all equally likely, and four keys, also equally likely.



Here $p_C(M)=p(M)=1/4$, and $p_M(C)=p(C)=1/4$ for all M and C . A cryptanalyst intercepting one of the ciphertext messages C_1, C_2, C_3 or C_4 would have no way of determining which of the four keys was used and, therefore, whether the correct message is M_1, M_2, M_3 or M_4 .

Mathematical Backgrounds

Perfect Secrecy

Perfect secrecy requires that the number of keys must be at least as great as the number of possible messages. Otherwise there would be some message M such that for given C , no K decipher C into M , implying $p_C(M)=0$. The cryptanalyst could thereby eliminate certain possible plaintext message from consideration, increasing the chances of breaking the cipher.

A cipher using a nonrepeating random key stream such as the one described in the preceding example is called a ***one-time pad***. One-time pads are the only ciphers that achieve perfect secrecy.

The implementation of one-time pads in computer systems is based on an ingenious device designed by Gilbert Verman in 1917. Letting $M=m_1m_2\dots$ denotes a plaintext bit stream and $K=k_1k_2\dots$ a key bit stream, the Verman cipher generates a ciphertext bit stream $C=E_K(M)=c_1c_2\dots$, where $c_i=(m_i+k_i) \bmod 2$, $i=1,2,\dots$. The Verman cipher is efficiently implemented in microelectronics by taking the “exclusive-or” of each plaintext/key pair $c_i=m_i+k_i$. Because $k_i+k_i=0$ for $k_i=0$ or 1 , deciphering is performed with the same operation: $c_i+k_i=m_i+k_i+k_i=m_i$.

Mathematical Backgrounds

Perfect Secrecy

Example $M=0111001101010101$, $K=0101011100101011$, here the key stream represent the stream of random bits with probabilities $p(0)=p(1)=0.5$.

Enciphering procedure: $C=M\oplus K=0111001101010101\oplus 0101011100101011=0010010001111110$.

Deciphering procedure: $M=C\oplus K=0010010001111110\oplus 0101011100101011= 0111001101010101$.

Mathematical Backgrounds

Complexity Theory

The strength of a cipher is determined by the computational complexity of the algorithms used to solve the cipher. The computational complexity of an algorithm is measured by its time T and space S requirements are expressed as function $f(n)$ of n , and n characterized the size of the input. This function is typically bounded as an “order-of-magnitude” of the form $O(n^t)$, where t can take any constant value.

For example if $f(n)$ is a polynomial of the form $f(n)=a_t n^t + a_{t-1} n^{t-1} + \dots + a_1 n^1 + a_0$ for constant t , then $f(n)=O(n^t)$; that is, all constants and low-order terms are ignored.

Measuring the time and space requirements of an algorithm by its order-of-magnitude allows to see how the time and space requirements grows as the size of the input increases. For example, if $T=O(n^2)$, doubling the size of the input quadruples the running time. Table 2.4.1 shows the running times of different classes of algorithms for $n=10^6$.

Mathematical Backgrounds

Complexity Theory

Class	Complexity	Number of operations for $n=10^6$	Real time
Polynomial			
Constant	$O(1)$	1	$1 \mu\text{sec}$
Linear	$O(n)$	10^6	1 second
Quadratic	$O(n^2)$	10^{12}	10 days
Cubic	$O(n^3)$	10^{18}	27397 years
Exponential	$O(2^n)$	10^{301030}	$10^{301016} \text{ years}$

Mathematical Backgrounds

Complexity Theory

Complexity theory classifies a problem according to the minimum time and space needed to solve the hardest instances of the problem based on some abstract model of computation.

The class ***P*** consists of all problems solvable in polynomial time.

The class ***NP*** (nondeterministic polynomial) consists of all problems solvable in polynomial time on nondeterministic model of computation.

The class ***NP-complete*** has the property that if any one of the problems is in ***P***, then all ***NP*** problems are in ***P*** and ***P=NP***. Thus the ***NP-complete*** problems are the “hardest” problem in ***NP***. The fastest known algorithms for systematically solving these problems have worst-case time complexities exponential in the size n of the problem.

Mathematical Backgrounds

Complexity Theory

It has been shown that ***NP-complete*** problems might make excellent candidates for ciphers because they cannot be solved (systematically) in polynomial time by any known techniques. ***NP-complete*** problems could be adapted to cryptographic use. To construct such a cryptographic system, secret “***trapdoor***” information is inserted into a computationally hard problem that involves inverting a one-way function.

A function f is a ***one-way function*** if it is easy to compute $f(x)$ for any x in the domain of f , while, for almost all y in the range of f , it is computationally infeasible to compute $f^{-1}(y)$ even if f is known. It is a ***trapdoor one-way function*** if it is easy to compute $f^{-1}(y)$ given certain additional information. The additional information, usually is the secret deciphering key.

Теория Информации

В.Н. Ярмолик

Лекция 8

2017

Mathematical Backgrounds

Number Theory

The set of ***Natural Numbers*** is divided into two subset: ***Real Number*** and ***Integer Numbers***. The main subset of numbers to be used in the field of Data Security is the Integer Numbers.

Integer Number d is a divider of n if and only if $n=kd$. It can be notated as $d \mid n$. If there is no any k , d is not a divider of n , what can be expressed as $d \nmid n$.

Integer number p , $p > 1$ is the prime number if the only dividers for this number are 1 and p .

Theorem. (*Euclid's*) There is infinite set of prime numbers.

Proof: Suppose that this set is finite and consists of the prime numbers $p_1, p_2, p_3, \dots, p_k$, Then it is the contradiction that the number

$$\left(\prod_{i=1}^k p_i \right) + 1,$$

is not divided by any of prime number $p_1, p_2, p_3, \dots, p_k$, thus it is divided by 1 and itself, what means that this number is a prime number.
#

Mathematical Backgrounds

Number Theory

Theorem. For any big positive integer number $k > 1$, there is a possibility to determine k composite numbers, following in a row within the set of integer numbers.

Proof: The number $(k+1)! = 2 \cdot 3 \cdot 4 \cdot \dots \cdot (k+1)$ is divided by any of the following numbers $2, 3, 4, \dots, (k+1)$. Thus, the numbers following in a row within the set of integer numbers $(k+1)! + 2, (k+1)! + 3, (k+1)! + 4, \dots, (k+1)! + (k+1)$, are composite numbers due to the fact that first number is divided by 2, second by 3 and so on. #

Theorem. If the integer number $n (n > 1)$ is not divided by any prime number no greater than $n^{1/2}$ means that it is a prime number.

Proof: Suppose that n is a composite number, and can be expressed as $n = ab$, $1 < a < n$; $1 < b < n$. Numbers a and b can not be greater than $n^{1/2}$ simultaneously. #

Mathematical Backgrounds

Number Theory

Theorem. (Eratosfen's)

1) If in a set of integer numbers $2, 3, 4, \dots, N$, delete all numbers divided by the first r prime numbers $2, 3, 5, 7, \dots, p_r$, then the first is not deleted number is a prime number.

2) If in a set of integer numbers $2, 3, 4, \dots, N$, delete all numbers divided by the prime numbers less or equal to $\sqrt{N}$, such a way that $p_r \leq \sqrt{N} \leq p_{r+1}$, then all remaining numbers will be the prime numbers p within the set

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
	3		5		7		9		11		13		15		17		19		21		23		25	
			5		7				11		13				17		19				23		25	
					7				11		13				17		19				23			

Mathematical Backgrounds

Number Theory

The distribution of Primes

At the present, the most practical method of selecting primes suitable for use in the RSA algorithm is to test randomly selected integers until the required number of primes have been found. The approach works only because the proportion of primes to non-primes is high enough.

By actual count, one finds that each group of 100 numbers, from 1 to 1000 (1 to 100, 101 to 200, etc.) contains respectively, the following number of primes: 25, 21, 16, 16, 17, 14, 16, 14, 15, 14. In each group of 100 numbers from 1,000,001 to 1,001,000, the corresponding frequency of primes is: 6, 10, 8, 8, 7, 7, 10, 5, 6, 8, and from 10,000,001 to 10,001,000 the corresponding frequency is 2, 6, 6, 6, 5, 4, 7, 10, 9, 6.

Mathematical Backgrounds

Number Theory

#	1 -1000
25	1-100
21	101-200
16	201-300
16	301-400
17	401-500
14	501-600
16	601-700
14	701-800
15	801-900
14	901-1000

#	10 000 001-10 001 000
2	10 000 001-10 000 100
6	10 000 101-10 000 200
6	10 000 201-10 000 300
6	10 000 301-10 000 400
5	10 000 401-10 000 500
4	10 000 501-10 000 600
7	10 000 601-10 000 700
10	10 000 701-10 000 800
9	10 000 801-10 000 900
6	10 000 901-10 001 000

Mathematical Backgrounds

Number Theory

According to the ***prime number theorem***, the ratio of $\pi(x)$, the number of primes in the interval from 2 to x and $x/\ln(x)$ approaches 1 as x becomes very large, that is

$$\lim_{x \rightarrow \infty} \frac{\pi(x)}{x / \ln(x)} = 1 \quad \text{Where } \ln(x) \text{ is the natural logarithm of } x.$$

x	$\pi(x)$	$x/\ln(x)$	$\lim_{x \rightarrow \infty} \frac{\pi(x)}{x / \ln(x)}$
1,000	168	145	1.159
100,000	9,592	8,686	1.104
10,000,000	664,579	620,421	1.071
1,000,000,000	50,847,476	48,254,942	1.054

Mathematical Backgrounds

Number Theory

Fermat's prime numbers: 3, 5, 17, 257, 65537, generated by the equation

$$2^{2^k} + 1.$$

Euler's prime numbers can be generated according to the equation $x^2 - x + 41$, for all integer x , $0 \leq x \leq 40$.

A *Mersenne prime* is a Mersenne number that is prime. It is known that if $2^p - 1$ is prime then p is prime.

$$(2^{ab} - 1) = (2^a - 1)(1 + 2^a + 2^{2a} + 2^{3a} + \dots + 2^{(b-1)a})$$

Mersenne prime numbers can be generated according to $2^p - 1$
For prime $p = 2, 3, 5, 7, 13, 17, 19, 31, 61$.

Mathematical Backgrounds

Number Theory

47 Mersenne primes are known. The largest known prime number ($2^{43\,112\,609} - 1$) is a Mersenne prime (October 2009).

Since 1997, all newly-found Mersenne primes have been discovered by the "Great Internet Mersenne Prime Search" (GIMPS), a distributed computing project on the Internet.

One of the the greatest known prime number is a Mersenne (35) number $2^{1398269} - 1$ with 420921 digits.

Mathematical Backgrounds

Number Theory

Composite numbers can be represented in a canonical form

$$a = \prod_{i=1}^k p_i^{\alpha_i}$$

where p_i are the prime numbers.

Example. $a=120=2^3 \cdot 3^1 \cdot 5^1$

Common Divisor of the numbers $a_1, a_2, a_3, \dots, a_n$, is an integer d , that $d \mid a_1, d \mid a_2, d \mid a_3, \dots, d \mid a_n$.

Greatest Common Divisor of the numbers $a_1, a_2, a_3, \dots, a_n$, is a greatest integer divisor d , that can be divided by any common divisor of this numbers $(a_1, a_2, a_3, \dots, a_n) = d$.

Example. $(6, 15, 27) = 3$.

Mathematical Backgrounds

Number Theory

Theorem. If

$$a_1 = \prod_{i=1}^k p_i^{\alpha_i} \quad a_2 = \prod_{i=1}^k p_i^{\beta_i} \quad \dots \quad a_n = \prod_{i=1}^k p_i^{\gamma_i}$$

then the greatest common divisor (g.c.d.) is

$$(a_1, a_2, \dots, a_n) = \prod_{i=1}^k p_i^{\min \{\alpha_i, \beta_i, \dots, \gamma_i\}}$$

Example. If $6 = 2^1 \cdot 3^1 \cdot 5^0$, $15 = 2^0 \cdot 3^1 \cdot 5^1$, $27 = 2^0 \cdot 3^3 \cdot 5^0$, then
 $(6, 15, 27) = 2^{\min\{1, 0, 0\}} \cdot 3^{\min\{1, 1, 3\}} \cdot 5^{\min\{0, 1, 0\}} = 2^0 \cdot 3^1 \cdot 5^0 = 3$.

Theorem. If

$$a_1 = \prod_{i=1}^k p_i^{\alpha_i} \quad a_2 = \prod_{i=1}^k p_i^{\beta_i} \quad \dots \quad a_n = \prod_{i=1}^k p_i^{\gamma_i}$$

then the least common multiplier (l.c.m.) is

$$l.c.m.(a_1, a_2, \dots, a_n) = \prod_{i=1}^k p_i^{\max \{\alpha_i, \beta_i, \dots, \gamma_i\}}$$

Example. If $6 = 2^1 \cdot 3^1$, $15 = 3^1 \cdot 5^1$, $27 = 3^3$, then
 $l.c.m.(6, 15, 27) = 2^{\max\{1, 0, 0\}} \cdot 3^{\max\{1, 1, 3\}} \cdot 5^{\max\{0, 1, 0\}} = 2^1 \cdot 3^3 \cdot 5^1 = 270$.

Mathematical Backgrounds

Number Theory

Theorem. If $a=bq+r$, then $(a,b)=(b,r)$.

Proof: Let $d=(a,b)$, then from the statement that $d \mid a$ and $d \mid b$ we can conclude that d is a divider for $a-bq=r$.#

Theorem. (Euclid Algorithm) For any integer numbers $a>0$ and $b>0$ that $a>b$, and b is not a divider of a for some s exists integer numbers $q_0, q_1, q_2, \dots, q_s$ and $r_0, r_1, r_2, \dots, r_s$ that $b>r_0>r_1>r_2>\dots>r_s>0$ and $a=bq_0+r_1$, $b=r_1q_1+r_2$, $r_1=r_2q_2+r_3$, ..., $r_{s-2}=r_{s-1}q_{s-1}+r_s$, $r_{s-1}=r_sq_s$ and $(a,b)=r_s$

Euclid's Algorithm

begin

$g_0:=a; g_1:=b;$

$i:=1;$

while $g_i \neq 0$ **do**

begin

$g_{i+1}:= g_{i-1} \bmod g_i;$

$i:=i+1;$

end

$gcd:= g_{i-1}$

{gcd-Greatest Common Divisor}

end

Mathematical Backgrounds

Number Theory

Example. Determine the greatest common divisor for integers 1173 and 323, $(1173, 323) = ?$.

Solution: $1173 = 323 \cdot 3 + 204$; $323 = 204 \cdot 1 + 119$; $204 = 119 \cdot 1 + 85$;
 $119 = 85 \cdot 1 + 34$; $85 = 34 \cdot 2 + 17$; $34 = 17 \cdot 2$;

$$\begin{aligned} g_{i+1} &:= g_{i-1} \bmod g_i; \\ 204 &:= 1173 \bmod 323; \\ 119 &:= 323 \bmod 204; \\ 85 &:= 204 \bmod 119; \\ 34 &:= 119 \bmod 85; \\ 17 &:= 85 \bmod 34; \\ 0 &:= 34 \bmod 17. \end{aligned}$$

Mathematical Backgrounds

Number Theory

Binary Algorithm This algorithm is an extension of Euclid's Algorithm and is based on the following statements:

1. If both a and b are even, then $(a,b)=2(a/2,b/2)$;
2. If a even, and b odd, then $(a,b)=(a/2,b)$;
3. According to the last Theorem $(a,b)=(b,a-b)$;
4. If both a and b are odd, then $a-b$ is even.

Example. Determine the greatest common divisor for integers 1173 and 323, $(1173,323)=?$.

Solution:

$$\begin{aligned}(1173,323) &= (323,850) = (323,425) = (323,102) = (323,51) = (51,272) \\ &= (51,136) = (51,68) = (51,34) = (51,17) = (17,34) = (17,17) = 17.\end{aligned}$$

Теория Информации

В.Н. Ярмолик

Лекция 9

2017

Mathematical Backgrounds

Number Theory

Numbers $a_1, a_2, a_3, \dots, a_n$, are ***relatively prime*** if and only if $(a_1, a_2, a_3, \dots, a_n) = 1$. Numbers $a_1, a_2, a_3, \dots, a_n$, are ***pair wise relatively prime*** if and only if for any i and $j \neq i$ $(a_i, a_j) = 1$.

Theorem. If $(a, b) = 1$, then for any integer numbers n and m $(a^n, b^m) = 1$, as well as, *if* $(a^n, b^m) = 1$, for any integer numbers n and m then $(a, b) = 1$.

Proof: If $(a, b) = 1$, then if canonical representation of $a = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_k^{\alpha_k}$ has $\alpha_i > 0$ it means that $\gamma_i = 0$ for canonical representation for $b = p_1^{\gamma_1} p_2^{\gamma_2} \dots p_k^{\gamma_k}$, as well as in a case of $n\alpha_i > 0$ we have $\gamma_i = 0$. #

Mathematical Backgrounds

Number Theory

CONGRUENCES

Two integer numbers a and b are said to be congruent **modulo** m , if the difference $a-b$ is divided by m , what can be written as:

$$a \equiv b \bmod m.$$

Last expression is called **congruence**.

Example. $32 \equiv 5 \bmod 9$; $48 \equiv 12 \bmod 9$; $17 \equiv 7 \bmod 5$.

In a case when $b < m$, the b is a **residue** of a by **modulo** m .

Mathematical Backgrounds

Number Theory

There are a set of useful lemmas for congruence:

1. If $a \equiv b \pmod m$, then for any integer k we have $ka \equiv kb \pmod m$.
2. If $ka \equiv kb \pmod m$, and $(k, m) = 1$, then $a \equiv b \pmod m$.
3. If $ka \equiv kb \pmod{km}$, where k and m are any integer numbers then $a \equiv b \pmod m$.
4. If $a \equiv b \pmod m$, and $c \equiv d \pmod m$, then $a + c \equiv b + d \pmod m$.
5. If $a_1 \equiv b_1 \pmod m$, and $a_2 \equiv b_2 \pmod m, \dots, a_n \equiv b_n \pmod m$, then $a_1 + a_2 + a_3 + \dots + a_n \equiv b_1 + b_2 + b_3 + \dots + b_n \pmod m$.
6. If $a \equiv b \pmod m$, and $c \equiv d \pmod m$, then $a \cdot c \equiv b \cdot d \pmod m$.
7. If $a_1 \equiv b_1 \pmod m$, and $a_2 \equiv b_2 \pmod m, \dots, a_n \equiv b_n \pmod m$, then $a_1 \cdot a_2 \cdot a_3 \cdot \dots \cdot a_n \equiv b_1 \cdot b_2 \cdot b_3 \cdot \dots \cdot b_n \pmod m$.
8. If $a \equiv b \pmod m$, then for any integer $k > 0$ we have $a^k \equiv b^k \pmod m$.

Mathematical Backgrounds

Number Theory

Principle of Modular Arithmetic:

Modular arithmetic is based on the following lemma:

$$9. (a * b) \bmod m = [(a \bmod m) * (b \bmod m)] \bmod m.$$

Where $*$ is any of the following operations “+”, “-” or “·”.

The preceding theorem shows that evaluating $(a * b) \bmod m$ in modular arithmetic gives the same result as evaluating it in ordinary integer arithmetic and reducing the result $\bmod m$.

Example. $7 \cdot 9 \bmod 5 = [(7 \bmod 5) \cdot (9 \bmod 5)] \bmod 5.$

Note that the principle of modular arithmetic also applies to exponentiations because exponentiation is equivalent to repeated multiplications:

Mathematical Backgrounds

Number Theory

Example. Consider the expression $3^5 \bmod 7$. This can be computed by rising 3 to the power 5 and then reducing the result ***mod*** 7 as shown next:

- | | |
|--------------------------------|--------------------|
| 1. Square 3: | $3 \cdot 3 = 9$ |
| 2. Square the result: | $9 \cdot 9 = 81$ |
| 3. Multiply by 3: | $81 \cdot 3 = 243$ |
| 4. Reduce <i>mod</i> 7: | $243 \bmod 7 = 5.$ |

Alternatively, the intermediate results of the computations can be reduce ***mod*** 7.

- | | |
|-----------------------|--------------------------|
| 1. Square 3: | $3 \cdot 3 \bmod 7 = 2$ |
| 2. Square the result: | $2 \cdot 2 \bmod 7 = 4$ |
| 3. Multiply by 3: | $4 \cdot 3 \bmod 7 = 5.$ |

Mathematical Backgrounds

Number Theory

Fast Exponentiation Algorithm

begin “return $x = a^z \bmod m$ ”

$a_1 := a; z_1 := z;$

$x := 1;$

while $z_1 \neq 0$ **do** “ $x(a_1^{z_1} \bmod m) = a^z \bmod m$ ”

begin

while $z_1 \bmod 2 = 0$ **do**

begin “square a_1 while z_1 is even”

$z_1 := z_1 \operatorname{div} 2;$

$a_1 := (a_1 \cdot a_1) \bmod m;$

end;

$z_1 := z_1 - 1;$

$x := (x \cdot a_1) \bmod m$ “multiply”

end;

$\text{fastexp} := x;$

end

Mathematical Backgrounds

Number Theory

Example. Consider the the calculation of $x = 5^{10} \bmod 7 = 5^{(10^{10})} \bmod 7$. This can be computed by the *Fast Exponentiation Algorithm*.

$a_1 := 5; z_1 := 10; x := 1;$
 $z_1 \neq 0; (10 \neq 0);$
 $z_1 \bmod 2 = 0; (10 \bmod 2 = 0);$
 $z_1 \div 2 = 5; (10/2 = 5);$
 $a_1 := a_1 \cdot a_1 \bmod m = 4; (5 \cdot 5 \bmod 7 = 4);$
 $z_1 \bmod 2 \neq 0; (5 \bmod 2 \neq 0);$
 $z_1 := z_1 - 1 = 4; (5 - 1 = 4);$
 $x := (x \cdot a_1) \bmod m = 4; (1 \cdot 4 \bmod 7 = 4);$
 $z_1 \neq 0; (4 \neq 0);$
 $z_1 \bmod 2 = 0; (4 \bmod 2 = 0);$
 $z_1 \div 2 = 2; (4/2 = 2);$
 $a_1 := a_1 \cdot a_1 \bmod m = 2; (4 \cdot 4 \bmod 7 = 2);$
 $z_1 \neq 0; (2 \neq 0);$
 $z_1 \bmod 2 = 0; (2 \bmod 2 = 0);$
 $z_1 \div 2 = 1; (2/2 = 1);$
 $a_1 := a_1 \cdot a_1 \bmod m = 4; (2 \cdot 2 \bmod 7 = 4);$
 $z_1 \bmod 2 \neq 0; (1 \bmod 2 \neq 0);$
 $z_1 := z_1 - 1 = 0; (1 - 1 = 0);$
 $x := (x \cdot a_1) \bmod m = 2; (4 \cdot 4 \bmod 7 = 2);$

$5^2 \bmod 7 = 4;$
 $5^4 \bmod 7 = 4 \cdot 4 \bmod 7 = 2;$
 $5^8 \bmod 7 = 2 \cdot 2 \bmod 7 = 4;$
 $5^{10} \bmod 7 = 4 \cdot 4 \bmod 7 = 2.$

Mathematical Backgrounds

Number Theory

If $a=r \bmod m$, ($0 \leq r < m$), then $m \mid (a-r)$. Hence, there is $a-r=qm$ or $a=qm+r$. The remainder r is called a **residue** of $a \pmod m$. A set of m integers $\{a_i\}=\{a_1, a_2, \dots, a_m\}$ is said to form a complete set of m integers in congruent **modulo** m for any r_i in the residue system $\{0,1,2,\dots,m-1\}$ in some order. This set is called **Complete Residue System modulo m** . Thus, for any integer a , there exists a congruence

$$a=r \bmod m$$

where r is a unique one among the numbers in a complete set of residues.

Example. The set of integer numbers $\{16,12,19,48,65\}$ is a complete residue system modulo 5. Really, $16=1 \bmod 5$, $12=2 \bmod 5$, $19=4 \bmod 5$, $48=3 \bmod 5$, $65=0 \bmod 5$ and we have got complete set of residues $\{0,1,2,3,4\}$.

A set of integers $\{a_i\}=\{a_1, a_2, \dots, a_n\}$ is said to form the **residue class modulo m** if all residues for a given numbers are the same and is equal r .

Example. The set of integer numbers $\{16,21,56,91,106\}$ is residue class modulo 5. Really, $16=1 \bmod 5$, $21=1 \bmod 5$, $56=1 \bmod 5$, $91=1 \bmod 5$, $106=1 \bmod 5$ and we have got the same residue 1.

Mathematical Backgrounds

Number Theory

Fermat's Theorem. If p is a prime number and $(a,p)=1$, where a is an integer, then

$$a^{p-1} = 1 \pmod{p}.$$

Proof: For a given p and a , $(a,p)=1$ let consider $p-1$ positive products $a, 2a, 3a, \dots, (p-1)a$. Any pair ia, ja ($i \neq j$) of the products is not comparable by *modulo* p , namely (see slide № 4)

$$ia \not\equiv ja \pmod{p},$$

As the result every product has their own unique nonzero residue r_i , ($1 \leq r_i \leq p-1$). This residues $\{1, 2, 3, \dots, (p-1)\}$ are in some order and create the Complete Residue System. Where $a = r_\alpha \pmod{p}$, $2a = r_\beta \pmod{p}$, $3a = r_\chi \pmod{p}$, $\dots$, $(p-1)a = r_\lambda \pmod{p}$, where $\{r_\alpha, r_\beta, r_\chi, \dots, r_\lambda\} = \{1, 2, 3, \dots, (p-1)\}$. Based on the lemma for congruence (see slide № 4) we can get the product for the last congruence $a = r_\alpha \pmod{p}$, $2a = r_\beta \pmod{p}$, $3a = r_\chi \pmod{p}$, $\dots$, $(p-1)a = r_\lambda \pmod{p}$ as

$$a \cdot 2a \cdot 3a \cdot \dots \cdot (p-1) \cdot a = r_\alpha \cdot r_\beta \cdot r_\chi \cdot \dots \cdot r_\lambda \pmod{p};$$

$$a \cdot 2a \cdot 3a \cdot \dots \cdot (p-1) \cdot a = 1 \cdot 2 \cdot 3 \cdot \dots \cdot (p-1) \pmod{p};$$

$$a^{p-1} \cdot (p-1)! = (p-1)! \pmod{p};$$

Due to $(p-1)!$ and p are relatively prime $((p-1)!, p) = 1$, then $a^{p-1} = 1 \pmod{p}$.#

Mathematical Backgrounds

Number Theory

Euler's Function $\psi(n)$ for $n \geq 1$ is the number of integers less than n and relatively prime with n . $\psi(1)=0$, $\psi(2)=1$, $\psi(3)=2$, $\psi(4)=2$, $\psi(5)=4$, $\psi(6)=2$, $\psi(7)=6$, $\psi(8)=4$, $\psi(9)=6$, $\psi(10)=4$, $\psi(11)=10$, If n is a prime number p , then $\psi(p)=p-1$.

Theorem. If $n=pq$, where p and q are the prime numbers, then $\psi(n)=\psi(p)\psi(q)=(p-1)(q-1)$.

Proof: Let consider the complete set of residues $\{0,1,2,\dots,pq-1\}$ by modulo $n=pq$. All this residues are relatively prime with $n=pq$, except $(p-1)$ elements $\{q,2q,3q,\dots,(p-1)q\}$, $(q-1)$ elements $\{p,2p,3p,\dots,(q-1)p\}$, and 0. Thus, $\psi(pq)=pq-1-(p-1)-(q-1)=pq-p-q+1=(p-1)(q-1)$.#

Example. $\psi(10)=\psi(2 \cdot 5)=\psi(2) \cdot \psi(5)=1 \cdot 4=4$.

Theorem. If p is a prime number, and $k > 0$ integer number, then

$$\psi(p^k)=p^k-p^{k-1}=p^{k-1}(p-1).$$

Mathematical Backgrounds

Number Theory

Proof: The set of integers which are less than p^k and are not relatively prime with p^k , includes the numbers $\{p, 2p, 3p, \dots, (p^{k-1}-1)p\}$. It means that among p^k-1 numbers less than p^k there are $p^{k-1}-1$ integers are not relatively prime with p^k . Thus, $\psi(p^k) = p^k - 1 - (p^{k-1} - 1) = p^k - p^{k-1}$. #

Example. $\psi(8) = \psi(2^3) = 2^3 - 2^2 = 8 - 4 = 4$.

Theorem. Function $\psi(n \cdot m)$ is multiplicative function $\psi(n \cdot m) = \psi(n) \cdot \psi(m)$, when $(n, m) = 1$.

When $a = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_r^{\alpha_r}$, then
 $\psi(a) = \psi(p_1^{\alpha_1}) \psi(p_2^{\alpha_2}) \dots \psi(p_r^{\alpha_r}) = (p_1^{\alpha_1} - p_1^{\alpha_1-1}) (p_2^{\alpha_2} - p_2^{\alpha_2-1}) \dots (p_r^{\alpha_r} - p_r^{\alpha_r-1}) = a(1 - 1/p_1)(1 - 1/p_2) \dots (1 - 1/p_r)$.

Example. $\psi(2700) = ?$ $2700 = 2^2 3^3 5^2$. $\psi(2700) = 2700(1 - 1/2)(1 - 1/3)(1 - 1/5) = 720$.

Mathematical Backgrounds

Number Theory

Euler's Theorem. If $n \geq 0$ is an positive integer number, and $(a,n)=1$, where a is an integer, then

$$a^{\psi(n)} = 1 \bmod n.$$

Proof: let $\{r_1, r_2, r_3, \dots, r_{\psi(n)}\}$ is a reduced residue system modulo n , then for $(a,n)=1$ numbers $ar_1, ar_2, ar_3, \dots, ar_{\psi(n)}$ will organize the same reduced residue system, such a way that $ar_1 = r_\alpha \bmod n$; $ar_2 = r_\beta \bmod n$; $ar_3 = r_\chi \bmod n, \dots, ar_{\psi(n)} = r_\lambda \bmod p$, where $\{r_\alpha, r_\beta, r_\chi, \dots, r_\lambda\}$ is a permutation of residues $\{r_1, r_2, r_3, \dots, r_{\psi(n)}\}$. Multiplying the right and left part of the last congruence we will get:

$$a^{\psi(n)} r_1 \cdot r_2 \cdot r_3 \cdot \dots \cdot r_{\psi(n)} = r_\alpha \cdot r_\beta \cdot r_\chi \cdot \dots \cdot r_{\psi(n)} \bmod n;$$

Taking into account, that $\{r_1, r_2, r_3, \dots, r_{\psi(n)}, m\} = 1$, then

$$a^{\psi(n)} = 1 \bmod n. \#$$

Mathematical Backgrounds

Number Theory

Example. $3^{10} \bmod 11 = ?$. According to the Fermat's theorem $3^{10} = 1 \bmod 11$, where $p=11$, and $a=3$.

Example. $3^{12} \bmod 26 = ?$. According to the Euler's theorem $3^{12} \bmod 26 = 1$, where $n=26$, $\psi(26) = \psi(2 \cdot 13) = \psi(2) \cdot \psi(13) = 1 \cdot 12 = 12$.

Linear Congruencies

The linear congruence is

$$ax = b \bmod n, b < n.$$

There are three possibilities for the linear congruence solution x , namely, no solutions, one solution and a set of solutions satisfying this linear congruence.

Theorem. If $(a, n) = d$ is not a divider of b , then there are not solutions of linear congruence $ax = b \bmod n$.

Proof: Suppose there is a solution x_0 , that satisfy the linear congruence $ax_0 = b \bmod n$. According to the condition of the theorem d is a divider of a and n , what means that d have to be a divider of ax_0 and nq , as well as a divider of $ax_0 - nq = b$. This implies a contradiction. #

Mathematical Backgrounds

Number Theory

Example. Linear congruence $2x=1 \bmod 4$ does not have a solution.

Theorem. If $(a,n)=1$ there is one solution of linear congruence $ax=b \bmod n$.

Proof: Let's take a complete residue system $\{0,1,2,\dots,n-1\}$ by modulo n . Due to the fact that a and n are relatively prime the integer numbers $\{0\cdot a, 1\cdot a, 2\cdot a, \dots, (n-1)\cdot a\}$ create the complete residue system modulo n . Among all integer numbers there is one ax_0 and only one with residue equals to b .#

Example. Linear congruence $2x=1 \bmod 3$ has a solution $x_0=2$.

Mathematical Backgrounds

Number Theory

Computing Inverses a^{-1}

For $b=1$ linear congruence is $ax=1 \bmod n$, where $x=a^{-1}$, then $aa^{-1} = 1 \bmod n$.

Let take two congruence $ax=1 \bmod n$ and $1=a^{\psi(n)} \bmod n$ (*Euler's theorem*), then multiply left and right part of this relations. As the result we will get:

$$ax=a^{\psi(n)} \bmod n \Rightarrow x=a^{\psi(n)-1} \bmod n,$$
$$\text{for prime } n \Rightarrow x=a^{n-2} \bmod n.$$

Example. Find a solution of linear congruence $3x=1 \bmod 7$. 7 is prime number, then $x=a^{n-2} \bmod n=3^{7-2} \bmod 7=3^5 \bmod 7=5$.

Example. Find a solution of linear congruence $4x=1 \bmod 9$. $\psi(9)=6$, then $x=a^{\psi(n)-1} \bmod n=4^{6-1} \bmod 9=4^5 \bmod 9=7$.

Similar for case of computing inverse it have been shown that

Mathematical Backgrounds

Number Theory

Euclid's algorithm extended to Computing Inverses a^{-1}

begin “Return x such that $ax \bmod n = 1$, where $0 < a < n$ ”

$g_0 := n; g_1 := a;$

$u_0 := 1; v_0 := 0;$

$u_1 := 0; v_1 := 1;$

while $g_i \neq 0$ **do** “ $g_i = u_i n + v_i a$ ”

begin

$y := g_{i-1} \operatorname{div} g_i;$

$g_{i+1} = g_{i-1} - y * g_i;$

$u_{i+1} = u_{i-1} - y * u_i;$

$v_{i+1} = v_{i-1} - y * v_i;$

$i := i + 1$

end;

$x := v_{i-1};$

if $x \geq 0$ **then** $a^{-1} := x$ **else** $a^{-1} := x + n$

Mathematical Backgrounds

Number Theory

Euclid's algorithm extended to Computing Inverses a^{-1}

The algorithm computes (a,n) by computing $g_{i+1} = g_{i-1} \bmod g_i$ for $i=1,2,\dots$ until $g_i=0$, where $g_0 = n$, $g_1 = a$, and “ $g_i = u_i n + v_i a$ ” is the loop invariant. When $g_i=0$, $g_{i-1} = (a,n)$. If $(a,n)=1$, then $g_{i-1}=1$ and $v_{i-1}a - 1 = u_{i-1}n$, giving $v_{i-1}a = 1 \bmod n$. Thus, $x = v_{i-1}$ is an inverse of $a \bmod n$. Now x will be in the range $-n < x < n$. If x is negative, $x+n$ gives the solution in the range $0 < x < n$.

The following illustrates the execution of the algorithm to solve the equation $3x \bmod 7 = 1$.

i	g_i	u_i	v_i	y
0	7	1	0	
1	3	0	1	2
2	1	1	-2	3
3	0			

Because $v_2 = -2$ is negative, the solution is $x = -2 + 7 = 5$.

Mathematical Backgrounds

Number Theory

Solution $ax \equiv b \pmod n$, $b < n$; for the case $(a, n) = 1$.

$x \equiv ba^{\varphi(n)-1} \pmod n$, for prime $n \Rightarrow x \equiv ba^{n-2} \pmod n$.

Exempl. Linear congruence $3x \equiv 3 \pmod 7$. Taking into account that $(3, 7) = 1$ and 7 is a prime number $x \equiv ba^{n-2} \pmod n = 3 \cdot 3^{7-2} \pmod 7 = 3^5 \pmod 7 = 5$.

Theorem. If $(a, n) = d$, and $d \mid b$, then there are d solutions of linear congruence $ax \equiv b \pmod n$.

Proof: According to the condition of the theorem d is a divider of a , n and b . Then from congruence $ax \equiv b \pmod n$ we can get $a_1 dx \equiv b_1 d \pmod{n_1 d}$, or what the same the congruence $a_1 x \equiv b_1 \pmod{n_1}$, where $(a_1, n_1) = 1$. The last congruence $a_1 x \equiv b_1 \pmod{n_1}$, has one solution x_0 . Integers of the same class by modulo n/d will be the solutions for the congruence $ax \equiv b \pmod n$. Namely, $x_1 = x_0 \pmod n$, $x_2 = x_0 + n/d \pmod n$, $x_3 = x_0 + 2n/d \pmod n$, ..., $x_d = x_0 + (d-1)n/d \pmod n$.#

Mathematical Backgrounds

Number Theory

Example. Find the solutions for the following linear congruence $6x \equiv 4 \pmod{10}$.

Taking into account that $(6, 10) = 2$ and 2 is a divider of 4, we will get the congruence $3x \equiv 2 \pmod{5}$. Then the solution of the last congruence is number $x_0 = ba^{n-2} \pmod{n} = 23^{5-2} \pmod{5} = 23^3 \pmod{5} = 4$.

$$x_1 = x_0 \pmod{n} = 4 \pmod{10} = 4;$$
$$x_2 = x_0 + n/d \pmod{n} = 4 + 10/2 \pmod{10} = 9.$$

Теория Информации

В.Н. Ярмолик

Лекция 10

2017

Public Key Cryptosystem

Classical Cryptography: Secret-Key or Symmetric Cryptography

Alice and Bob agree on an encryption method and a shared *key*.

Alice uses the key and the encryption method to *encrypt* (or *encipher*) a message and sends it to Bob.

Bob uses the same key and the related decryption method to *decrypt* (or *decipher*) the message.

Public Key Cryptosystem

Advantages of Classical Cryptography

There are some *very fast classical encryption* (and decryption) algorithms

Since the speed of a method varies with the length of the key, faster algorithms allow one to use longer key values.

Larger key values make it harder to guess the key value -- and break the code -- by brute force.

Public Key Cryptosystem

Disadvantages of Classical Cryptography

Requires secure transmission of key value

Requires a separate key for each group of people that wishes to exchange encrypted messages (readable by any group member)

For ***example***, to have a separate key for each pair of people, 100 people would need 4950 different keys.

Public Key Cryptosystem

Definitions

Public-key/two-key/asymmetric cryptography involves the use of **two** keys:

a ***public-key***, which may be known by anybody, and can be used to ***encrypt messages***, and ***verify signatures***

a ***private-key***, known only to the recipient, used to ***decrypt messages***, and ***sign*** (create) ***signatures***

is ***asymmetric*** because

those who encrypt messages or verify signatures **cannot** decrypt messages or create signatures

Public Key Cryptosystem

Starting point

Public Key Cryptosystem address two key issues:
key distribution – how to have secure communications in general without having to trust a KDC with your key

digital signatures – how to verify a message comes intact from the claimed sender

public invention due to ***Whitfield Diffie & Martin Hellman*** at Stanford U. in 1976

known earlier in classified community

Public Key Cryptosystems

Public Key Distribution Systems

The *public key distribution algorithm* proposed by **Deffie** and **Hellman** appears to be difficult for computing logarithms over $GF(q)$ with q elements $\{0,1,2,\dots,q-1\}$. Consider a pair of inverse functions

$$C = \alpha^M \pmod{q};$$

$$M = \log_{\alpha} C \pmod{q},$$

where $0 < M, C < q$, respectively, q is prime, and α is primitive element of $GF(q)$. Calculation of C from M is easy, but computation of M from C is more difficult, because computing logarithms over $GF(q)$ is believed to be much harder. The possible solution to a public-key distribution is

1. Each user A and B generates an independent random number M_i and M_j chosen uniformly from the set of integers $\{1,2,\dots,q-1\}$, and keep this numbers secret.

2. Both users make the following calculations A calculate $C_i = \alpha^{M_i} \pmod{q}$, and B calculate $C_j = \alpha^{M_j} \pmod{q}$ and place the values of C_i, C_j in a public file.

Public Key Cryptosystems

Public Key Distribution Systems

3. User A computes K_{ij} by obtaining C_j from the public file and letting

$$K_{ij} = C_j^{M_i}(\text{mod } q) = (\alpha^{M_j})^{M_i}(\text{mod } q) = \alpha^{M_j M_i}(\text{mod } q)$$

user B obtains the common key K_{ij} in a similar fashion as

$$K_{ji} = C_i^{M_j}(\text{mod } q) = (\alpha^{M_i})^{M_j}(\text{mod } q) = \alpha^{M_i M_j}(\text{mod } q)$$

We can see that $K_{ij} = K_{ji}$, that is why both users have the same key.

The other user must compute the $K_{ij} = K_{ji}$ from C_j and C_i by computing

$$K_{ij} = K_{ji} = C_i^{(\log \alpha C_j)}(\text{mod } q)$$

Public Key Cryptosystems

Public Key Distribution Systems

Example. Consider a prime field $GF(q)$ with a prime number q . Let a be a fixed primitive element of $GF(q)$ such that the powers of a produce all nonzero elements $1, 2, \dots, q-1$ of $GF(q)$. Let us pick $a = 2$ and $q = 11$. The powers of $a = 2 \pmod{11}$ produce the following table.

l	0	1	2	3	4	5	6	7	8	9	10	...
a^l	2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}	...
$a^l \pmod{11}$	1	2	4	8	5	10	9	7	3	6	1	...

To initiate communication, user **A** chooses a random number $M_i = 5$ from the integer set $2^l \pmod{11} = \{1, 2, 3, \dots, 10\}$ in the above table and keeps it secret.

Then he calculates $C_i = a^{M_i} \pmod{q} = 2^5 \pmod{11} = 10$, and **B** generates secret $M_j = 7$ to calculate $C_j = a^{M_j} \pmod{q} = 2^7 \pmod{11} = 7$.

Public Key Cryptosystems

Public Key Distribution Systems

Finally both users A and B send their results $C_i=10$ and $C_j=7$ of calculation to partner to obtain the common key

$$K_{ij} = C_j^{M_i} (\bmod q) = (a^{M_j})^{M_i} (\bmod q) = 7^5 (\bmod 11) = 10$$

user B obtains the common key K_{ij} in a similar fashion as

$$K_{ji} = C_i^{M_j} (\bmod q) = (a^{M_i})^{M_j} (\bmod q) = 10^7 (\bmod 11) = 10$$

Thus, each user completes computation of the common key 10.

Public Key Cryptosystems

Public Key Distribution Systems

$GF(q)$, $q=19$ and
 $a=2,3,10,13,14,15$

a	a^2	a^3	a^4	a^5	a^6	a^7	a^8	a^9	a^{10}	a^{11}	a^{12}	a^{13}	a^{14}	a^{15}	a^{16}	a^{17}	a^{18}
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
2	4	8	16	13	7	14	9	18	17	15	11	3	6	12	5	10	1
<u>3</u>	<u>9</u>	<u>8</u>	<u>5</u>	<u>15</u>	<u>7</u>	<u>2</u>	<u>6</u>	<u>18</u>	<u>16</u>	<u>10</u>	<u>11</u>	<u>14</u>	<u>4</u>	<u>12</u>	<u>17</u>	<u>13</u>	<u>1</u>
10	5	12	6	3	11	15	17	18	9	14	7	13	16	8	4	2	1
13	17	12	4	14	11	10	16	18	6	2	7	15	5	8	9	3	1
14	6	8	17	10	7	3	4	18	5	13	11	2	9	12	16	15	1
15	16	12	9	2	11	13	5	18	4	3	7	10	17	8	6	14	1
4	16	7	9	17	11	6	5	1	4	16	7	9	17	11	6	5	1
7	11	1	7	11	1	7	11	1	7	11	1	7	11	1	7	11	1

Public Key Cryptosystems

Public Key Distribution Systems

Example. Consider the key exchange problem in $GF(2^m)$ for $m=3$. A primitive polynomial $p(x)$ of degree $m=3$ over $GF(2)$ is $p(x)=1+x+x^3$. If α be a root of $p(x)$ over $GF(2)$, then the field elements of $GF(2^3)$ generated by $p(\alpha)=1+\alpha+\alpha^3=0$ are shown below

<i>Power</i>	<i>Polynomial</i>	<i>Vector</i>
α^0	1	1 0 0
α^1	α	0 1 0
α^2	α^2	0 0 1
α^3	$1 + \alpha$	1 1 0
α^4	$\alpha + \alpha^2$	0 1 1
α^5	$1 + \alpha + \alpha^2$	1 1 1
α^6	$1 + \alpha^2$	1 0 1
α^7	1	1 0 0

Public Key Cryptosystems

Public Key Distribution Systems

Suppose users A and B select $M_i=2$ and $M_j=5$, respectively. Both $M_i=2$ and $M_j=5$ are kept secret, but $C_i=\alpha^{M_i}(\bmod p(x))=\alpha^2(\bmod (1+x+x^3))=001$ and $C_j=\alpha^{M_j}(\bmod p(x))=\alpha^5(\bmod (1+x+x^3))=111$ are placed in the public file. User A can communicate with user B by taking $C_j=111$ from the public file and computing their common key K_{ij} as follows

$$K_{ij}=(C_j)^{M_i}(\bmod p(x))=(\alpha^5)^2(\bmod p(x))=\alpha^{10}(\bmod p(x))=\alpha^3=110$$

Public Key Cryptosystems

Knapsack Cryptosystems

The knapsack problem is a mathematically attractive proportion for cryptography. The **Merkle-Hellman** scheme based on the trapdoor knapsack problem is a public-key asymmetric cryptosystem.

Assume a key (*things*) $K = \{k_1, k_2, k_3, \dots, k_n\}$, where k_i are integers for $i = 1, 2, \dots, n$. In a simplest way the knapsack problem can be defined as follows. C is the size of the knapsack and k_i is the size of the i -th *thing*.

Let $C = 1524$ and the possible *things* within this C are

$$K = (123, 763, 37, 1451, 830, 333, 621, 745, 971, 201).$$

Public Key Cryptosystems

Knapsack Cryptosystem I

Then the knapsack cryptosystem can be regarded as the enciphering the n -bit plaintext $M = \{x_1, x_2, x_3, \dots, x_n\}$, into the ciphertext according to the following formula:

$$C = KM = k_1x_1 + k_2x_2 + \dots + k_nx_n.$$

Calculation of C is as simple as seen from above equation, but recovery of M from C and K involves solving a knapsack problem and is generally difficult when n is large and K is randomly chosen.

Such interpretation of the Knapsack Problem can not be used for Cryptography.

If the key K is chosen such that each element of K is larger than the sum of the preceding elements, the corresponding knapsack problem becomes very simple.

Public Key Cryptosystems

Knapsack Cryptosystem II

That is $k_i > k_1 + k_2 + k_3 + \dots + k_{i-1}$. Letting

$$c_1 = k_1 x_1,$$

$$c_2 = k_1 x_1 + k_2 x_2,$$

...

$$C = c_n = k_1 x_1 + k_2 x_2 + \dots + k_n x_n.$$

where c_n represents the ciphertext C i.e. $C = c_n$. Now, the plaintext M can be recovered from c_n for $i=1,2,\dots,n$ and the key K according to the following procedure.

If $c_n < k_n$, then set $x_n = 0$ and $c_{n-1} = c_n$.

If $c_n > k_n$, then set $x_n = 1$ and $c_{n-1} = c_n - k_n$. Using the computed value c_{n-1} we can find x_{n-1} and c_{n-2} in a similar fashion. The recovery procedure continues until $M = \{x_1, x_2, \dots, x_n\}$, has been completely recreated.

Public Key Cryptosystems

Knapsack Cryptosystem II

Example. Suppose the plaintext $M=\{11001\}$ and the key $K=\{151, 187, 426, 1091, 2412\}$, the ciphertext C becomes

$$C = K \times M = 1 \times 151 + 1 \times 187 + 0 \times 426 + 0 \times 1091 + 1 \times 2412 = 2750$$

Since $C=c_5=2750$ is the ciphertext, the recovery of x_5 from y_5 and k_5 is $x_5=1$ because $c_5 = 2750 > k_5 = 2412$. Hence the procedure for recovery of x_i for $i=1,2,\dots,5$ will continue as shown below.

$$y_5=2750 > k_5 = 2412, x_5 = 1;$$

$$y_4=y_5 - k_5 = 338 < k_4 = 1091, x_4 = 0;$$

$$y_3=338 < k_3 = 426, x_3 = 0;$$

$$y_2=338 > k_2 = 187, x_2 = 1;$$

$$y_1=y_2 - k_2 = 151 = k_1, x_1 = 1.$$

Thus the recovered plaintext is $M=\{11001\}$ as expected.

However, this simple knapsack vector K cannot be used as a public enciphering key because someone can easily recover M from C .

Public Key Cryptosystems

Knapsack Cryptosystem III

One way function with *trap door*

The public key is a vector $K_p = wK_s \bmod m$ ($K_p = \{k_1, k_2, \dots, k_n\}$) which is also published by the user as his *public key*, while the parameters:

$w, v = w^{-1} \bmod m$ and m kept secret as his *private keys*.

K_p is generated by multiplying each component of the knapsack secret vector (*secret key*) $K_s = \{k_1^*, k_2^*, \dots, k_n^*\}$, where $k_i^* > k_1^* + k_2^* + \dots + k_{i-1}^*$, by w modulo m such that

$$k_i = wk_i^* \bmod m, i=1,2,\dots,n$$

where a secret quantity $v = w^{-1}$ is calculated from m and w , where $m > w$ and $(w, m) = 1$, moreover, m has to be larger than $k_1^* + k_2^* + \dots + k_n^*$ and

$$wv = 1 \bmod m \text{ or } ww^{-1} = 1 \bmod m$$

Public Key Cryptosystems

Knapsack Cryptosystem III

Then the knapsack cryptosystem can be regarded as the enciphering the n -bit plaintext $M = \{x_1, x_2, x_3, \dots, x_n\}$, into the ciphertext according to the following formula:

$$C = K_p \times M = k_1x_1 + k_2x_2 + \dots + k_nx_n.$$

Recovering the original plaintext M from the ciphertext C requires the transformation of C into C^* via v which is the multiplicative inverse of w modulo m . That is,

$$\begin{aligned} C^* &= vC \bmod m = w^{-1}C \bmod m = w^{-1}(K_p \times M) \bmod m = \\ &= w^{-1}(k_1x_1 + k_2x_2 + \dots + k_nx_n) \bmod m. \end{aligned}$$

Taking into account, that $k_i = wk_i^* \bmod m$,

$$C^* = (w^{-1}wk_1^*x_1 \bmod m + w^{-1}wk_2^*x_2 \bmod m + \dots + w^{-1}wk_n^*x_n \bmod m) \bmod m.$$

Then due to $w^{-1}w = 1$, and $k_1^* + k_2^* + \dots + k_n^* < m$,

$$\begin{aligned} C^* &= (k_1^*x_1 \bmod m + k_2^*x_2 \bmod m + \dots + k_n^*x_n \bmod m) \bmod m = \\ &= (k_1^*x_1 + k_2^*x_2 + \dots + k_n^*x_n) \bmod m = (k_1^*x_1 + k_2^*x_2 + \dots + k_n^*x_n). \end{aligned}$$

Public Key Cryptosystems

Knapsack Cryptosystem Design

1. First of all the *secret key* should be generated.

$K_s = \{k_1^*, k_2^*, \dots, k_n^*\}$ with $k_i^* > k_1^* + k_2^* + k_3^* + \dots + k_{i-1}^*$.

2. The random value m (*secret*) is chosen such that

$m > k_1^* + k_2^* + \dots + k_n^*$, what is important requirement for deciphering procedure.

3. Then the random value of w (*secret*) is chosen such that $w < m$, $(w, m) = 1$.

4. And one more *secret* parameter v is calculated as $v = w^{-1}$, according to $wv = 1 \bmod m$

5. Then *open key* is generated as $K_p = wK_s \bmod m$, where $k_i = wk_i^* \bmod m$, $i = 1, 2, \dots, n$.

6. Open key $K_p = \{k_1, k_2, \dots, k_n\}$ is available for everybody

Public Key Cryptosystems

Knapsack Cryptosystem Design Example

1. First of all the *secret key* is generated as

$K_s = \{k_1^*, k_2^*, \dots, k_n^*\} = \{1, 3, 5, 11, 21, 44, 87, 175, 349, 701\}$ with $k_i^* > k_1^* + k_2^* + k_3^* + \dots + k_{i-1}^*$.

2. The random value $m = 1590 > k_1^* + k_2^* + \dots + k_n^* = 1397$ is selected.

3. Then the random value of $w=43$ (*secret*) is chosen such that $w=43 < m=1590$, $(w, m) = (43, 1590) = 1$.

4. *Secret* parameter v is calculated as inverse value of w from the linear equation $43v = 1 \bmod 1590$. The result is $v = w^{-1} = 37$

5. Then *open key* is generated as $k_i = wk_i^* \bmod m$, $i=1, 2, \dots, n$.

$$k_1 = 43 \times 1 \bmod 1590 = 43;$$

$$k_2 = 43 \times 3 \bmod 1590 = 129;$$

...

$$k_n = 43 \times 701 \bmod 1590 = 473;$$

6. Open key is $K_p = \{k_1, k_2, \dots, k_n\} = \{43, 129, 215, 903, 302, 561, 1165, 697, 1523, 473\}$

Теория Информации

В.Н. Ярмолик

Лекция 11

2017

Public Key Cryptosystems

Exponentiation Ciphers

In 1978, ***Pohling*** and ***Hellman*** published an encryption scheme based on computing exponentials over a finite field. At about the same time, *Rivest*, *Shamir* and *Adleman* published a similar scheme, but with a slight twist - a twist that gave the MIT group a method for realizing public-key encryption as put forth by *Deffie* and *Hellman*.

The *Pohling-Hellman* and *RSA* schemes both encipher a message block $M=0,1,...,n-1$ by computing the exponential

$$C=M^e \bmod n,$$

where e and n are the key to the enciphering transformation. M is restored by the same operation, but using a different exponent d for the key:

$$M=C^d \bmod n.$$

Enciphering and deciphering can be implemented using the fast exponentiation algorithm shown earlier, so $C=\text{fastexp}(M,e,n)$ and $M=\text{fastexp}(C,d,n)$.

Public Key Cryptosystems

Exponentiation Ciphers

The enciphering and deciphering transformations are based on Euler's generalization of *Fermat's* Theorem, which states that for every M relatively prime to n :

$$M^{\phi(n)} \bmod n = 1.$$

This property implies that if e and d satisfy the relation

$$ed = 1 \bmod \phi(n),$$

then deciphering restores the original plaintext message. This result is proved in the following theorem:

Public Key Cryptosystems

Exponentiation Ciphers

Theorem. Given e and d satisfying $ed \bmod \phi(n)=1$ and a message $M=0,1,\dots,n-1$ such that $\gcd(M,n)=1$, then

$$(M^e \bmod n)^d \bmod n = M.$$

Proof: We have

$$(M^e \bmod n)^d \bmod n = M^{ed} \bmod n.$$

Now, $ed \bmod \phi(n)=1$ implies that $ed = t\phi(n)+1$ for some integer t .

Thus,

$$\begin{aligned} M^{ed} \bmod n &= M^{t\phi(n)+1} \bmod n = M M^{t\phi(n)} \bmod n = M (M^{t\phi(n)} \bmod n) \bmod n = \\ &= M (M^{\phi(n)} \bmod n)^t \bmod n = M 1^t \bmod n = M. \end{aligned}$$

By symmetry, enciphering and deciphering are commutative and mutual inverses; thus,

$$(M^d \bmod n)^e \bmod n = M^{de} \bmod n = M.$$

Public Key Cryptosystems

Exponentiation Ciphers

Given $\phi(n)$, it is easy to generate a pair (e,d) satisfying $ed \bmod \phi(n)=1$. This is done by first choosing d relatively prime $\phi(n)$, and then using the extended version of *Euclid's* algorithm to compute its inverse:

$$e = \text{inv}(d, \phi(n)).$$

Because e and d are symmetric, we could also pick e and compute $d = \text{inv}(e, \phi(n))$.

In the ***Pohlig-Hellman Scheme***, the modulus n is chosen to be large prime p . The enciphering and deciphering functions are thus given by

$$C = M^e \bmod p$$

$$M = C^d \bmod p,$$

where all arithmetic is done in the Galois field $\mathbf{GF}(p)$. Because $n=p$ is prime, $\phi(n) = p-1$, which is trivially derived from p . Thus the scheme can be used for conventional encryption, where e and d are both kept secret.

Public Key Cryptosystems

Exponentiation Ciphers

Example. Let $p=11$, whence $\phi(p)=p-1=10$. Choose $d=7$ and compute $e=\text{inv}(7,10)=3$. Suppose, $M=5$. Then M is enciphered as:

$$C = M^e \bmod p = 5^3 \bmod 11 = 4.$$

Similarly, C is deciphered as:

$$M = C^d \bmod p = 4^7 \bmod 11 = 5.$$

The security of the scheme rests on the complexity of computing discrete logarithms in $\mathbf{GF}(p)$.

Public Key Cryptosystems

Riverst-Shamir-Adleman (RSA) Scheme

In the RSA scheme, the modulus n is the product of two large primes p and q :

$$n=pq.$$

Thus, $\phi(n)=(p-1)(q-1)$. The enciphering and deciphering functions are

$$C=M^e \bmod n$$

$$M=C^d \bmod n,$$

Example. Let $p=5$, $q=7$ whence $n=pq=35$ and $\phi(n)=(5-1)(7-1)=24$. Choose $d=11$. and compute $e=\text{inv}(11,24)=11$. Suppose, $M=2$. Then M is enciphered as:

$$C=M^e \bmod n=2^{11} \bmod 35=2048 \bmod 35 = 18.$$

and deciphering as:

$$M=C^d \bmod n=18^{11} \bmod 35=2.$$

Public Key Cryptosystems

Riverst-Shamir-Adleman (RSA) Scheme

Example. Let $p=53$, $p=61$ whence $n=pq=53\times 61=3233$ and $\phi(n)=(53-1)(61-1)=3120$. Letting $d=791$, we get $e=71$.

$$ed=791\times 71=1 \text{ mod } 3120$$

To encipher the message $M=RENAISSANCE$, we break it into blocks of 2 digits each, where $A=00$, $B=01, \dots, Z=25$, and blank =26. We thus get

$$M = RE \quad NA \quad IS \quad SA \quad NC \quad E$$

$$M=1704 \ 1300 \ 0818 \ 1800 \ 1302 \ 0426$$

The first block is enciphered as $1704^{71}=3106$. The entire message is enciphered as

$$C=3106 \ 0100 \ 0931 \ 2691 \ 1984 \ 2927.$$

Public Key Cryptosystems

Procedure RSA's Key Generation

1. Select two random *secret* primes, p and q , where
 - a) p and q differ by a few digits;
 - b) both $p-1$ and $q-1$ should contain large prime factors;
 - c) $\gcd(p-1, q-1)$ should be small.
2. Compute the product $n=pq$, which is the *public* modulus, where p and q are randomly selected *secret* values.
3. Calculate Euler's function, $\phi(n)=(p-1)(q-1)$, which is *secret*.
4. Select the key e or d , which is relatively prime to $\phi(n)$.
5. Calculate the multiplicative inverse of either d or e according to the equation $ed \bmod \phi(n)=1$.

Public Key Cryptosystems

Comparison of Secret Key (SK) and Public Key (PK) cryptosystems

<i>DISTINCT FEATURES</i>	<i>SECRET KEY</i>	<i>PUBLIC KEY</i>
NUMBER OF KEYS	Single key.	Pair of keys.
TYPES OF KEYS	Key is secret.	One key is private, and one key is public.
SIZE OF KEY	50-250 bits	500-2500 bits
RELATIVE SPEEDS	Faster.	Slower.

Public Key Cryptosystems

Application of Public Key (PK) cryptosystems

Three important application of public-key algorithms:

Public-Key Distribution Schemes (PKDS) - where the scheme is used to securely exchange a single piece of information (whose value depends on the two parties, but cannot be set). This value is normally used as a session key for a private-key scheme.

Signature Schemes - used to create a digital signature only, where the private-key signs (create) signatures, and the public-key verifies signatures.

Public Key Schemes (PKS) - used for encryption, where the public-key encrypts messages, and the private-key decrypts messages.

Теория Информации

В.Н. Ярмолик

Лекция 12

2017

Stream Cipher Cryptosystem

Stream Ciphers

There are two types of stream ciphers - one for which the key-bit stream is independent of the plaintext and the other for which the key-bit stream is a function of either the plaintext or the ciphertext depending on the feedback connection. The former is often called a ***synchronous cipher*** because it requires synchronization between the key-bit stream and the ciphertext for successful decipherment. The latter is often referred to as a ***self-synchronizing cipher*** or auto-key cipher because an erroneously added or deleted bit causes only a fixed number of errors in the deciphered plaintext, after which the correct plaintext is again restored.

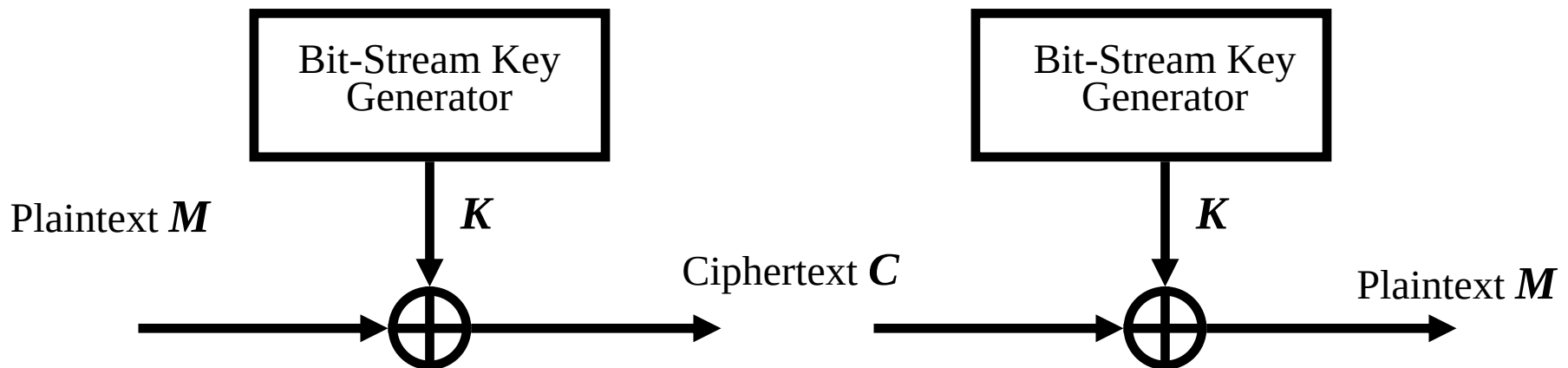


Fig. Stream Cipher

Stream Cipher Cryptosystem

Stream Cipher

Some of the differences between block and stream cipher are:

- 1.The block cipher enciphers a single data block at one time, but the stream cipher requires to encipher or decipher on a bit-by-bit bases;
- 2.In the block cipher, every ciphertext bit is a complex function of plaintext and key bits via intersymbol dependence, but in the stream cipher every cipher bit is produced by Exclusive Or operation of the plaintext bit and key bit;
- 3.The block cipher may not require an initial seed vector, but the stream cipher does;
- 4.Block cipher like DES used in the commercial sector, but stream ciphers are used for military purposes.

Stream Cipher Cryptosystem

Stream Cipher

The main part of any stream cipher is key generation algorithm. More often used algorithm for various application is ***Multiplicative Congruent Algorithm***

$$\underline{x_{t+1}} = \underline{(ax_t + c) \bmod N};$$

Where for Pentium PC is a very popular module

$N = 2^{31} - 1 = 2147483647$, and “magic”

values of a are: 16807, 630360016, 1078318381, 1203248318, 397204094, 2037812808, 1323257245, 764261123, 112817,

Stream Cipher Cryptosystem

Stream Cipher

Pseudorandom Number Generators

1. $x_{t+1} = (1176x_t + 1476x_{t-1} + 1776x_{t-2}) \bmod (2^{32}-5);$

2. $x_{t+1} = (2^{13}(x_t + x_{t-1} + x_{t-2})) \bmod (2^{32}-5);$

3. $x_{t+1} = (1995x_t + 1998x_{t-1} + 2001x_{t-2}) \bmod (2^{32}-849);$

4. $x_{t+1} = (2^{19}(x_t + x_{t-1} + x_{t-2})) \bmod (2^{32}-1629);$

5. $x_{t+1} = (5115x_t + 1776x_{t-1} + 1492x_{t-2} + 2111111111x_{t-3} + c_t) \bmod 2^{32};$

$c_t = \lfloor (5115x_{t-1} + 1776x_{t-2} + 1492x_{t-3} + 2111111111x_{t-4} + c_{t-1}) / 2^{32} \rfloor.$

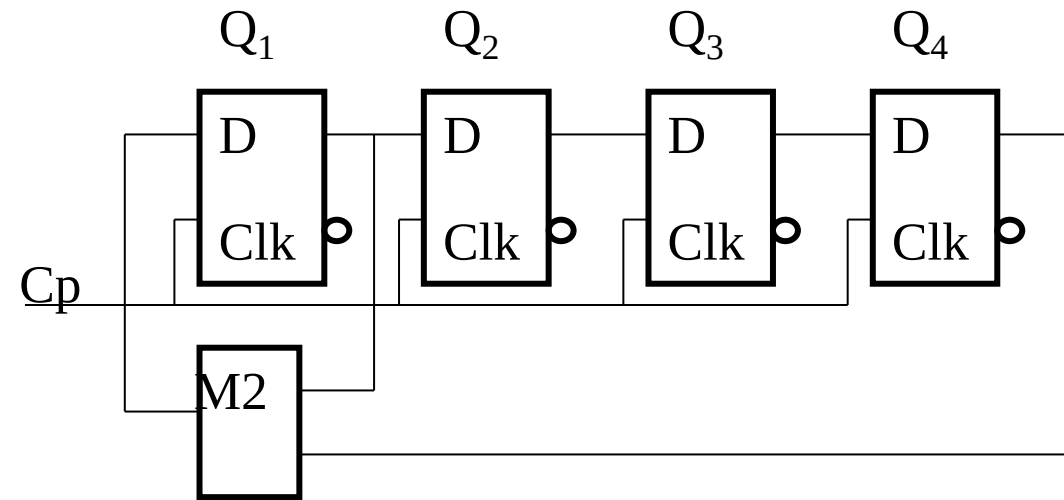
6. COMBO: $z_n = (x_n + y_n) \bmod 2^{32}, x_n = (x_{n-1} * x_{n-2}) \bmod 2^{32}, y_n = (y_{n-1} + c_n) \bmod 2^{16}, c_n = \lfloor (y_{n-1} + c_{n-1}) / 2^{16} \rfloor.$

7. KISS: $z_n = (x_n + y_n + u_n) \bmod 2^{32}, x_n = (69069x_{n-1} + 1) \bmod 2^{32}, y_n = y_{n-1}(I_{32} + L^{13})(I_{32} + R^{17})(I_{32} + L^5), u_n = (2u_{n-1} + u_{n-2} + c_n) \bmod 2^{32}, c_n = \lfloor (2u_{n-2} + u_{n-3} + c_{n-1}) / \bmod 2^{32} \rfloor.$

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

A **linear feedback shift register (LFSR)** is often used as the cryptographic key encoder for a stream cipher system. One of the main reason for this is that LFSR is easily obtainable and comparatively inexpensive. The encoder that generates the key-bit stream must be deterministic so that the key-bit stream may be reproduced for decipherment. According to the primitive polynomial $\phi(x)=1+x+x^4$ the following block diagram of the LFSR can be design



$$Q_1(k+1) = Q_1(k) \oplus Q_4(k)$$

$$Q_2(k+1) = Q_1(k)$$

$$Q_3(k+1) = Q_2(k)$$

$$Q_4(k+1) = Q_3(k)$$

Fig. LFSR for the generating polynomial $\phi(x)=1+x+x^4$

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

State sequence for LFSR

State	Q ₁ Q ₂ Q ₃ Q ₄	State	Q ₁ Q ₂ Q ₃ Q ₄
0	1 0 0 0	8	1 1 0 1
1	1 1 0 0	9	0 1 1 0
2	1 1 1 0	10	0 0 1 1
3	1 1 1 1	11	1 0 0 1
4	0 1 1 1	12	0 1 0 0
5	1 0 1 1	13	0 0 1 0
6	0 1 0 1	14	0 0 0 1
7	1 0 1 0	15	1 0 0 0

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

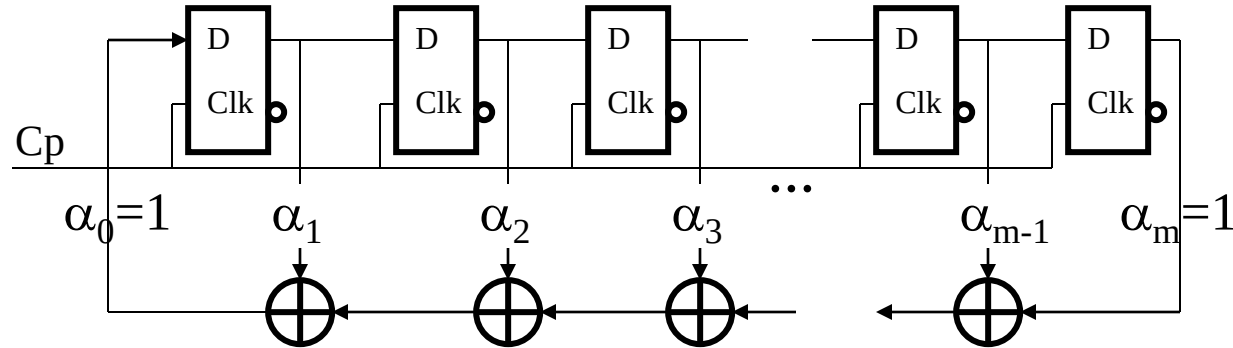
One primitive polynomial for each m from 1 to 28

$m=\deg \varphi(x)$	$\varphi(x)$	$m=\deg \varphi(x)$	$\varphi(x)$
1	$1+x$	15	$1+x+x^{15}$
2	$1+x+x^2$	16	$1+x^2+x^3+x^5+x^{16}$
3	$1+x+x^3$	17	$1+x^3+x^{17}$
4	$1+x+x^4$	18	$1+x^7+x^{18}$
5	$1+x^2+x^5$	19	$1+x+x^2+x^5+x^{19}$
6	$1+x+x^6$	20	$1+x^3+x^{20}$
7	$1+x+x^7$	21	$1+x^2+x^{21}$
8	$1+x+x^5+x^6+x^8$	22	$1+x+x^{22}$
9	$1+x^4+x^9$	23	$1+x^5+x^{23}$
10	$1+x^3+x^{10}$	24	$1+x^3+x^4+x^9+x^{24}$
11	$1+x^2+x^{11}$	25	$1+x^3+x^{25}$
12	$1+x^3+x^4+x^7+x^{12}$	26	$1+x+x^2+x^6+x^{26}$
13	$1+x+x^3+x^4+x^{13}$	27	$1+x+x^2+x^5+x^{27}$
14	$1+x+x^{11}+x^{12}+x^{14}$	28	$1+x^3+x^{28}$

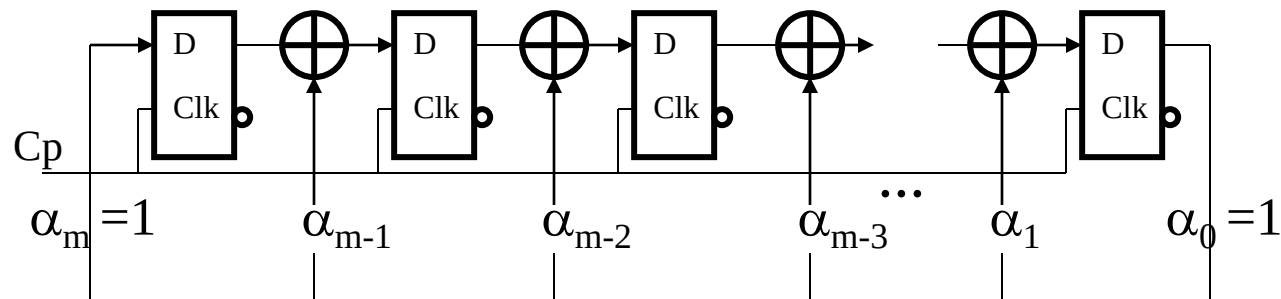
Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

$$\varphi(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \dots + \alpha_{m-1} x^{m-1} + \alpha_m x^m; \alpha_m = \alpha_0 = 1; \alpha_i \in \{0, 1\}$$



Standard form of LFSR Type 1

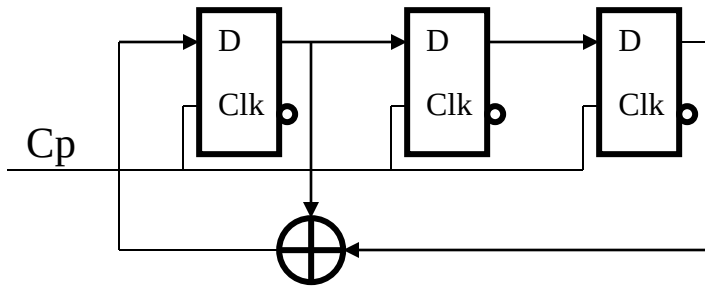


Standard form of LFSR Type 2

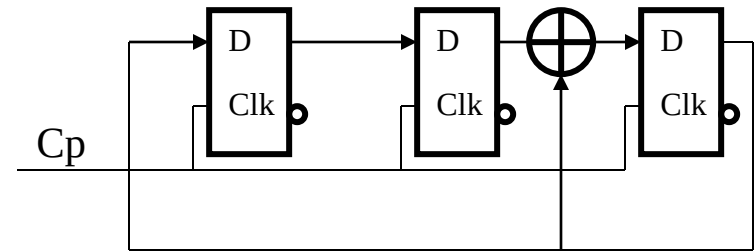
Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

Example $\varphi(x) = 1 + x + x^3$



1 0 0
1 1 0
1 1 1
0 1 1
1 0 1
0 1 0
0 0 1



1 0 0
0 1 0
0 0 1
1 0 1
1 1 1
1 1 0
0 1 1

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

Mathematical description of LFSR

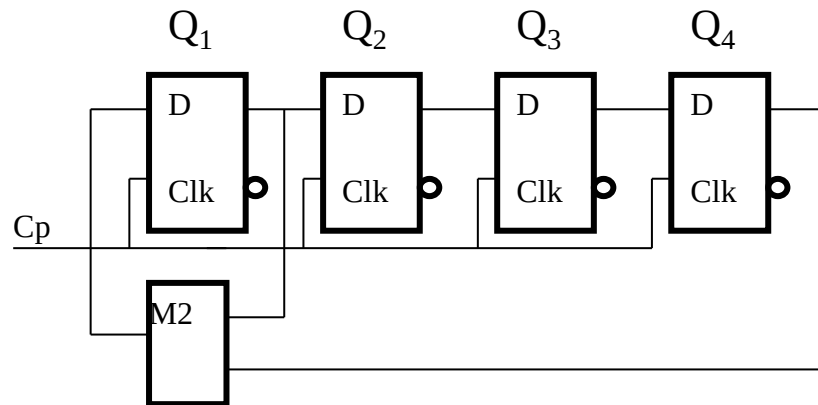
$$\begin{vmatrix} a_1(k+1) \\ a_2(k+1) \\ a_3(k+1) \\ \dots \\ a_m(k+1) \end{vmatrix} = \begin{vmatrix} \alpha_1 \alpha_2 \alpha_3 \dots \alpha_{m-1} \alpha_m \\ 1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 1 & 0 & \dots & 0 & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & 1 & 0 \end{vmatrix} \times \begin{vmatrix} a_1(k) \\ a_2(k) \\ a_3(k) \\ \dots \\ a_m(k) \end{vmatrix}$$

$$A(k+1) = V \times A(k)$$

$$a_1(k+1) = \sum_{i=1}^m \alpha_i a_i(k);$$

$$a_j(k+1) = a_{j-1}(k), j = \overline{2, m}, k = 0, 1, 2, \dots$$

Example



$$V = \begin{vmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{vmatrix}$$

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

Properties of pseudorandom sequences (M-sequences)

1. There are $\Phi(2^m-1)/m$ primitive polynomial for a given m , where Φ is a Euler function.
2. For a given polynomial $\varphi(x)$ there is an inverse polynomial which can be derived according to the following relation $\varphi^{-1}(x)=x^m\varphi(x^{-1})$.
3. The period L of a M-sequence depends on of the primitive polynomial degree only
 $L=2^m-1$.
4. For the specified polynomial $\varphi(x)$ there are L distinct M-sequences that are differ by phase shifts. Thus 15 M-sequences are associated with the polynomial $\varphi(x)=1+x+x^4$.

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

5. In M-sequence there is unpredictable (pseudorandom) sequence of ones and zeros.
Their probabilities is given by

$$p(a_k=1)=\frac{2^{m-1}}{2^m-1}=\frac{1}{2}+\frac{1}{2^{m+1}-2};$$

$$p(a_k=0)=\frac{2^{m-1}-1}{2^m-1}=\frac{1}{2}-\frac{1}{2^{m+1}-2};$$

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

6. There is one run of m (consecutive) 1's and one run of $m-1$ 0's. For $m-1 > r > 0$, there are $2^{m-(r+2)}$ runs of length r for 1's and the same number of runs of 0's.

For the M-sequence **000111101011001** generated according to the polynomial $\varphi(x)=1+x+x^4$ there are one run of 4 1's one run of 3 0's, one run each of 2 0's or 1's.

7. There is the autocorrelation property that measures the similarity between original M-sequence and a shifted version of the same sequence. Any pair of such sequence will be identical in $2^{m-1}-1$ positions and will differ in 2^{m-1} positions.

000111101011001
011110101100100

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

8. '*Shift-and-add*' property. For any s ($1 \leq s < L$) there are exist an $r \neq s$ ($1 \leq r < L$) such that $\{a_k\} \oplus \{a_{k-s}\} = \{a_{k-r}\}$.

$\{a_0\}$	<i>000111101011001</i>
$\{a_{-2}\}$	<i>011110101100100</i>
$\{a_{-9}\}$	<i>011001000111101</i>

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

9. Among the L M-sequences generated by the polynomial $\varphi(x)$ there exists a single sequence with the property $a_k = a_{2k}$, $k=0,1,2,\dots$ which is called ***characteristic*** and derived as follows. For the specified polynomial the system of linear equations

$$a_i = a_{2i}; \quad i=0 \text{ to } m-1;$$

As an example, we shall find characteristic sequence for the generating polynomial $\varphi(x)=1+x^3+x^4$. For this case the system of linear equations takes the form

$$a_0 = a_0$$

$$a_1 = a_2$$

$$a_2 = a_4 = a_0 \oplus a_1$$

$$a_3 = a_6 = a_2 \oplus a_3$$

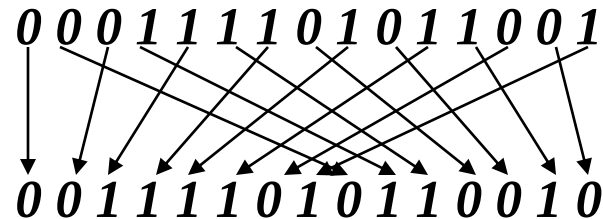
The solution is $a_1 a_2 a_3 a_4 = 1000$.

Stream Cipher Cryptosystem

LFSR-Linear Feedback Shift Register

10. Property of '**Decimation**'. Decimation of the M-sequence $\{a_i\}$ by q , ($q=1,2,3,\dots$) means the generation of another sequence $\{b_j\}$ as q th elements of original M-sequence $\{a_i\}$, i.e. $b_j = a_{qj}$.

With $(L,q)=1$ the period of $\{b_j\}$ is 2^m-1 , and decimation is said to be proper or normal.

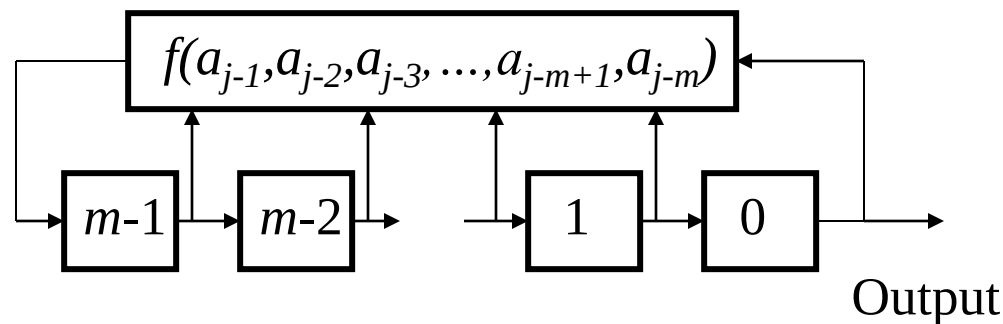


Stream Cipher Cryptosystem

NFSR - NonLinear Feedback Shift Registers

A general feedback shift register (FSR) of length m consists of m stages (or delay elements). During each unit of time the following operations are performed: (i) the contents of stage 0 is output and forms part of the output sequence; (ii) the contents of stage j is moved to stage $j-1$ for each j , $1 \leq j \leq m-1$; (iii) the new content of stage $m-1$ is the feedback bit $a_j = f(a_{j-1}, a_{j-2}, a_{j-3}, \dots, a_{j-m+1}, a_{j-m})$, where the feedback function f is a Boolean function and a_{j-r} is the previous content of the shift register.

Not that if the feedback function f is a linear function, then the FSR is an LFSR, otherwise the FSR is called a nonlinear FSR.



Example. De Bruin sequence for the case of $m=3$ and nonlinear feedback function $f(x_1, x_2, x_3) = 1 \oplus x_2 \oplus x_3 \oplus x_1 x_2$. The output sequence is the **de Bruijn** sequence with cycle 0,0,0,1,1,1,0,1

Stream Cipher Cryptosystem

NFSR - NonLinear Feedback Shift Registers

Converting a maximum-length generator (LFSR) to the de Bruin Generator

Let we have a LFSR of length m with (linear) feedback function $a_j = f(a_{j-1}, a_{j-2}, a_{j-3}, \dots, a_{j-m+1}, a_{j-m})$, then FSR with feedback function $g(a_{j-1}, a_{j-2}, a_{j-3}, \dots, a_{j-m+1}, a_{j-m}) = f(a_{j-1}, a_{j-2}, a_{j-3}, \dots, a_{j-m+1}, a_{j-m}) \oplus a_{j-1}^* a_{j-2}^* a_{j-3}^* \dots a_{j-m+1}^*$ is a ***de Bruijn FSR***. Here a_{j-1}^* denotes the complement of a_{j-1} .

Statement 1. For any M-sequence there is only one m -bit code $000\dots 01$, where $m = \deg \varphi(x)$ and $\varphi(x)$ is the primitive polynomial.

Statement 2. For any M-sequence there is the sequence of m -bit codes $000\dots 01, 100\dots 00$, where $m = \deg \varphi(x)$.

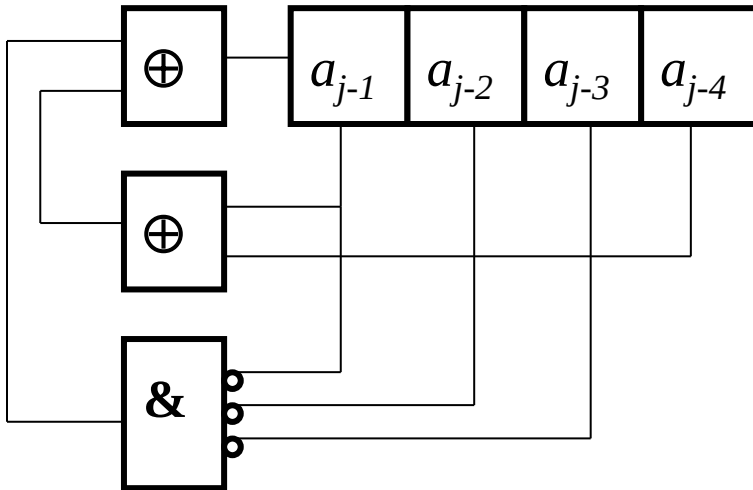
Statement 3. For any M-sequence there is the only one m -bit code with $m-1$ first zero, namely the code $000\dots 01$.

Stream Cipher Cryptosystem

NFSR - NonLinear Feedback Shift Registers

Example. De Bruin sequence for the case of $m=3$ and linear feedback function described by primitive polynomial $\phi(x)=1 \oplus x^1 \oplus x^4$, has a nonlinear function

$$g(a_1, a_2, a_3, a_4) = a_1 \oplus a_4 \oplus a_1^* a_2^* a_3^*.$$



a_3	a_2	a_1	a_0
1	0	0	0
1	1	0	0
1	1	1	0
1	1	1	1
0	1	1	1
1	0	1	1
0	1	0	1
1	0	1	0
1	1	0	1
0	1	1	0
0	0	1	1

1	0	0	1
0	1	0	0
0	0	1	0
0	0	0	1
0	0	0	0
1	0	0	0

Stream Cipher Cryptosystem

NFSR - NonLinear Feedback Shift Registers

Ford sequence

Ford sequences are a special class of de Bruijn sequences and due to the simplest algorithm for generation can be applied for bit stream generation.

Algorithm: A successive m -bit code $X_k = (b_2, b_3, b_4, \dots, b_{m+1})$ is generated on the basis of the preceding code $X_{k-1} = (b_1, b_2, b_3, \dots, b_m)$, where the value of $b_{m+1} \in \{0, 1\}$ is determined as follows.

1. The code of the form $X'_{k-1} = (b_2, b_3, b_4, \dots, b_m, 1)$ is generated.
2. All its cyclic shifts are obtained.
3. Among which code $X''_{k-1} = (b_i, b_{i+1}, b_{i+2}, \dots, b_m, 1, b_2, b_3, \dots, b_{i-1})$, which is the largest m -bit number is chosen.
4. If $b_2 = b_3 = \dots = b_{i-1} = 0$, then $b_{m+1} = b_1 \oplus 1$, otherwise $b_{m+1} = b_1$.

Stream Cipher Cryptosystem

NFSR - NonLinear Feedback Shift Registers

Example. Ford sequence for the case of $m=4$

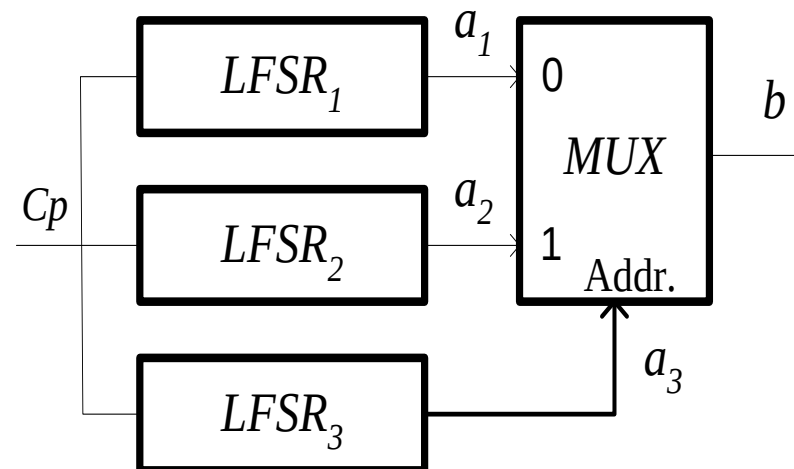
k	X_{k-1}	X''_{k-1}	b_{m+1}
1	0000	1000	$b_1 \oplus 1 = 0 \oplus 1 = 1$
2	0001	1100	$b_1 \oplus 1 = 0 \oplus 1 = 1$
3	0011	1110	$b_1 \oplus 1 = 0 \oplus 1 = 1$
4	0111	1111	$b_1 \oplus 1 = 0 \oplus 1 = 1$
5	1111	1111	$b_1 \oplus 1 = 1 \oplus 1 = 0$
6	1110	1110	$b_1 = 1$
7	1101	1110	$b_1 = 1$
8	1011	1110	$b_1 \oplus 1 = 1 \oplus 1 = 0$
9	0110	1110	$b_1 = 0$
10	1100	1100	$b_1 = 1$
11	1001	1100	$b_1 \oplus 1 = 1 \oplus 1 = 0$
12	0010	1010	$b_1 \oplus 1 = 0 \oplus 1 = 1$
13	0101	1110	$b_1 = 0$
14	1010	1010	$b_1 \oplus 1 = 1 \oplus 1 = 0$
15	0100	1100	$b_1 = 0$
16	1000	1000	$b_1 \oplus 1 = 1 \oplus 1 = 0$

Stream Cipher Cryptosystem

NFSR - NonLinear Feedback Shift Registers

Geffe generator

The ***Geffe generator***, as well as ***Stop and Go generator*** is defined by three maximum length *LFSRs* whose length m_1 , m_2 and m_3 are pair wise relatively prime, and the key stream length is $(2^{m_1}-1)(2^{m_2}-1)(2^{m_3}-1)$.

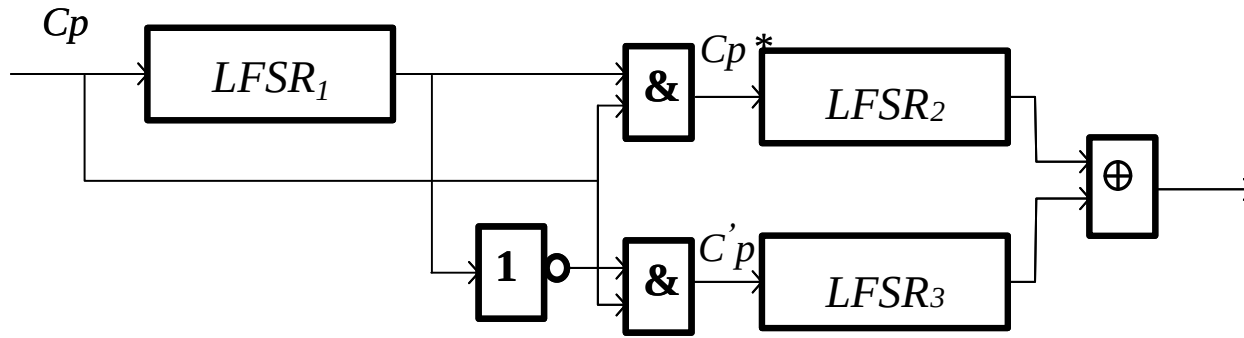
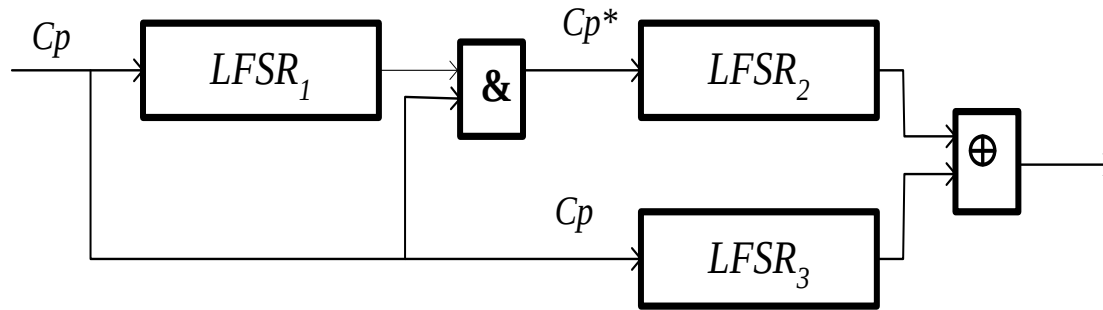


$$b = (a_2 a_3) + (a_1 (1 \oplus a_3)),$$

Stream Cipher Cryptosystem

NFSR - NonLinear Feedback Shift Registers

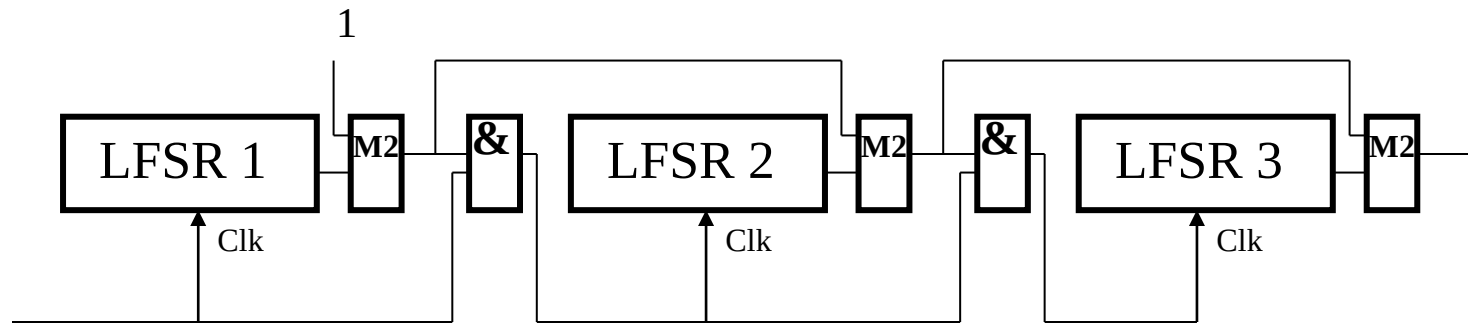
Stop & Go generator
Beth-Piper Stop-and-Go generator



Stream Cipher Cryptosystem

NFSR - NonLinear Feedback Shift Registers

Gollmann Cascaded Key stream generator

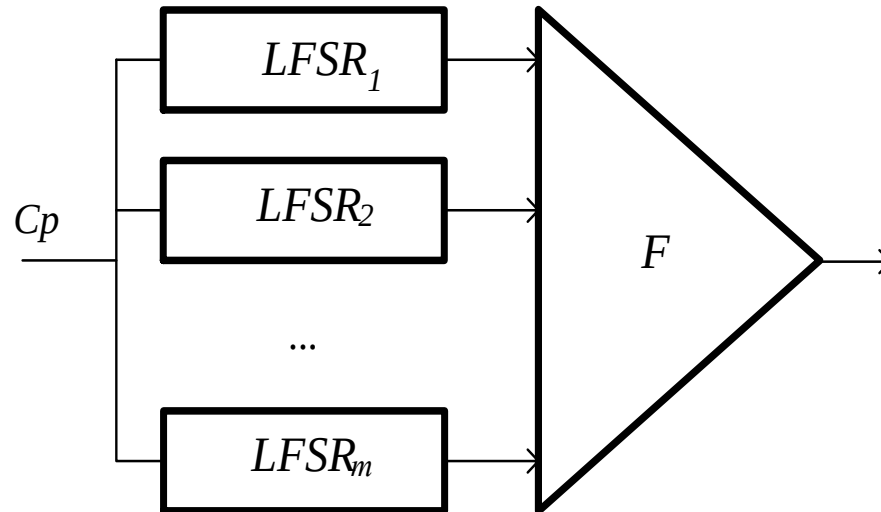


The ***Goldman Cascaded Key stream generator*** generates the sequence with the length is $(2^{m1}-1)(2^{m2}-1)(2^{m3}-1)$.

Stream Cipher Cryptosystem

NFSR - NonLinear Feedback Shift Registers

Threshold generator



Stream Cipher Cryptosystem

A5/1 Algorithm

A5/1 is a stream cipher used to provide over-the-air communication privacy in the GSM cellular telephone standard. It was initially kept secret, but became public knowledge through leaks and reverse engineering.

A GSM transmission is organised as sequences of ***bursts***. In a typical channel and in one direction, one burst is sent every 4.615 milliseconds and contains 114 bits available for information. A5/1 is used to produce for each burst a 114 bit sequence of keystream which is XORed with the 114 bits prior to modulation. A5/1 is ***initialized*** using a **64-bit key** together with a publicly-known **22-bit frame number**.

A5/1 is based around a combination of three ***Linear Feedback Shift Registers*** (LFSRs) with irregular clocking. The three shift registers are specified as follows:

Stream Cipher Cryptosystem

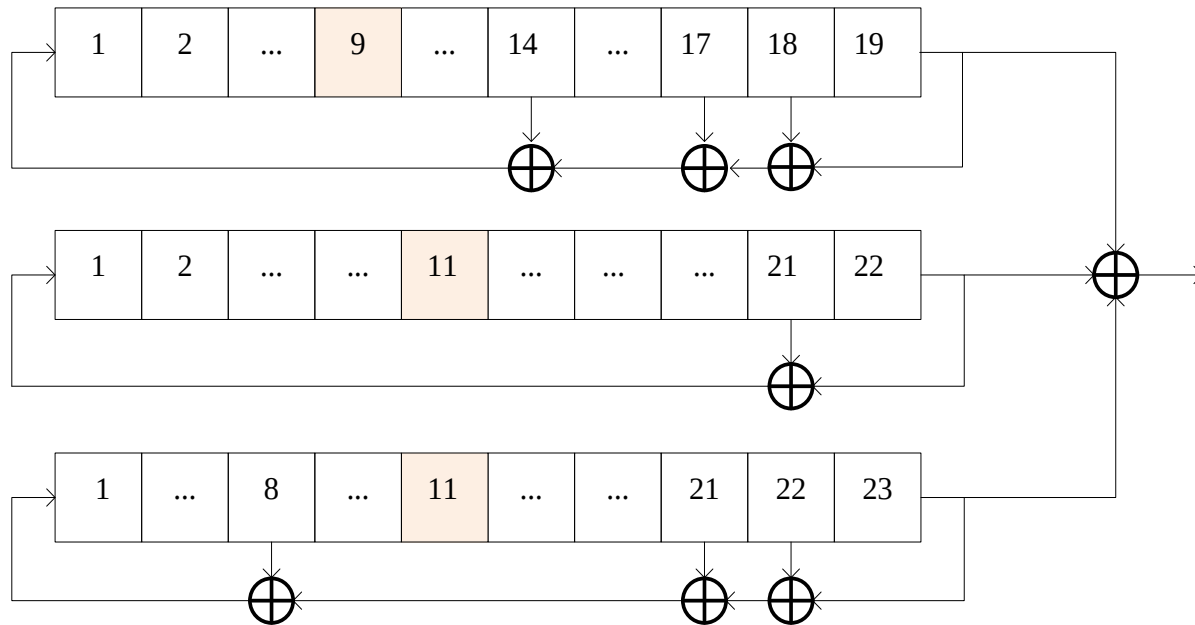
A5/1 Algorithm

LFSR number	Length in bits	Feedback polynomial	Clocking bit	Tapped bits
1	19	$x^{19} + x^{18} + x^{17} + x^{14} + 1$	9	14, 17, 18, 19
2	22	$x^{22} + x^{21} + 1$	11	21, 22
3	23	$x^{23} + x^{22} + x^{21} + x^8 + 1$	11	8, 21, 22, 23

The registers are clocked in a stop/go fashion using a majority rule. Each register has an associated clocking bit. At each cycle, the clocking bit of all three registers is examined and the majority bit is determined. A register is clocked if the clocking bit agrees with the majority bit. Hence at each step two or three registers are clocked, and each register stops with probability 3/4.

Stream Cipher Cryptosystem

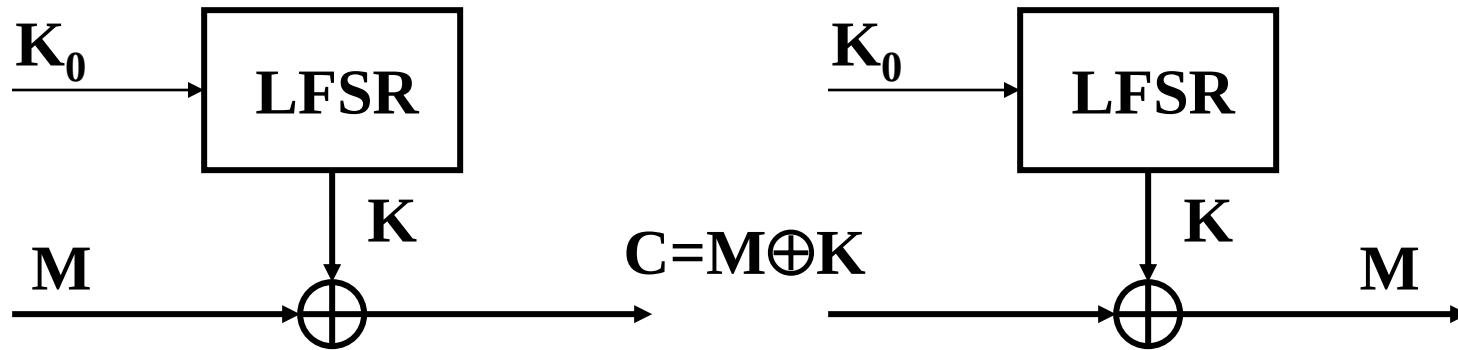
A5/1 Algorithm



Stream Cipher Cryptosystem

Synchronous Stream Cipher

Synchronous Stream Cipher can be implemented based on LFSR according to the following block diagram.

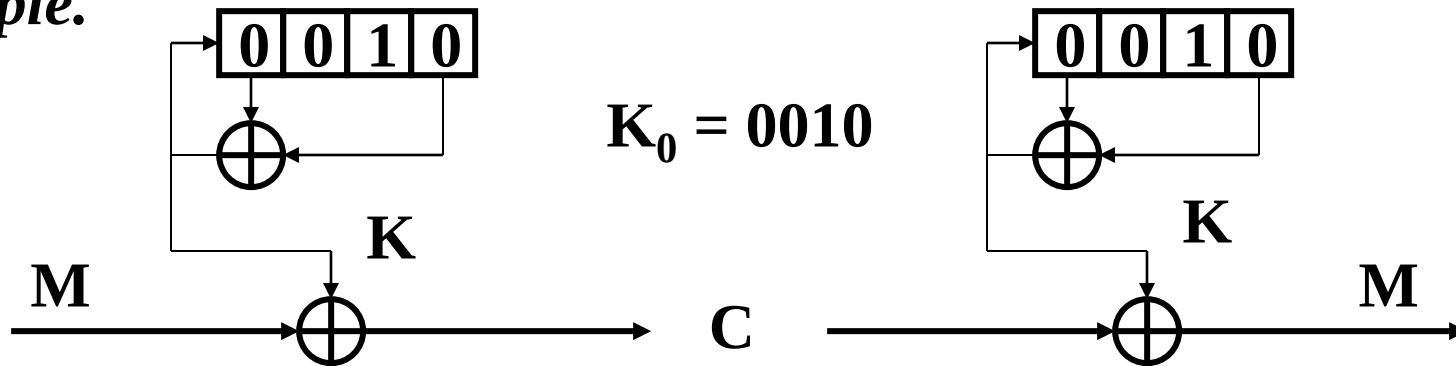


K_0 – Seed Vector; M – Plaintext; C – Ciphertext.

Stream Cipher Cryptosystem

Synchronous Stream Cipher

Example.



$M=101011111100001$
$K=011110101100100$
$C=110101010000101$

Stream Cipher Cryptosystem

Synchronous Stream Cipher

However, the LFSR is not always suitable for key generation because an opponent can easily derive the entire key stream from **$2m$** bits of plaintext-ciphertext pairs.

$$k_{m+1} = \alpha_1 k_m \oplus \alpha_2 k_{m-1} \oplus \alpha_3 k_{m-2} \oplus \dots \oplus \alpha_m k_1;$$

$$k_{m+2} = \alpha_1 k_{m+1} \oplus \alpha_2 k_m \oplus \alpha_3 k_{m-1} \oplus \dots \oplus \alpha_m k_2;$$

$$k_{m+3} = \alpha_1 k_{m+2} \oplus \alpha_2 k_{m+1} \oplus \alpha_3 k_m \oplus \dots \oplus \alpha_m k_3;$$

...

$$k_{2m} = \alpha_1 k_{2m-1} \oplus \alpha_2 k_{2m-2} \oplus \alpha_3 k_{2m-3} \oplus \dots \oplus \alpha_m k_m;$$

Stream Cipher Cryptosystem

Synchronous Stream Cipher

Example. To break stream cryptosystem shown on previous example first of all the pair of 8 bit plaintext-ciphertext have to be obtained. Let it be $M=10101111$ and $C=11010101$. Then, the modulo two summation of this M and C will result as a key stream K .

$$\begin{array}{rcccccccc}
 M & = & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\
 C & = & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\
 K & = & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\
 & & k_1 & k_2 & k_3 & k_4 & k_5 & k_6 & k_7 & k_8
 \end{array}
 \Rightarrow
 \begin{array}{l}
 k_5 = \alpha_1 k_4 \oplus \alpha_2 k_3 \oplus \alpha_3 k_2 \oplus \alpha_4 k_1; \\
 k_6 = \alpha_1 k_5 \oplus \alpha_2 k_4 \oplus \alpha_3 k_3 \oplus \alpha_4 k_2; \\
 k_7 = \alpha_1 k_6 \oplus \alpha_2 k_5 \oplus \alpha_3 k_4 \oplus \alpha_4 k_3; \\
 k_8 = \alpha_1 k_7 \oplus \alpha_2 k_6 \oplus \alpha_3 k_5 \oplus \alpha_4 k_4;
 \end{array}$$

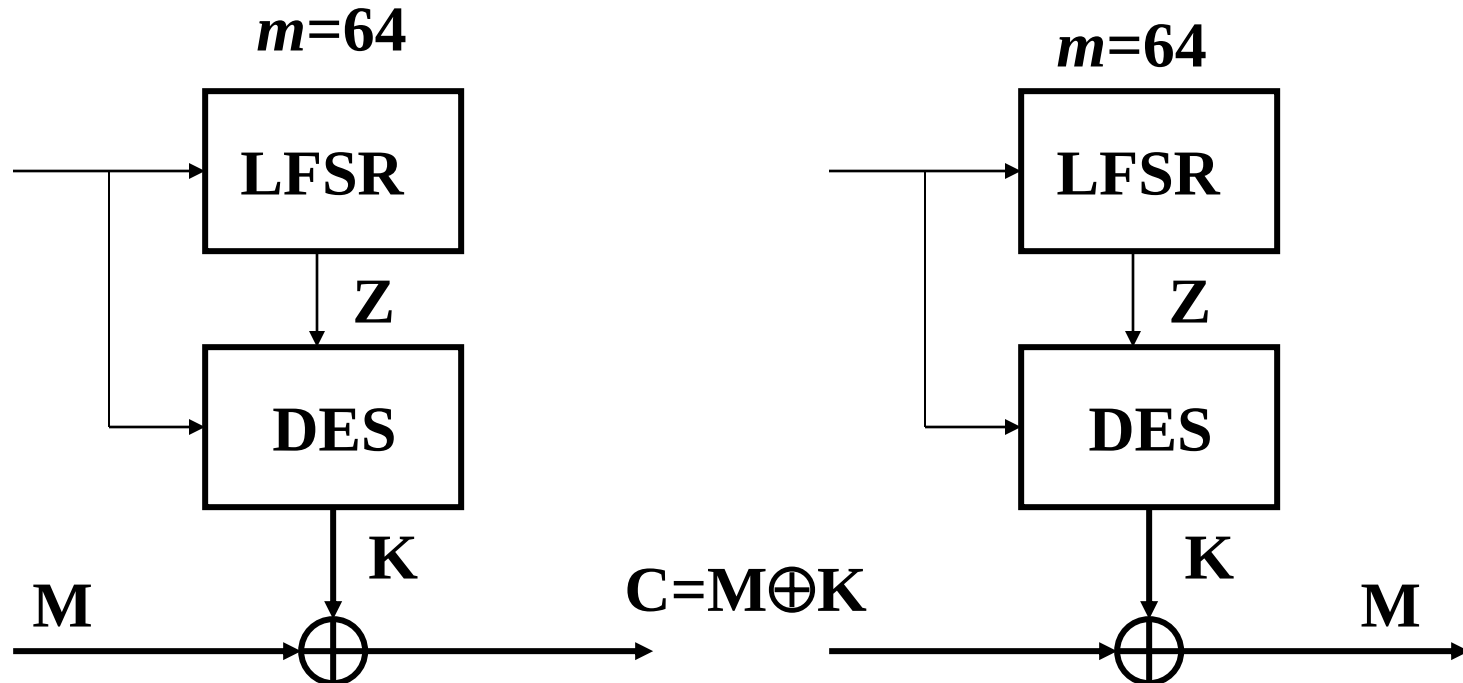
$$\begin{array}{l}
 1 = \alpha_1 \oplus \alpha_2 \oplus \alpha_3; \\
 0 = \alpha_1 \oplus \alpha_2 \oplus \alpha_3 \oplus \alpha_4; \\
 1 = \alpha_2 \oplus \alpha_3 \oplus \alpha_4; \\
 0 = \alpha_1 \oplus \alpha_3 \oplus \alpha_4;
 \end{array}$$

$$\alpha_4 = 1; \alpha_1 = 1; \alpha_2 = 0; \alpha_3 = 0;$$

Stream Cipher Cryptosystem

Synchronous Stream Cipher

To increase efficiency of the cryptosystem the combined approach can be used. As an example see below the combination of stream cipher and DES algorithm.

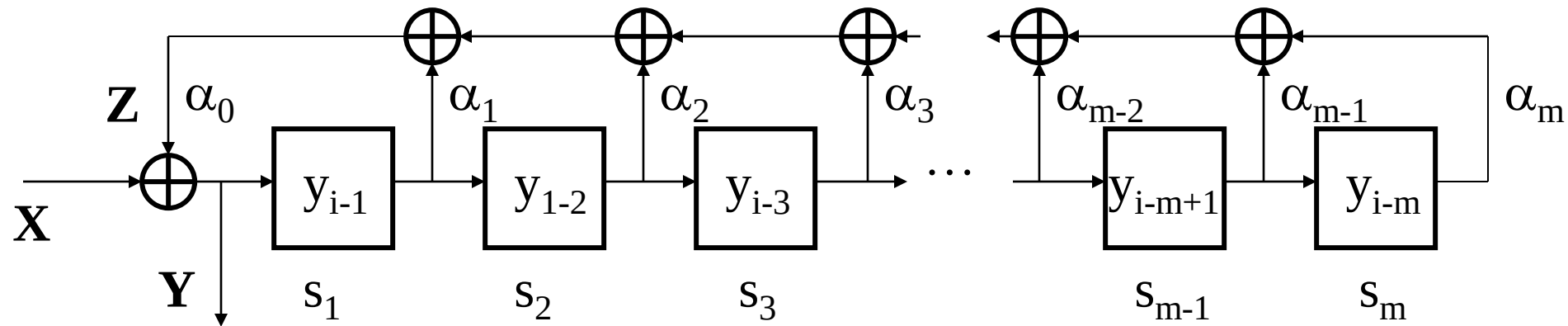


Stream Cipher Cryptosystem

Self-Synchronizing Stream Cipher

Self-synchronizing cipher systems are another class of stream cipher and are categorized as ***ciphertext autokey cipher*** and ***plaintext autokey cipher***.

Ciphertext Autokey Cipher



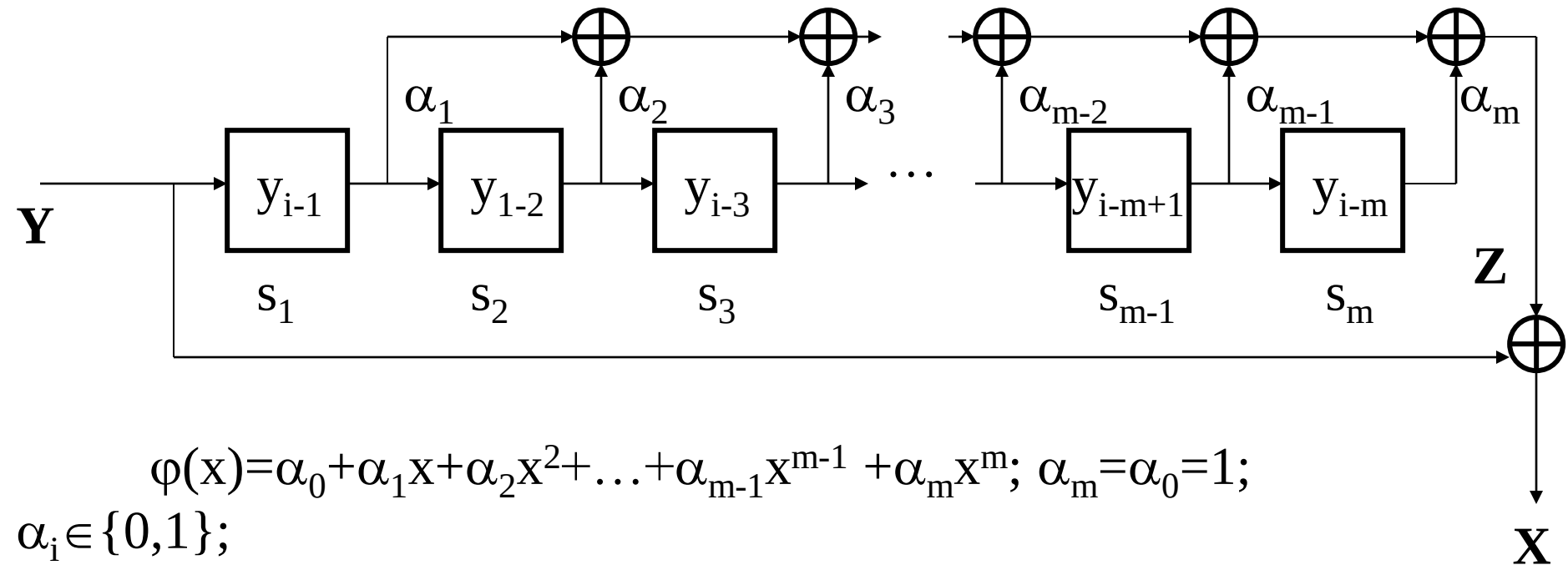
$\varphi(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \dots + \alpha_{m-1} x^{m-1} + \alpha_m x^m$; $\alpha_m = \alpha_0 = 1$; $\alpha_i \in \{0, 1\}$; Z -key stream; X plaintext; Y -ciphertext; S -seed vector.

$$y_i = \begin{cases} x_i + \sum_{j=1}^i \alpha_j y_{i-j} + \sum_{j=i+1}^m \alpha_j s_{j-i}; & 0 \leq i \leq m-1; \\ x_i + \sum_{j=1}^m \alpha_j y_{i-j}; & i \geq m. \end{cases}$$

6. Stream Cipher Cryptosystem

6.4. Self-Synchronizing Stream Cipher

Ciphertext Autokey Decipher



$$\varphi(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \dots + \alpha_{m-1} x^{m-1} + \alpha_m x^m; \quad \alpha_m = \alpha_0 = 1;$$

$$\alpha_i \in \{0, 1\};$$

Z-key stream; X plaintext; Y-ciphertext; S-seed vector.

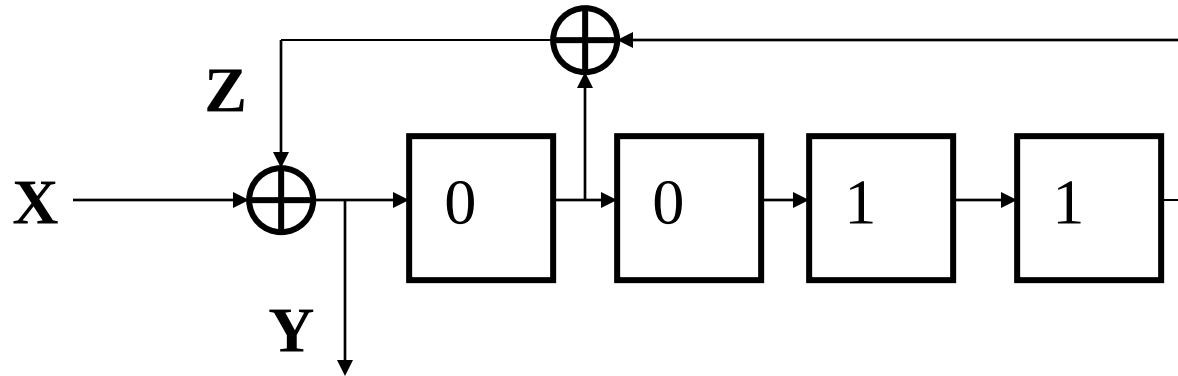
$$X_i = \begin{cases} y_i + \sum_{j=1}^i \alpha_j y_{i-j} + \sum_{j=i+1}^m \alpha_j s_{j-i}; & 0 \leq i \leq m-1; \\ y_i + \sum_{j=1}^m \alpha_j y_{i-j}; & i \geq m. \end{cases}$$

Stream Cipher Cryptosystem

Self-Synchronizing Stream Cipher

Example. $\phi(x)=1+x+x^4$; $X=11101001$; $S=(s_1s_2s_3s_4)=0011$.

Ciphertext Autokey Cipher



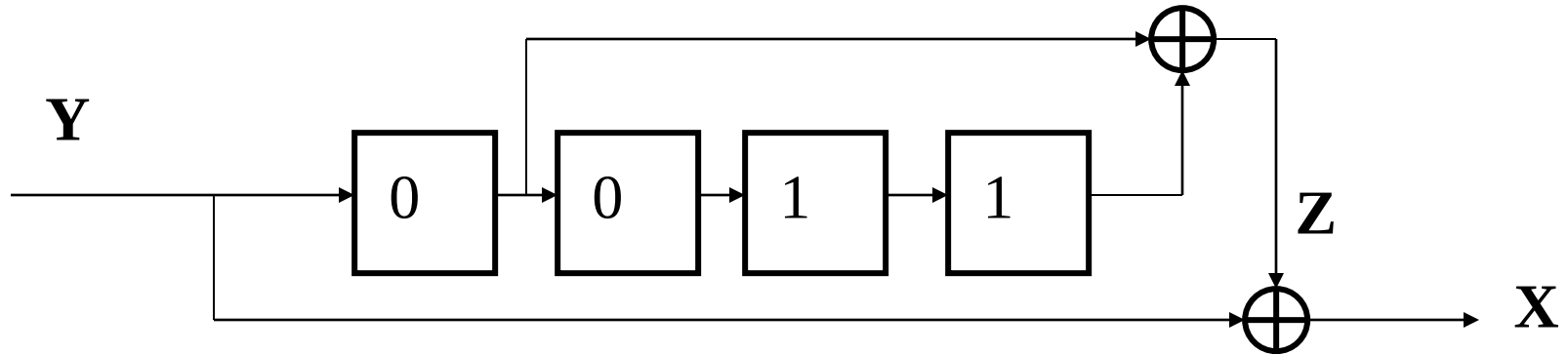
1	0	0	0	0	1
1	0	0	0	0	0
1	1	1	0	0	0
0	1	1	1	0	0
1	0	0	1	1	0
0	0	0	0	1	1
0	1	1	0	0	1
1	1	1	1	0	0

6. Stream Cipher Cryptosystem

6.4. Self-Synchronizing Stream Cipher

Example 6.5. Continuation. $\phi(x)=1+x+x^4$; $Y=00110011$; $S=(s_1s_2s_3s_4)=0011$.

Ciphertext Autokey Decipher

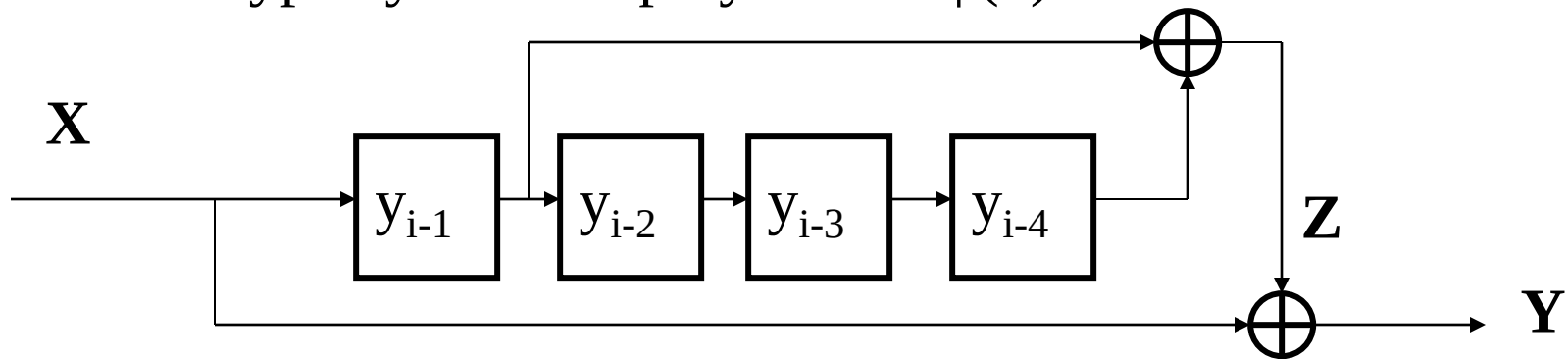


0	0	0	1	1	1
0	0	0	0	1	1
1	0	0	0	0	1
1	1	0	0	0	0
0	1	1	0	0	1
0	0	1	1	0	0
1	0	0	1	1	0
1	1	0	0	1	1

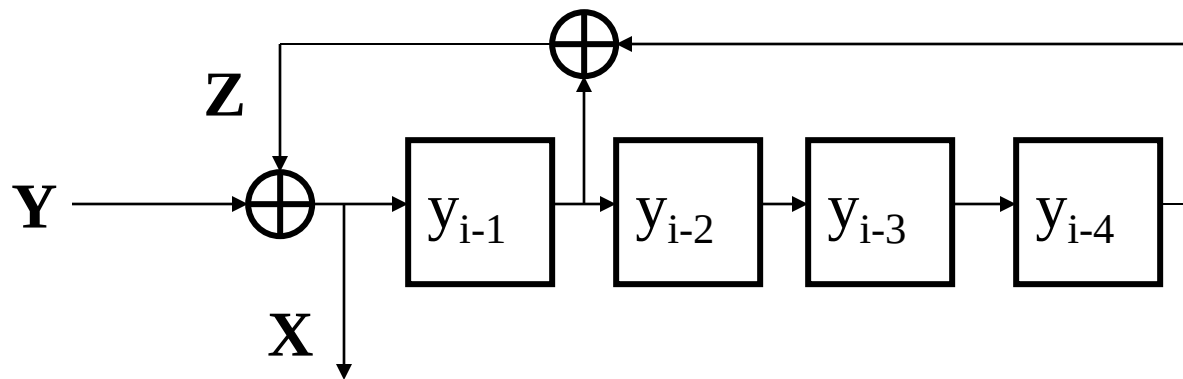
6. Stream Cipher Cryptosystem

6.4. Self-Synchronizing Stream Cipher

Plaintext autokey cipher are often called self-synchronizing cipher with plaintext input to the shift register. Let consider an example of such a type of the cryptosystem for polynomial $\phi(x)=1+x+x^4$



A self-synchronizing encipherer with plaintext excitation.



A autokey decipherer system with plaintext feedback.

Stream Cipher

RC4

Description of RC4

This stream cipher was invented in 1987 by Ron Rivest, one of the inventors of the RSA public key cryptography algorithm.

Even though the RC4 cipher is officially named "Rivest Cipher 4", it is also known as "Ron's Code 4" (RC2, RC5 and RC6 also exist).

RC4 had a really large success thanks to its simplicity and efficiency. It was used in many popular standards and protocols such as WEP, WPA, SSL or TLS.

The RC4 is used with a variable length cryptographic **key** (session key), typically between 40 and 256 bits

Stream Cipher

RC4

Principle

The RC4 algorithm generates a pseudo-random key stream that is then used to generate the ciphertext (by XORing it with the plaintext). It is called pseudo-random because it generates a sequence of numbers that only approximates the properties of random numbers. The sequence of bytes generated is not random since the output is always the same for a given input but it has to approximate random properties to make it harder to crack. The key stream is generated from a variable length key using an internal state composed of the following elements:

1. A 256 bytes array (denoted ***S***) containing a permutation of these 256 bytes
2. Two indexes ***i*** and ***j***, used to point elements in the ***S*** array.

Once the ***S*** array has been initialized and "shuffled" with the key-scheduling algorithm (***KSA***), it is used and modified in the pseudo-random generation algorithm (***PRGA***) to generate the key stream.

Stream Cipher

RC4

Key-scheduling algorithm (KSA)

The first step of this algorithm consist in initializing the **S** table with the identity permutation: the values in the array are equal to their index.

S	0	1	2	...	254	255
	0	1	2	...	254	255

Once the **S** array is initialized, the next step consist in shuffling the array using the **key** to make it a permutation array. To do so, we simply iterate 256 times the following actions after initializing **i** and **j** to 0:

1. Compute $\mathbf{j} = [\mathbf{j} + \mathbf{S}[\mathbf{i}] + \mathbf{key}[\mathbf{i} \bmod \mathbf{keylength}]] \bmod 256$
2. Swap **S**[**i**] and **S**[**j**]
3. Increment **i**

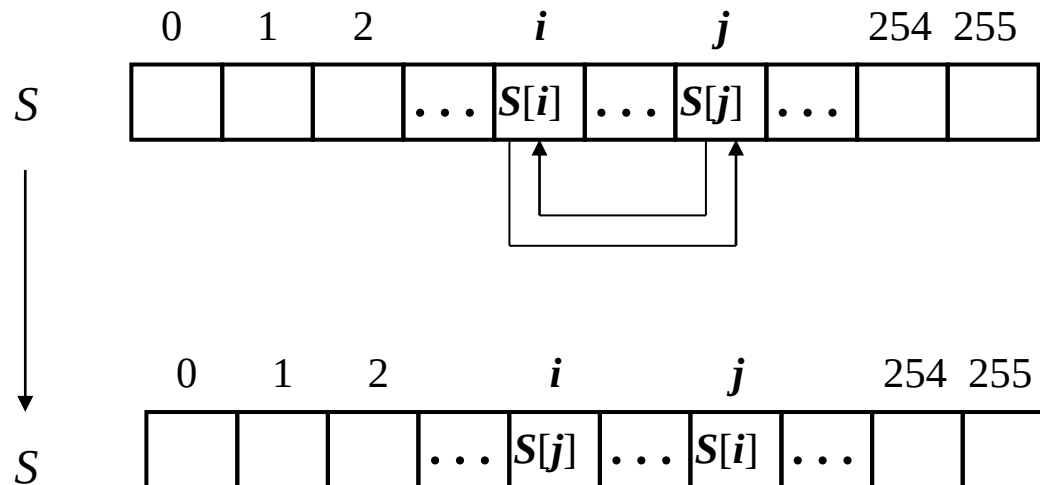
Stream Cipher

RC4

Key-scheduling algorithm (KSA)

Step n

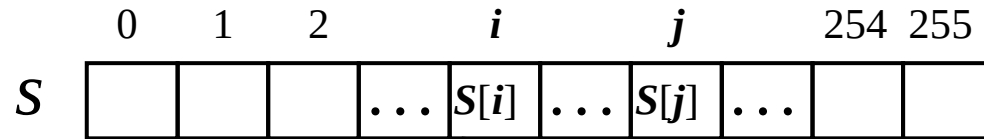
i and j already computed



Stream Cipher

RC4

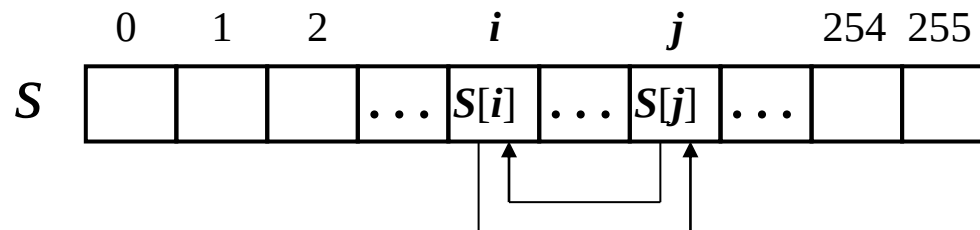
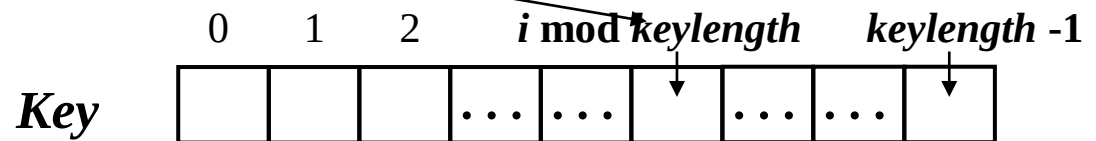
Key-scheduling algorithm (KSA)



Step $n+1$

$i = i + 1$

$j = [j + S[i] + \text{key}[i \bmod \text{keylength}]] \bmod 256$



Stream Cipher

RC4

Key-scheduling algorithm (KSA)

Here is some pseudo-code corresponding to the key-scheduling algorithm (KSA):

```
for i from 0 to 255  
     $S[i] := i$   
end for  
 $j := 0$   
for i from 0 to 255  
     $j := [j + S[i] + \text{key}[i \bmod \text{keylength}]] \bmod 256$   
    swap values of  $S[i]$  and  $S[j]$   
end for
```


Stream Cipher

RC4

The pseudo-random generation algorithm (PRGA)

This step of the algorithm consists in generating a key stream of the size of the message to encrypt. This algorithm enables us to generate a key stream of any size. To do so, we first initialize the two indexes (i, j) to 0 and we then start the generation of the key stream one byte at a time until we reached the size of the message to encrypt. For each new byte to compute we do the following actions:

1. Compute new value of i and j :

$$i := (i + 1) \bmod 256$$

$$j := (j + S[i]) \bmod 256$$

2. Swap $S[i]$ and $S[j]$ to have a dynamic state (it makes it obviously harder to crack than if the state was computed only once and use for the generation of the whole key stream)

3. Retrieve the next byte of the keystream from the S array at the index $(S[i] + S[j]) \bmod 256$

Stream Cipher

RC4

The pseudo-random generation algorithm (PRGA)

Here is some pseudo-code corresponding to the pseudo-random generation

algorithm:

$i := 0$

$j := 0$

while Generating Output:

$i := (i + 1) \bmod 256$

$j := (j + S[i]) \bmod 256$

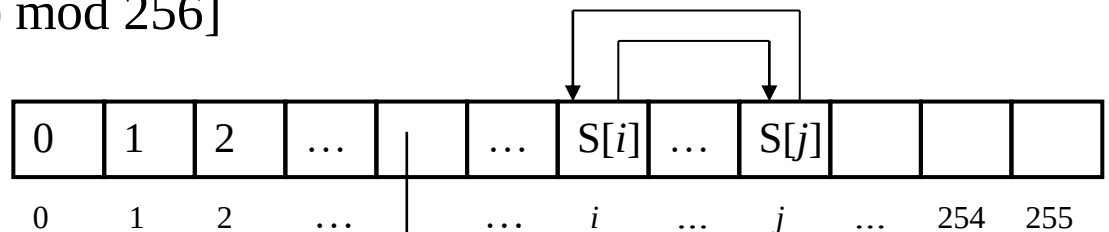
swap values of $S[i]$ and $S[j]$

$K := S[(S[i] + S[j]) \bmod 256]$

Output K

end while

S



Key stream byte
 $(S[i] + S[j]) \bmod 256$

Stream Cipher

RC4

The pseudo-random generation algorithm (PRGA)

This step of the algorithm consists in generating a key stream of the size of the message to encrypt. This algorithm enables us to generate a key stream of any size. To do so, we first initialize the two indexes (i, j) to 0 and we then start the generation of the key stream one byte at a time until we reached the size of the message to encrypt. For each new byte to compute we do the following actions:

1. Compute new value of i and j :

$$i := (i + 1) \bmod 256$$

$$j := (j + S[i]) \bmod 256$$

2. Swap $S[i]$ and $S[j]$ to have a dynamic state (it makes it obviously harder to crack than if the state was computed only once and use for the generation of the whole key stream)

3. Retrieve the next byte of the keystream from the S array at the index $(S[i] + S[j]) \bmod 256$

Stream Cipher

RC4

The pseudo-random generation algorithm (PRGA)

Here is some pseudo-code corresponding to the pseudo-random generation

algorithm:

$i := 0$

$j := 0$

while Generating Output:

$i := (i + 1) \bmod 256$

$j := (j + S[i]) \bmod 256$

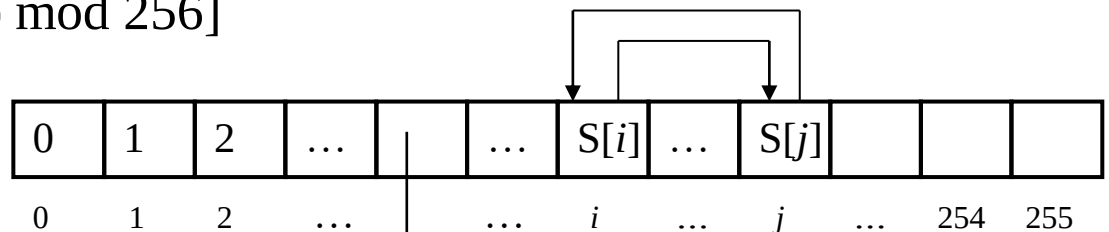
swap values of $S[i]$ and $S[j]$

$K := S[(S[i] + S[j]) \bmod 256]$

Output K

end while

S



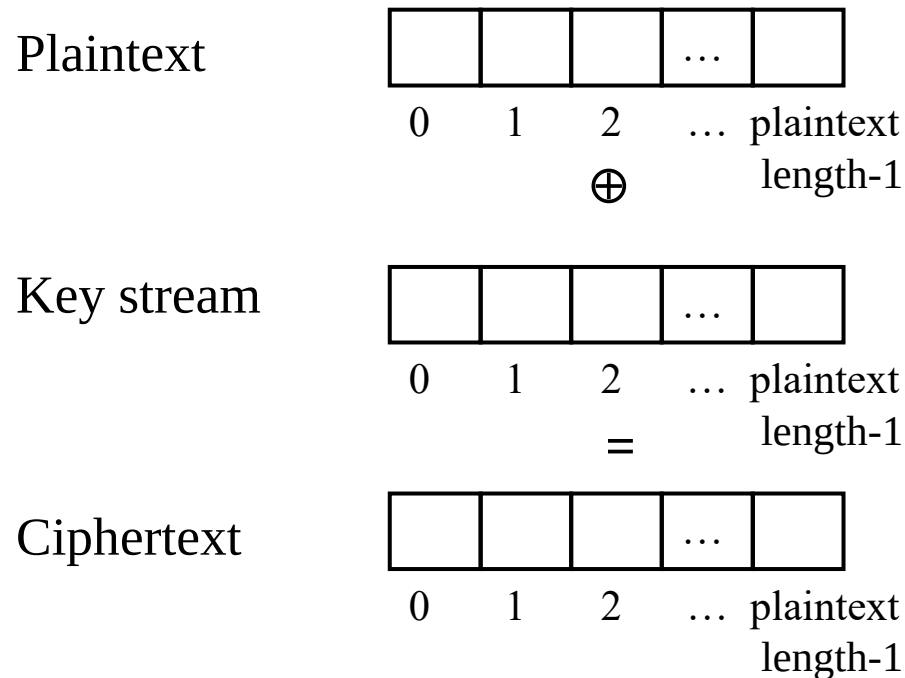
Key stream byte
 $(S[i] + S[j]) \bmod 256$

Stream Cipher

RC4

Encryption and decryption

Once the key stream has been generated, the encryption of the plaintext is really simple: it simply consists of a XOR between the plaintext and the key stream. See below an illustration of the encryption:



Теория Информации

В.Н. Ярмолик

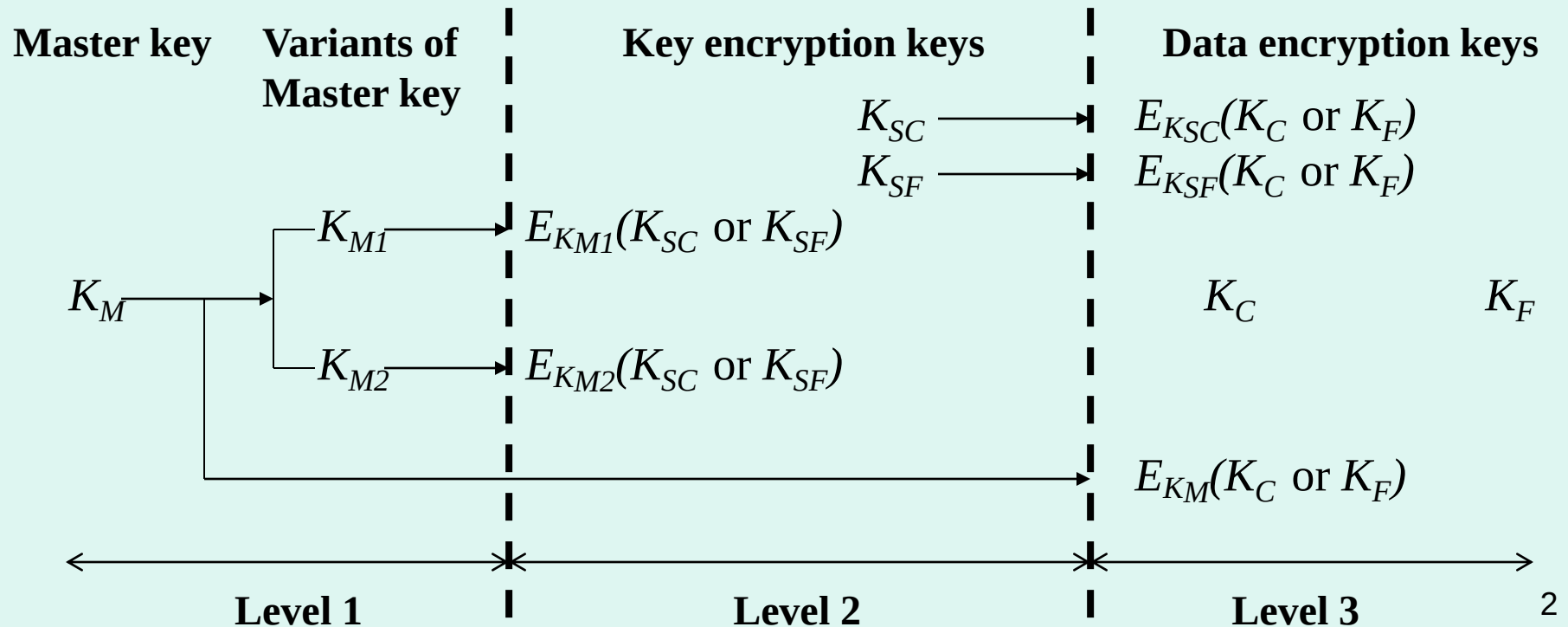
Лекция 13

2017

Key Management

Key Hierarchy

There are key layers at each host, i.e. one **master key** and two or more of its variants at the highest level, **key encryption keys** at the second level, and **data encryption keys** at third as shown in following diagram. At highest level (Level 1) of key organization which creates one master key K_M and two variants K_{M1} and K_{M2} . The second level comprises secondary communication keys K_{SC} and secondary file keys K_{SF} , which are stored as cryptograms obtained by applying K_{M1} and K_{M2} . The third level comprises data encryption keys K_C or K_F which are protected by applying the key encryption keys or the host master key.



Key Management

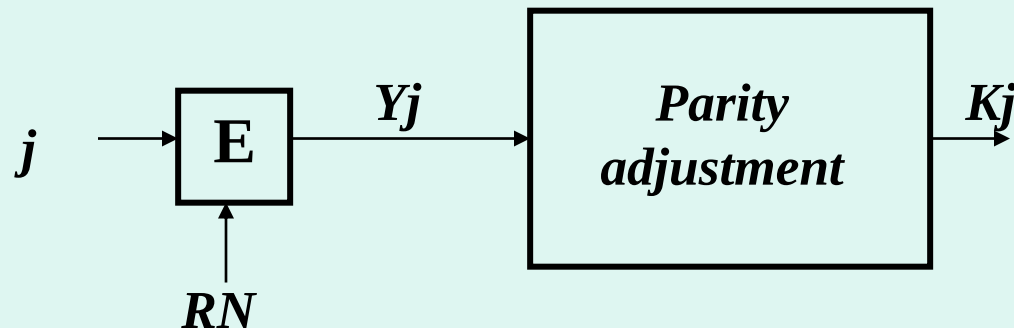
Key Generation

Master key is generated by a random process such as tossing coins or throwing dice. It is desirable that master key remains unchanged for a relatively long time. Since this key protects all other keys stored at the host processor, special care should be taken to ensure that it is generated and installed in the cryptographic facilities in a secure manner.

The master key is used to encrypt the key encryption keys when they are required to be stored outside a secure cryptographic facility, such as disk.

Key generation procedure for key encryption keys based on DES

Let RN be a 64-bit random number produced by *DES*.

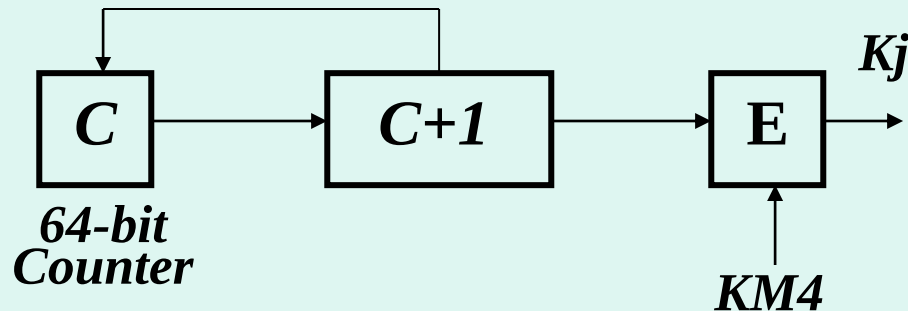


j -index number; E -encryption algorithm; K_j – the i th cryptographic key.

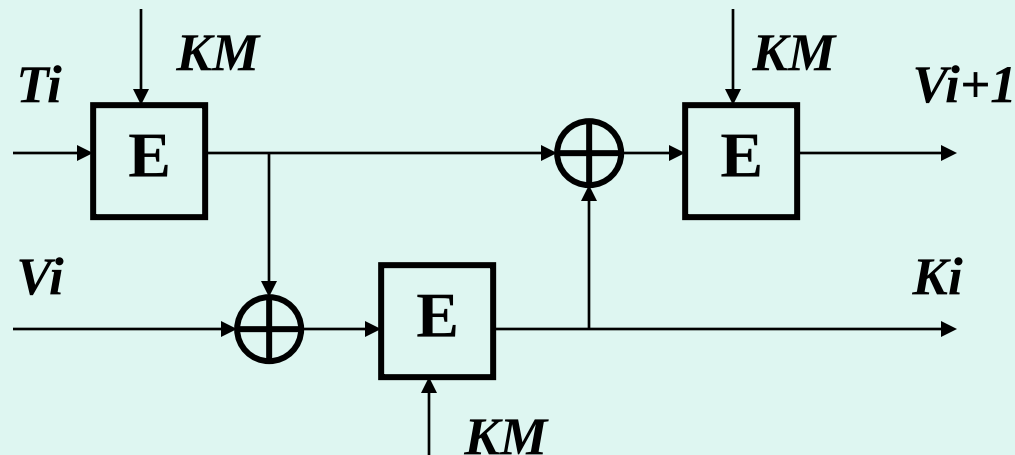
Key Management

Key Generation

One way to generate RNi which represent enciphered data encryption keys is using a nonresettable counter that can be incremented only by the cryptographic facility (CF). Since the counter is accessed only by the CF, an opponent cannot reset its values and therefore the prior data encryption keys cannot be regenerated.



The new value of the key Kj is generated by incrementing the counter and enciphering the result value $C+1$ with the special variant of the host master key $KM4$



ANSI X 9.17

E - DES Algorithm;

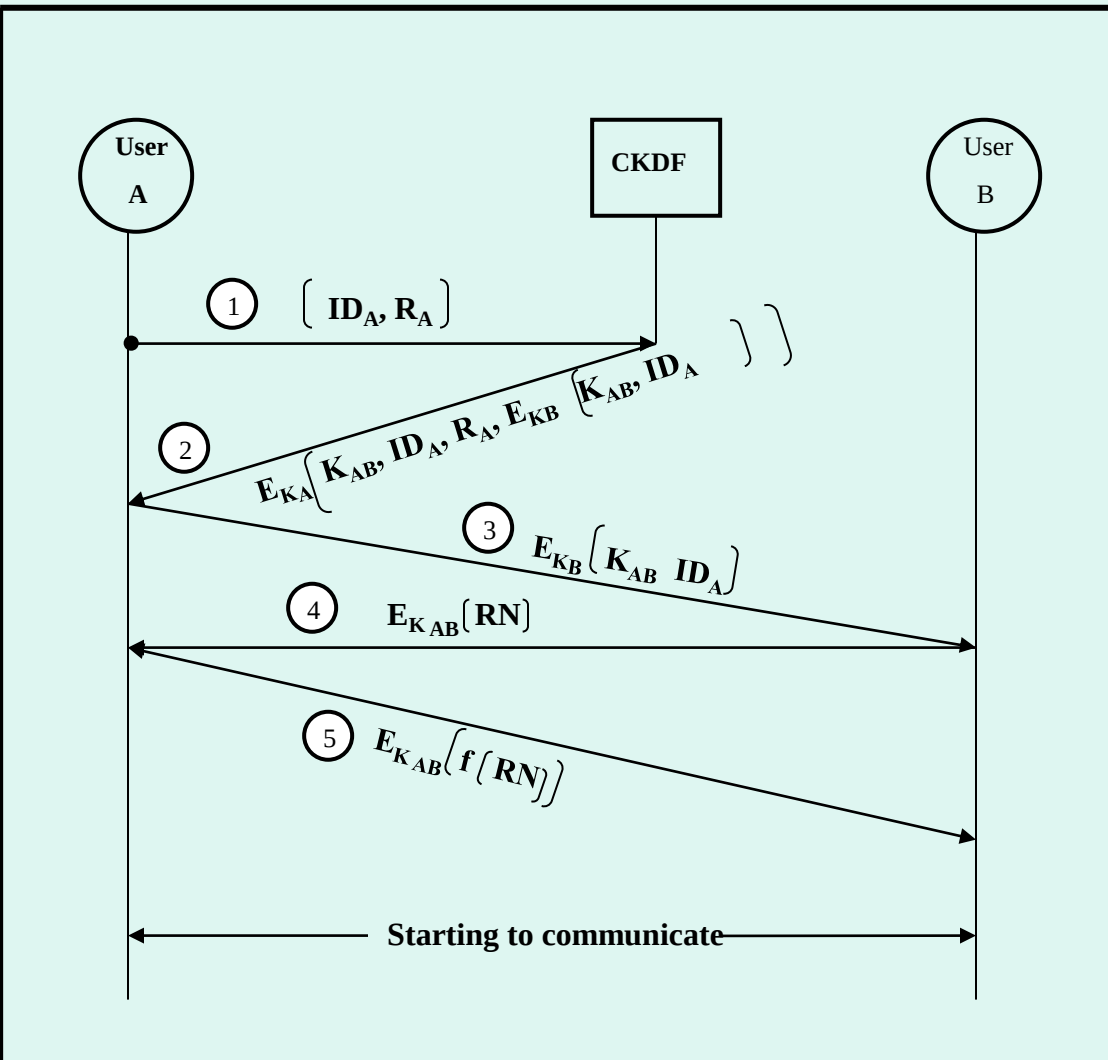
$V0$ - Secret 64-bit initial number

K - master key;

T - time stamp.

Key Management

Key Distribution



Key Distribution protocol for Symmetric Algorithms

This Protocol can be apply for the group of n users to eliminate the problem of generating and managing of

$$\binom{n}{2} = n(n-1)/2$$

possible key pairs which would be enormous.

To implement this protocol a Centralized Key Distribution Facility (CKDF) have to be used to organize the communication in between user A and B. All users have to have the keys for communication with CKDF. User A the key K_A and user B - K_B

Key Management

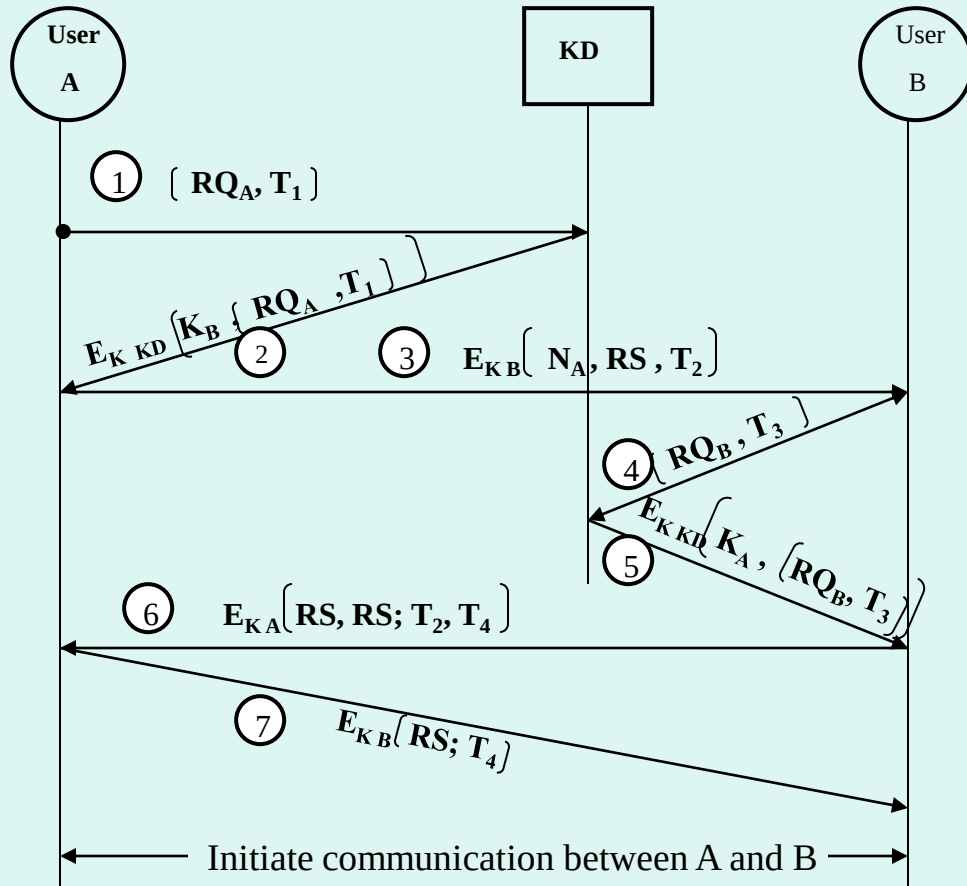
Key Distribution

Key Distribution protocol for Symmetric Algorithms

1. User A sends a suitable request R_A along with A's identity ID_A to CKDF in clear form.
2. CKDF sends to user A the following cryptogram: $E_{K_A}(K_{AB}, ID_A, R_A, E_{K_B}(K_{AB}, ID_A))$ where K_{AB} is a cross-domain key protected transmission between users A and B; K_A and K_B are the secondary communication keys between A and CKDF and between B and CKDF, respectively; and $E_{K_A}(K_{AB}, ID_A)$ is a cryptogram of K_{AB}, ID_A obtained under B's secret key K_s .
3. Upon receiving the cryptogram, A decipher it and checks whether the identifier ID_A and the request R_A are identical to the original. If so, A forwards the cryptogram $E_{K_B}(K_{AB}, ID_A)$ to the user B.
4. B deciphers the received cryptogram under the key K_B and recovers the clear form of both K_{AB} and ID_A . As a result, users A and B can have the same key K_{AB} , he initiates an authentication procedure in order to eliminate any possibility of false cryptogram. Thus, B directs the cryptogram $E_{K_{AB}}(RN)$, i.e. encryption of a random number RN with K_{AB} , to A.
5. A deciphers the received cryptogram to obtain RN and modified it in some predetermined way to get $f(RN) = (RN)'$ which depends on the time. The sequence $f(RN)$ is encrypted under K_{AB} and returns $E_{K_{AB}}((RN)')$ to user B.
6. If A confirms receipt of the legitimate RN and the use of K_{AB} , with the handshake, A can begin transmitting the message to B.

Key Management

Key Distribution



Key Distribution protocol for Asymmetric Algorithms

In asymmetric cryptosystems, a public key can be sent via an insecure channel without endangering the security of a secret key. But any user receiving of key should check whether the key is authentic.

Using the time-stamping technique, let us design a key distribution protocol between two users A and B belonging to the same key directory. Let K_A be the public key of user A and K_B that for user B. They are registered in the Key Directory (KD). Assume also that public key K_{KD} of the KD is known to all its users. The protocol for establishing a key distribution between A and B is as follows:

7. Key Management

7.3. Key Distribution

1. User A initiates the procedure to obtain the public key of B , K_B , by sending the request RQ_A , along with the current time T_1 in clear form to the KD .
2. The KD then finds K_B on the basis of the request RQ_A and transmits K_B , along with a copy (RQ_A, T_1) to A in the form of the cryptogram $E_{K_{KD}}[K_B, (RQ_A, T_1)]$, enciphered under a public key of the KD .
3. Knowing K_{KD} A can decipher the cryptogram and extract an authentic public key K_B , a copy of the request RQ_A and the time T_1 . The pair (RQ_A, T_1) is then compared with the original. Now, having the public key K_B , A sends B a cryptogram where N_A is A 's name, RS is a random sequence chosen by A , and T_2 is the current time.
4. Upon receiving the cryptogram, B recreates the clear form of messages using the secret key K_S . Since B knows the name N_A , B can ask for A 's authentic public key from the KD by sending the appropriate request RQ_B and the current time T_3 .
5. In response, the KD transmits to B the cryptogram $E_{K_{KD}}[K_A, (RQ_B, T_3)]$, enciphered under the KD 's public key.
6. After deciphering the cryptogram, B obtains the authentic public key of A , but the pair (RQ_B, T_3) must be compared with the original. Next, B creates the cryptogram $E_{K_A}(RS, RS', T_2, T_4)$ enciphered under A 's public key, where the random sequence RS is obtained from the user A , a random sequence RS' is self-generated, and T_4 is the current time. Subsequently, this cryptogram is sent to A .
7. A decrypts the cryptogram and compares (RS, T_2) with the original. If these sequences are the same, A is sure that user B is authentic. Finally, A retransmits the random sequence RS' in the form of the cryptogram $E_{K_B}(RS', T_4)$ to B .
8. B compare the original RS' to confirm the authenticity of A . Then start.

Key Management

Key Sharing

Shamir showed how to implement secret sharing of a robust key based on a (k,n) threshold scheme, where k is the minimum number of entities (users) required to construct the cryptographic key K ; and n is the number of total entities. Dividing the cryptographic key K into n pieces (shadows) $K_1, K_2, \dots, K_n$ in such a way that K is easily reconstructible from any k pieces of it. The so-called **(k,n) threshold scheme** has the following features:

1. Knowledge of any k or more K_i pieces makes K easily computable.
2. Knowledge of any $k-1$ or fewer K_i pieces leaves K completely undetermined.

Threshold schemes are ideally suited to applications in which a group of mutually suspicious individuals with conflicting interests must cooperate.

According to the Shamir's threshold schemes the shadows are derived from a random polynomial of degree $k-1$:

$$h(x) = (a_{k-1}x^{k-1} + a_{k-2}x^{k-2} + a_1x^1 + a_0) \bmod P$$

with constant coefficients a_i where $a_0 = K$.

Arithmetic is done in the Galois field $GF(P)$, where P is a prime number larger than both K and n . For a given $h(x)$, the key K is easily computed by

$$K = a_0 = h(0)$$

The n shadows are computed by evaluating $h(x)$ at n distinct values $x_1, x_2, \dots, x_n$. $K_i = h(x_i)$, $i=1, 2, \dots, n$. The values $x_1, x_2, \dots, x_n$ need not be secret, and could be user identifiers or the number 1 through to n . Because k uniquely determine a polynomial of degree $k-1$, $h(x)$ and, therefore, K can be reconstructed from k shadows. There is not enough information, however, to determine $h(x)$ or K from fewer than k shadows.

Key Management

Key Sharing

Example Let $(k,n)=(3,5)$ and $P=17$, as well as the key $K=13$. Polynomial with the random coefficients is

$$h(x) = (2x^2 + 10x + 13) \bmod 17$$

Evaluating $h(x)$ at $x_i=1,2,3,4,5$ we get five shadows:

$$K_1 = h(1) = (2 + 10 + 13) \bmod 17 = 8;$$

$$K_2 = h(2) = (8 + 20 + 13) \bmod 17 = 7;$$

$$K_3 = h(3) = (18 + 30 + 13) \bmod 17 = 10;$$

$$K_4 = h(4) = (32 + 40 + 13) \bmod 17 = 0;$$

$$K_5 = h(5) = (50 + 50 + 13) \bmod 17 = 11;$$

We can reconstruct $h(x)$ from any three of the shadows. Using K_1, K_4 , and K_5 we have

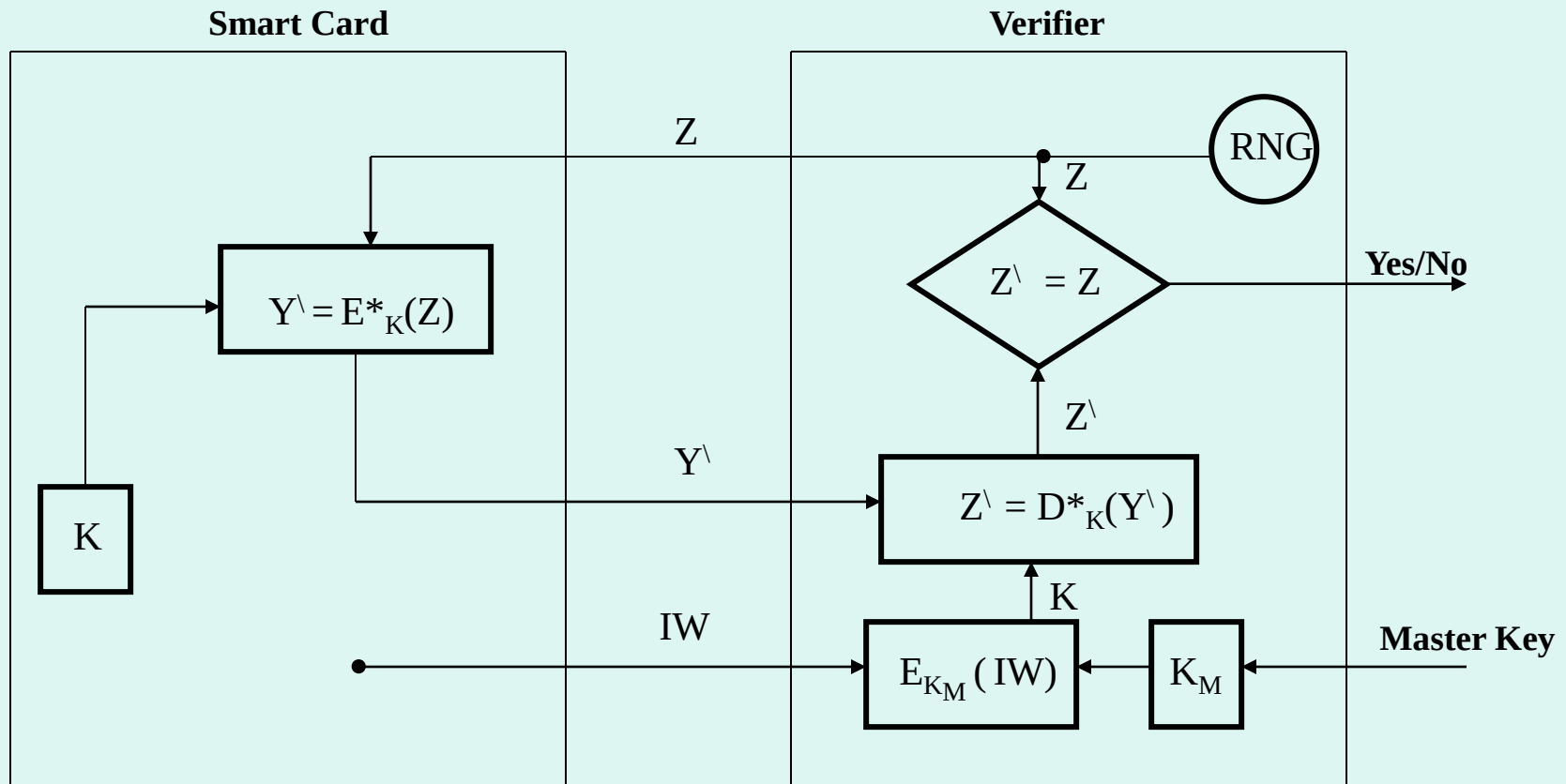
$$\begin{array}{l} 8 = \alpha_2 + \alpha_1 + K \bmod 17; \\ 0 = 16\alpha_2 + 4\alpha_1 + K \bmod 17; \\ 11 = 25\alpha_2 + 5\alpha_1 + K \bmod 17; \end{array} \longrightarrow \begin{array}{l} 9 = 15\alpha_2 + 3\alpha_1 \bmod 17; \\ 3 = 24\alpha_2 + 4\alpha_1 \bmod 17; \end{array} \longrightarrow \begin{array}{l} 36 = 60\alpha_2 + 12\alpha_1 \bmod 17; \\ 9 = 72\alpha_2 + 12\alpha_1 \bmod 17; \end{array}$$

$$\begin{array}{l} 2 = 9\alpha_2 + 12\alpha_1 \bmod 17; \\ 9 = 4\alpha_2 + 12\alpha_1 \bmod 17; \end{array} \longrightarrow 10 = 5\alpha_2 \bmod 17; \longrightarrow 5\alpha_2 = 10 \bmod 17;$$

Key Management

Key Authentication

Let us consider the Authentication problem based on the Smart Card Authentication Scheme. Priory the Smart Card should get the secret key K , to be written into protected memory on Card, along with the Identification Ward (IW). The key K is generated based on cryptographic algorithm E , according to the relation $K = E_{K_M}(IW)$, where K_M is a master key.



Key Management

Key Authentication

Smart Card (SC) Authentication Algorithm by Verifier (V).

1. Pseudorandom number (PRN) Z is generated by Verifier.
2. The number Z is transmitted to the SC.
3. According the encryption algorithm E and key K smart card generates the cryptogram $Y' = E_K(Z)$.
4. Cryptogram Y' and Identification Word IW is sending to verifier.
5. Verifier regenerates the value of the Smart Card key K , based on its IW and their own value of the master key K_M .
6. Then applying deciphering algorithm D and Y' as a cryptogram with the key K verifier generates the number Z' .
7. Verifier compares two numbers Z and Z' . In a case of equality it makes the conclusion that this Card is a valid card.

Теория Информации

В.Н. Ярмолик

Лекция 14

2017

Elliptic curve cryptography

Elliptic curve

Elliptic curve does not mean ellipse. They are just only the curve described by the cubic equations, look like the equations have been used for ellipses description.

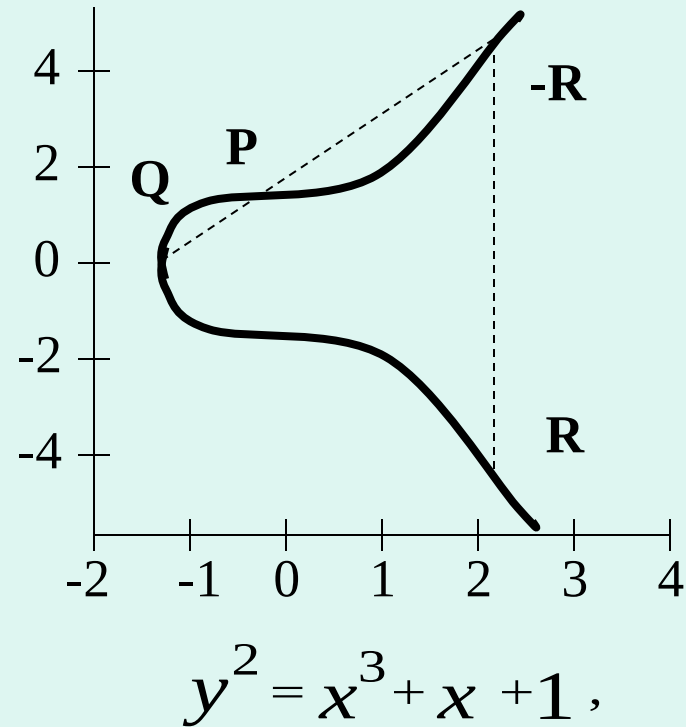
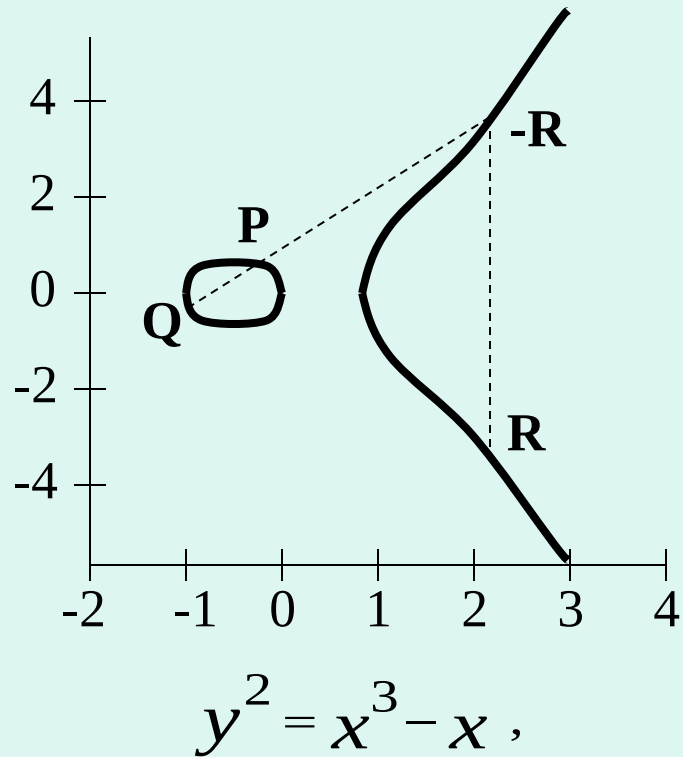
For the general case the cubic relation has the form

$$y^2 + a x y + b y = x^3 + c x^2 + d x + e ,$$

where a, b, c, d and e are real numbers, which have to satisfy some simple requirements. The definitions of the elliptic curve includes as well some element which O named as infinite element or zero element. There are two examples of such a type of curve are shown in the following Figures.

Elliptic curve cryptography

Elliptic curve



Elliptic curve cryptography

Elliptic curve

According to the definition of Elliptic curve there is the following property: If three points of elliptic curve are on the line, the their sum equals to O .

This property allow to define the rules for addition over the points on the elliptic curve.

1. The object O is a zero additive element that there is an equality $O = -O$ and for any P on elliptic curve we have $P + O = P$.
2. The line through P and $-P$ is vertical line which does not intersect the elliptic curve at a third point; thus the points P and $-P$ cannot be added as previously. It is for reason that the elliptic curve group includes the point at infinity O . By definition, $P + (-P) = O$ in the elliptic curve group.
3. Adding distinct points P and Q .

Suppose that P and Q are two distinct points on an elliptic curve, and the P is not $-Q$. To add the points P and Q , a line is drawn through the two points. This line will intersect the elliptic curve in exactly one more point, call $-R$. The point $-R$ is reflected in the x -axis to the point R . The law for addition in an elliptic curve group is $P + Q = R$.

Elliptic curve cryptography

Elliptic curve

4. To add a point P to itself, a tangent line to the curve is drawn at the point P . If y_P is not 0, then the tangent line intersects the elliptic curve at exactly one other point R . This operation is called doubling the point P ; the law for doubling a point on an elliptic curve group is defined by: $P+P=2P=R$.

5. Multiplication of the point P by positive integer k is defined as a sum of k copies of P , so $kP=P+P+P+\dots+P$.

An elliptic curve over real numbers may be defined as the set of points (x,y) which satisfy an elliptic curve equation of the form:

$$y^2 = x^3 + ax + b,$$

where x , y , a and b are real numbers. For a and b the following relation should be true $4a^3 + 27b^2 \neq 0$

Each choice of the numbers a and b yields a different elliptic curve. For example $a=-4$ and $b=0,67$ gives the elliptic curve with equation

$$y^2 = x^3 - 4x + 0,67$$

Elliptic curve cryptography

Elliptic curve

1. If two points $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$ are different and $P \neq -Q$, then $P+Q=R$, where $R=(x_R, y_R)$, and

$$\begin{aligned}s &= (y_P - y_Q) / (x_P - x_Q); \\ x_R &= s^2 - x_P - x_Q; \\ y_R &= -y_P + s(x_P - x_R).\end{aligned}$$

2. Doubling the point P gives the result $2P = R$

$$\begin{aligned}s &= (3x_P^2 + a) / (2y_P); \\ x_R &= s^2 - 2x_P; \\ y_R &= -y_P + s(x_P - x_R).\end{aligned}$$

Value of a , have been used in previous equation is one of the parameter of elliptic curve $y^2 = x^3 + ax + b$.

Elliptic curve cryptography

Elliptic groups

If two positive integer numbers a and b satisfies to the following relation

$$4a^3 + 27b^2 \not\equiv 0 \pmod{M},$$

where M is a prime number and $a < M$ and $b < M$, then the elliptic curve can be used to form of the ***elliptic group***.

The elliptic group $E_M(a,b)$ consists of the set of pairs (x,y) of the positive integer numbers less than M , which satisfies to the relation:

$$y^2 = x^3 + ax + b \pmod{M},$$

Integer positive pairs of two numbers from $(0,0)$ till $(M-1,M-1)$, which satisfies to the following relation

$$y^2 = x^3 + ax + b \pmod{M},$$

can be used as elliptic group elements (***points***).

Elliptic curve cryptography

Elliptic groups

The rule to generate all elements of the elliptic group consists of the next steps

1. For all values of x , $0 \leq x < M$, the value of $x^3 + ax + b \pmod{M}$ should be calculated.

2. For each value from the previous step the square root modulo M have to be determined. In a case of the negative answer there are not the element within the elliptic group $E_M(a,b)$. If the root exists the there are two value of y and two elements in a group.

Example For example, let $M=5$ and $y^2 = x^3 + 4 \pmod{5}$, what can be describe as $E_5(0,4)$. In this case $a=0$, $b=4$ and $4 \times 0^3 + 27 \times 4^2 \pmod{5} = 1 \neq 0$, it means that we can construct the elliptic group $E_5(0,4)$.

Let start with $x=0$; $y^2 = 0^3 + 4 \pmod{5} = 4$; $y=+2$ and $y=-2$, Then there are two points $(0,2)$ and $(0,-2) = (0,3)$ within the elliptic group $E_5(0,4)$. Entire set of point are shown below

$$E_5(0,4) = \{O, (0,2), (0,3), (1,0), (3,1), (3,4)\}.$$

Elliptic curve cryptography

Elliptic groups

Example. For example, let $M=23$ and $y^2=x^3+x$. In this case $a=1$, $b=0$ and $4 \times 1^3 + 27 \times 0^2 = 4 \neq 0$, it means that we can construct the elliptic group $E_{23}(1,0)$.

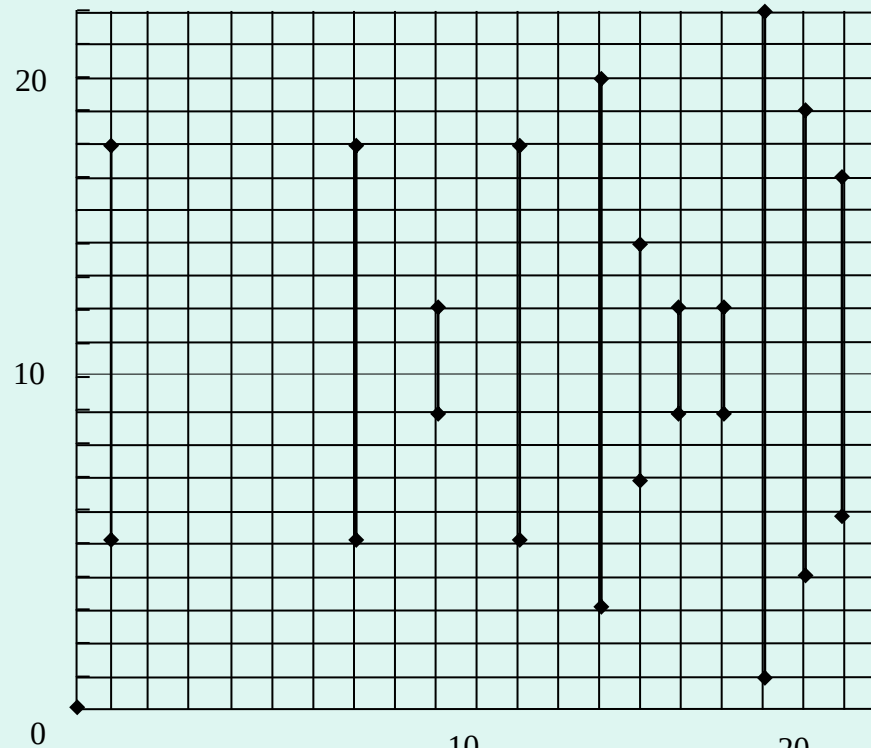
The point $(9,5)$ belongs to this group due to their coordinates satisfies to the elliptic curve equation $y^2=x^3+x \bmod 23$. Really, substituting the values $x=9$ and $y=5$, the next calculation can be done

$$\begin{aligned}5^2 \bmod 23 &= 9^3 + 9 \bmod 23, \\25 \bmod 23 &= 729 + 9 \bmod 23, \\25 \bmod 23 &= 738 \bmod 23, \\2 &= 2.\end{aligned}$$

The elliptic group $E_{23}(1,0)$ has 23 following points
 $\{O (0,0) (1,5) (1,18) (9,5) (9,18) (11,10) (11,13) (13,5) (13,18) (15,3)$
 $(15,20) (16,8) (16,15) (17,10) (17,13) (18,10) (18,13) (19,1) (19,22)$
 $(20,4) (20,19) (21,6) (21,17)\}.$

Elliptic curve cryptography

Elliptic Curve group



Note that there is two points for every x value. Even though the graph seems random, there is still symmetry about $y=11,5$. Recall that elliptic curves over real numbers, there exists a negative point for each point which is reflected through the x -axis. over the finite field, the negative components in the y values are taken modulo 23, resulting in a positive number as a difference from 23. Here $-P=(x_P,(-y_P \bmod 23))$. If $P=(1,5)$, then $-P=(1,(-5 \bmod 23))=(1,18)$.

Elliptic curve cryptography

Elliptic Curve group

The number of points in elliptic group is bounded according to the inequality **Hassego**. Consider the next example of the elliptic curve group.

$$M+1-2M^{1/2} \leq \#E_M(a,b) \leq M+1+2M^{1/2}.$$

Consider the next example of the elliptic curve group.

Example For example, let $M=5$ and $y^2=x^3+1 \pmod{5}$, what can be describe as $E_5(0,1)$. In this case $a=0$, $b=1$ and $4 \times 0^3 + 27 \times 1^2 \pmod{5} = 2 \neq 0$, it means that we can construct the elliptic group $E_5(0,4)$ with the six points.

$$E_5(0,1)=\{O, (0,1), (0,4), (2,2), (2,3), (4,0)\}.$$

$$5 + 1 - 2 \cdot 5^{1/2} = 2 \leq \#E_5(0,1) = 6 \leq 5 + 1 + 2 \cdot 5^{1/2} = 10$$

Elliptic curve cryptography

Arithmetic in an Elliptic Curve group

Elliptic curve group have a finite number of points, which is a desirable property for cryptographic purposes. Since these curves consists of a few discrete points, it is not clear how to “connect the dots” to make their. It is not clear how geometric relationships can be applied. However, the algebraic rules for elliptic curves group.

1. Adding distinct points P and Q .

If P and Q are distinct points such that P is not $-Q$, then $P+Q=R$

where

$$s = (y_P - y_Q) / (x_P - x_Q) \bmod M;$$

$$x_R = s^2 - x_P - x_Q \bmod M;$$

$$y_R = -y_P + s(x_P - x_R) \bmod M;$$

2. Doubling the point P . Provided that y_P is not 0, $2P=R$ where

$$s = (3x_P^2 + a) / (2y_P) \bmod M;$$

$$x_R = s^2 - 2x_P \bmod M;$$

$$y_R = -y_P + s(x_P - x_R) \bmod M;$$

Recall that a is one of the parameters chosen with the elliptic curve.

Elliptic curve cryptography

Arithmetic in an Elliptic Curve group

Example. In the elliptic group defined by

$$y^2 = x^3 + 9x + 17 \pmod{23}$$

For the $P=(16,5)$ determine $2P=P+P$.

$$s = (3x_P^2 + a)/(2y_P) \pmod{M} = (3 \times 16^2 + 9)/(2 \times 5) \pmod{23}; \text{ then } 10s = 18 \pmod{23};$$

$$s = 18 \times 10^{\phi(23)-1} \pmod{23} = 18 \times 10^{21} \pmod{23} = 11.$$

$$x_R = s^2 - 2x_P \pmod{M} = (11^2 - 2 \times 16) \pmod{23} = 20;$$

$$y_R = -y_P + s(x_P - x_R) \pmod{M} = -5 + 11(16 - 20) \pmod{23} = -49 \pmod{23} = -3 \pmod{23} = 20;$$

As the result $2P=(20,20)$.

For two points $P=(16,5)$ and $Q=2P=(20,20)$ determine

$$R=P+Q=3P.$$

$$s = (y_P - y_Q)/(x_Q - x_P) \pmod{M} = (5 - 20)/(20 - 16) \pmod{23}; \text{ then } 4s = -15 \pmod{23} = 8 \pmod{23};$$

$$s = 8 \times 4^{\phi(23)-1} \pmod{23} = 8 \times 4^{21} \pmod{23} = 2.$$

$$x_R = s^2 - x_P - x_Q \pmod{M} = (2^2 - 16 - 20) \pmod{23} = -9 \pmod{23} = 14;$$

$$y_R = -y_P + s(x_P - x_R) \pmod{M} = -5 + 2(14 - 16) \pmod{23} = -9 \pmod{23} = 14;$$

As the result $R=3P=(14,14)$.

Elliptic curve cryptography

Arithmetic in an Elliptic Curve group

Example. Consider the addition of two points $P=(0,1)$ and $Q=(2,2)$, belonging to the elliptic group $E_5(0,1)$. The following conditions are holds true $(0,1) \neq (2,2)$ and $(2,2) \neq -(0,1)$, the first case for addition should be used. Then $s=(y_P-y_Q)/(x_P-x_Q) \bmod M = (1-2)/(0-2) \bmod 5$. As the result $2s=1 \bmod 5$ and $s=3$.

The resulting point $R=(x_R, y_R)$ has the coordinates:

$$x_R = s^2 - x_P - x_Q \bmod M = (3^2 - 0 - 2) \bmod 5 = 2;$$

$$y_R = -y_P + s(x_P - x_R) \bmod M = -1 + 3(0 - 2) \bmod 5 = -7 \bmod 5 = 3.$$

The final result is $R=(x_R, y_R)=(2,3)$.

Example. Consider the doubling of the point $P = (2,2)$, belongs to group $E_5(0,1)$. Due to $(2,2)=(2,2)$, the second case for addition should be chosen. Then $s=(3x_P^2+a)/(2y_P) \bmod M=(3 \times 2^2+0)/(2 \times 2) \bmod 5=3$. The resulting point $2P=R=(x_R, y_R)$ is:

$$x_R = s^2 - 2x_P \bmod M = (3^2 - 2 \times 2) \bmod 5 = 0;$$

$$y_R = -y_P + s(x_P - x_R) \bmod M = -2 + 3(2 - 0) \bmod 5 = 4.$$

$$R=(x_R, y_R)=(0,4).$$

Elliptic curve cryptography

Arithmetic in an Elliptic Curve group

Example. Consider the addition of two points $P=(2,2)$ and $-P=(2,3)$, belonging to the elliptic group $E_5(0,1)$. The following conditions is true $(2,2)=- (2,3)$. The resulting point is O . Formaly this results can be recognized from first step of calculation:

$$s=(y_P-y_Q)/(x_P-x_Q) \bmod M = (2-3)/(2-2) \bmod 5=\infty.$$

Then $(2,2) + (2,3) = O$.

+	O	(0,1)	(0,4)	(2,2)	(2,3)	(4,0)
O	O	(0,1)	(0,4)	(2,2)	(2,3)	(4,0)
(0,1)	(0,1)	(0,4)	O	(2,3)	(4,0)	(2,2)
(0,4)	(0,4)	O	(0,1)	(4,0)	(2,2)	(2,3)
(2,2)	(2,2)	(2,3)	(4,0)	(0,4)	O	(0,1)
(2,3)	(2,3)	(4,0)	(2,2)	O	(0,1)	(0,4)
(4,0)	(4,0)	(2,2)	(2,3)	(0,1)	(0,4)	O

Elliptic curve cryptography

Elliptic curve group generation

The finite field (Elliptic curve group) needs to be chosen so that the prime order of the fixed point is sufficient large, say 150-300 bits. If the field is $GF(M)$ where M is a large prime, xy term is omitted, leaving us with

$$y^2 = x^3 + a x + b ,$$

There is a requirement that $4a^3 + 27b^2 \bmod M$ is non-zero.

If the field is $GF(2^m)$ then we have to include the xy term to get

$$y^2 + xy = x^3 + a x + b ,$$

There is a requirement that b is non-zero.

If we carry on computing $P+P+P+\dots$ for long enough, since the number of curve points is finite, we must eventually get a result O . We will certainly have $aP=bP$ for some a and b with $b>a$. This implies that $cP=O$ where $c=b-a$. The least c for which this is true is called the ***order of the point***.

Elliptic curve cryptography

Elliptic curve group generation

+	O	(0,1)	(0,4)	(2,2)	(2,3)	(4,0)
O	O	(0,1)	(0,4)	(2,2)	(2,3)	(4,0)
(0,1)	(0,1)	(0,4)	O	(2,3)	(4,0)	(2,2)
(0,4)	(0,4)	O	(0,1)	(4,0)	(2,2)	(2,3)
(2,2)	(2,2)	(2,3)	(4,0)	(0,4)	O	(0,1)
(2,3)	(2,3)	(4,0)	(2,2)	O	(0,1)	(0,4)
(4,0)	(4,0)	(2,2)	(2,3)	(0,1)	(0,4)	O

Results of the addition points
 $P+P+P+\dots$ for $E_5(0,1)$

$cP=O$
 Point $P=(0,1)$ has the
 order $c=3$

+	(0,1)	(0,4)	(2,2)	(2,3)	(4,0)
P	(0,1)	(0,4)	(2,2)	(2,3)	(4,0)
$2P$	(0,4)	(0,1)	(0,4)	(0,1)	O
$3P$	O	O	(4,0)	(4,0)	(4,0)
$4P$	(0,1)	(0,4)	(0,1)	(0,4)	O
$5P$	(0,4)	(0,1)	(2,3)	(2,2)	(0,4)
$6P$	O	O	O	O	^{17}O

Elliptic curve cryptography

Elliptic curve group generation

For good security, the curve and fixed point are chosen so that the order of the fixed point G is a large prime number. This is quite easy to do once we know the order of the curve, provided it has a large prime factor. However finding the order of an arbitrary curve is tricky – it involves what is known as ***Schoof’s algorithm*** – and getting this algorithm to run in a reasonable time takes a lot of tricks.

There are two alternative methods which instead construct curves of known order, one based on ***Complex Multiplication*** and other based on “***Weil’s theorem***”, which is much quicker to code.

If the order of the fixed point G is an n -bit prime, then computing k from kG and G takes roughly $2^{n/2}$.

Elliptic curve cryptography

Elliptic Curve Discrete Logarithm Problem

In the multiplicative group, the discrete logarithm problem is:

Given elements r and q of the group, and a prime p , find a number k such that $r = q^k \bmod p$.

If the elliptic curve groups is described using multiplicative notation, then the elliptic curve discrete logarithm problem is:

Given points P and Q in the group $E_M(a,b)$, find a number k that $kP=Q$; k is called the discrete logarithm of Q to the base P

$$k = \log_P Q \bmod M$$

Example In the elliptic group defined by

$$y^2 = x^3 + 9x + 17 \bmod 23,$$

What is the discrete logarithm k of $Q=(4,5)$ to the base $P=(16,5)$ in $E_{23}(9,17)$

One (naive) way to find k is to compute multiples of P until Q is found. The first multiples of P are: $P=(16,5)$, $2P=(20,20)$, $3P=(14,14)$, $4P=(19,20)$, $5P=(13,10)$, $6P=(7,3)$, $7P=(8,7)$, $8P=(12,17)$, $9P=(4,5)$. Since $9P=(4,5)=Q$, the discrete logarithm of Q to the base P is $k=9$. **In real application, k would be large enough such that it would be infeasible to determine k in this manner.**

Elliptic curve cryptography

Elliptic Curve Discrete Logarithm Problem

Example. Based on elliptic curve $y^2 = x^3 + 10x + 10$ and $M=23$ define the elliptic group $E_{23}(10,10)$ with the generating point $G = G(x,y) = G(5,1)$. The result of the operation kG in $E_{23}(10,10)$ are shown below

k	1	2	3	4	5	6	7	8	9
kG	(5,1)	(8,21)	(11,5)	(10,11)	(12,8)	(7,20)	(15,19)	(9,1)	(9,22)
k	10	11	12	13	14	15	16	17	18
kG	(15,4)	(7,3)	(12,15)	(10,12)	(11,18)	(8,2)	(5,22)	(O)	(5,1)

For chosen generating point $G(5,1)$ the order is 17.

Elliptic curve cryptography

Key Exchange in Elliptic Curve Group

First of all the prime number $M \approx 2^{180}$ as well as the parameters of a and b for the elliptic curve to define the elliptic group $E_M(a,b)$. Then the generating point $G = G(x,y)$ should be chosen such a way that c for which $cG=O$ is very large prime number. The parameters $E_M(a,b)$ and G are open for everybody.

Key exchange procedure in between the user A and B.

1. User **A** takes the integer number n_A , that $n_A < c$. n_A is **secret key** for user **A**. Then **A** generate an **open key** $P_A = n_A G$, Note, that P_A is the point over $E_M(a,b)$.

2. User **B** repeats the same to get their own secret key n_B and open P_B key.

3. User **A** sends to **B** his open key P_A , and **B** to **A** his P_B .

4. User **A** generates common secret key as $K = n_A P_B$, as well as user **B** generate the same common secret key as $K = n_B P_A$.

It is easy to show that $n_A P_B = n_A (n_B G) = n_B (n_A G) = n_B P_A$.

To brake this system the third part should calculate the value k (n_A or n_B) based on G and kG .

Elliptic curve cryptography

Key Exchange in Elliptic Curve Group

Example. Let $M=23$ and $E_{23}(10,10)$ and generating point is $G=G(5,1)$ with the order c equal to 17.

1. Secret key for user **A** is $n_A=2 < c = 17$, the open key $P_A = n_A G = 2(5,1) = (8,21)$.
2. Secret key for user **B** is $n_B=3 < c = 17$, the open key $P_B = n_B G = 3(5,1) = (11,5)$.
3. Based the open communication channel the user **A** sends to **B** $P_A = (8,21)$ and **B** to **A** his key $P_B = (11,5)$.
4. Common secrete key $K = n_A P_B = n_B P_A = 2(11,5) = 3(8,21) = (7,20)$.

Elliptic curve cryptography

Elliptic Curve Group Cryptosystem

There are several approaches to apply the Elliptic Curve Group to modern Cryptographic systems. The most straightforward solution can be described as follows.

First step is the coding the message to be send m as a point $P_m = P_m(x, y)$ of the Elliptic group. The first problem is the limitation of the Elliptic group point values. There are not all possible meaning of x and y within the defined Elliptic group $E_M(a, b)$.

As in the key exchange algorithm: The parameters $E_M(a, b)$ and G are open for everybody. Both users (**A** and **B**) generate their keys: secret (n_A, n_B) , and public (P_A, P_B) , where $P_A = n_A G$, and $P_B = n_B G$.

Enciphering

To send the message $P_m = P_m(x, y)$ from **A** to **B**.

1. User **A** generate random integer number k .
2. User **A** determine the ciphertext C_m based on two points P_m and P_B , which also consists of two points. Then $C_m = (kG, P_m + kP_B)$.

Elliptic curve cryptography

Elliptic Curve Group Cryptosystem

Deciphering

To decipher ciphertext C_m **B** have to calculate.

1. User **B** multiply the first point kG of C_m by their secret key n_B to get $n_B(kG)$.
2. User **B** subtract the result $n_B(kG)$ from the second point of C_m . Then,

$$P_m + kP_B - n_B(kG) = P_m + k(n_B G) - n_B(kG) = P_m$$

The user **A** masked their message P_m by the addition to the point kP_B

Elliptic curve cryptography

Elliptic Curve Group Cryptosystem

Example. Let $M=23$ and $E_{23}(10,10)$ and generating point is $G=G(5,1)$ with the order c equal to 17. User **A** have to sent to **B** the message encoded as a point $P_m=(15,4)$.

User **B Secret key** is $n_B = 3$, and **Public key** – $P_B = n_B G = 3(5,1) = (11,5)$

Enciphering

1. User **A** randomly generates $k = 7 < c = 17$.
2. Using point G **A** obtained $kG = 7(5,1) = (15,19)$.
3. Based on user's **B** public key $P_B = (11,5)$ user **A** calculate $kP_B = 7(11,5) = (10,11)$.
4. User **A** determine the value $P_m + kP_B = (15,4) + (10,11) = (11,18)$.
5. User **A** sends to user **B** ciphertext $C_m = ((15,19), (11,18))$

Elliptic curve cryptography

Elliptic Curve Group Cryptosystem

Deciphering

To decipher ciphertext $C_m = ((15,19), (11,18))$ **B** have to calculate.

1. User **B** based on the first point $kG = (15,19)$ elliptic group $E_{23}(10,10)$ and his own secret key $n_B=3$ determine $n_B(kG)=3(15,19)=(10,11)$.

2. User **B** subtract the result $n_B(kG)=(10,11)$ from the second point of C_m . Then,

$$(P_m + kP_B) - (n_B(kG)) = (11,18) - (10,11) = (11,18) + (10,-11) = \\ = (11,18) + (10,12) = (15,4)$$

Теория Информации

В.Н. Ярмолик

Лекция 15

2017

Quantum Cryptography

Main definitions

Quantum cryptography uses our current knowledge of physics to develop a cryptosystem that is not able to be defeated - that is, one that is completely secure against being compromised without knowledge of the sender or the receiver of the messages. The word *quantum* itself refers to the most fundamental behavior of the *smallest particles* of *matter* and *energy*: quantum theory explains everything that exists and nothing can be in violation of it. Key distribution in standard systems can be compromised by eavesdropping.

Quantum Cryptography is an encryption method that can detect eavesdropping. Using optical transmission to send a secret key to the other side, quantum cryptography draws on the inherent properties of photons, which become slightly altered if they are observed by an intruder. When an alteration is detected, the receiver knows the sender's key has been compromised.

Quantum Cryptography

Main definitions

Quantum Cryptography based on the magic of

HEISENBERG'S UNCERTAINTY PRINCIPLE

(Published by [Werner Heisenberg](#) in 1927)

“Locating a particle in a small region of space makes the momentum of the particle uncertain; and conversely, measuring the momentum of a particle precisely makes the position uncertain.”

This means that it is possible to detect eavesdropping and compensate for it.

Quantum Cryptography

Main definitions

Elements of the Quantum Theory

Light waves are propagated as discrete quanta called photons.

They are massless and have energy, momentum and angular momentum called spin.

Spin carries the polarization.

If on its way we put a polarization filter a photon may pass through it or may not.

We can use a detector to check if a photon has passed through a filter.

Quantum Cryptography

Heisenberg Uncertainty Principle

Certain pairs of physical properties are related in such a way that measuring one property prevents the observer from knowing the value of the other.

When measuring the polarization of a photon, the choice of what direction to measure affects all subsequent measurements.

If a photon passes through a vertical filter it will have the vertical orientation regardless of its initial direction of polarization.

Quantum No-cloning Theorem. The polarization of a single photon can not be cloned.

Quantum Cryptography

Polarization by a filter

A pair of *orthogonal filters*, for example, such as vertical/horizontal is called a basis.

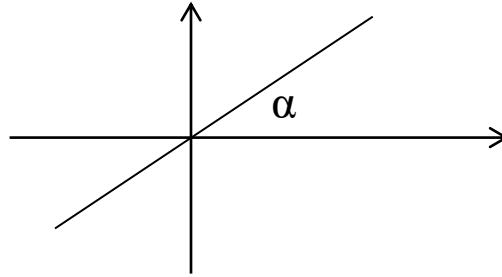
A pair of basis is conjugate (Сопряженный) if the measurement in the first basis completely randomizes the measurements in the second basis.

Usually one basis is *orthogonal* with the 0-90 degree (+) and the second one *circular* with 45-135 degree basis (×).

The observations are interpreted using a binary scheme: *right-circular* (/) or *vertical* is (|) 0 *left-circular* (\) or *horizontal* (—) is 1.

Quantum Cryptography

Polarization by a filter



The probability the photon polarization is horizontal (—)
can be estimated as

$$P_1 = \cos^2 \alpha;$$

And vertical according to

$$P_2 = 1 - P_1 = \sin^2 \alpha.$$

$$P_1 + P_2 = \sin^2 \alpha + \cos^2 \alpha = 1.$$

$$\cos 45^\circ = \sin 45^\circ = 1/(2^{1/2}).$$

$$\cos^2 45^\circ = \sin^2 45^\circ = 1/2.$$

Quantum Cryptography

Polarization by a filter

Polarization	↑	→	/	\	↑	→	/	\
Basis	+	+	+	+	x	x	x	x
Result	↑	→	↑ or → (50/50)	↑ or → (50/50)	/ or \ (50/50)	/ or \ (50/50)	/	\

Photon measurement with different bases.

Quantum Cryptography

Photon receiving

Sender-receiver of photons Alice and Bob

Suppose Alice uses 0-deg/90-deg (*orthogonal*) polarizer sending photons to Bob. But she does not reveal which.

Bob can determine photons by using filter aligned to the same basis.

But if he uses 45deg/135 deg (*circular*) polarizer to measure the photon he will not be able to determine any information about the initial polarization of the photon.

The result of his measurement will be completely random

Eavesdropper Eve

If Eve uses the filter aligned with Alice's she can recover the original polarization of the photon.

If she uses the misaligned filter she will receive no information about the photon .

Also she will influence the original photon and be unable to retransmit it with the original polarization.

Bob will be able to deduce Eve's presence.

Quantum Cryptography

BB84 Protocol

Quantum Key Distribution Protocol (QKD)

Stephen Wiesner – early 1970s wrote paper "Conjugate Coding"

Paper by *Charles Bennett* and *Gilles Brassard* in 1984 is the basis for *QKD* protocol BB84. Prototype developed in 1991.

Original Paper: *Bennett*: "Quantum cryptography using any two nonorthogonal states", *Physical Review Letters*, Vol. 68, No. 21, 25 May 1992, pp 3121-3124

Quantum Cryptography

BB84 Protocol

1. Alice generates the random sequence of bits.
2. For all random bits Alice generate randomly one of two basis *orthogonal or circular*.
3. Alice transmits a polarized beam in short bursts. The polarization in each burst is randomly modulated to one of four states (*horizontal* (1), *vertical* (0), *left-circular* (1), or *right-circular* (0)), according to random sequence of bits.
4. Bob measures photon polarizations in a random sequence of basis (*orthogonal or circular*).
5. Bob tells the sender publicly what sequence of bases were used.
6. Alice tells the receiver publicly which bases were correctly chosen.
7. Alice and Bob discard all observations not from these correctly-chosen basis.

Quantum Cryptography

BB84 Protocol

Example. Here is the process Alice and Bob went through to generate their Shared secret key:

Alice's random bits	0	1	1	0	1	0	0	1
Alice's sending basis	+	+	×	+	×	×	×	+
Photon Polarization Alice sends		—	\		\	/	/	—
Bob's random measuring basis	+	×	×	×	+	×	+	+
Photon Polarization Bob measures		/	\	/	—	/	—	—
Bob's received bits	0	0	1	0	1	0	1	1
Discussion of Basis	yes	no	yes	no	no	yes	no	yes
Shared secret key	0		1			0		1

Quantum Cryptography

Intercept and Resend Attak

Eve receives Alice's encoded photon. If she guesses the base correctly, then she just has to encode a new photon and send it on to Bob.

If Eve guesses incorrectly, she will just generate a new randomly encoded photon to send to Bob.

Therefore, the probability an intercepted photon generates an error in the key string is **50% x 50% = 25%**

If n bits are compared, the number of bits required to detect an eavesdropper will be 72 key bits.

$$P_d = 1 - \left(\frac{3}{4}\right)^n$$

Quantum Cryptography

Intercept and Resend Attak

Example. Intercept and Resend Attack (Men in the Middle):

Allic's random bits	0	1	1	0	1	0	0	1
Allic's random sending basis	+	+	×	+	×	×	×	+
Photon polarization Alice sends		−	\		\	/	/	−
Eve's random measuring basis	+	×	+	+	×	+	×	+
Eve's random measures and sends		/	−		\	−	/	−
Bob's random measuring basis	+	×	×	×	+	×	+	+
Photon polarization Bob measures		/	/	\	−	/		−
Bob's received bits	0	0	0	1	1	0	0	1
Public discussion of Basis	yes	no	yes	no	no	yes	no	yes
Shared secret key	0		0			0		1
Error in key			*					

Quantum Cryptography

Raw Key Generation

For privacy amplification, the first part of the protocol works exactly as before:

1. Alice sends Bob photons over a quantum channel, then the two exchange information over a public channel about which measurement bases they used.

2. As before, they delete the results for which they used different measurement bases. Now, however, they also must delete bit slots in which Bob should have received a photons, but didn't, either because of Eve's intrusion, or from *dark counts* at Bob's detector.

3. Bob transmits the location of dark counts to Alice over the public channel.

4. Next, Alice and Bob publicly compare small parts of their raw keys to estimate the error rate, then delete these publicly disclosed bits from their key, leaving the *tentative final key*.

If the error rate exceeds a pre-determined error threshold, indicating possible interception by Eve, they start over from the beginning to attempt a new key.

Quantum Cryptography

Raw Key Correction

If the error rate is below the threshold, they remove any remaining errors from the rest of the raw key, to produce the *reconciled key* by using parity checks of sub-blocks of the tentative final key.

To do this:

1. They partition the key into blocks of length l such that each block is unlikely to contain more than one error.
2. Then they each compute parity on all blocks and publicly compare results, throwing away the last bit of each compared block.
3. If parity does not agree for a block, they divide the block into two, then compare parity on the sub-blocks, continuing in this binary search fashion until the faulty bit is found and deleted. This step is repeated with different random partitions until it is no longer efficient to continue.

After this process, they select randomly chosen subsets of the remaining key, computing parity, discarding faulty bits and the last bit of each partition as before.

This process continues for some fixed number of times to ensure with high probability that the key contains no error.

Because physical imperfections are inevitable in any system, it must be assumed that Eve may be able to obtain at least partial information.

Теория Информации

В.Н. Ярмолик

Лекция 16

2017

Physical Cryptography

Main definitions

Physical cryptography is a different and very recent approach to cryptography and security which is based on inherent complexity of nanoscale electronic and photonic systems.

R. Pappu, B. Recht, J. Taylor, N. Gershenfeld, Physical One-Way Functions, Science, vol. 297, pp. 2026-2030, 20 September 2002.

The main category belongs to physical cryptography is ***Physical Unclonable Function (PUF)***, which firstly have been call as ***Physical One-Way Function (POWF)***, ***Physical Random Functions (PRF)***. While the two last terms have been coined first, today mainly the expression ***Physical Uncloneable Function*** or PUF is in use. The notion of a ***Physical One-Way Function*** has been defined by *R. Pappu*. It definitely owns the merit of being the first formalization attempt in the field. One of the widely used now definition of PUFs has been proposed by *Tuyls et al.*

Physical Cryptography

Main definitions

Definition 1. *Physical Unclonable Function* consist of inherently unclonable physical systems. They inherit their unclonability from the fact that they consist of many random components that are present in the manufacturing process and can not be controlled. When a stimulus is applied to the system, it reacts with a response. Such a pair of a stimulus (*challenge*) C and a *response* R is called a *challenge-response pair* (CRP). In particular, a PUF is considered as a function that maps challenges to responses.

According to this definition *Physical Unclonable Functions* or PUFs have been introduced as challenge response entities that are undetachably embedded in a physical system, possibly a silicon integrated circuit. The physical system is, explicitly or implicitly, produced in such a way that it contains uncontrollable random elements.

Physical Cryptography

Main definitions

More general definition of PUFs as Super High Information Content systems (SHICs) have been made by *U.Ruhrmair*.

Definition 2. *Physical Unclonable Functions* is the *complex disordered physical systems* with extraordinarily high amount of structural information which satisfy to the next conditions:

1. The information content of the system can be extracted reliably and repeatedly through measurement with different *challenge* C_i and obtaining the resulting *response (answer)* R_i .
2. The number of possible challenges C is so large that the values of all corresponding responses R cannot be determined for all possible challenges C , within limited time.
3. Due to high information content in the system it must also be impossible to model, computationally learn, simulate or otherwise numerically predict the challenge-response pair (C_j, R_j) based on known pair (C_l, R_k) .
4. It must be prohibitively difficult to physically reproduce or clone PUF as physical systems.

Physical Cryptography

PUF's main properties

Physical Unclonable Function (PUF) [Pappu, Gassend et al 2002]

PUF Security is based on

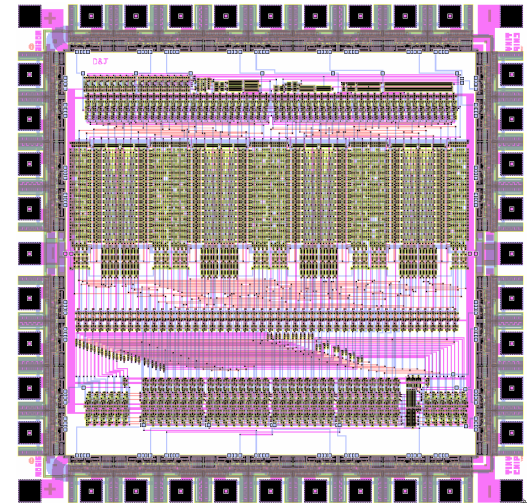
- *wire delays*
- *gate delays*
- *quantum mechanical fluctuations*

PUF characteristics

- *uniqueness*
- *reliability*
- *unpredictability*

PUF Assumptions

- *Infeasible to accurately model PUF*
- *Pair-wise PUF output-collision probability is constant*
- *Physical tampering will modify PUF*



Physical Cryptography

Delay-Based Silicon PUF

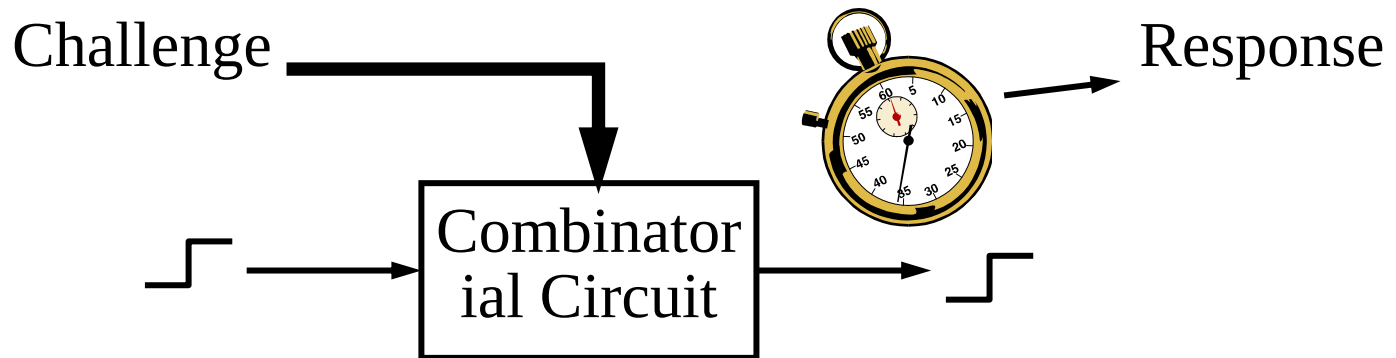
concept

Because of random process variations, no two Integrated Circuits even with the same layouts are identical

- Variation is inherent in fabrication process
- Hard to remove or predict
- Relative variation increases as the fabrication process advances

Delay-Based Silicon PUF concept

- Generate secret keys from unique delay characteristics of each processor chip



Physical Cryptography

PUF for IC

PUF implementations for silicon integrated circuits

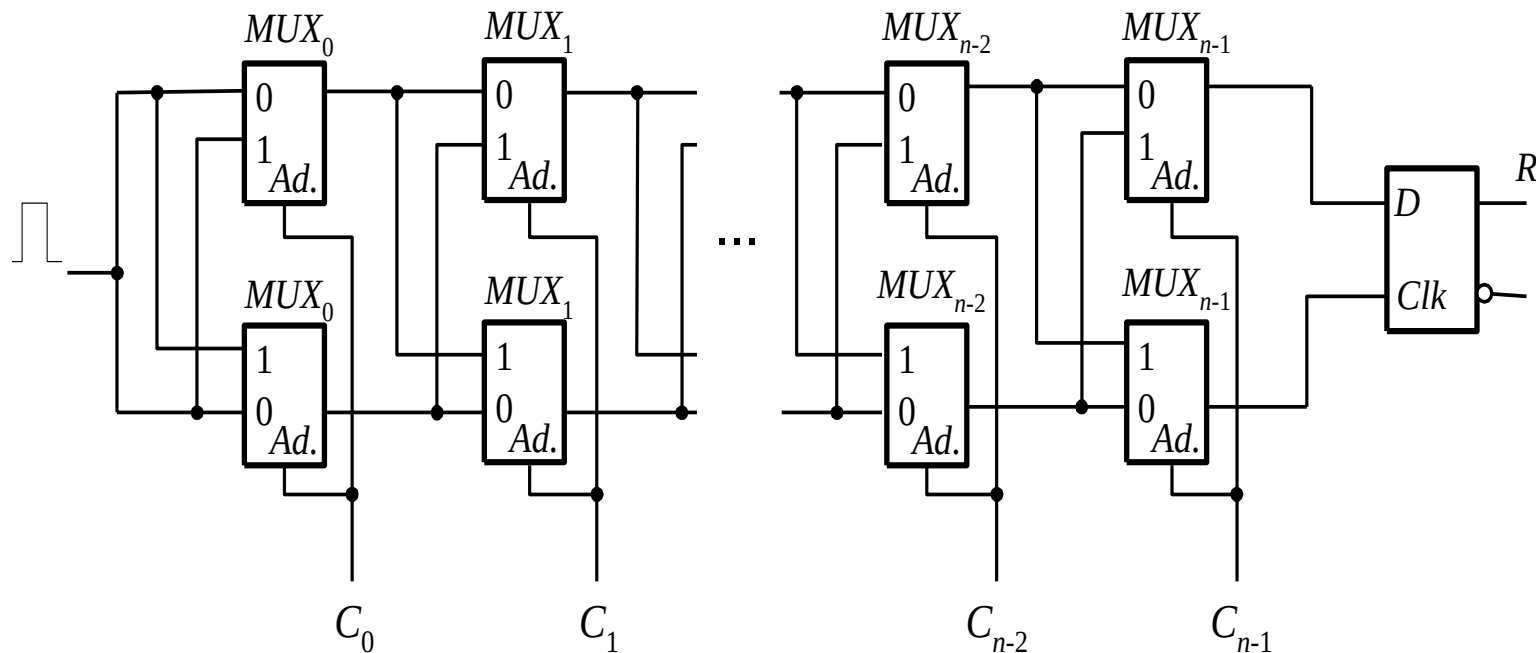
The first *PUFs* embedded in silicon devices that were suggested are based on delay measurements of digital paths. Because of process variations, two identically designed digital paths on two *ICs*, or on two distinct places on the same *IC*, will not have exactly the same delay from their input to their output. Very well known method have been proposed to measure the random delay variations have been call as ***Arbiter PUF***.

Arbiter PUFs

According to Arbiter PUF a race condition is introduced by feeding two symmetrical paths into an arbiter. Two simultaneous pulses on the inputs of the paths will in general not arrive at the outputs simultaneously due to small random delay variations. The arbiter decides which of the two paths was the fastest.

Physical Cryptography

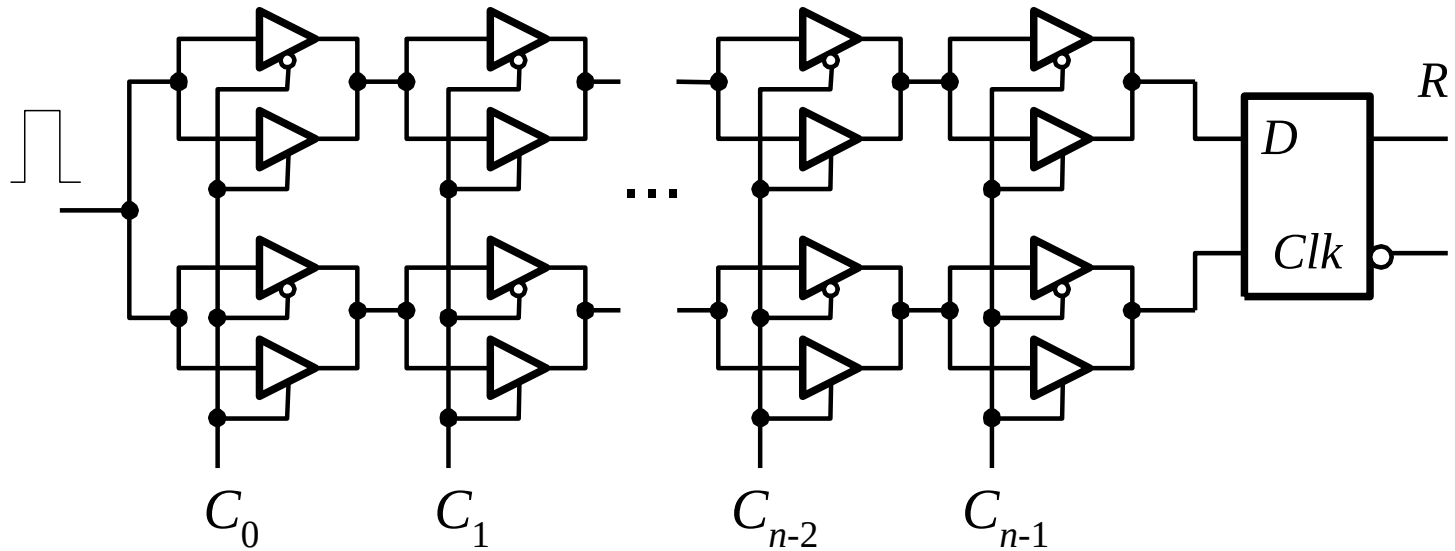
MUX based Arbiter PUF



The *challenge* bits C determine the delay paths by controlling the *MUXes*. To evaluate the response R for a particular challenge C , a rising signal is given to the inputs of both paths at the same time, the signals race through the two delay paths, and the *arbiter* (D -latch) at the end of two paths decides which signal is faster.

Physical Cryptography

Tristate Buffers PUF



The proposed *PUF* circuit is utilizing the delay variances of tristate buffer. In this case the tristate buffers just replaced *MUXes* of original *Arbiter PUF* presented in previous slide. The simulation results presented in show that the *Tristate Buffer* based *PUF* consumes less power and requires less area overhead compare to the *MUX* based version.

Physical Cryptography

MUX based Arbiter PUF

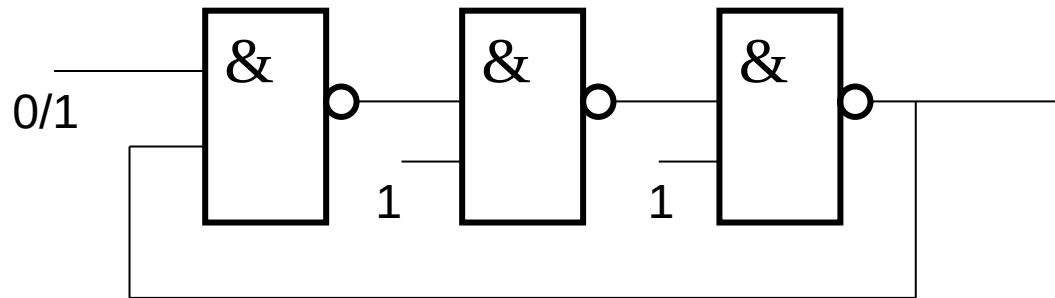
The *MUX*-based one-output ($m=1$) Arbiter *PUF* circuit with $n=64$ stages (64-bit challenge C) has been fabricated and tested in TSMC's $0.18\mu\text{m}$, single-poly, 6-level metal process. The experimental results show that two identical *PUF* circuits with identical challenges C on two different chips have different responses R with a probability of 23% (*interchip variation*). For ideal *PUFs* with pure symmetric layout for the circuit shown in Figures 1 and 2 the interchip variation would be close to 50%. Within the same framework, multiple measurements on the same chip showed that responses R were different with the probability equals to 0.7% (*intrachip variation*). For ideal *PUFs* this probability should be equal 0.0%, what means that the values of responses R have to be constant and can be extracted reliably and repeatedly through measurement for all possible challenges C .

Physical Cryptography

Ring Oscillator PUFs

Ring Oscillator PUFs

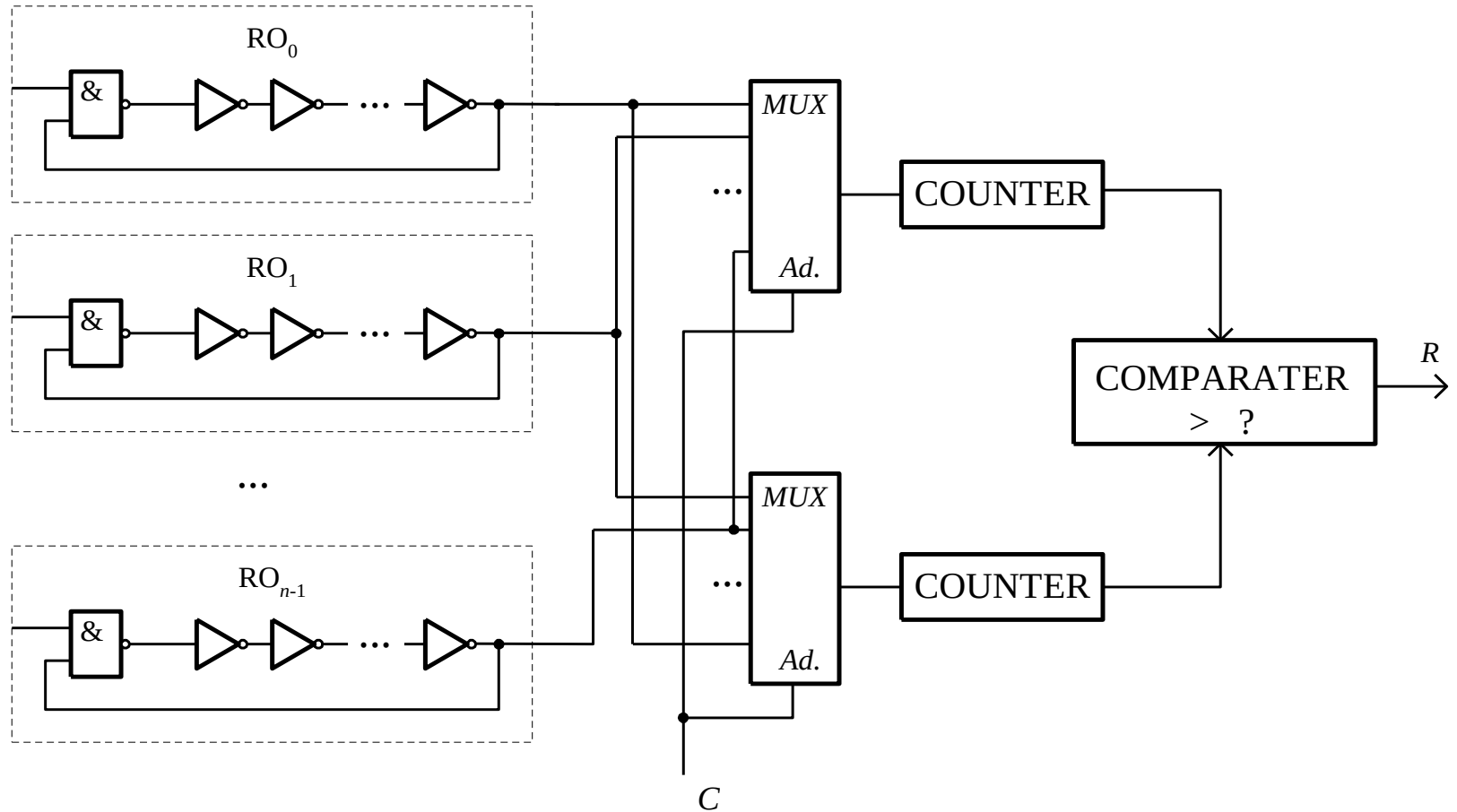
This type of *PUFs* is based on ring oscillators (delay loops). Each ring oscillator (*RO*) is a simple circuit with odd number of invertors with feedback loop that due to manufacturing variation oscillates with a particular frequency.



Ring Oscillator

Physical Cryptography

Ring Oscillator PUFs



Physical Cryptography

Ring Oscillator PUFs

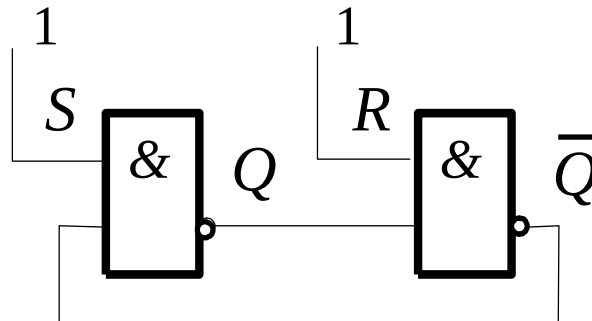
In a case of *RO PUF* presented in last slide each comparison of a pair of *RO* generates one bit response R . For n *RO* there are $n(n - 1)/2$ distinct pairs of *RO*. As have been shown the response bits obtained from pair-wise comparisons are correlated. For example, if oscillator RO_i is faster than oscillator RO_j , the comparison will yield $R=1$. If RO_j is in turn faster than RO_r , the comparison will yield $R=1$. It is clear that when RO_i is compared with RO_r that the comparison will be same $R=1$ - these bits are correlated. Fortunately, it is possible to derive the maximum entropy of this circuit assuming pair-wise comparisons, i.e., the number of independent bits that can be generated by the circuit as a function of n , the number of *RO*. There are $n!$ different orderings of ring oscillators based on their frequencies. In a case when the orderings are equally likely, the entropy will be $\log_2(n!)$ bits. For example, 128 oscillators can produce 716 bits.

Physical Cryptography

SRAM based PUF

SRAM based PUFs

Static Random Access Memory or *SRAM* is a very common type of volatile storage in integrated circuits. A typical *SRAM*-cell consists of six transistors in CMOS-technology. The actual storage element comprising four transistors implements two cross-coupled inverters. The cross-coupled storage element is bi-stable, meaning it has two stable states, and hence can store one bit of information by residing in one of both states. When the cell is powered on, it will quickly converge into one out of two stable states. A cross-coupled circuit is constructed such that it provides a positive feedback loop to store the required bit value within the loop. An example of such a circuit is a simple R-S latch based on two 2NAND gates is shown below.



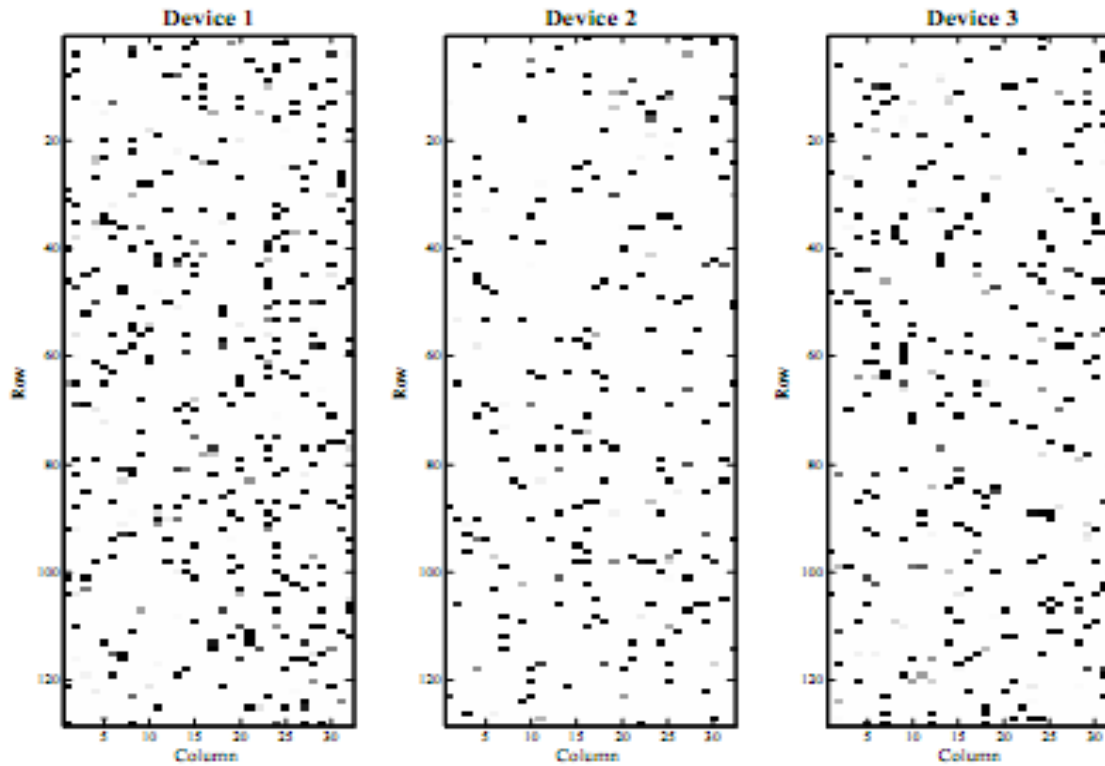
Physical Cryptography

SRAM based PUF

As have been shown earlier the amount of randomness present in the power-up values of the flip-flops is limited. This is due to the strong favor of most flip-flops to start up in '0', rather than a more or less fifty-fifty chance. The experimental investigations have been made on three different Virtex-II Pro FPGAs. The power-up values of 4096 flip-flops (32 columns $\times$ 128 rows) were measured after 101 consecutive power-ups. The result shows that more than 90% of the measured flip-flops always power-up as '0' versus little less than 10% which always start up as '1'. Less than 1% will start up as '0' some of the times and '1' the other times. This results allows to make the conclusion that *SRAM based PUF* is not reliable, especially for FPGA implementation. To overcome this problem the concepts of the butterfly *PUF* which behave similarly to a *SRAM* cell during the startup phase have been proposed.

Physical Cryptography

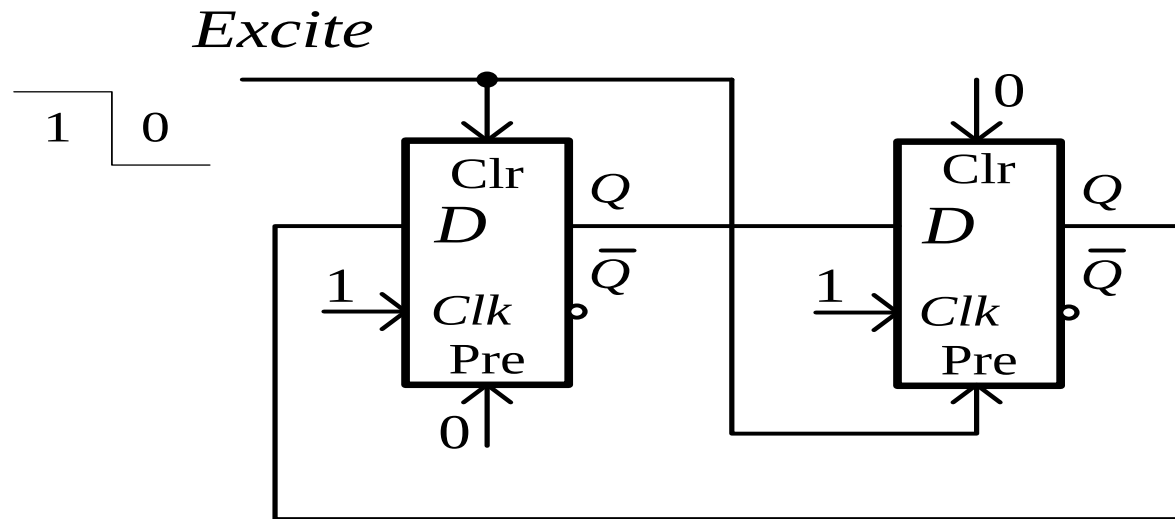
SRAM based PUF



Physical Cryptography

The Butterfly PUF

The *Butterfly PUF*, proposed by Kumar et.al, is a technique that aims to emulate the behavior of an *SRAM PUF*. This technique is based on the simulation of cross-coupled combinational loop using latches presented in the FPGA. The structure of the Butterfly PUF cell is constructed as symmetrically as possible.



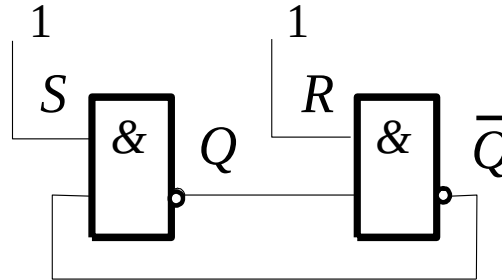
Physical Cryptography

The Butterfly PUF

The *Butterfly PUF* consists of two latches, each with a preset (*Pre*) and clear (*Clr*) inputs. In *Butterfly PUF* the signal at the *Pre* input of first latch and the signal at the *Clr* input for the second latch are always set to 0. The *Excite* input is connected to *Clr* of latch 1 and *Pre* of latch 2. The *Clk* signal in both latches is always high. To start the *PUF* operation, the *Excite* signal is set to high. This brings the *Butterfly PUF* circuit to an unstable operating point. After some period of time *Excite* signal is set to low. This initiates the process of the *Butterfly PUF* circuit to attain either one of the two possible stable states, 0 or 1 at the Q output of the second latch. The final state is determined by the random delay mismatch in the pair of feedback paths and the *Excite* signal paths due to process variation.

Physical Cryptography

R-S Latch PUF



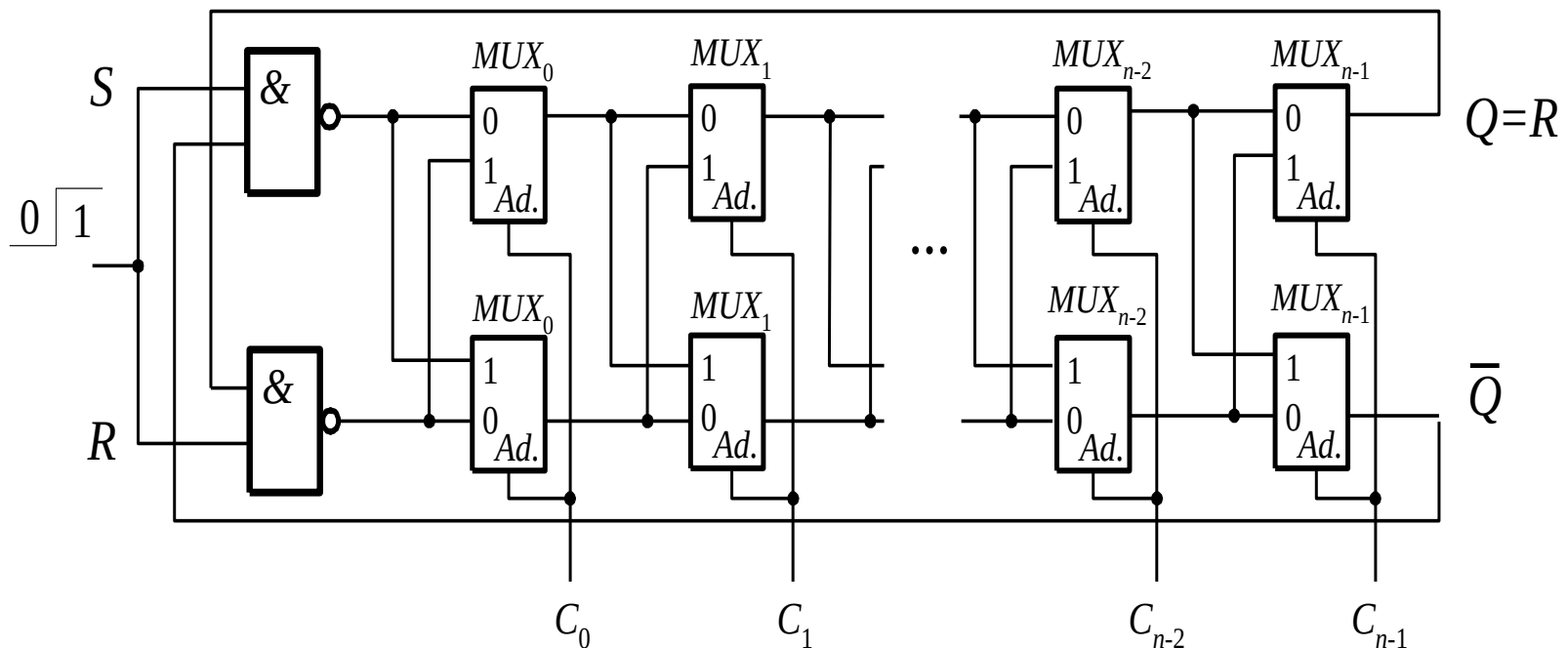
R-S Latch truth table

Current inputs $S R$	Next inputs $S R$	Current state $Q \bar{Q}$	Next state $Q \bar{Q}$
$S R = XX$	0 0	$Q \bar{Q}$	1 1
$S R = XX$	0 1	$Q \bar{Q}$	1 0
$S R = XX$	1 0	$Q \bar{Q}$	0 1
$S R \neq 00$	1 1	$Q \bar{Q}$	$Q \bar{Q}$
$S R = 00$	1 1	$Q \bar{Q}$	Random value 1 0 or 0 1

Physical Cryptography

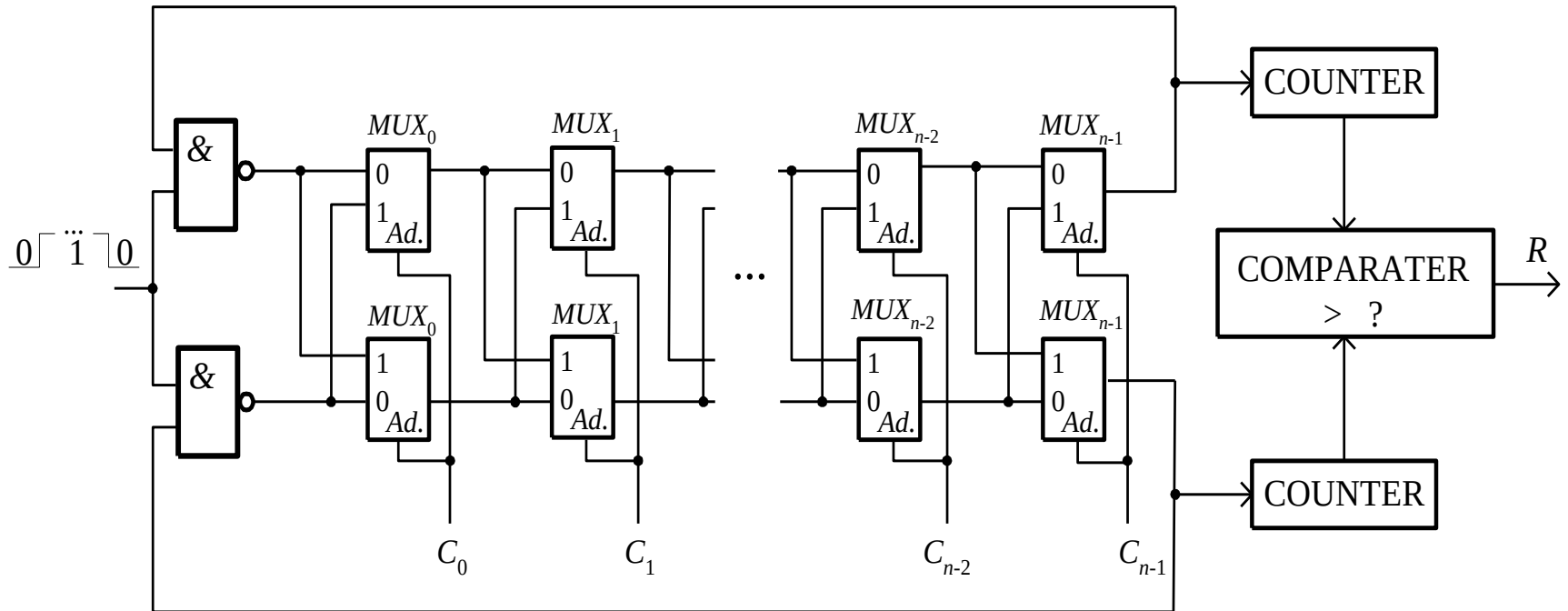
Arbiter&RS PUF

To increase the variations of random value of the delay time the combined implementation of PUF can be proposed. This solution *Arbiter&RS PUF* is united two approaches, namely *Arbiter PUF* and *R-S Latch PUF*, as is shown in Figure.



Physical Cryptography

RO&Arbiter PUF



Physical Cryptography

PUF Efficiency

Comparison of PUF With Digital Hash Functions

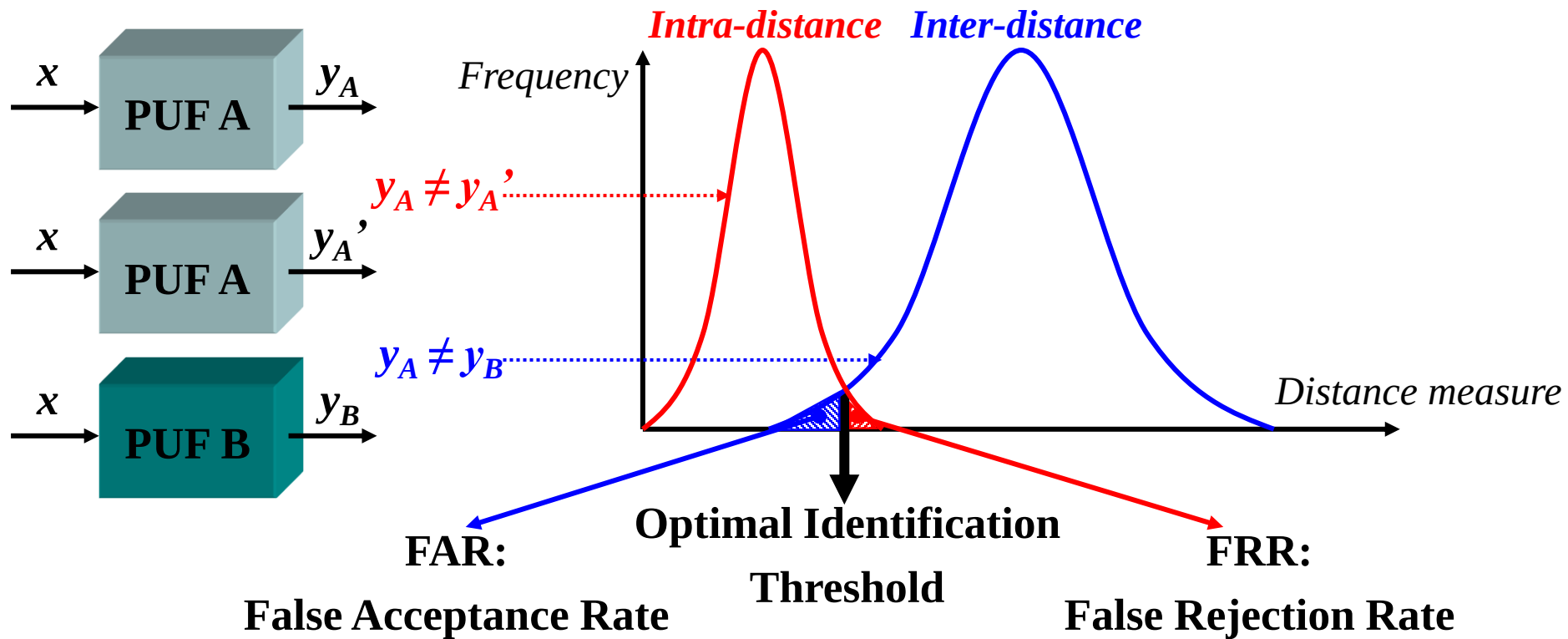
Algorithm	MD4	MD5	SHA-256	AES	Yuksel	<i>PUF</i>
# of gates	7350	8400	10868	3400	1701	<i>545</i>

- Reference PUF: 545 gates for 64-bit input
 - 6 to 8 gates for each input bit
 - 33 gates to measure the delay
- Low gate count of PUF has a cost
 - probabilistic outputs
 - difficult to characterize analytically
 - non-unique computation
 - extra back-end storage
- Different attack target for adversaries
 - model building rather than key discovery
- Physical security
 - hard to break tag and remain undetected

Physical Cryptography

PUF Application

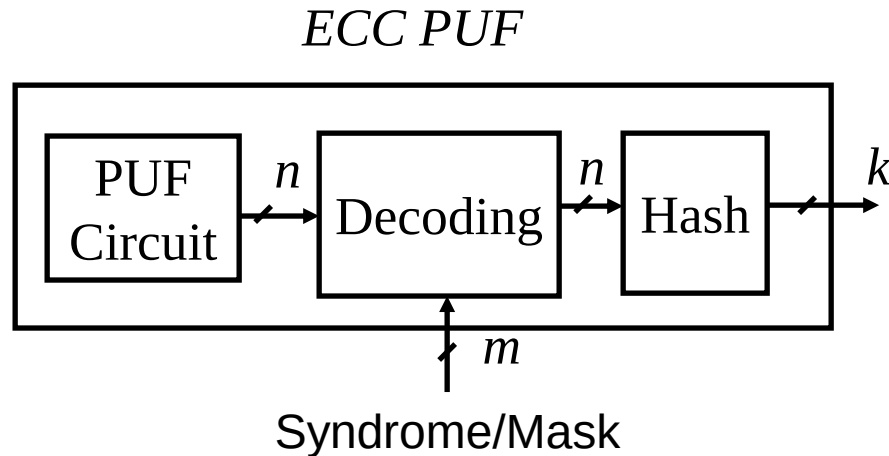
System Identification



Physical Cryptography

PUF Application

***In the Field:
Symmetric
Key Generation***



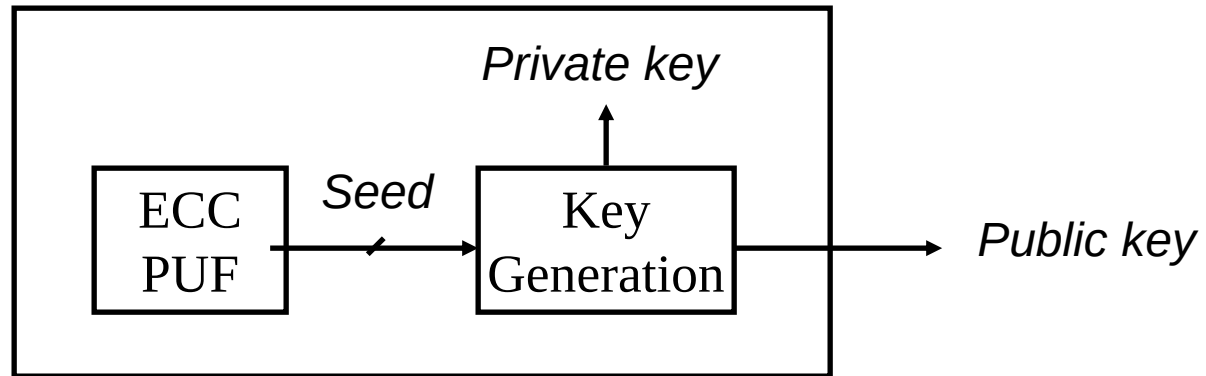
In the field, the syndrome will be used to re-generate the same PUF reference output from the circuit

- This output is unique and unpredictable for each chip
- Can be directly used as a symmetric key

Physical Cryptography

PUF Application

Private/Public Key Pair Generation



PUF response is used as a random seed to a private/ public key generation algorithm

- No secret needs to be handled by a manufacturer

A device generates a key pair on-chip, and outputs a public key

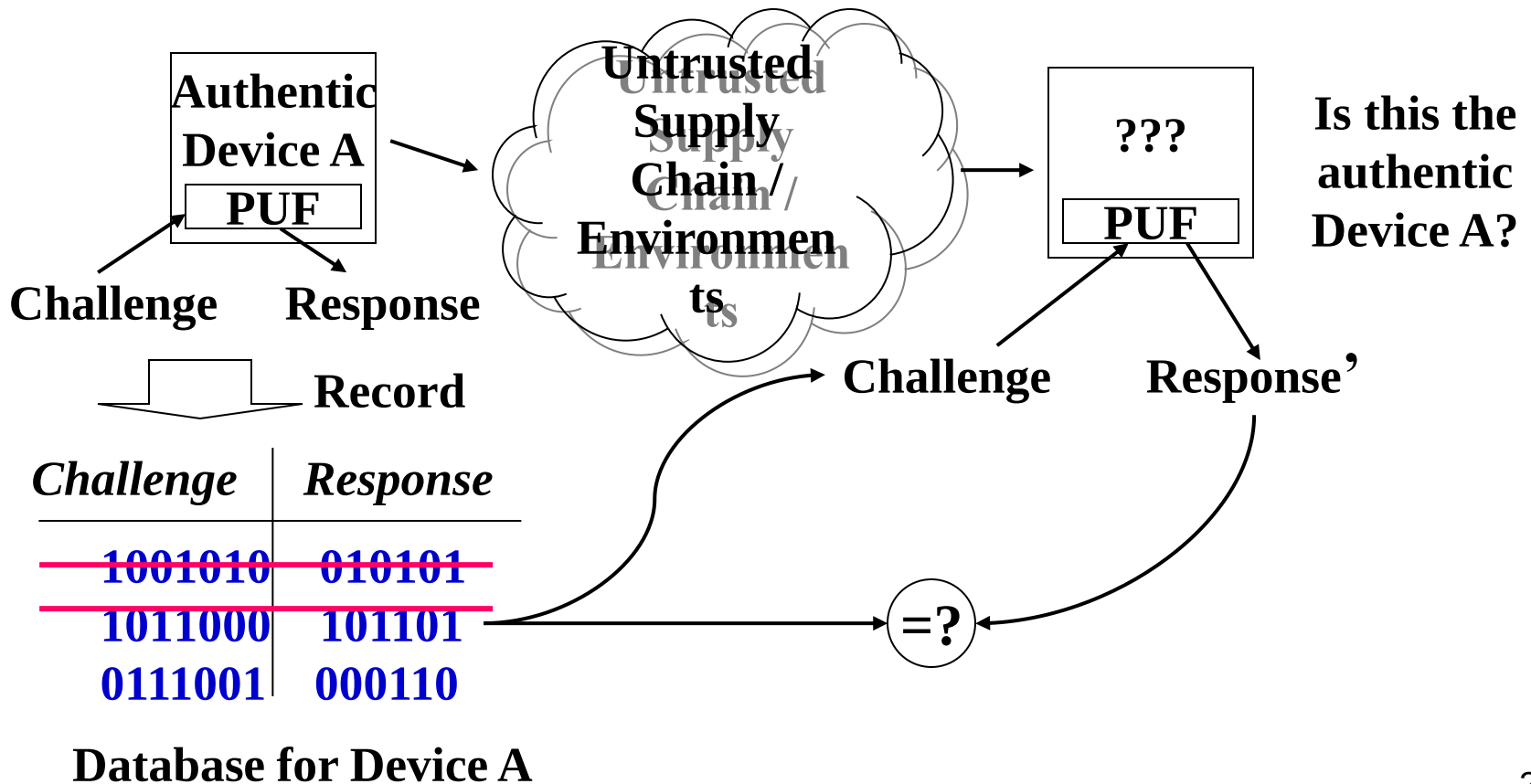
- The public key can be endorsed at any time

Physical Cryptography

PUF Application

Low-Cost Authentication

Protect against IC/FPGA substitution and counterfeits without using cryptographic operations



Теория Информации

В.Н. Ярмолик

Лекция 17

2017

Digital Signatures

Main Definitions

A *digital signature* is a property private to a user or process that is used for signing message. Let ***B*** be the recipient of a message ***M*** signed by ***A***. Then ***A***'s signature must satisfy these requirements:

1. ***B*** must be able to validate ***A***'s signature on ***M***.
2. It must be impossible for anyone, including ***B***, to *forge* ***A***'s signature.
3. In a case ***A*** should disavow signing a message ***M***, it must be possible for a judge or third party to resolve a dispute arising between ***A*** and ***B***.

A digital signature, therefore, establishes *sender authenticity*; it is analogous to an ordinary written signature. By condition (2), it also establishes *data authenticity*.

Digital Signatures

Main Definitions

Public-key authentication systems provide a simple scheme for implementing digital signatures. Because the transformation D_A is private to A , D_A serves as A 's digital signature. The recipient B of a message M signed by A (i.e., transformed by D_A) is assured of both sender and data authenticity. It is impossible for B or anyone else to forge A 's signature on another message, and impossible for A to disclaim a signed document (assuming D_A has not been lost or stolen). Because the inverse transformation E_A is public, the receiver B can readily validate the signature, and a judge can settle any disputes arising between A and B . Summarizing,

1. A signs M by computing $C = D_A(M)$.
2. B validates A 's signature by checking that $E_A(C)$ restore M .
3. A judge resolve a dispute arising between A and B by checking whether $E_A(C)$ restore M in the same way as B .

General Digital Signature Scheme

A digital signature scheme typically consists of three steps (phases):

1. ***Preparation phase*** consisting on different parameters are needed for signing and verification signature generation, like private and public keys generation, Hash function selection and so on.
2. A ***signing*** algorithm (signature generation) that, given a message and a private key, produces a signature.
3. A ***signature verifying*** algorithm that, given a message, public key and a signature, either accepts or rejects the message's claim to authenticity.

Digital Signature

Digital Signature Based on Conventional Algorithms

Diffie and Lamport approach

An approach invented by *Diffie* and *Lamport*, which is based upon a conventional algorithm such as DES.

1. Preparation phase

In order that an n -bit message may be signed, the sender randomly generates in advance $2n$ 56-bit cryptographic keys $k1_0, k1_1, k2_0, k2_1, \dots, kn_0, kn_1$, which are kept *secret*. The receiver is given in advance two sets of corresponding *nonsecret* 64-bit validation parameters $u1_0, u1_1, u2_0, u2_1, \dots, un_0, un_1$ and $v1_0, v1_1, v2_0, v2_1, \dots, vn_0, vn_1$, where

$$vi_0 = E_{ki_0}(ui_0), vi_1 = E_{ki_1}(ui_1); i=1,2,\dots,n.$$

The validation parameters are also recorded by the sender in an established public registry with recognized and accepted integrity. It is the receiver's responsibility to ensure that the validation parameters received from the sender are genuine.

Digital Signature

Digital Signature Based on Conventional Algorithms

Diffie and Lamport approach

2. Signature generation (Signing)

Later, when message M is sent, the digital signature is generated by selecting $k1_0$ or $k1_1$ depending on whether the first bit m_1 of M is 0 or 1, respectively; $k2_0$ or $k2_1$ depending on whether the second bit m_2 of M is 0 or 1, respectively; and so forth. For example, if $M=0,1,1,0, \dots$, the signature would contain the following keys: $k1_0, k2_1, k3_1, k4_0, \dots$.

3. Signature verification

The receiver validates the digital signature by ensuring that the first 56-bit key in the signature will encipher validation parameter $u1_0$ into $v1_0 = E_{k1_0}(u1_0)$ if the first bit of M is 0, or that it will encipher $u1_1$ into $v1_1 = E_{k1_1}(u1_1)$ if the first bit of M is 1, the second 56-bit key in the signature will encipher validation parameter $u2_0$ into $v2_0 = E_{k2_0}(u2_0)$ if the first bit of M is 0, or that it will encipher $u2_1$ into $v2_1 = E_{k2_1}(u2_1)$ if the first bit of M is 1, and so forth.

Digital Signature

Digital Signature Based on Conventional Algorithms

Diffie and Lamport approach

The above approach demonstrates that digital signatures can be obtained with symmetric algorithm and it is simple and easy to understand. An obvious *disadvantage* is that a *separate key* must be included in the signature for *each bit* in the message. This could result in very large signature depending upon the size of the message to be signed. If data compression techniques are used, smaller signatures are possible.

Let compression encoding be a function ***H*** (***Hash Function***) that maps variable length messages ***M*** of ***n*** bits to fixed length bit patterns of ***H(M)=m*** bits, where ***m*** and ***H(M)*** are such that it is computationally infeasible to find two different messages, ***M*** and ***M'***, for which ***H(M)=H(M')***.

Digital Signature

Digital Signature Based on Conventional Algorithms

Diffie and Lamport approach

There are many way in which a suitable Hash Function could be defined using the DES algorithm. First let M be divided into n 64-bit blocks, $X_1, X_2, \dots, X_n$, and define $H(M)$ as a function that transforms $n \times 64$ bits of data, $M=(X_1, X_2, \dots, X_n)$, into 64 bits of data $H=H_K(M)$, where K is a cryptographic key and H is computed as follows:

$$H_K(X_1)=Y_1;$$

$$H_K(X_2 \oplus Y_1)=Y_2;$$

...

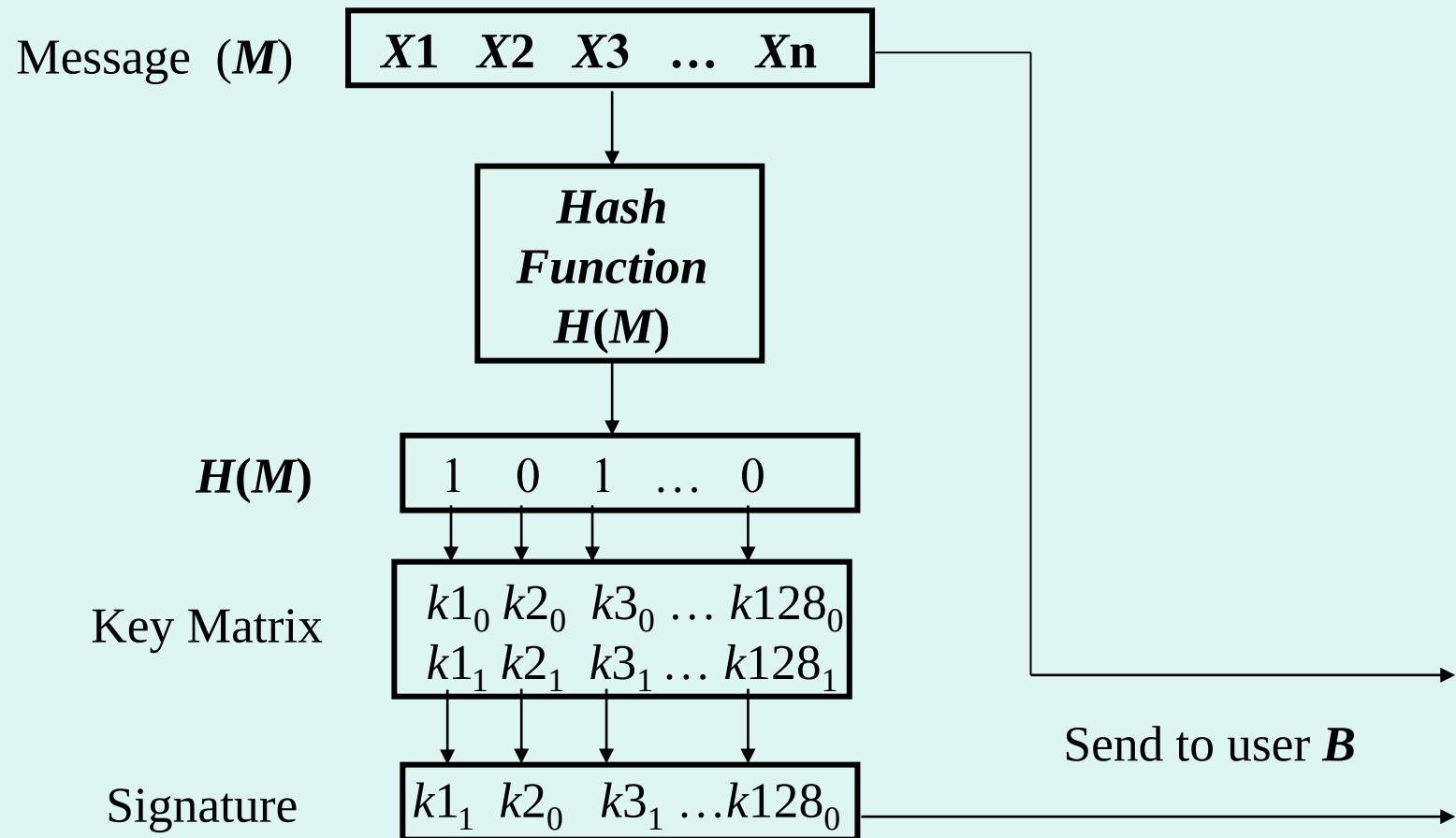
$$H_K(X_n \oplus Y_{n-1})=Y_n;$$

$$H_K(X_1 \oplus X_2 \oplus \dots \oplus X_n \oplus Y_n)=H;$$

Resulting value is a so called *Message Digest (MD)*.

Digital Signature

Digital Signature Based on Conventional Algorithms *Diffie and Lamport approach*



Digital Signature

Digital Signature Based on Conventional Algorithms

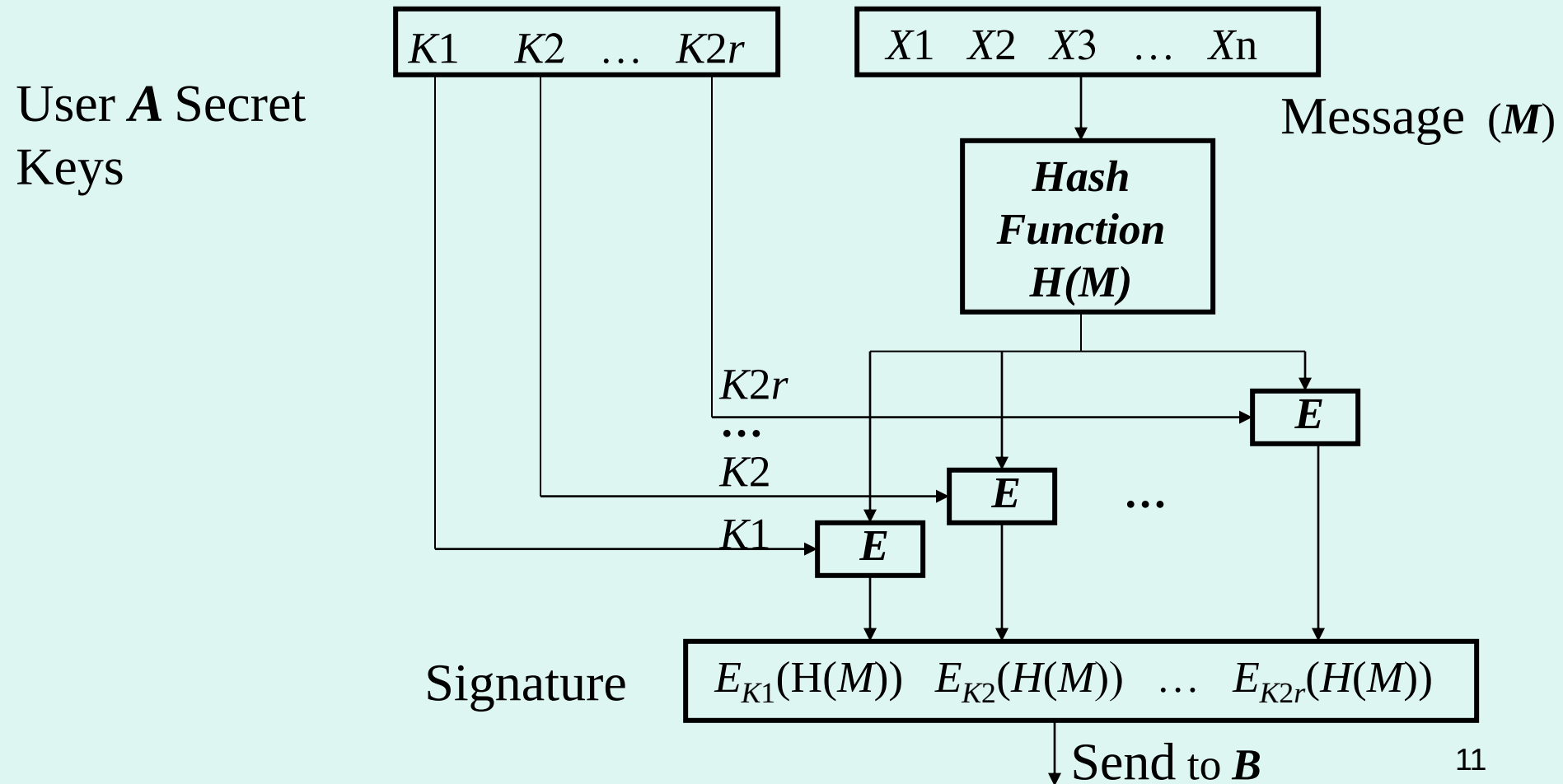
Diffie and Lamport approach

The signature is validated against the 128-bit compressed encoding of the received message via the technique described in method presented above. However, the size of the signature can be reduced from 128 to 64 keys if the compressed encoding is divided into 64 2-bit segment instead of 128 1-bit segment, and if the keys are arranged in a 4 by 64 table instead of 2 by 128 table.

Digital Signature

Rabin's Digital Signature Algorithms

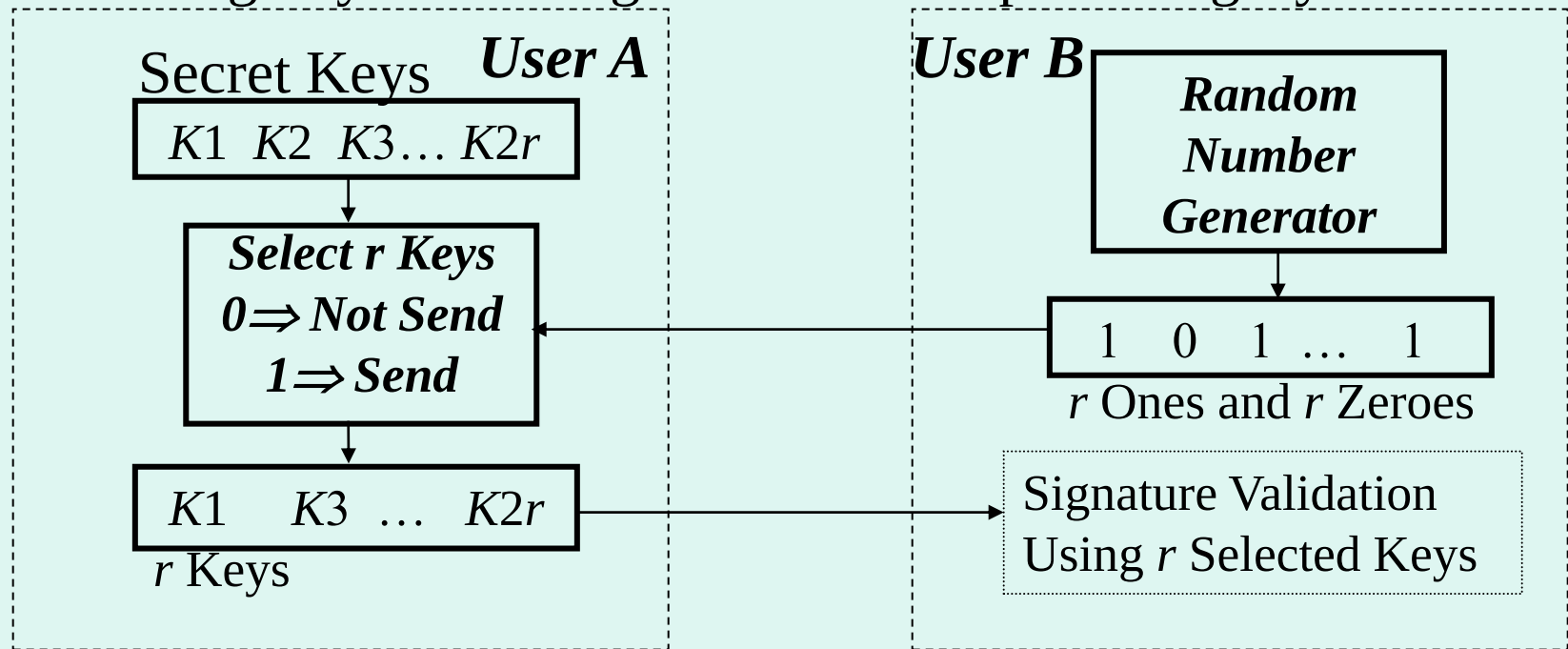
Rabin's approach, is based upon a conventional algorithm such as DES. A digital signature composed of a list of secret cryptographic keys. The sender randomly generates $2r$ cryptographic keys $K_1, K_2, \dots, K_{2r}$.



Digital Signature

Rabin's Digital Signature Algorithms

The receiver is given, in advance, the corresponding validation parameters $U1, U2, \dots, U_{2r-1}, U_{2r}$ and $E_{K1}(U1), E_{K2}(U2), \dots, E_{K_{2r-1}}(U_{2r-1}), E_{K_{2r}}(U_{2r})$, which have also been recorded by the sender in an established registry with recognized and accepted integrity.



Signature Creation according to the Rabin method

The receiver randomly generates a vector of numbers, in which there are exactly r ones and r zeros, and sends a copy to the sender. Once the r keys have been received, the signature is validated by ensuring that they confirm to the validation parameters.

Digital Signature

Matyas and Meyer Digital Signature Algorithms

1. Preparation phase

Matyas and Meyer digital signature is a list of 31 cryptographic keys that are selected from 31×31 matrix of secret keys

Initial Keys	$k(1,1)$	$k(1,2)$	...	$k(1,31)$
	$k(2,1)$	$k(2,2)$	...	$k(2,31)$

			...	
Final Keys	$k(31,1)$	$k(31,2)$	...	$k(31,31)$

Row 1 of the matrix contains 31 initial **secret** keys, denoted $k(1,1), k(1,2), \dots, k(1,31)$ which are produced via a standard generator of pseudo-random numbers. For example, the initial keys for the n th message could be produced with the DES algorithm using a single **secret key** K , via the relation

$$k(1,j) = E_K(31(n-1) + j),$$

where j is column number, and n message number.

Digital Signature

Matyas and Meyer Digital Signature Algorithms

In addition to the key matrix, the following 30×31 matrix of ***nonsecret code words*** is produced via a generator of pseudo-random numbers, according to the relation

$$u(i,j) = E_U(31^2(n-1) + 31(i-1) + j),$$

where i is row number, j – column number, n message number, and U is ***nonsecret seed key***.

$$\begin{array}{cccc} u(1,1) & u(1,2) & \dots & u(1,31) \\ u(2,1) & u(2,2) & \dots & u(2,31) \\ & & \dots & \\ u(30,1) & u(30,2) & \dots & u(30,31) \end{array}$$

Then, the keys in row 2 through 31 are obtained via repeated steps of encipherment using initial keys

$$k(i+1,j) = E_{k(i,j)}(u(i,j)), \quad i=1,2,\dots,30, \quad j=1,2,\dots,31.$$

Digital Signature

Matyas and Meyer Digital Signature Algorithms

The key in row 31 of the matrix are called the *final keys*. These final keys, which are prepared in advance by the sender and sent to the receiver, represent the *validation pattern*. *Nonsecret keys* $a_1, a_2, \dots, a_{31}$ could be established universal constant that are recorded in a public registry.

2. Signature generation

The first step in forming the signature is to obtain the compressed encoding of the message M that is to be signed. For the purposes of illustration, assume that the Hash function has the value $H(M)=(H_1, H_2)$, where H_1 and H_2 are 64-bit quantities. The values of H_1 and H_2 is used together with 31 nonsecret keys $a_1, a_2, \dots, a_{31}$ to produce 31 unique code words, $b_1, b_2, \dots, b_{31}$, as follows: $E_{a_1}(H_1) \oplus E_{a_1}(H_2) = b_1$, $E_{a_2}(H_1) \oplus E_{a_2}(H_2) = b_2, \dots, E_{a_{31}}(H_1) \oplus E_{a_{31}}(H_2) = b_{31}$. The 31 b -values are now sorted into numerical sequence

b_1	b_2	b_3	$\dots$	b_{31}	<i>Unsorted</i>
b_5	b_{20}	b_{11}	$\dots$	b_7	<i>Sorted</i>

Digital Signature

Matyas and Meyer Digital Signature Algorithms

Then the signature is a sequence of keys $k(5,1)$, $k(20,2)$, $k(11,3), \dots, k(7,31)$, what guarantees that one and only one key is selected from each row and column.

3. Signature Verification

The receiver, except for one additional step, checks the signature by repeating the same steps performed by the sender. First, the compressed encoding of the message is obtained. Then the 31 **b**-values are computed and sorted into numerical order. This allows the 31 keys in the signature vector to be reinserted into empty 31×31 key matrix. The receiver then uses each signature key to encipher the standard, nonsecret code word to form each lower key in the same column of the matrix, including the final keys that make up the validation pattern. If the final keys (row 31) of the key matrix are equal to the validation pattern previously received from the sender, then the message and signature are accepted as valid; otherwise, the message and signature are rejected.

Теория Информации

В.Н. Ярмолик

Лекция 18

2017

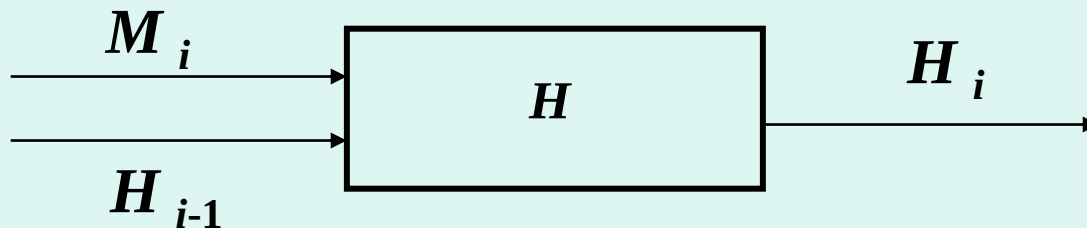
Digital Signature

Hash functions

Hash function $H(M)$ has the message M to be encrypted as argument. The length of the message have to be an arbitrary, as well as the size of the hash function is fixed by m bits. It is important to emphasize that function $H(M)$ is one way function, it means that recovering of plaintext based on the value of the function is impossible.

Most hash functions is calculated based on the following recurrence relation

$$H_i = H(M_i, H_{i-1})$$



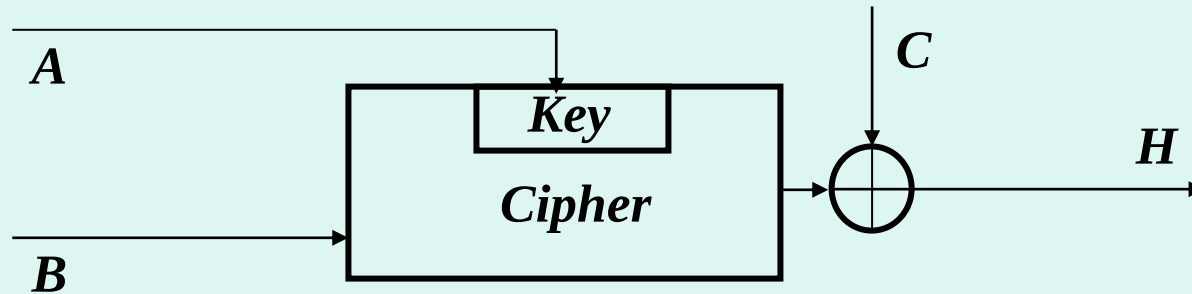
where M_i is consecutive block of the message.

For the case, when the length of hash value H_i and message block are equal, then the following diagram for calculation can be applied.

Digital Signature

Hash functions

For the case, when the length of hash value H_i and message block are equal, then the following diagram for calculation can be applied.



H_0 -IV – Initial Vector; $H_i = E_A(B) \oplus C$, where A , B , and C can take the following meaning M_i , H_{i-1} and $(M_i \oplus H_{i-1})$ or be a constant values. where M_i is consecutive block of the message.

Digital Signature

Hash functions

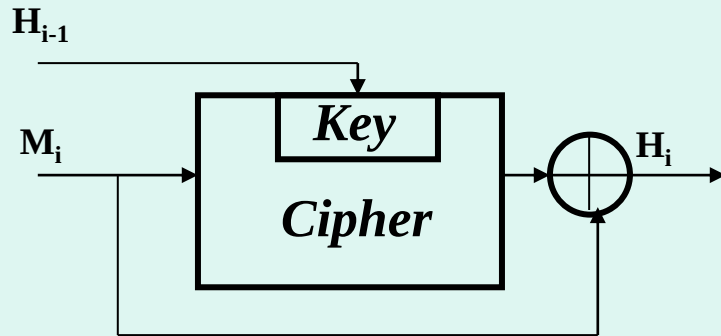
There are 64 possible implementation of the Hash function according to the last block diagram. Some of them are shown below.

1	$H_i = E_{H_{i-1}} (M_i) \oplus M_i$
2	$H_i = E_{H_{i-1}} (M_i \oplus H_{i-1}) \oplus M_i \oplus H_{i-1}$
3	$H_i = E_{H_{i-1}} (M_i) \oplus H_{i-1} \oplus M_i$
4	$H_i = E_{H_{i-1}} (M_i \oplus H_{i-1}) \oplus M_i$
5	$H_i = E_{M_i} (H_{i-1}) \oplus H_{i-1}$
6	$H_i = E_{M_i} (M_i \oplus H_{i-1}) M_i \oplus H_{i-1}$
7	$H_i = E_{M_i} (H_{i-1}) M_i \oplus H_{i-1}$
8	$H_i = E_{M_i} (M_i \oplus H_{i-1}) \oplus H_{i-1}$
9	$H_i = E_{M_i \oplus H_{i-1}} (M_i) \oplus M_i$
10	$H_i = E_{M_i \oplus H_{i-1}} (H_{i-1}) \oplus H_{i-1}$
11	$H_i = E_{M_i \oplus H_{i-1}} (M_i) \oplus H_{i-1}$
12	$H_i = E_{M_i \oplus H_{i-1}} (H_{i-1}) \oplus M_i$

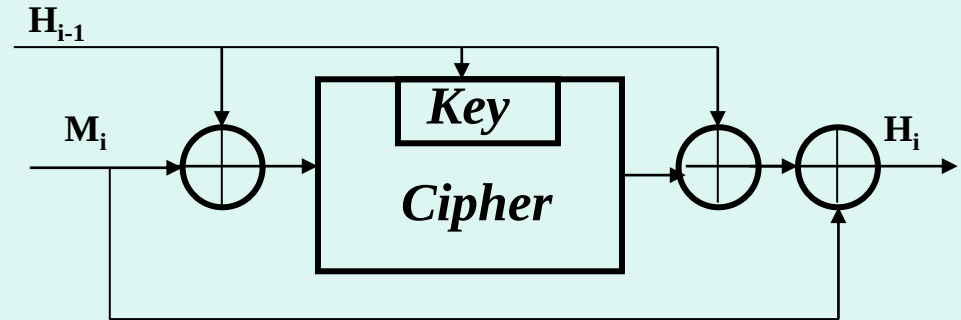
Digital Signature

Hash functions

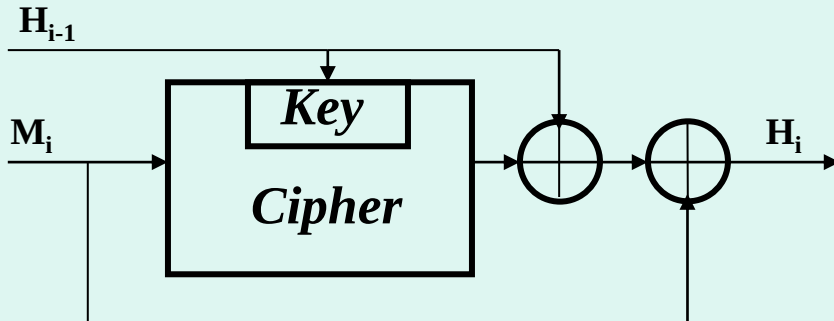
The most often used hash functions.



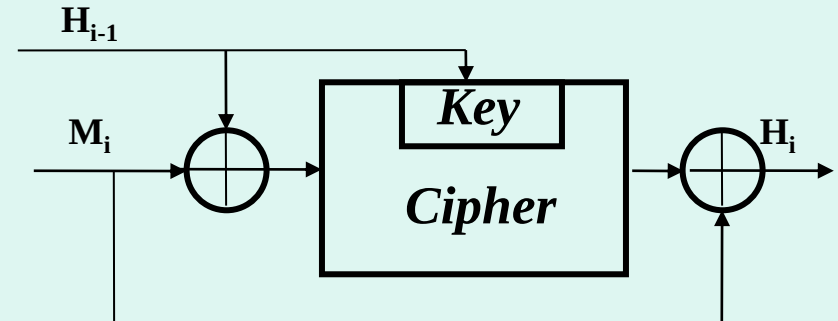
(1)



(2)



(3)



(4)

Digital Signature

Hash functions SHA

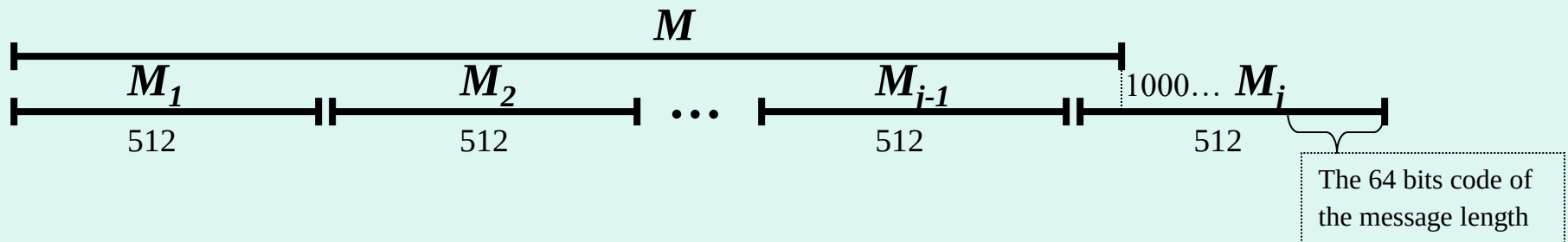
Secure Hash Algorithm (SHA) have been developed within the framework of developing **Secure Hash Standard** in 1992. This secure hash algorithm is an integral part of the **Digital Signature Algorithm (DSA)**.

The size of the message M to be encrypted have to be less then 2^{64} .

Hash function $H(M)$ is called **Message Digest (MD)** and has the size $m=160$ bits.

Determination of the $H(M)$ according to the Secure Hash Algorithm has the following steps:

1. The message M is divided into the blocks M_i with the standard size – 512 bits.



2. The five 32 bits variable takes their initial value.

A=0X67452301

B=0XEFCDA89

C=0X98BADCFE

D=0X10325476

E=0XC3D2E1F0

Digital Signature

Hash functions SHA

3. The main cycle is calculated to perform 512 bit block M_i , starting from M_1 , then M_2 , ..., and up to M_j . For each block M_i the standard procedure have to be performed. The five 32 bit variable A , B , C , D and E are copied into internal variable a , b , c , d , and e . The main cycle can be described as:

For $t=0$ **to** 79

$$\mathbf{TEMP} = (a \lll 5) \oplus f_t(b, c, d) \oplus e \oplus W_t \oplus K_t$$

$$e = d$$

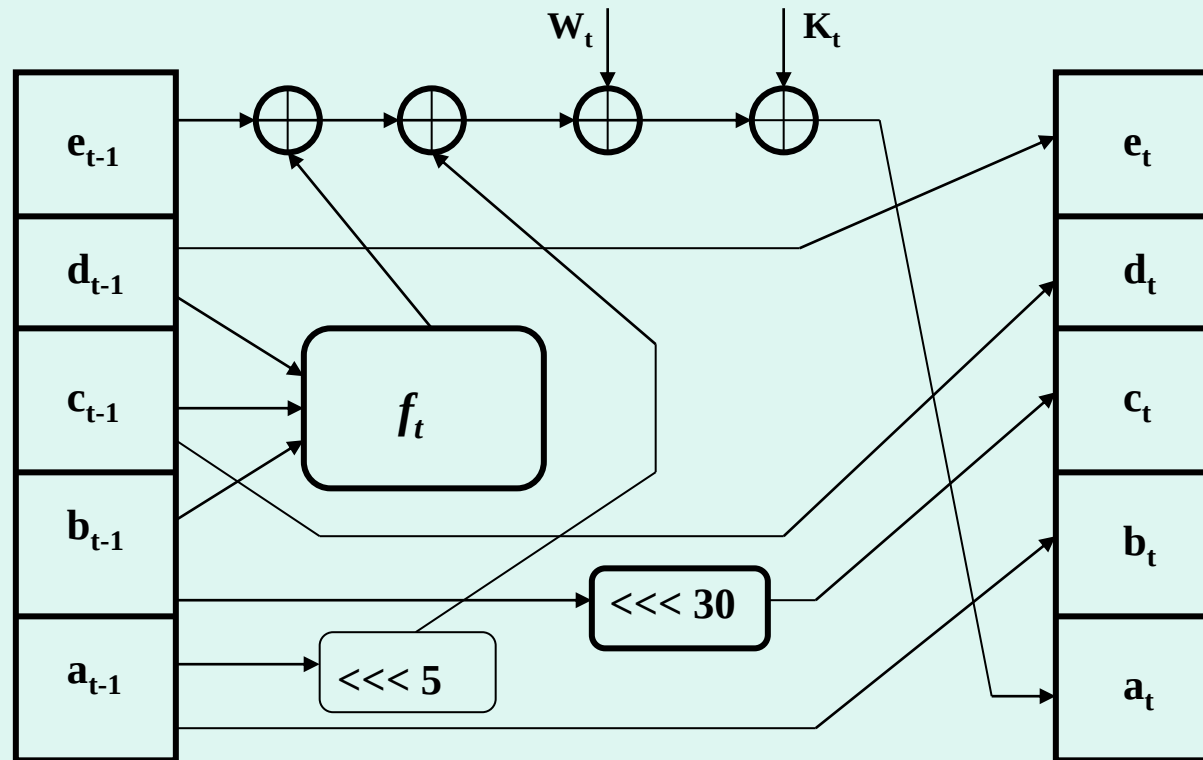
$$d = c$$

$$c = (b \lll 30)$$

$$a = \mathbf{TEMP}$$

Digital Signature

Hash functions SHA



Digital Signature

Hash functions SHA

Nonlinear Boolean functions $f_t(a,b,c)$ are:

$$f_t(a,b,c)=(a \cdot b) + (\overline{a} \cdot c); \quad t=\overline{0,19};$$

$$f_t(a,b,c)=a \oplus b \oplus c; \quad t=\overline{20,39};$$

$$f_t(a,b,c)=(a \cdot b) + (a \cdot c) + (b \cdot c); \quad t=\overline{40,59};$$

$$f_t(a,b,c)=a \oplus b \oplus c; \quad t=\overline{60,79};$$

There are four constant values $K_t=0X5A827999$,
 $K_t=0X6ED9EBA1$, $K_t=0X8F1BBCDC$, $K_t=0XCA62C1D6$ for the same
values of t as in a case of Boolean functions.

Digital Signature

Hash functions SHA

Block M_i consisting from sixteen 32 bit word ($M_0, M_1, \dots, M_{15}$) are converted into eighty 32-bit words ($W_0, W_1, \dots, W_{79}$), according to the relations

$$W_t = M_t; \quad t = \overline{0, 15};$$

$$W_t = (W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}) \lll 1; \quad t = \overline{16, 79};$$

where $\lll s$, is a cyclic shift to the left on s positions.

4. At the end of main cycle the values of variables a, b, c, d , and e are added to the values of corresponding variables A, B, C, D , and E .

The final values is generated as a concatenation of variables A, B, C, D , and E .

Теория Информации

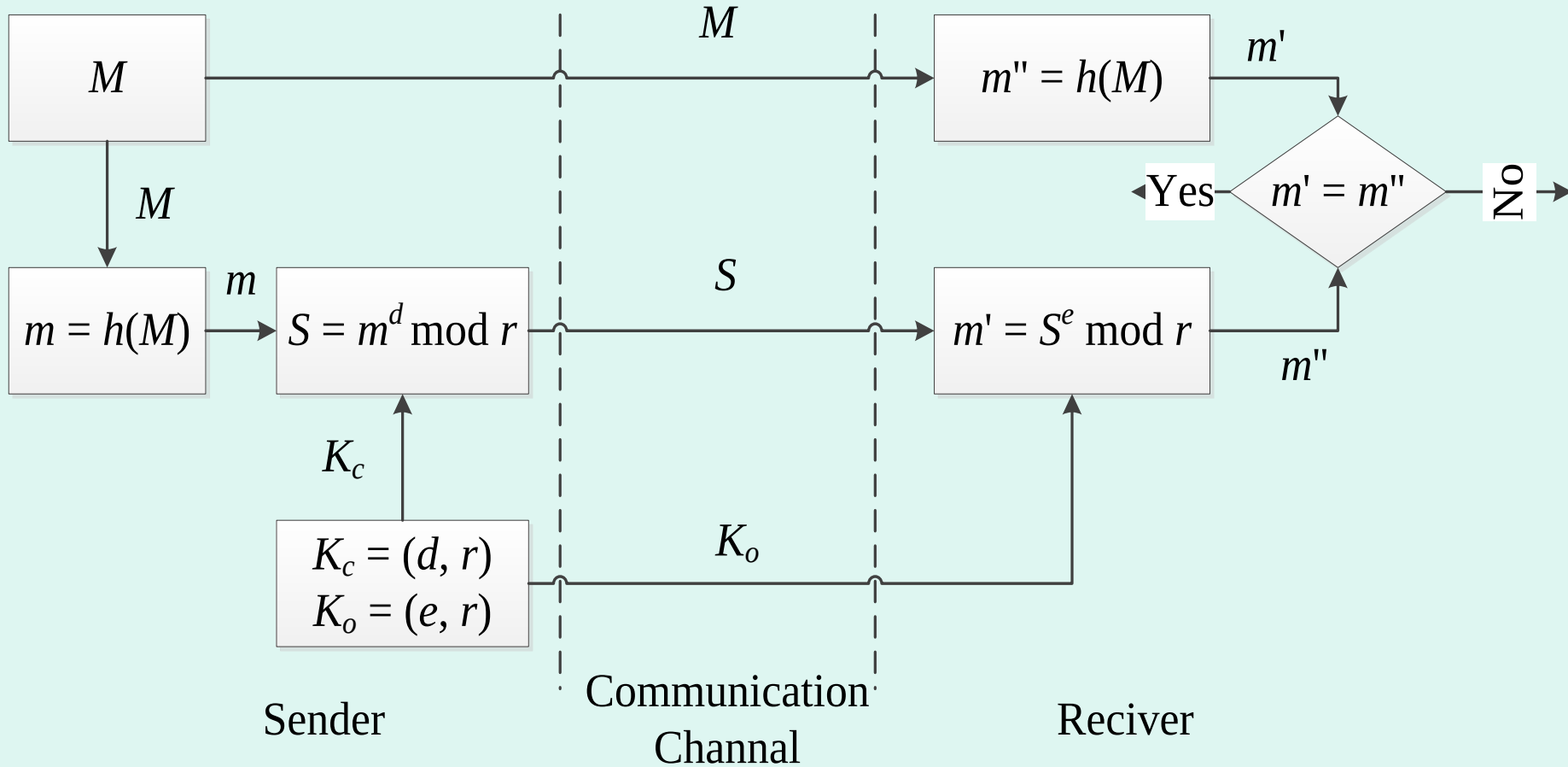
В.Н. Ярмолик

Лекция 19

2017

Digital Signature

RSA signature scheme



Digital Signature

ElGamal signature scheme

The *ElGamal signature scheme* is a digital signature scheme which is based on the difficulty of computing discrete logarithms. It was described by **Taher ElGamal** in 1984.

One of the variant developed at NSA and known as the Digital Signature Algorithm (DSA) is much more widely used. There are several other variants.

1. Preparation phase

1. $H(M)$ a collision-resistant hash function for message compression have been chosen.

2. P is a large prime such that computing discrete logarithms modulo P is difficult.

3. $\alpha < P$ is a randomly chosen primitive element (generator) of the multiplicative group of integers modulo P .

All these parameters have to be shared between users.

Digital Signature

ElGamal signature scheme

1. Preparation phase cont.

Key generation

4. Randomly select a secret key X with $1 < X < P-1$.
5. Compute $Y = \alpha^X \bmod P$.

Public key is (P, α, Y) .

Secret key is X .

2. Signature generation (Signing)

To sign a message M the signer performs the following steps.

1. Compress the message $H(M)$ to get the Hash $m = H(M)$.
2. Choose a random K such that $0 < K < P-1$ and $(K, P-1) = 1$.
3. Compute $W = \alpha^K \bmod P$.
4. Compute $KV = (m - XW) \bmod (P-1)$.

If $V=0$ start again.

Then the pair (W, V) is the digital signature of M . The signer repeats these steps for every signature.

Digital Signature

ElGamal signature scheme

3. Signature verification

The receiver get the message M with the signature (W,V) . Then the message M is verified as follows.

1. Generate the hash value $H(M)=m$.
2. Compare $0 < W < P$ and $0 < V < P-1$.
3. Check the equality $\alpha^m = Y^W W^V \bmod P$.

The verifier accepts a signature if all conditions are satisfied and rejects it otherwise.

$$\begin{aligned}\alpha^m &= Y^W W^V \bmod P \\ &= \alpha^{XW} \alpha^{KV} \bmod P. \\ \alpha^{m-XW-KV} &= 1 \bmod P\end{aligned}$$

Digital Signature

Digital Signature Algorithm

The ***Digital Signature Algorithm*** (DSA) is a United States Federal Government standard for digital signatures. It was proposed by the National Institute of Standards and Technology (NIST) in August 1991 for use in their ***Digital Signature Standard (DSS)***, and adopted in 1993. A minor revision was issued in 1996, in 2000 and again in 2009.

DSA is covered by U.S. Patent 5,231,668, filed July 26, 1991, and attributed to David W. Kravitz, a former NSA employee. This patent was given to "The United States of America as represented by the Secretary of Commerce, Washington, D.C." and the NIST has made this patent available worldwide royalty-free.

Digital Signature

Digital Signature Algorithm

1. Preparation phase

Preparation phase has two steps. The ***first step*** is a choice of ***algorithm parameters*** which may be shared between different users of the system:

1. $H(M)$ a collision-resistant hash function for message compression have been chosen. Usually it is *SHA*.
2. Specify L and N length pairs of (1024,160), (2048,224), (2048,256), and (3072,256).
3. Choose an N -bit prime q . N must be less than or equal to the hash output length, for example 160 bit.
4. Generate P is a large prime such that $P-1$ is a multiple of q . Computing discrete logarithms modulo P is difficult.
5. Choose G , a number whose multiplicative order modulo P is q . This may be done by setting $G = h^{(P-1)/q} \bmod P$ for some arbitrary h ($1 < h < P-1$), and trying again with a different h if the result comes out as 1. Most choices of h will lead to a usable G ; commonly $h=2$ is used.

The ***algorithm parameters*** (P, G, q) is the public values.

Digital Signature

Digital Signature Algorithm

1. *Preparation phase cont.*

The second step computes private and public keys for user:

6. Choose the private key X by some random method, where $0 < X < q$.

7. Calculate the public key $Y = G^X \bmod P$.

Public values and ***key*** are (P, G, q, Y) .

Private key is X .

Digital Signature

Digital Signature Algorithm

2. Signature generation (Signing)

To sign a message M the signer performs the following steps.

1. Sender determine the Hash function $m=H(M)$ on electronic documents M , where $1 < m < q$.
2. Sender randomly generate uniformly distributed no correlated K such that $0 < K < q$.
3. Calculate $r = (G^K \bmod P) \bmod q$.
4. Then with the help of the secret key X sender calculates the integer number S : **$S = [(m + r \cdot X) / K] \bmod q$** .

Recalculate the signature in the unlikely case that $r = 0$ or $S = 0$

The pair of numbers r and S creates the Digital signature, thus the signed message represents the three elements, namely

(M, r, S) .

Digital Signature

Digital Signature Algorithm

3. Signature verification

First of all the receiver based on (M, r, S) have to:

1. Verify the inequalities $0 < r < q$, $0 < S < q$. In a case negative result of the verification procedure the digital signature is not valid.
2. Obtain the Hash value $m = h(M)$.
3. Then receiver accomplish the following calculation $w = S^{-1} \bmod q$.
4. Calculate the numbers $u_1 = (m \cdot w) \bmod q$ and $u_2 = (r \cdot w) \bmod q$.
5. Then with the help of the open key Y receiver perform the following calculations $v = ((G^{u_1} \cdot Y^{u_2}) \bmod P) \bmod q = ((G^{m/S} \cdot G^{X(r/S)}) \bmod P) \bmod q = ((G^{(m+Xr)/S}) \bmod P) \bmod q = (G^K \bmod P) \bmod q$. In a case if $v=r$ digital signature is valid.

Digital Signature

Blind Signature

Blind signature, was introduced by David Chaum, as form of digital signature in which the content of a message is closed (*blinded*) before it is signed. The resulting blind signature can be publicly verified against the original, unblinded message in the manner of a regular digital signature. Blind signatures are typically employed in privacy-related protocols where the signer and message author are different parties.

Examples include cryptographic ***election systems*** and ***digital cash*** schemes.

Digital Signature

Blind Signature

Blind signature schemes can be implemented using a number of common public key signing schemes, for instance RSA and DSA.

1. To perform such a signature, the message is first "blinded", typically by combining it in some way with a random "blinding factor".

2. The blinded message is passed to a signer, who then signs it using a standard signing algorithm.

3. The resulting message, along with the blinding factor, can be later verified against the signer's public key. In some blind signature schemes, such as RSA, it is even possible to remove the blinding factor from the signature before it is verified. In these schemes, the final output (message/signature) of the blind signature scheme is identical to that of the normal signing protocol.

Digital Signature

RSA Blind Signature

There are three party within this scheme, namely:

A the *author* of the message ***m*** to be sign;

N the *signing authority*, who have to sign the message;

B the *verifier*.

1. *Preparation phase*

The *signing authority* have to generate the public key (D, n) and secret key (E, n) , where $n=pq$.

The signer have to sign $s=m^E \bmod n$ the message ***m***, without the knowledge of ***m***.

2. *Signature generation (Signing)*

To sign a message ***m*** the following steps have to be perform.

1. The author **A** generate the random value ***r***, such that ***r*** is relatively prime to ***n***, $(r, n) = 1$.

- 2 The author **A** is calculated $r^D \bmod n$, and the resulting value $r^D \bmod n$ is used as a blinding factor.

Digital Signature

RSA Blind Signature

2. Signature generation (Signing) cont.

3. The author **A** of the message computes the product of the message m and blinding factor, $r^D \bmod n$ i.e.

$$m^* = m r^D \bmod n.$$

4. **A** sends the resulting value m^* to the signing authority **N**.

5. The signing authority **N** then calculates the blinded signature s^* as:

$$s^* = (m^*)^E \bmod n = (mr^D)^E \bmod n.$$

6. s^* is sent back by **N** to the author **A** of the message.

7. **A** can then remove the blinding factor to reveal s , the valid RSA signature of m :

$$s = s^* r^{-1} \bmod n = (mr^D)^E r^{-1} \bmod n = m^E r^{DE} r^{-1} \bmod n = m^E \bmod n.$$

Digital Signature

Proxy Signature

In 1996 *Mambo*, *Usuda* and *Okamoto* proposed the *proxy signature* scheme. The user *A*, called *original signer*, delegate her signing capability to another user, *B* called the *proxy signer*.

Proxy signer *B* can sign the message on behalf of *A*. A *verifier V* can validate signature correctness and distinguish between a normal signature and proxy signature. As the result, the verifier can be convinced of the original signer's agreement on the signed message.

Proxy signature scheme have been suggested for use in a number of application, including *electronic commerce*, *mobile agents*, *distributed shared object systems* and so on.

Digital Signature

Proxy Signature

Proxy signature scheme is based on the ***Discrete Logarithm Problem (DLP)*** for a cyclic multiplicative group G_p generated by α of order p . There are not probabilistic polynomial-time algorithm to get the value x from $\alpha^x \bmod p$.

1. Preparation phase

The following parameter have to define:

1. Public Hash function $H(M)$.
2. p is large prime number.
3. α generating element.

The ***original signer A***

1. The original signer **A** generate his private key x and calculate the public key $y = \alpha^x \bmod p$.
2. The original signer randomly generate the value k and make the calculation $r = \alpha^k \bmod p$.

Digital Signature

Proxy Signature

1. Preparation phase

The *original signer A* cont.

3. The original signer **A** calculate $s = x + kr \bmod p-1$.
2. **A** sends (r, s) to the proxy signer **B**.

The *proxy signer B*.

1. The proxy signer **B** checks the validity of (r, s) using next equality

$$\alpha^s = y r^r \bmod p$$

$$\alpha^{(x+kr) \bmod p-1} = \alpha^x (\alpha^k)^r \bmod p$$

$$\alpha^{(x+kr) - ((x+kr) \bmod (p-1))} = 1 \bmod p$$

If the equality holds, the proxy signer accepts (r, s) as valid proxy secret key. If not, the protocol stops.

Digital Signature

Proxy Signature

2. Signature generation (Signing)

1. The proxy signer **B** signs the message M representing as $m=H(M)$ based on the value $s=x+kr \bmod p-1$. As signature generation algorithm the *ElGamal* algorithm can be used.
2. After that, the proxy signer sends the message M with the signature (S_{pr}, r) to the verifier **V**.

3. Signature verification

The verifier **V** based on (M, r, S_{pr}) have to:

1. Verifier recovered the public key y_{pr} of the proxy signer according to the following calculations $y_{pr}=yr^r \bmod p=\alpha^x(\alpha^k)^r \bmod p=\alpha^s \bmod p$.

After that, the verifier **V** performs the verification phase of the normal signature scheme.

Digital Signature

Group Signature

A ***Group signature scheme*** is a method for allowing a member of a group to anonymously sign a message on behalf of the group. The concept was first introduced by ***David Chaum*** and ***Eugene van Heyst*** in 1991.

For example, a group signature scheme could be used by an employee of a large company where it is sufficient for a verifier to know a message was signed by an employee, but not which particular employee signed it.

Essential to a group signature scheme is a *group manager*, who is in charge of adding group members and has the ability to reveal the original signer in the event of disputes. In some systems the responsibilities of adding members and revoking signature anonymity are separated and given to a membership manager and revocation manager respectively.

Digital Signature

Ring Signature

A *ring signature* is a type of digital signature that can be performed by any member of a group of users that each have keys. Therefore, a message signed with a ring signature is endorsed by someone in a particular group of people. One of the security properties of a ring signature is that it should be difficult to determine *which* of the group members' keys was used to produce the signature. Ring signatures are similar to group signatures but differ in two key ways: first, there is no way to revoke the anonymity of an individual signature, and second, any group of users can be used as a group without additional setup. Ring signatures were invented by Ron Rivest, Adi Shamir and Yael Tauman, and introduced at ASIACRYPT in 2001. The name "ring signature" comes from the ring-like structure of the signature algorithm.

Теория Информации

В.Н. Ярмолик

Лекция 20

2016

Zero-Knowledge Proof

Main definitions

A know the value of p and q , where $n = pq$.

How does **A** proceed to convince **B** without revealing the factorization? In other words, is it possible for **A** to prove to **B** the assertion that it has knowledge of the factorization of n without yielding anything beyond the validity of the assertion?

Example. The proof $n = 670592745 = 12345 \times 54321$ is not a *zero-knowledge* proof that n is not a prime.

Informally. A *zero-knowledge* proof is an interactive proof protocol that provides highly convincing evidence that a statement is true or that Prover has certain knowledge (of a secret) and that Prover knows a (standard) proof of it while providing not a single bit of information about the proof (knowledge or secret). (In particular, Verifier who got convinced about the correctness of a statement cannot convince the third person about that.)

Zero-Knowledge Proof

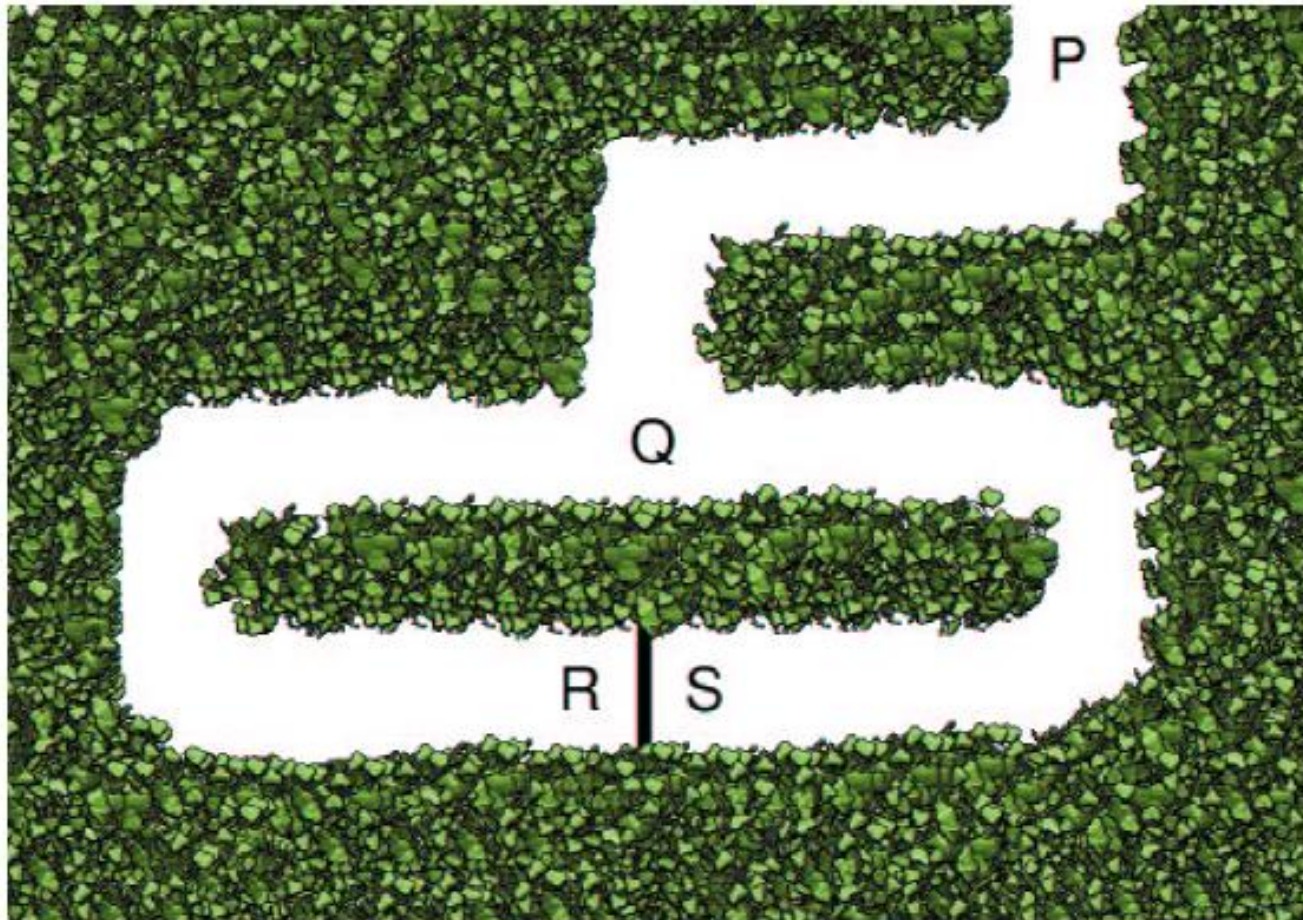
Main definitions

More formally A *zero-knowledge* proof of a theorem T is an interactive two party protocol, in which **Prover** is able to convince **Verifier** who follows the same protocol, by the overwhelming statistical evidence, that T is true, if T is indeed true, but no **Prover** is not able to convince **Verifier** that T is true, if this is not so. In additions, during interactions, **Prover** does not reveal to **Verifier** any other information, except whether T is true or not. Consequently, whatever **Verifier** can do after he gets convinced, he can do just believing that T is true.

Similar arguments hold for the case **Prover** possesses a secret.

Zero-Knowledge Proof

Main definitions



The Ali Baba's Cave.

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Feige-Fiat-Shamir Identification Protocol

In the following protocol, a *prover A*, who wants to prove knowledge of secret S_A to a verifier **B**. Also, the acronym *TTP* will mean a *Trusted Third Party*.

Setup stage:

1. In the scheme, *TTP* published a module n which is the product of two large primes p and q of the form $4r+3$ for which the $x^2 \bmod n$ has solutions. Also, a parameter $\alpha \in \mathbb{N}$ is chosen.
2. **A** and **B** select, respectively, a secret $S_A, S_B \leq n-1$. Then $T_A = S_A^2 \pmod n$, and $T_B = S_B^2 \pmod n$, are computed for **A** and **B**, respectively by the *TTP*. They then register T_A , and T_B with the *TTP* as public keys.

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Zero-Knowledge Proof Protocol:

1. **A** selects an $m \in \mathbb{N}$, called a *commitment* such that $m \leq n-1$ and sends $w = m^2 \bmod n$, called the *witness*, to **B**.
2. **B** chooses $c \in \{0,1\}$, called a *challenge*, and sends it to **A**.
3. **A** computes $r = mS_A^c \bmod n$, called the *response*, and sends it to **B**.
4. **B** computes $r^2 \bmod n$. If $r^2 = wT_A^c \bmod n$, then reset α 's value to $\alpha-1$ and go to step 1 if $\alpha > 0$. If $\alpha = 0$, then terminate the protocol with **B** accepting the proof. If $r^2 \neq wT_A^c \bmod n$, then terminate the protocol with **B** rejecting the proof.

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Suppose that an enemy E tries to impersonate A . Then E could cheat by selecting any $m \leq n-1$ and sending the witness $w = m^2 \bmod n$, to B . If B then sends the challenge $c=0$, E would send the correct answer $r=m$, which passes step 4. However, if $c=1$ is sent as a challenge, then E , who does not know the square root S_A of T_A , cannot respond correctly. Hence, E has a probability $1/2$ of escaping detection in the first iteration, $1/4$ in the second, and so on. To diminish this kind of arbitrary cheating, to an acceptably small probability of $2^{-\alpha}$ one performs iterations of the protocol for sufficiently high value of α . Thus, the Feige-Fiat-Shamir Protocol is a Zero-Knowledge Proof of Knowledge of a modular square root of T_A , namely the secret S_A .

Zero-Knowledge Proof

Main definitions

Zero-Knowledge Proof

Motivation for the topics in this section may be derived from the age-old cryptography problem of supplying mutually mistrustful entities with a means of interchanging messages, where each entity desires to reveal as little as possible about its own secrets.

Very informally. An interactive “proof” protocol at which a ***Prover*** tries to convince a ***Verifier*** about the truth of a statement, or about possession of a knowledge, is called “***zero-knowledge***” protocol if the ***Verifier*** does not learn from communication anything more except that the statement is true or that ***Prover*** has knowledge (secret) she claims to have.

For instance, suppose that entity ***A*** wants to convince entity ***B*** that it has knowledge of the factorization of an *RSA* modulus $n=pq$. It is certainly simple to do this, since ***A*** can merely reveal p and q (an instance of what is called a ***maximum disclosure proof***).

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Example:

Setup stage:

1. An RSA modulus $n=pq=5\times 7=35$ is chosen by the *TTP* and made public, but the $p=5$ and $q=7$ are kept secret. Also, a parameter $\alpha=2$ is chosen.

2. **A** select, respectively, a secret $S_A=11$. Then *TTP* computes for **A** $T_A=S_A^2 \pmod n=11^2 \pmod{35}=16$. They then register $T_A=16$ with the *TTP* as their public keys.

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Zero-Knowledge Proof Protocol:

1. **A** selects an $m=3$, called a *commitment* such that $m=3 \leq n-1=34$ and sends $w=m^2 \bmod n=3^2 \bmod 35=9$ called the *witness*, to **B**.
2. **B** chooses $c=0$ called a *challenge*, and sends it to **A**.
3. **A** computes $r=mS_A^c(\bmod n)=3 \times 11^0 \bmod 35 = 3$ called the *response*, and sends it to **B**.
4. **B** computes $r^2(\bmod n)=3^2 \bmod 35=9$. As the result $wT_A^c(\bmod n)=9 \times 16^0 \bmod 35=r^2=9$, then reset α 's value to $\alpha-1=2-1=1$ and go to step 1 if $\alpha=1>0$.

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Zero-Knowledge Proof Protocol:

1. **A** selects an $m=23$, called a *commitment* such that $m=23 \leq n-1=34$ and sends $w=m^2 \bmod n=23^2 \bmod 35=4$ called the *witness*, to **B**.
2. **B** chooses $c=1$ called a *challenge*, and sends it to **A**.
3. **A** computes $r=mS_A^c(\bmod n)=23 \times 11^1 \bmod 35=8$ called the *response*, and sends it to **B**.
4. **B** computes $r^2(\bmod n)=8^2 \bmod 35=29$. As the result $wT_A^c(\bmod n)=4 \times 16^1 \bmod 35=29=r^2=29$, then reset α 's value to $\alpha-1=1-1=0$. If $\alpha=0$, then terminate the protocol with **B** accepting the proof.

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Generalization

This protocol is representative of a general structure for a larger class of what are called ***Three-move, Zero-knowledge Protocols***. In such protocols, **A** sends a *witness*, **B** replies with a *challenge*, and **A** replies with a *response*. One iteration of these “moves” is called a *round*. There is a hidden randomization factor built into the protocol when *prover A* selects a *private commitment* from some pre-determined set. Then **A** computes the associated (*public*) *witness*. This initial randomness guarantees that the iterations in the protocol are sequential and independent, namely that there is a variation from one protocol round to another. Also, this establishes a set of “*questions*”, which **A** lays claim to being able to answer, so naturally **A**’s subsequent responses are restricted. The given design of the protocol means that only the *prover A* with knowledge of the secret S_A is capable of answering all of the challenges with correct responses, none of which provide information about S_A .

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Generalization

The process by which **A** determines the set of questions and **B** chooses the questions, (1)-(2) in the Feige-Fiat-Shamir Protocol for example, is an instance of what is called a ***Cut and Choose Protocol***. This terminology arises from the classic technique for dividing anything fairly between two people, namely **A** cuts the object (say a piece of fruit) in “half”, and **B** chooses one of the halves, leaving the other half for **A**. Also, the process involved in (2)-(3) of Feige-Fiat-Shamir Protocol, given a witness from (1), is an instance of ***Challenge-Response Protocol***. In this type of protocol it is essential, for security reasons, that **A** responds to at most one challenge for a given witness, and should never reuse a witness.

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Limitation

The main restriction of the previous algorithm is two requirements for selection the values of the public keys T_A and T_B .

The values T_A and T_B have to satisfy to the equation $x^2 = T_j \bmod n$.

There is an inverse $T_j^{-1} \bmod n$ value for T_j .

Let us examine the following the case for $n=35$.

All possible values of T_j are $\{1,4,9,11,14,15,16,21,25,29,30\}$, It should be mentioned that values $T_j \in \{14,15,21,25,30\}$ have not the inverse values.

Zero-Knowledge Proof

Feige-Fiat-Shamir Identification Protocol

Extension

*An extension of the algorithm is the application by user **A** of several public as well as secret keys.*

*Then the user **A** have to calculate*

*$r = m \times (S_{A1}^{c1} \times S_{A2}^{c2} \times S_{A3}^{c3} \times \dots \times S_{Ak}^{ck}) \pmod{n}$, based on k secret keys as well as random bit string $c1, c2, c3, \dots, ck$ called the *response*, and sends it to **B**.*

*The user **B** calculates*

$r^2 \pmod{n}$ and $w \times (T_{A1}^{c1} \times T_{A2}^{c2} \times T_{A3}^{c3} \times \dots \times T_{Ak}^{ck}) \pmod{n}$, then makes the comparison

$$r^2 \pmod{n} = w \times (T_{A1}^{c1} \times T_{A2}^{c2} \times T_{A3}^{c3} \times \dots \times T_{Ak}^{ck}) \pmod{n}.$$

Zero-Knowledge Proof

Guillou-Quisquater Identification Protocol

Let **A** is a smart card, which have to prove their validity to the verifier **B**. Identification of the **A**'s represents the bit string I –public key for the given card. *Open parameters* are: $n=pq$, where p and q are *secret*, and the value V , along with I . *Secret value* is G , which satisfies to the equality

$$I \times G^V = 1 \bmod n$$

The smart card A have to prove that it knows the meaning of G .

Protocol:

Smart card A generates random integer number r , where $1 < r \leq n-1$.

1. Then calculate

$$T = r^V \bmod n,$$

and sends it to **B**.

Zero-Knowledge Proof

Guillou-Quisquater Identification Protocol

Verifier **B** generates random integer number d , where $1 < d \leq n-1$, and sends it to **A**.

Smart card **A** calculates

$$D = r \times G^d \mod n,$$

and sends it to verifier **B**.

Verifier **B** calculates

$$T' = D^V \times I^d \mod n,$$

Due to the following relations

$$T' = D^V \times I^d \mod n = (rG^d)^V \times I^d \mod n = r^V (I \times G^V)^d \mod n = r^V \mod n = T$$

This equality is true for the case when $I \times G^V = 1 \mod n$.

Теория Информации

В.Н. Ярмолик

Лекция 21

2016

Steganography

Main definitions

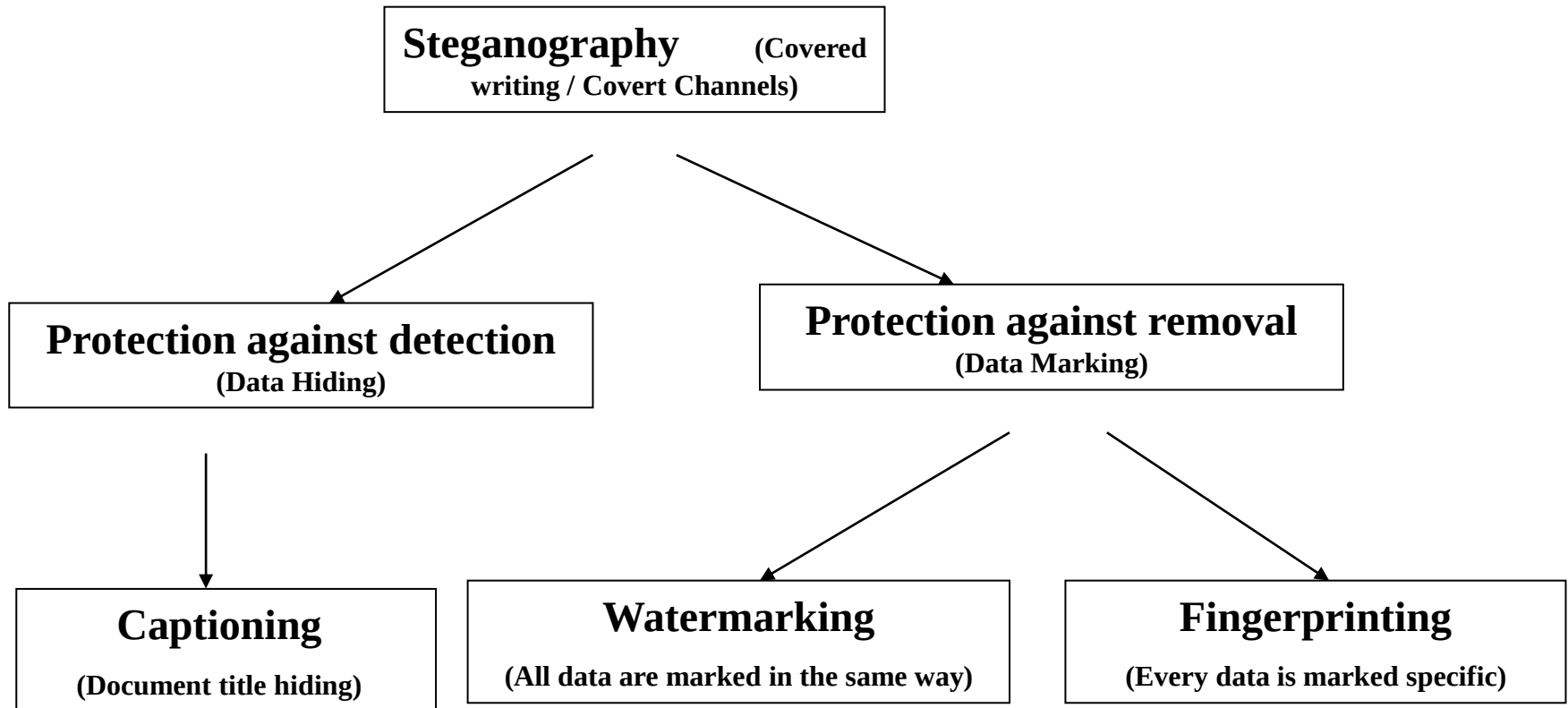
Steganography is the art and science of hiding messages inside others messages. In Greek, steganography means covered or secret writing. (***Stegano***=covered ***graphy***= writing).

Cryptography is a distant cousin of steganography. Cryptography scrambles a message so it can not be understood but there is knowledge of a message being there. While steganography's intent is to hide the existence of a hidden message.

The ***Digital watermarking*** and ***fingerprinting*** of audio and video industries are starting to recognize the possible uses of steganography.

Steganography

Main definitions

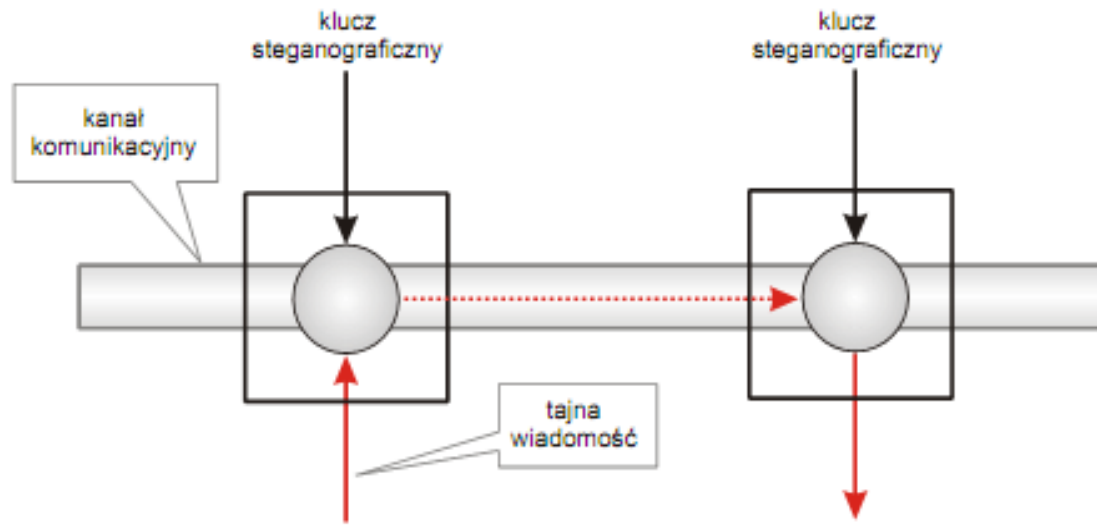


Steganography

Main definitions and terms

Cover data (*Cover, vessel* or *container*) is the term used to describe the original, innocent message, data, audio, still, video, etc.

Embedded data is the information hidden in the cover data. The **stego data** is the data containing both the cover signal and the embedded information.

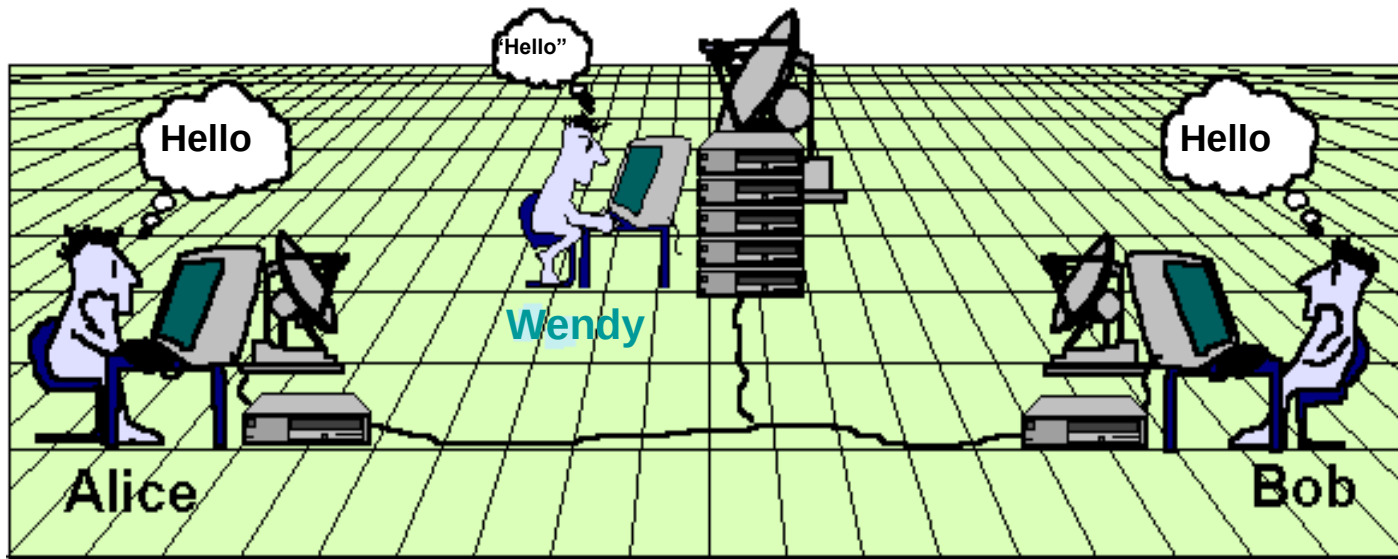


**Cover medium + embedded message + stegokey =
= stego medium;**

Stego medium + stegokey = embedded message

Steganography

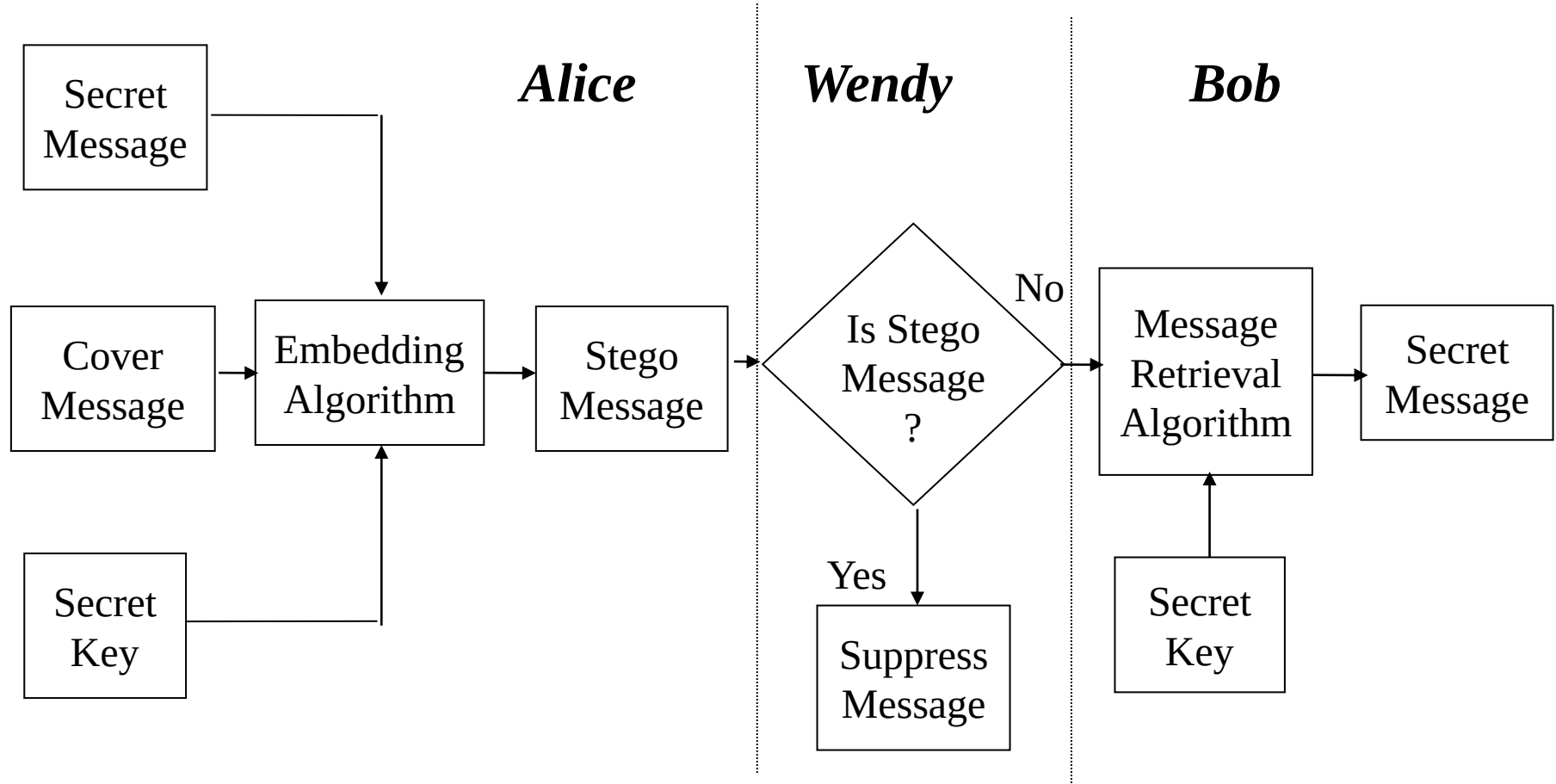
The prisoner problem



- Alice and Bob are in jail and wish to hatch an escape plan.
- Alice's and Bob's communication pass through Wendy.
- Alice's and Bob's goal is to hide their ciphertext in innocuous looking way so that Wendy will not become suspicious.
- If Wendy is a passive warden he will not do any thing to Alice's and Bob's communication.
- If Wendy is an active warden he will alter the data being sent between Alice and Bob.

Steganography

Terminology



Steganography

Main definitions and terms

Steganalyst is one who applies steganalysis in an attempt to detect the existence of hidden information. A Steganalyst is a type of cyberforensics. A few other terms to keep in mind are ***capacity*** and ***robustness***.

Capacity refers to the amount of information that can be hidden before distortion, or the amount of information that can be inputted into audio with out eavesdroppers to detect.

Robustness is the amount of modification the cover medium can withstand before the hidden information is destroyed.

Steganography

History

The earliest records of steganography were recorded by the Greek historian *Herodotus*. When the Greek tyrant Histiaeus was held prisoner by King Darius in Susa **during the 5th century**, he had to send a secret message to his son-in law Aristagoras in Miletus. Histiaeus shaved the head of a slave and **tattooed a message on his scalp**. When the slave's hair had grown long he was dispatched to Miletus.

Another historical example comes from *Herodotus* as well. **During the Greek era** they used wax tablets to write text on. *Demeratus*, a Greek, needed to notify Sparta that Xerxes intended to invade Greece. To avoid capture, he scraped the wax off of the tablets and **wrote the message on the underlying wood**. He covered the tablet with wax again and it would appear to be blank and unused.

Steganography

History

In Ancient Roman, they used invisible ink to write between already written texts. The invisible ink there would use is substances like fruit juices, urine and milk. These substances would darken, and become legible when heated. (Sellars, 2001)

In *the 12th century*, Saladin, a Muslim leader who fought the Crusaders, often used codes written in Arabic to pass messages to followers. Certain uses of the religious writing from the Koran, which is Islam's holy book. Using material derived from the Koran seems to be harmless religious writing, yet holds orders for terrorists. The same use of the Koran is being researched today on the Internet.

Steganography

History

Kahn tells of a trick used in China of embedding a code ideogram at a prearranged position in a dispatch; a similar idea led to the grille system used in Europe, where a wooden template would be placed over a innocuous text, highlighting an embedded secret message.

During WWII the grille method or some variants were used by spies. In the same period, the Germans developed **microdot technology**, which prints a clear, good quality photograph shrinking it to the size of a dot.

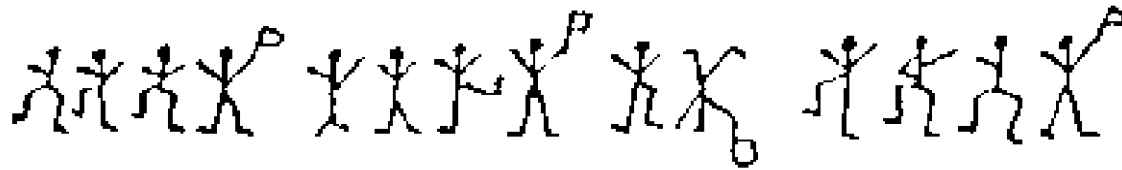
There are rumors that during the 1980's **Margareth Thatcher**, then Prime Minister in UK, became so irritated about press leaks of cabinet documents, that she had the word processors programmed to encode the identity of the writer in the word spacing, thus being able to trace the disloyal ministers.

During the "Cold War" period, **US** and **USSR** wanted to hide their sensors in the enemy's facilities. These devices had to send data to their nations, without being spotted.

Steganography

Early Steganography

Pictographs: e.g., Sherlock Holmes's Dancing Men.



“Come Here At Once”

Steganography

Null-Cipher

Message sent by a German spy during World war-I:

PRESIDENT'S EMBARGO RULING SHOULD HAVE
IMMEDIATE NOTICE. GRAVE SITUATION AFFECTING
INTERNATIONAL LAW. STATEMENT FORESHADOWS
RUIN OF MANY NEUTRALS. YELLOW JOURNALS
UNIFYING NATIONAL EXCITEMENT IMMENSELY.

PRESIDENT'S EMBARGO RULING SHOULD HAVE
IMMEDIATE NOTICE. GRAVE SITUATION AFFECTING
INTERNATIONAL LAW. STATEMENT FORESHADOWS
RUIN OF MANY NEURALS. YELLOW JOURNALS
UNIFYING NATIONAL EXCITEMENT IMMENSELY.

Pershing sails from NY June 1.

Steganography

Computer Steganography

Computer Steganography is based on two principles.

The first one is that the files containing digitized text, images or sound can be altered to a certain extent without losing their functionality.

The other principle deals with the human inability to distinguish minor changes in image color or sound quality, which is especially easy to make use of in objects that contain redundant information, be it 16-bit sound, 8-bit or even better 24-bit image. The value of the least significant bit of the pixel color won't result in any perceivable change of that color.

Steganography

Textual Staganography

Dear Friend ; We know you are interested in receiving cutting-edge announcement . If you are not interested in our publications and wish to be removed from our lists, simply do NOT respond and ignore this mail ! This mail is being sent in compliance with Senate bill 1627 ; Title 3 , Section 305 . This is NOT unsolicited bulk mail . Why work for somebody else when you can become rich within 14 days . Have you ever noticed society seems to be moving faster and faster and most everyone has a cellphone ! Well, now is your chance to capitalize on this ! WE will help YOU decrease perceived waiting time by 130% plus sell more . You can begin at absolutely no cost to you ! But don't believe us . Mr Jones who resides in Georgia tried us and says "I was skeptical but it worked for me" . We are licensed to operate in all states ! For God's sake, order now ! Sign up a friend and you'll get a discount of 90% . God Bless !

Text file with embedded message “Steganography”

(www.spammimic.com)

Steganography

Textual Staganography

A textual use of steganography can be outlined by technique called ***Brassil*** et al. *Brassil* and others are concerned of the illegal distribution of documents, that include people you are not paying royalties to original author and use documents that are have a copyright. *Brassil* designed a method of marking printable documents with a unique codeword and not visible to most. This is basically a form of ***watermarking***. The code words could be a binary number that is embedded in the document by altering particular textual features. There are 3 specific methods he designed; ***line-shift coding***, ***word-shift coding***, and ***feature coding***.

Line-shift coding is a method that moves the vertical text line making the document unique. For example one could take the original document created and move every second line 1/300 of an inch up or down. This method is more detectable than the others.

Steganography

Textual Staganography

Line-shift coding is a method that moves the vertical text line making the document unique. For example one could take the original document created and move every second line $1/300$ of an inch up or down. This method is more detectable than the others.

This is a method of altering a document by vertically shifting the locations of text lines to uniquely encode the document. This method provides the highest reliability for detection of the embedded code in images degraded by noise. To demonstrate that this technique is not visible to the casual reader, we have applied line-shift encoding to this paragraph.

Steganography

Textual Staganography

Word-shift coding embeds a codeword into a document by shifting the horizontal locations of words within text lines, while maintaining natural spacing appearance. This method is only applicable to documents with variable spacing between adjacent words. One way this technique can be used is by taking each text-line and locating the largest and smallest spaces between words. To code, the largest spacing is reduced by a certain amount and the smallest is extended by the same amount. This maintains the line length and there is little visible change.

Steganography

Textual Staganography

Now	is	the	time	for	all	men/women	to ...
Now	is	the	time	for	all	men/women	to ...

(a)

Now	is	the	time	for	all	men/women	to ...
Now	is	the	time	for	all	men/women	to ...

(b)

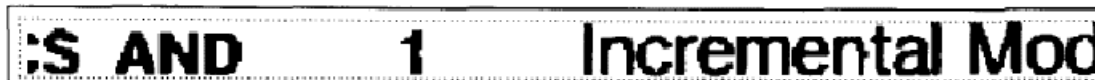
The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog.

Embedded message is 01000001:
-0,5 → 0; +0,5 → 1.

Steganography

Textual Staganography

Feature coding is *Brassils* third method of watermarking. The author would use this technique by altering certain text features. An example would be extending or shortening the upward vertical endlines of letters such as ***t, l, b, d, h***, or altering the endline lengths.



The diagram shows the text "S AND 1 Incremental Mod" within a rectangular border. The letter 't' in the word "Incremental" has a significantly longer upward vertical stroke (ascender) compared to the other letters in the word, which are shorter and aligned with the top of the word's main body.

(a)



The diagram shows the text "S AND 1 Incremental Mod" within a rectangular border. The letter 'l' in the word "Incremental" has a significantly longer upward vertical stroke (ascender) compared to the other letters in the word, which are shorter and aligned with the top of the word's main body.

(b)



The diagram shows the text "S AND 1 Incremental Mod" within a rectangular border. The letter 'b' in the word "Incremental" has a significantly longer upward vertical stroke (ascender) compared to the other letters in the word, which are shorter and aligned with the top of the word's main body.

Steganography

Lexical Staganography

Textual steganography can also be done through alternative methods, for example ***syntactic*** and ***semantic methods***. Syntactic is a method that utilizes punctuations and contradictions. Another name for Semantic is called substitution. This method of concealing hidden messages is a distant form of steganography. For example I could say *the apples in Washington are known to be the best*. When you read this email you know that the word apple means bomb and "know to be the best" refers to a major landmark. This technique is almost impossible to detect and retract the hidden message.

Steganography

Lexical Staganography

Lexical steganography is the encoding of data in blocks of text on the lexical, or word, level. For instance, pretend that you are a secret agent and that I am your faithful husband, waiting pensively at home for news that you are safe. I might receive a letter from you saying, "*Dear Keith - I am doing fine. I can't tell you where I am, because it's top secret, but be sure that I'm just dandy.*" How can I know if this is really true, or if you're being forced to write this by the Evil Enemy? Well, one way to tell would be if we had some preexisting agreement that if you start your letter with, "*I am doing fine,*" it means you're doing fine, but if you start with, "*I am doing well,*" it means you're in trouble and I'd better send James Bond in to save you.

Through this sort of method, you've encoding information without it being directly visible on the surface. Both, "*I am doing fine,*" and "*I am doing well,*" are reasonable sentences and would likely pass through the E.V.I.L. censors, but we can take advantage of this redundancy in the English language to encode information on a different level.

Steganography

Lexical Staganography

Lexical Steganography

The most straightforward subliminal channel in natural language is probably the choice of words. On the word-level, meaning is traditionally linked to the lexical relation of ***synonymy***. For example, consider the following set of covers:

$$C = \{\text{Midshire is a nice little city,}$$
$$\text{Midshire is a fine little town,}$$
$$\text{Midshire is a great little town,}$$
$$\text{Midshire is a decent little town,}$$
$$\text{Midshire is a wonderful little town}\}$$

Steganography

Lexical Staganography

We can use synonymy to encode ten states in the above sentences, since two information-carrying symbols are available, one of which can take on five values, whereas the other one can take on two values. The first is the choice for either wonderful, decent, nice, fine or great and the other is for either city or town:

$$\text{Midshire is a } \left\{ \begin{array}{c} \text{wonderful} \\ \text{decent} \\ \text{fine} \\ \text{great} \\ \text{nice} \end{array} \right\} \text{ little } \left\{ \begin{array}{c} \text{city} \\ \text{town} \end{array} \right\} .$$

We call sets of words that can be used interchangeably ***synonymy sets***.

$$\{\text{wonderful, decent, fine, great, nice}\}, \\ \{\text{city, town}\}.$$

Steganography

Lexical Staganography

A ***mixed-radix*** number is one way to encode a secret state in a sentence, if we think of state in numeric terms. Mixed-radix numbers can be written using positional notation in the following way:

$$\begin{bmatrix} \dots, & a_3, & a_2, & a_1, & a_0 \\ \dots, & b_3, & b_2, & b_1, & b_0 \end{bmatrix}$$

for $0 \leq a_i < b_i$. The numeric interpretation of a mixed radix number is

n

$$\dots + a_3 b_2 b_1 b_0 + a_2 b_1 b_0 + a_1 b_0 + a_0.$$

Given the bases of an n -digit mixed-radix number $b_{n-1}, \dots, b_2, b_1, b_0$ we can always use this numerical identity to uniquely write each number in the range from 0 to $\prod b_i - 1$ as the sequence $a_{n-1}, \dots, a_2, a_1, a_0$ where $0 \leq a_i < b_i$

Steganography

Lexical Steganography

A final category of data hiding in text involves changing the words themselves. ***Semantic*** methods are similar to the ***syntactic*** method. These methods assign two synonyms primary or secondary value. For example, the word “*big*” could be considered primary and “*large*” secondary value. When decoding, primary word will be read as ones, secondary words as zero. Word webs such as WordNet can be used to automatically generate synonym tables. For example, the choice between “*propensity*”, “*predilection*”, “*penchant*” and “*proclivity*” represents two bits of data. Counterexample “*Too*” may be replaced with “*also*” sometimes, but for the sentence “*This bed is too big*” it clearly cannot be. The reason is that adv. *too* has 2 senses and that is why belongs to two synonym sets {*Excessively, Overly, Too*} and {*Besides, Too, Also, Likewise, As well*}. This example shows that the distinct condition should be determined for the set of “***interchangeable***” words. Consider three words *XXX*, *YYY*, and *ZZZ*.⁵

Steganography

Lexical Staganography

Case 1. XXX and YYY are both single-sensed and are considered interchangeable;

Case 2. XXX and YYY are doubly-sensed, as such: $S1:\{XXX, YYY\}$; $S2:\{XXX, YYY, ZZZ\}$. We can still allow unconditional substitution between the two words.

Case 3. YYY is doubly-sensed, and YYY single-sensed: $S1:\{XXX, YYY\}$; $S2:\{XXX, ZZZ\}$. No substitution can be made here for any word.

Case 4. XXX and YYY are both single-sensed, and they are in the same synonym set $\{XXX, YYY\}$, but YYY has a space in it. We ignore these and do not allow them to be the target of substitutions.

Steganography

Lexical Staganography

The second problem associated with the naive algorithm is its inability to take advantage of the extra information-carrying capacity in regions of a synonym set beyond the highest power of 2. To take advantage of absolutely all of the information-carrying capacity in a document. The ***Ideal Coding*** can be used.

Let a document containing the word *AAA*, *MMM*, and *WWW*, where the relevant interchangeability sets involved are: {*AAA*(0), *BBB*(1), *CCC*(2)}; { *MMM*(0), *NNN*(1), *OOO*(2),*PPP*(3), *QQQ*(4)}; {*WWW*(0), *XXX*(1), *YYY*(2)}. By replacing the locations containing *AAA*,*MMM*, and *WWW* with some member of their interchangeability set, we can achieve $3 \times 5 \times 3 = 45$ different states, which enough to store 5 binary digits.

Steganography

Lexical Staganography

To mapping of these binary digits to states of the document is simple. By using the numbers tagged to each set element, can treat the document as a ***multi-base number*** – that is, its last digit signifies the “one’s place” and can run from 0 through 2, its middle digit can run from 0 to 4, and the first digit may nave the value starting from 0 to 2. The standard counting sequence in binary thus maps logically to our “counting sequence” for document stays as such:

Binary Stego-Digits	00000	00001	00010	00011	00100	...	01110	01111	...
Mixed-Base State Code	000	001	002	010	011	...	042	100	...

Steganography

Mimicry

Another wide spread lexical steganography in *mimicry*. According to this approach the real like text is generated which is described in ***Context Free Grammar (CFG)***.

Grammar CFG – is one of formal method to describe any language and consists from ***words phrases*** and ***rules***.

Mimicry design the ***binary tree*** based on particular ***CFG*** with the ***decision nodes*** and construct the text according to the ***embedded message*** bits.

Example: Embedded message is 10101 and binary tree is as the follows.

Steganography

Mimicry

Старт → существительное

существительное → Илья || Иван

глагол → поехал куда || пошел куда

куда → на работу, чтобы зачем. || домой, чтобы зачем.

зачем → забрать что || взять что

что → деньги || одежду.

Start → a noun

Noun → Ilya || Ivan

0||1

Verb → has droved where || has gone where

0||1

Where → for work, that what for || home, that what for

0||1

What for → to take away that || to take that

0||1

That → money || clothes.

0||1

Steganography

Mimicry

Decision nodes	Message bit	Result after message 1010 has been embedded
Старт	—	Старт → существительное
существительное	1	существительное → Иван
Иван глагол	0	глагол → поехал куда
Иван поехал куда	1	куда → домой, чтобы зачем
Иван поехал домой, чтобы зачем	0	зачем → забрать что
Иван поехал домой, чтобы забрать что	1	что → одежду

Теория Информации

В.Н. Ярмолик

Лекция 22

2016

Steganography

Hiding in images

Basically, any type of hidden message can be hidden in an image; text, or audio. A basic outline of the steganography process is

1. Identification of ***redundant bits*** in a ***cover medium***.

Redundant bits are those bits that can be modified without degrading the quality of the cover medium.

2. Selection of a subset of the redundant bit to be replaced with data from a secret message. The stego medium is created by replacing the selected redundant bits with message bits.

Steganography

Hiding in images

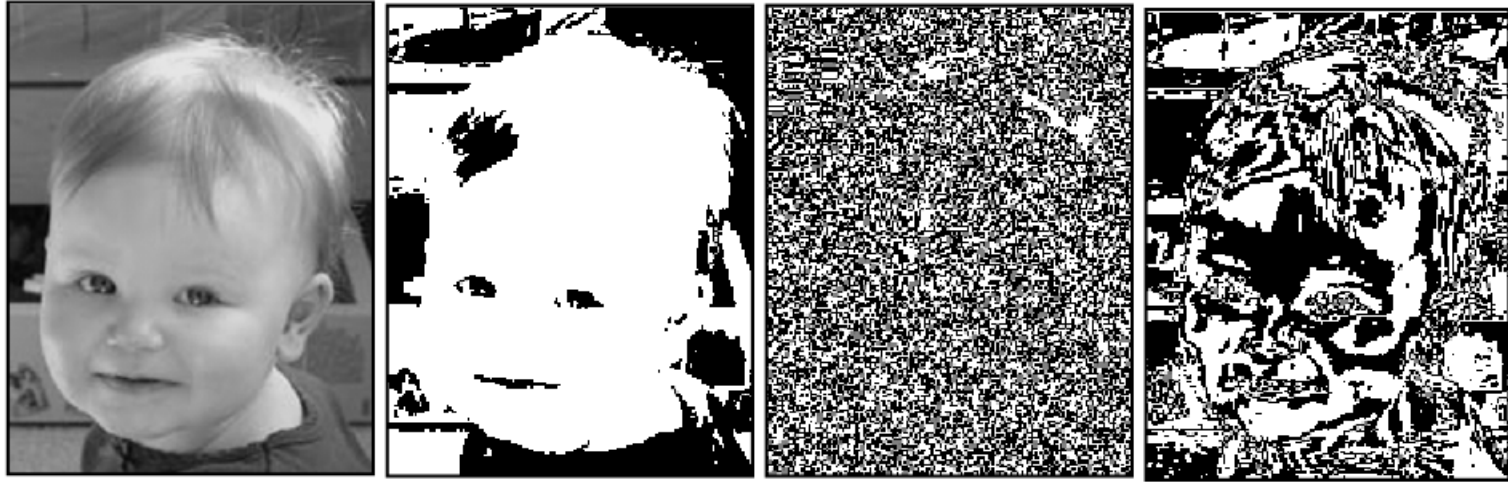
The advancement of computers is creating an incredible ability to use steganography in images and detecting that there is even a secret message is almost impossible. "To a computer, an image is an array of numbers that represent light intensities at various points, or pixels. The pixels make up the image's raster data. An image size of 640 by 480 pixels, utilizing 256 colours (8 bits per pixel) is fairly common. Such an image would contain around 300 Kilobits of data." Digital images are typically stored in either 24-bits or 8-bit per pixel file." Normally 10% of an image can be used until physical distortions can be detected.

There are three common techniques; *least significant bit (LSB) insertion*, *masking* and *filtering algorithms* and *transformations*. (Sellars, 2001)

The least significant bit insertion method is the most well known steganography technique. It is the easiest to use but the most vulnerable to attacks. If the LSB is applied to each byte of a 24-bit image, three bits can be encoded into each pixel. There are three bytes in each pixel and any changes made to the pixel bits would be unrecognizable to the human eye.

Steganography

Hiding in images



(a)

(b)

(c)

(d)

White indicates an ON bit and black an OFF bit

(a) 8-bit grayscale source image

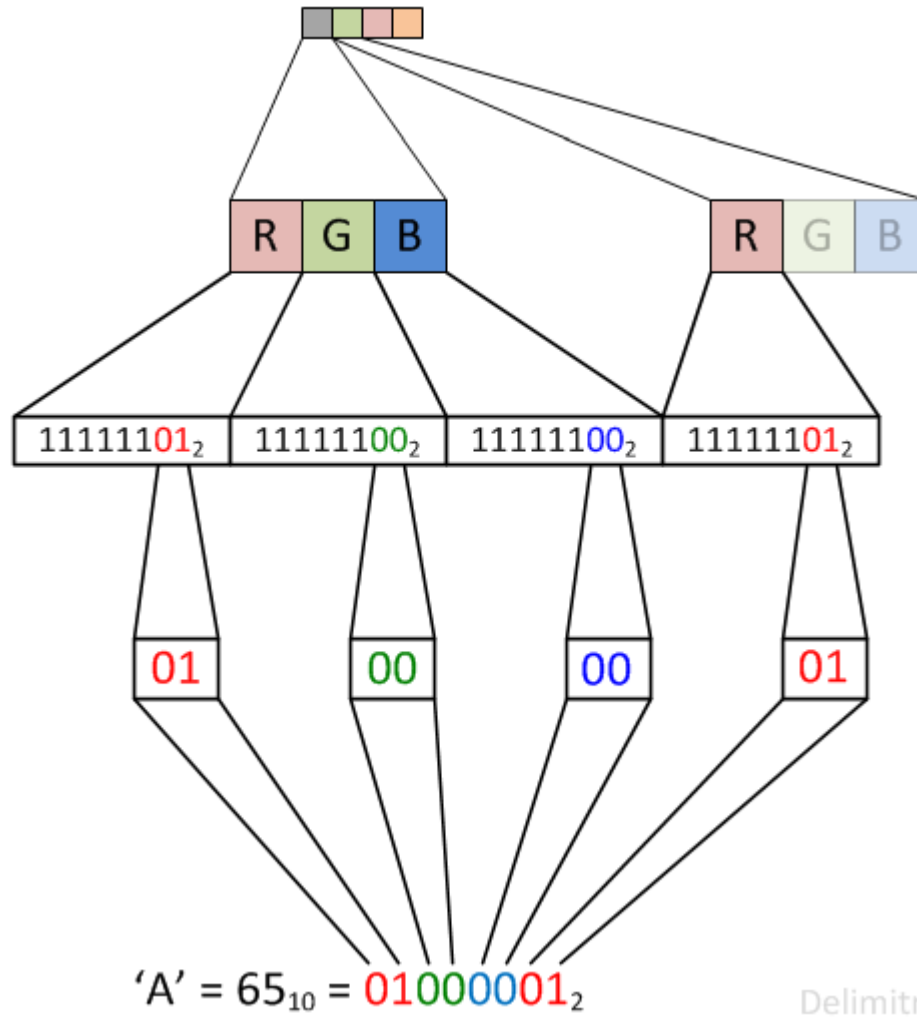
(b) Most significant bit plane (a_7) of the source

(c) Least significant bit plane (a_0) of the source

(d) Bit plane a_4 of the source image

Steganography

Hiding in images



Steganography

Hiding in images

LSB method - ***Least Significant Bit Method*** is very common method in media based steganography to cover the information as least significant bits of media vessel. For graphic media, for example grayscale graphic, when the string of the pixels is represented by bytes

10101010 10001100 11100001 11000011 00010011 01101100 10011001 11111000

The letter "H" is represented as the binary value of "01001000". To hide this, the above presented stream would be changed as follows:

10101010 10001101 11100000 11000010 00010011 01101100 10011000 11111000

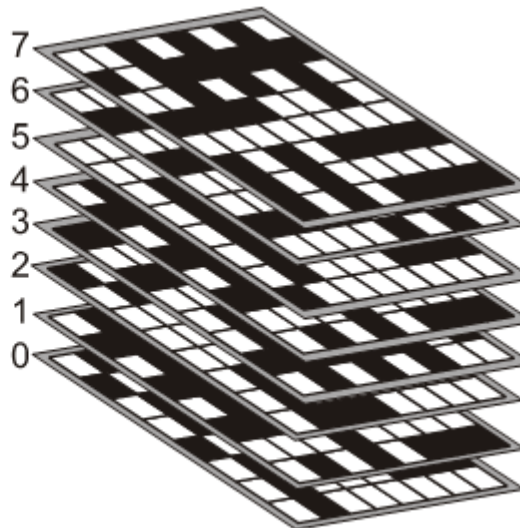
For RGB format every pixel should be represented by 3 bytes (Red Green Blue) 24 bits. Experiments shows the up to 50% bits have the random nature that is why can be replaced. In this case not only least significant but up to four last bits, and 1024×768 image can hold up to 1MB of data.

Steganography

BPCS Steganography

BPCS Steganography is the *Bit Plain Complexity* Steganography.

Digital images are categorized as either binary (black-and-white) or multi-valued pictures despite actual color. Then an n -bit image can be decomposed into a set of n binary images by bit-slicing operation.



Steganography

BPCS Steganography

The most steganography techniques, as well as BPCS are based on the category of the region (local area) of the image. Usually the size of the region equals to 8×8 binary patterns are belonging to some 8×8 pixels of the image. There are *continuance* and *random* region.

BPCS is based on the 8×8 continuance binary patterns.

The BPCS method makes use of the more complex regions of an image to embed data. Within this method the black-and-white border image complexity have been adopted. The length of the black-and-white border in a binary image is a good measure for image complexity. If the border is long, the image is complex, otherwise it is simple. The total length of the black-and-white border equals to the summation of the number of color-changes along the row and columns in an image. For example, a single black pixel surrounded by white background pixels has the boarder length of 4.

Steganography

BPCS Steganography

The image complexity α is defines as:

$$\alpha = (\text{Real number changes}) / (\text{The max. possible Black and White changes in the image})$$

$$= k / (2 \times 2^n \times (2^n - 1))$$

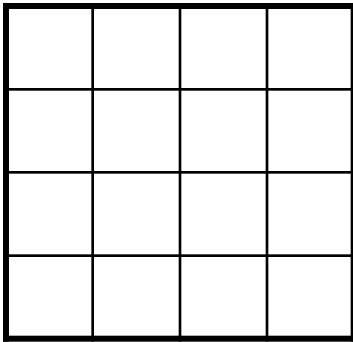
$$0 \leq \alpha \leq 1$$

We assume the image frame is always square having $2^n \times 2^n$ pixels (Practical image size $n=2 \div 12$). This characteristic is calculated over the whole image area. It gives us the global complexity of a binary images. However, we can use α for the region or the image block with 8×8 pixels. or bits for our algorithm.

Steganography

BPCS Steganography

W



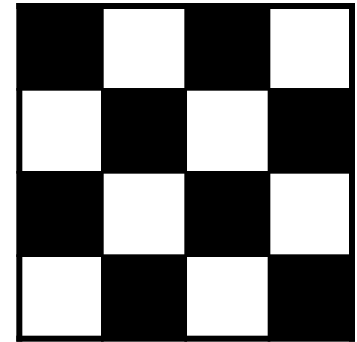
$$\alpha = 0/24 = 0$$

B



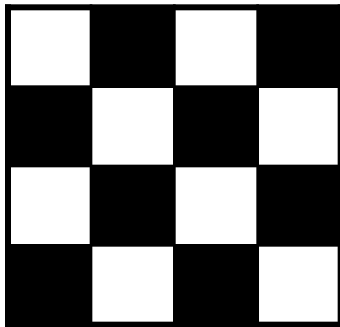
$$\alpha = 0/24 = 0$$

B-W



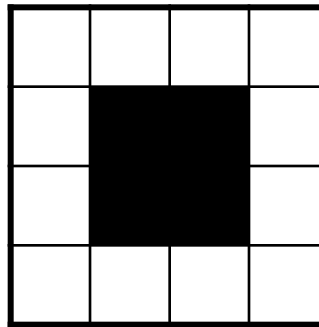
$$\alpha = 24/24 = 1$$

W-B



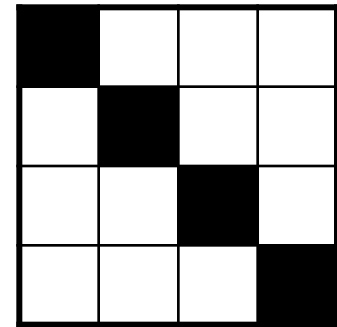
$$\alpha = 24/24 = 1$$

P



$$\alpha = 8/24 = 1/3$$

P



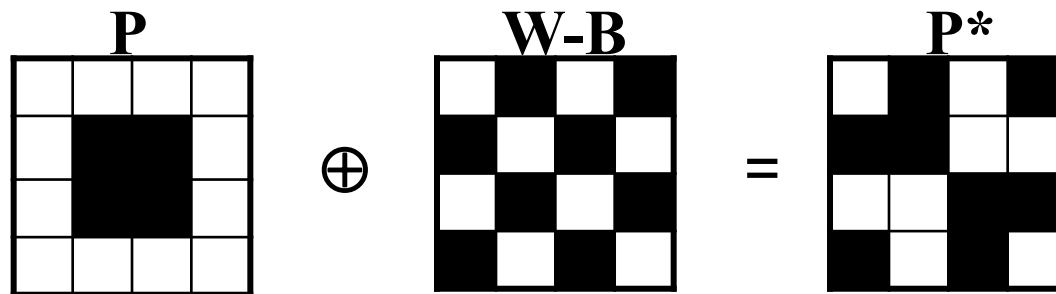
$$\alpha = 12/24 = 1/2$$

Steganography

BPCS Steganography

Conjugation of binary image

We regard black and white pixels have logical value “1” and “0”.
Now we define P^* as the conjugate of P which satisfies: $P^* = P \oplus (W-B)$;
 $(P^*)^* = P$; $P^* \neq P$ where $\oplus$ means Exclusive OR operation



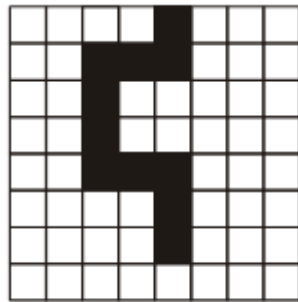
The most important property about conjugation is the following:

Let $\alpha(P)$ be the complexity of a given image P , then we have

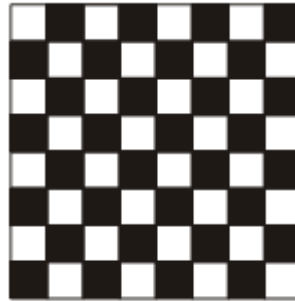
$$\alpha(P^*) = 1 - \alpha(P)$$

Steganography

BPCS Steganography

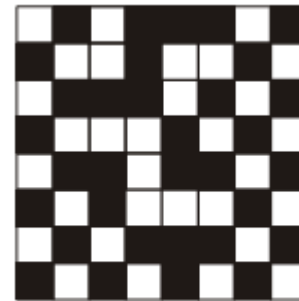


$\alpha=23/112$

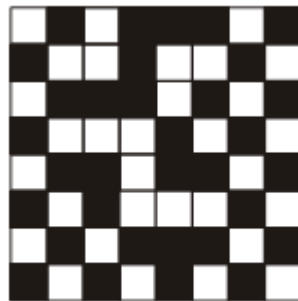


$\alpha=112/112$

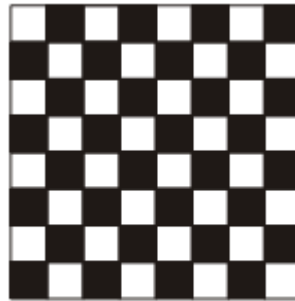
=



$\alpha=89/112$

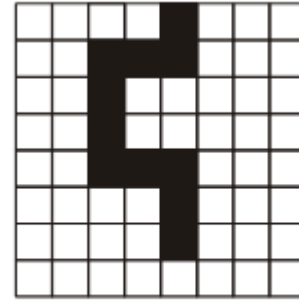


$\alpha=89/112$



$\alpha=112/112$

=



$\alpha=23/112$

Steganography

BPCS Steganography

Criterion to segment a bit-plane into informative and noise-like regions

First proposition is the size 8×8 bits is a good size for local area. Secondly, our next question is how much image data we can discard without deteriorating image quality, or how small informative data are indispensable to keep image information. To discard data means to replace local image areas in a bit-plane with random noise patterns. If we replace all the local areas having complexity value $\alpha_L \leq \alpha$, yet it still maintains a good quality, then we can discard more. If it is not more a good quality, we can not discard that much. If $\alpha = \alpha_L$ is the minimum complexity value to be good, such α_L is the threshold value. To be “*informative*” for the image means the following. If the image data is still “*picture-like*” after we have discarded (randomized) certain amount of image data for such an α_L and if we discard more, then it becomes only “*noise-like*”. Then, that α_L is regarded as the limit of the informative image complexity.

Steganography

BPCS Steganography

There are 99,9999999999999333% noise-like patterns among of all 8×8 binary patterns for $\alpha_L = 0.3$.

Surprising enough, it is only $6,67 \times 10^{-140}\%$ informative patterns

BPCS Steganography Algorithm

The vessel is a color image in BMP format, which hides (or, embeds) the secret information (file in any format). Secret file is segmented into a series of blocks having 8 bytes of data each. These block (secrete blocks) are regarded as 8×8 image patterns.

- 1.Transform the vessel image from PBC to CGC system.
- 2.Segment each bit-plane of the vessel image into informative and noise-like regions by using a threshold value α_L A typical value is $\alpha_L = 0.3$.

Steganography

BPCS Steganography

3.Group the bytes of the secrete file into a series of secrete blocks

S.

4.If a block S is less complex than the threshold α_L , then conjugate it to make it a more complex block S^* . The conjugated block must be more complex due to $\alpha(S^*)=1-\alpha(S)$.

5.Embed each secret block into the noise-like regions of the bit-planes (or, replace all the noise-like regions with a series of secret blocks). If the block is conjugated, then record this fact in a “conjugation map”.

6.Also embed the conjugation map as was done with the secret blocks.

7.Convert the embedded vessel image from CGC back to PBC.

The decoding algorithm is just the reverse procedure of the embedding steps.

Steganography

BPCS Steganography

Embedding Capacity:

The numerous experiments show that most color images taken by digital camera can be used as vessel images. In almost all cases, the information hiding capacity was nearly 50% of the size of each vessel image. Next table shows the capacity changes with threshold for the digital image of the authors of the BPCS Steganography.

Threshold α	Capacity Bytes	Percent of original %
20/64	499432	44
25/64	469480	41
30/64	437240	38
35/64	400848	35
40/64	361800	32
45/64	315432	27

Steganography

BPCS Steganography

There are two metrics for image complexity estimation within the BPCS Steganography. The first one α based on the weighted length k of the B-W border within the image block.

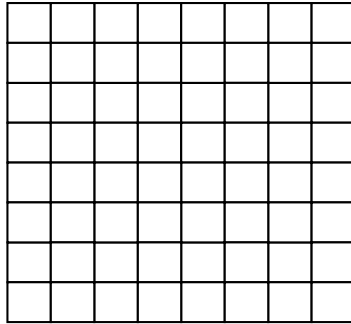
$$\alpha = \frac{k}{2 \times 2^n \times (2^n - 1)};$$

The second one β represents the weighted number m of the regions within the block with black or white color.

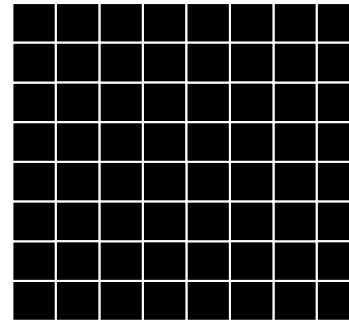
$$\beta = \frac{m}{2^n \times 2^n};$$

Steganography

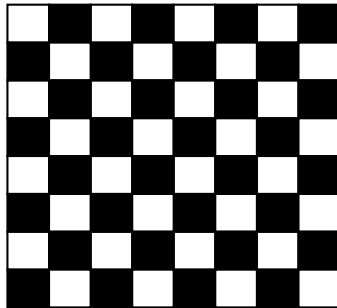
BPCS Steganography



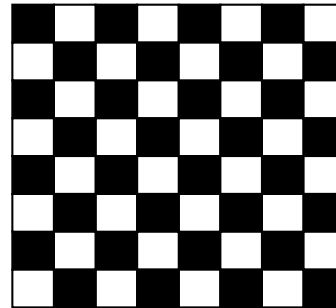
$W: \alpha=0, \beta=0,015625$



$B: \alpha=0, \beta=0,015625$



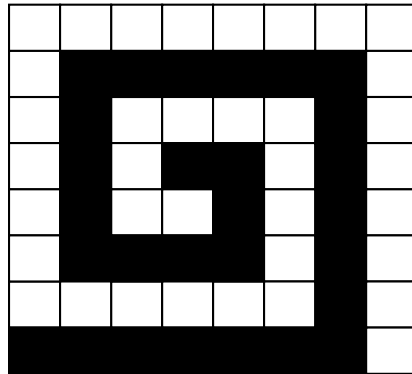
$W_C: \alpha=\beta=1$



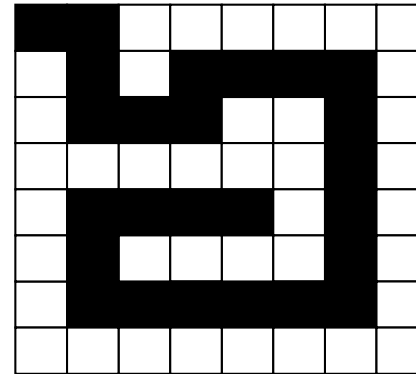
$B_C: \alpha=\beta=1$

Steganography

BPCS Steganography



$$\alpha=50/112, \beta=2/64$$



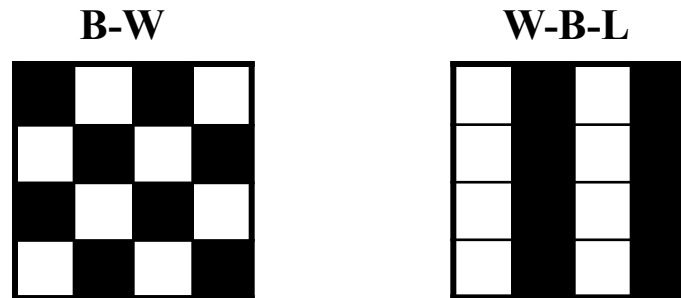
$$\alpha=49/112, \beta=2/64$$

Steganography

ABCDE Steganography

ABCDE Steganography – A Block Complexity based Data Embedding can be regarded as an extension of BPCS steganography as based on three measures of complexity: ***black-and-white border complexity measure*** (the same as for BPCS); ***run-length irregularity***; ***border noisiness***.

For the example of chessboard pattern (B-W) the measure $\alpha=1$, i.e. the maximum of α . This block, however, has a regular periodical pattern, and cannot be regarded as complex one.



A possible workaround here is to introduce a upper limit threshold α_U and to accept only blocks those have α belongs to interval $\alpha_L \leq \alpha \leq \alpha_U$. Unfortunately, even if we adopted this new criterion, we may still encounter blocks like W-B-L, for which $\alpha=0.5$

Steganography

ABCDE Steganography

Run-Length Irregularity – is defined based on the histogram of the run-lengths of both black and white pixels along a row or a column. Suppose that we have a binary pixel shown below



It consists of: a run of one black pixels; a run of two white pixels; a run of two black pixels and a run of three white pixels. Here we find that $h[1]=1$, $h[2]=2$, $h[3]=1$, Where $h[i]$ is the frequency of runs of i pixels in black or white.

The following h_s is now introduced to measure the *irregularity* of a binary pixel sequence:

$$h_s = - \sum_{i=1}^n h[i] \cdot \log_2 p_i$$

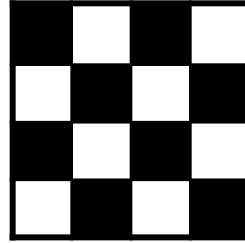
$$p_i = \frac{h[i]}{\sum_{j=0}^n h[j]}$$

where n is the longest run-length possible

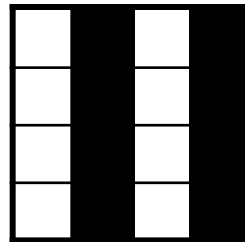
Steganography

ABCDE Steganography

For the **W-B** region we have eight of one white and eight of one black runs, so $h[1]=16$, and there are not any runs with more pixels. That is why $p_1=16/16=1$, and $h_s=0$.



For the **W-B-L** region we have two of four white and two of four black runs, so $h[4]=4$, and there are not any runs with more pixels. That is why $p_4=4/4=1$, and $h_s=0$.



If a sequence contains runs of the same length only, it would reveal a regular periodicity. On other hand, if a sequence contains runs of various lengths, it would not have any distinct periodicity, and h_s would become large.

Steganography

ABCDE Steganography

The ***border noisiness*** complexity measure is computed based on the differences between adjacent binary pixel sequences in a block. Let the block size be $n \times n$ ($n > 1$). Let r_i and c_j be i -th row and j -th column of a block respectively. The border noisiness γ of a block is defined as follows:

$$\gamma = \min \{E_f(P_x(r)), E_f(P_x(c))\} / (n-1)$$

Where $P_x(r) = \{g(r_0 \oplus r_1), \dots, g(r_{n-2} \oplus r_{n-1})\}$;
 $P_x(c) = \{g(c_0 \oplus c_1), \dots, g(c_{n-2} \oplus c_{n-1})\}$, symbol $\oplus$ denotes bit wise exclusive-or, $g(x)$ is the number of ones in a binary sequence x , and

$$E_f(X) = (1.0 - \text{Var}(X) / \max_X \{\text{Var}(X)\}) \text{Aver}(X)$$

$X = \{x_0, \dots, x_{m-1}\}$; $\text{Var}(X)$ is the variance of X ; and $\text{Aver}(X)$ is average of X .

The definition of the border noisiness γ is similar to that of the run-length irregularity β . The number of the black-and-white pixel borders are counted for every pairs of adjacent rows and columns. They are represented by the set $P_x(r)$ for rows and $P_x(c)$ for columns in the above definition. A weighted average E_f is then computed against these sets.

Steganography

Run-length mechanism

A *run-length mechanism* for hiding data into binary images is based on the concept of visibility; i.e., the edge portion of host binary images is used for hiding secure data, because the modification of edge portion is more difficult to be recognized by human eyes. To embed a single bit, for example “1”, of secure data, we let the running length be odd, and vice versa.

Embedded Procedure

First of all the binary vessel image is represented by a bit stream using the raster scanning order. Let O denote this image, $O = \{o_i | o_i \in \{0,1\}, i=1,2,\dots,H \times W\}$ where H and W denote the height and width of the host image, respectively. The data to be embedded into the host image is also represented by a bit stream $D = \{d_i | d_i \in \{0,1\}, i=1,2,\dots,L\}$, where L denotes the length (in bit) of D , and its value is depended on O and quality control parameter. Next, we examine o_i , for $i=1,2,\dots,H \times W$, and count the length (called run-length) of the continued pixels with the same bit value. When a pixel's value has varied, a run-length r_j is obtained.

Steganography

Run-length mechanism

And each run-length will embed a bit d_j , where $j=1,2,\dots,L$. We illustrate the definition of run-length by the following:

O	1	1	1	1	1	1	0	0	0	0	0	1	1	...
	$r_j=7$						v_j	r_{j+1}				v_{j+1}		

If $d_j \neq r_j \bmod 2$, the pixel v_j 's value should be changed, i.e., the value of r_j will increase one. Otherwise, there is no pixel required to change for embedding d_j .

Теория Информации

В.Н. Ярмолик

Лекция 23

2016

Steganography

CPT Scheme

CPT Scheme is the method proposed by Y.Y.Chen, H.K.Pan, Y.C.Tseng for a secure data hiding for two-color images. The CPT scheme is block-based. A binary container image F is regularly partitioned into small pixel blocks F_i of a size $m \times n$ (for simplicity, we assume that F 's size is a multiple of $m \times n$). The scheme is able to hide as many as $r \leq \lfloor \log_2(mn+1) \rfloor$ bits of the secret message in each host block by modifying at most 2 bits in the block. The secret key has two components:

- K : a randomly selected binary matrix of size $m \times n$.
- W : a weight matrix which is an integer matrix of size $m \times n$. W satisfies the condition that $\{[W]_{i,j} | i=1, \dots, m; j=1, \dots, n\} = \{1, 2, \dots, 2^r - 1\}$. Note that since $2^r \leq 2^{\lfloor \log_2(mn+1) \rfloor} \leq 2^{\log_2(mn+1)} \leq mn+1$, it is trivial to find a matrix that can serve as a weight matrix.

Steganography

CPT Scheme

In the fact, the number of choices for W can be calculated as:

$C_{2^r-1}^{mn} \times (2^r-1)! \times (2^r-1)^{mn-(2^r-1)}$ For instance, if $m=n=8$ and $r=5$, there are $C_{31}^{64} \times 31! \times 31^{33}$ possible W 's . This number should be large enough to prevent a brute-force attack.

The data hiding is achieved by modifying some bits of F . Below (next slide) we show how to hide a bit stream $b_1b_2,\dots,b_r$ into an $m \times n$ host block, say F_i .

CPT Scheme *hiding algorithm*.

S1. Compute $F_i \oplus K$, where $\oplus$ means the bit wise Exclusive-Or of two equal-size binary matrices.

S2. Compute $SUM((F_i \oplus K) \otimes W)$, where $\otimes$ means the pair wise multiplication of two equal-size matrices and SUM means the sum of all elements in matrix.

Steganography

CPT Scheme

S3. From the matrix $F_i \oplus K$, compute for each $w=1, \dots, 2^r-1$ the following set: $S_w = \{(j,k) | ([W]_{j,k} = w \wedge [F_i \oplus K]_{j,k} = 0 \vee ([W]_{j,k} = 2^r - w \wedge [F_i \oplus K]_{j,k} = 1))\}$. Intuitively, S_w is the set containing every matrix index (j,k) such that if we complement $[F_i]_{j,k}$, we can increase the sum in step S2 by w . There are actually two possibilities to achieve this:

(i) if $[W]_{j,k} = w$ and $[F_i \oplus K]_{j,k} = 0$, then complementing $[F_i]_{j,k}$ will increase the weight by w , and

(ii) if $[W]_{j,k} = 2^r - w$ and $[F_i \oplus K]_{j,k} = 1$, then complementing $[F_i]_{j,k}$ will decrease the weight by $2^r - w$, or equivalently increase the sum by w (under ***mod*** 2^r). Also, define $S_w = S$ for any $w = w' \bmod 2^r$.

S4. Define a weight difference $d = (b_1 b_2, \dots, b_r) - \text{SUM}((F_i \oplus K) \otimes W) \bmod 2^r$. If $d = 0$, there is no need to change F_i . Otherwise, we run the following program to transform F_i to F'_i .

Steganography

CPT Scheme

Program to *transform* F_i into F'_i .

$$(hd \bmod 2^r) + (-(h-1)d \bmod 2^r) = d$$

- a) Randomly pick an $h \in \{1, \dots, 2^r - 1\}$ such that $S_{hd} \neq \emptyset$ and $S_{-(h-1)d} \neq \emptyset$. Indexes hd and $-(h-1)d$ are used by *mod* 2^r .
 - b) Randomly pick a $(j, k) \in S_{hd}$ and complement the bit $[F_i]_{j,k}$.
 - c) Randomly pick a $(j, k) \in S_{-(h-1)d}$ and complement the bit $[F_i]_{j,k}$.
- To summarize, the above steps ensure the following invariant:

$$SUM((F'_i \oplus K) \otimes W) = b_1 b_2, \dots, b_r \bmod 2^r$$

Steganography

CPT Scheme

In the above step S4, we pick two nonempty sets S_{hd} and $S_{-(h-1)d}$. Since these sets indicate the locations where we can complement F_i to increase the weight by hd and $-(h-1)d$, respectively. The overall effect is an increase of the weight by d . It is proved that step S4 will always succeed. However, there are logical flaws in the above program, which we left out intentionally for ease of presentation. First, the set S_0 (and similarly S_{2r} and so on) is not yet defined. Like other S_w 's we can regard S_0 as the set of indices such that complementing these locations in F_i will result in an increase of weight by 0. Since this can be achieved by changing nothing on F_i , we can always regard S_0 as nonempty and whenever the statement

Steganography

CPT Scheme

First, F is partitioned into two 4×4 blocks, F_1 and F_2 . We can hide $r \leq \lfloor \log_2(4 \times 4 + 1) \rfloor$

$$b_1 b_2 b_3 = 001$$

$$b_1 b_2 b_3 = 001$$

S1

$$F_1 \oplus K =$$

1	0	0	1
1	1	0	0
0	1	1	0
0	0	1	0

$$F_2 \oplus K =$$

0	1	0	0
1	1	1	0
1	0	0	1
0	1	0	1

S2

$$SUM((F_1 \oplus K) \otimes W) \bmod 2^3 =$$

1	0	0	1
1	1	0	0
0	1	1	0
0	0	1	0

×

1	2	3	4
5	6	7	1
2	3	4	5
6	7	1	2

$$= (1+4+5+6+3+4+1) \bmod 2^3 = 0$$

Steganography

CPT Scheme

S3. $S_1=\{(2,4)\}$; $S_2=\{(1,2), (3,1), (4,4), (2,2)\}$; $S_3=\{(1,3), (2,1)\}$; $S_4=\{(1,4), (3,3)\}$; $S_5=\{(3,4), (3,2)\}$; $S_6=\{(4,1)\}$; $S_7=\{(2,3), (4,2), (1,1), (4,3)\}$.

S4. The weight difference $d=(b_1b_2b_3)-SUM((F_1\oplus K)\otimes W)=001-0=1-0=1$. If $d\neq 0$ there is need to change F_1 . We run the following program to transform F_1 to F_1' .

a) For any h there is the possibility to use one S_w out of all possible.

b) Randomly we are picking a $(j,k)=(2,4)$ and complement the bit $[F_1]_{2,4}$ to new value 1.

c) Due to $[W]_{2,4}$ has the value 1 what exactly equals to $d=1$ we should not make any changes for F_1 yet.

Steganography

CPT Scheme

For F_2 , since $\text{SUM}((F_2 \oplus K) \otimes W) \bmod 2^3 = (2+5+6+7+2+5+7+2) \bmod 2^3 = 4$. The weight difference $d = 1 - 4 = -3 = 5$.

Thus, swapping $[F_2]_{2,2}$ and $[F_2]_{3,2}$ will increase the weight by $[W]_{2,2} = -6 = 2$ and $[W]_{3,2} = 3$, respectively. The resulting F with modified bits are shown below.

F'	0	1	0	1
	1	0	0	1
	1	0	0	0
	0	0	0	0
F_1'				
	1	1	1	0
	0	0	1	1
	0	1	1	1
F_2'				

Steganography

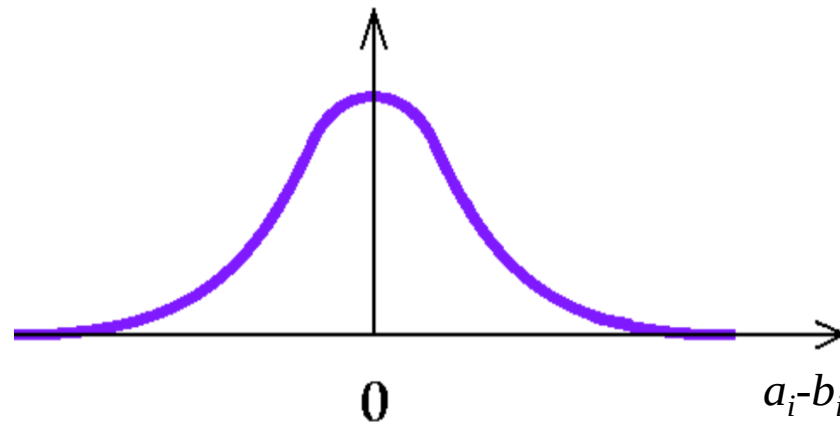
Patchwork Method

Patchwork Method (W.Bender, 1996),

Patchwork method is based on the peculiarity of the graphic images that for large enough number n pair of pixels A and B with the values a and b , the parameter

$$S_n = \sum_{i=1}^n (a_i - b_i) \approx 0.$$

has the value very close to zero. The distribution of $S = a - b$ is shown below.



Steganography

Patchwork Method

The ***Patchwork*** method consists of changing of brightness of random chosen set of pixels pairs (a_i, b_i) , such a way that the value of S'_n , for this set will be sufficiently high compare with zero.

Patchwork watermark embedding algorithm:

1. Based on psedorandom number generator the pair (A,B) of pixel with the brightness (a_i, b_i) should be chosen;
 2. The brightness a_i is increasing on value δ ;
 3. The brightness b_i is decreasing on value δ ;
- Steps 1 – 3 repeats n times.*

Then the value of S'_n is determined as

$$S'_n = \sum_{i=1}^n ((a_i + \delta) - (b_i - \delta)) \approx 2\delta n.$$

Теория Информации

В.Н. Ярмолик

Лекция 24

2016

Software Protection

Software Ownership Violation

Software piracy, the illegal copying and resale of applications, is a 30 billion dollar per year industry. Piracy is therefore a major concern for anyone who sells software. In the early days of the personal computer revolution, software developers experimented vigorously with various forms of technical protection against illegal copying. The main goal is to make ***piracy*** more difficult.

Many software developers also worry about their applications ***being reverse engineered***. Several court cases have been tried in which a valuable piece of code was extracted from an application and incorporated into a competitor's code. Such threats have recently become more of a concern since, more and more, programs are distributed in easily decompilable formats rather than native binary code. Important examples include the Java class file format and ANDF.

Software Protection

Software Ownership Violation

A related threat is *software tampering*. Many mobile agents and *e-commerce* application programs must, by their very nature, contain encryption keys or other secret information. Pirates who are able to extract, modify, or otherwise tamper with this information can incur significant financial losses to the intellectual property owner.

There are three types of attack (*software piracy*, malicious *reverse engineering*, and *tampering*) It should be noted that it is much more difficult to defend a client than it is to defend a host. To defend a host against a malicious client, all that is needed is to restrict the actions that the client is allowed to perform. Unfortunately, no such defence is available to protect a client against a host attack. Once the client code resides on the host machine, the host can make use of any conceivable technique to extract sensitive data from the client, or to otherwise violate its integrity.

Software Protection

Software Ownership

Protection

Recently, have shown that some degree of protection can be achieved based on software ***watermarking***, ***tamper-proofing*** and ***obfuscation*** have emerged as feasible technical means for the intellectual property protection of software. (Other promising techniques, such as *traitor tracing*, *secret sharing*, *reference states*, and *secure evaluation*, are still in the hands of theorists.)

Software watermarking embeds a copyright notice in the software code to allow the owners of the software to assert their intellectual property rights. ***Software fingerprinting*** is a similar technique that embeds a unique customer identification number into each distributed copy of an application in order to facilitate the tracking and prosecution of copyright violators.

Software Protection

Software Ownership

Protection

Tamper-proofing causes a program to malfunction when it detects that it has been modified.

Obfuscation attempts to transform a program into an equivalent one that is harder to reverse engineer.

There are three types of defences against the software copyrights violations (software ***watermarking***, ***obfuscation***, and ***tamper-proofing***).

In this presentation, we avoid the vexed question by dismissing the traditional underlying assumption of a benign host. Instead, we assume that any host may be malicious, as we analyse computer security in this presentation from the *viewpoint* of the *owner of the client software*.

Software Protection

Watermarking

Watermarking embeds a secret message into a cover message. In media ***watermarking***, the secret is usually a copyright notice and the cover a digital image or an audio or video production. ***Watermarking*** an object discourages intellectual property theft or, when such theft has occurred, allows us to prove ownership.

Fingerprinting is similar to ***watermarking***, except a different secret message is embedded in every distributed cover message. This may allow us not only to detect when theft has occurred, but also to trace the copyright violator. A typical fingerprint includes a vendor, product, and customer identification numbers.

Software Protection

Watermarking

We can describe the software *watermarking* problem as follows:

Embed a structure W (the watermark) into a program P such that:

- W can be reliably located and extracted from P (the embedding is resilient to dewatermarking attacks).
- W is large (the embedding has a high data rate).
- Embedding W into P does not adversely affect the performance of P (the embedding is cheap).
- Embedding W into P does not change any statistical properties of P (the embedding is stealthy).
- W has a mathematical property that allows us to argue that its presence in P is the result of deliberate actions.

Software Protection

Static Watermarks

Software watermarks come in two flavours, ***static*** and ***dynamic***. ***Static watermarks*** are stored in the application executable itself; whereas, ***dynamic watermarks*** are constructed at runtime and stored in the dynamic state of the program.

Collberg and ***Thomborson*** present ***subtractive attack***, ***distortive attack***, and ***additive attack*** as three possible ways of corrupting a watermark, where corruption means the watermark can no longer be used to prove ownership.

Subtractive Attack: Eliminate the presence of the watermark,

Distortive Attack: Introduce distortion via transformations such that the watermark can no longer be recovered.

Software Protection

Static Watermarks

Additive Attack: Insert information to completely cover the original watermark or add information so that the original watermark cannot be trusted as the original watermark.

Moskowitz and ***Cooperman*** describe a *static* data **watermarking** method in which the watermark is embedded in an image using one of the many media **watermarking** algorithms. This image is then stored in the static data section of the program. ***Davidson*** and ***Myhrvold*** describe a static code watermark in which a ***fingerprint*** is encoded in the basic block sequence of a program's control flow graphs.

Software Protection

Static Watermarks

There two basic types of *static watermarks*: *code watermarks* which are stored in the section of the executable that contains instructions, and *data watermarks* which are stored in any other section, including heads, string section, debugging information section, etc.

Static Data Watermarks

Data watermarks are very common since they are easy to construct and recognize, for example:

CONST C=“Copyright (C) 2003, Yarmolik&Company Ltd.”

The JPEG group’s copyright notice can be easily extracted from the *Netscape* binary.

Software Protection

Static Watermarks

Example

Char V;

```
switch e {  
  case 1 : V='C'  
  case 5 : V='O'  
  case 6 : V='P'  
  case 8 : V='Y'  
  case 9 : V='R'  
  ...  
}
```

Software Protection

Static Watermarks

Static Watermarks

Code Watermarks can be constructed in a similar way, since object code also contains redundant information. For example, if there are not data or control dependencies between two adjacent statements $S1$ and $S2$, they can be flipped in either order. A watermarking bit could then be encoded in whether $S1$ and $S2$ are in lexicographic order or not. There are many variation of this technique. When litigating against software pirates who copied their PC-AT ROM, IBM argued that the order in which registers were pushed and popped constituted a signature of their software.

Software Protection

Static Watermarks

Similarly, by reordering the branches of an ***m***-branch case statement we can encode

$\log_2(m!) \cong \log_2((2\pi m)^{1/2})(m/e)^m) = O(m \log_2 m)$ watermarking bits

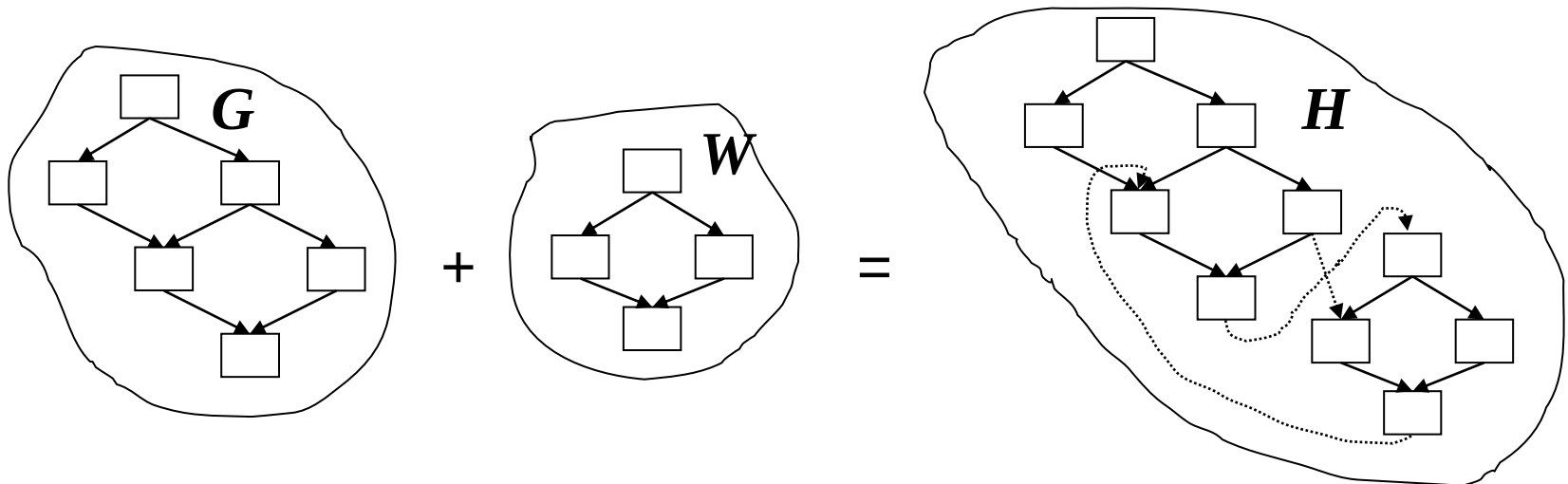
For example if $m=3$ we have *S1*, *S2* and *S3* branches and can encode $\lfloor \log_2 3! \rfloor = 2$ watermarking bits as for example

<i>Order of branches</i>	S1,S2,S3	S1,S3,S2	S2,S1,S3	S2,S3,S1
<i>Watermark bits</i>	00	01	10	11

Software Protection

Static Watermarks

Venkatesan et. al. present what appears to be the strongest known **static software watermarking** technique. The idea is to treat the source program as a control flow graph G of basic blocks, to which a watermark graph W is added forming a new control flow graph H . Next Figure shows how G and W are merged by adding code to the watermarked program that introduces new control flow edges between the two graphs. These edges (dashed in the figure) can be realized, for example, by using **opaque predicates**.



Software Protection

Static Watermarks

To detect the watermark of Venkatesan et. al., the extractor needs to:

- a) reconstruct the control flow graph of the watermarked program;
- b) identify which of the nodes of the control flow graph belong to the watermark graph (or, at least identify most of these nodes), and;
- c) reconstruct the watermark graph itself. To identify the watermark nodes, the authors suggest storing one or more bits at a node that flags when a node is in W by using some padded data. This is a serious weakness of the algorithm.

Software Protection

Dynamic Watermarks

There are three kinds of *dynamic watermarks*. In each case, the application P is run with a predetermined input sequence $I=I_1, \dots, I_k$ which makes the application enter a state which represents the watermark. The methods differ in which part of the program state the watermark is stored, and in the way it is extracted. There are *Easter Egg Watermarks*, *Data Structure Watermarks* and *Execution Trace Watermarks*.

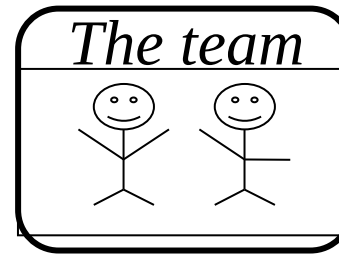
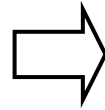
Ester Egg Watermarks

A watermark encoded in an *Easter Egg*, a piece of code that gets activated for a highly unusual input to the application. Typically, the code will display a copyright message or an unexpected image on the screen.

Software Protection

Dynamic Watermarks

```
If Input == I  
{  
  Display  
  (TeamPic)  
}
```



As an example for the application for sorting the integer array with $N=1000$ entries the highly unusual input $I=I_1, \dots, I_k$ may be $I=0, 0, 0, \dots, 0$.

Software Protection

Dynamic Watermarks

Dynamic Data Structure Watermark

Dynamic data structure Watermark scheme shown below being embedded within the state of a program ***P*** as it is being run with a particular input ***I*** (see example)

```
String V;
```

```
If Input == I {
```

```
    V[1]='C';
```

```
    V[3]='P';
```

```
    V[2]='O';
```

```
    V[4]='Y';
```

```
    V[6]='I';
```

```
    V[5]='R';
```

```
    .....
```

```
}
```

The watermark is extracted by examining the current values held in ***P***'s variables, after the end of the input sequence has been reached. This can be done using a dedicated watermark extraction routine which is linked in with the executing program or by running the program under a debugger.

Software Protection

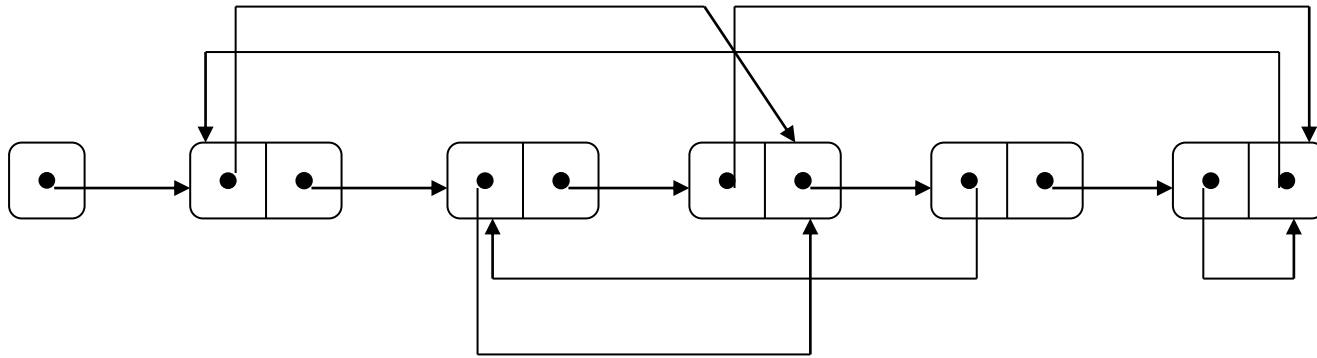
Dynamic Watermarks

Dynamic Graph Watermarking It is the most promising method among the dynamic data watermarking algorithms. The central idea of ***Dynamic Graph Watermarking*** is to embed a watermark in the topology of a dynamically built graph structure. The watermark is a product n of two large primes p and q .

Embedding the Watermark based on ***Radix- r encoding*** in a circular linked list. An extra pointer field encodes a base- r digit in the length of the path from the node back to itself. A null-pointer encodes a 0, a self-pointer a 1, a pointer to the next node encodes a 2, etc. A list of length m can encode any integer in the range $0, \dots, (m+1)^m - 1$. The list requires $2m+1$ extra words, if we assume no overhead heap cells. The dynamic bit-rate is $\log_2(m+1)^m / (2m+1) \cong (\log_2 m) / 2$. For $m=255$ we can hide $255 \times 8 = 2040$ bits in 511 words of storage, or 4 hidden bits per word. Below there is an example for $p=61$ and $q=73$, where the right pointer field holds the next field, the left pointer encodes a base- r digit.

Software Protection

Dynamic Watermarks



$$61 \times 73 = 3 \times 6^4 + 2 \times 6^3 + 3 \times 6^2 + 4 \times 6^1 + 1 \times 6^0$$

Software Protection

Dynamic Watermarks

Embedding the Watermark based on *Enumeration Encoding* uses result from graph enumeration. The idea is to let the watermark number n be represented by the index of the watermark graph G in some convenient enumeration. This requires us to be able to

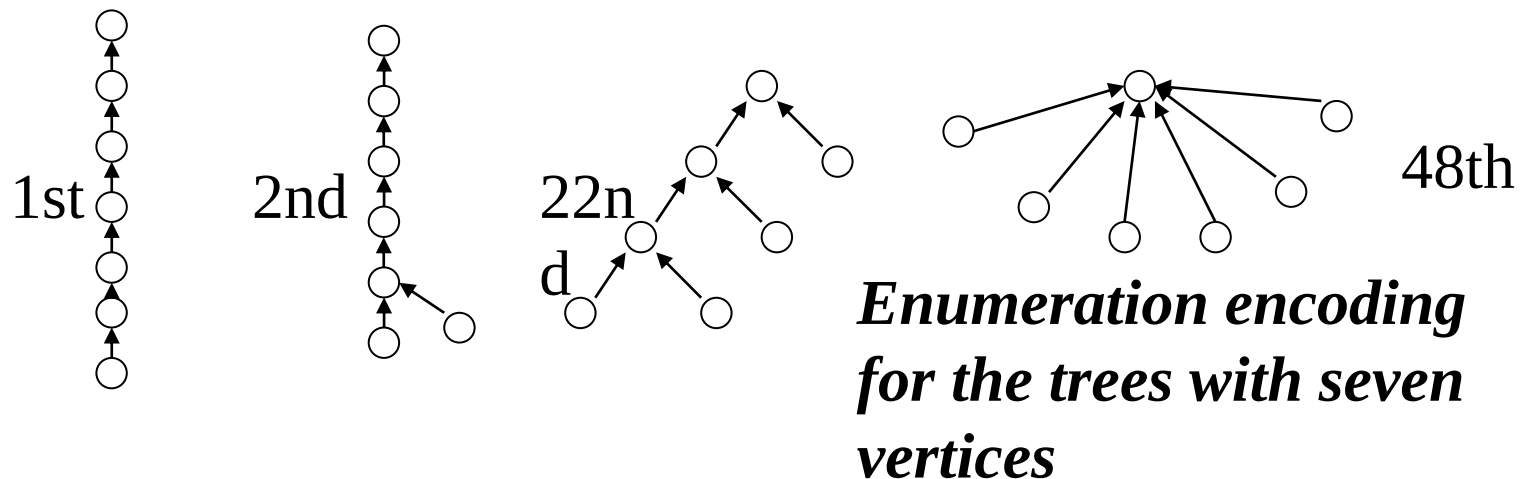
1. Given n , generate the n :th graph in the enumeration
2. Given G , extract its index n in the enumeration.

Both operation must be efficient, since we expect n to be large. Several restricted classes of graphs allow efficient enumeration and indexing. For example, we can let G be an oriented “parent-pointer” tree, in which case it is enumerable by the techniques described in Knuth. We construct an index n for any enumerable graph in the usual way, that is, by ordering the operations in the enumeration.

Software Protection

Dynamic Watermarks

For example, we might index the m -node trees in “largest sub tree first” order, in which case the path of length $m-1$ would be assigned index 1. Indices 2 through a_m-1 would be assigned to the other trees in which there is a single subtree connected to the root node. Indices $a_{m-1}+1$ through $a_{m-1}+a_{m-2}$ would be assigned to the trees with exactly two sub trees connected to the root node, such that one of the sub trees has exactly $m-2$ nodes and so on.

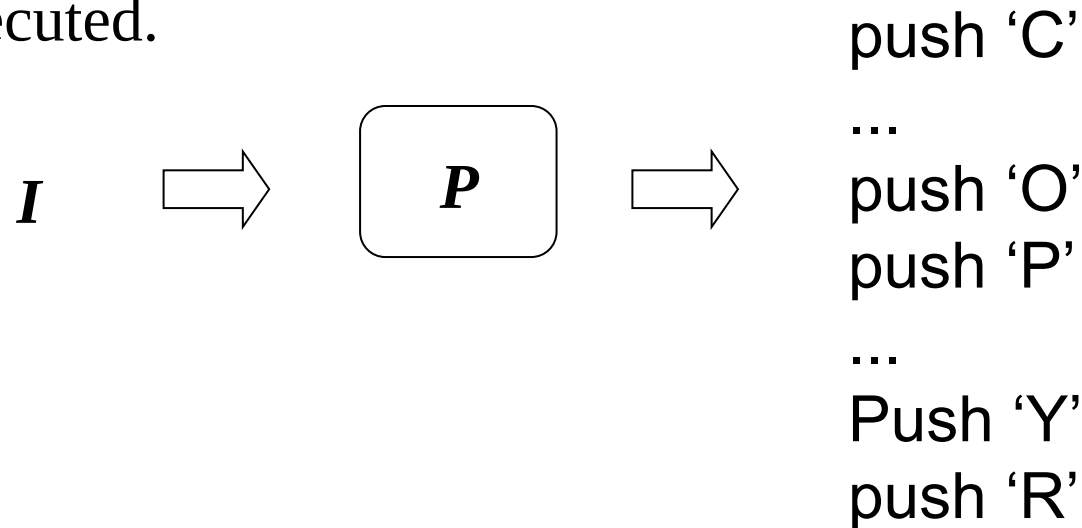


Software Protection

Dynamic Watermarks

Dynamic Execution Trace Watermark

This type of watermarks is embedded within the trace (either instruction or addresses, or both) of the program as it is being run with a particular input *I*. The watermark is extracted by monitoring some (possibly statistical) property of the address trace and/or the sequence of operators executed.



The obfuscation transformations that we have already presented, as well as many optimizing and translating transformation, will effectively obliterate any structure embedded in an instruction trace²³.

Software Protection

Easter Egg Watermarks Examples

[MS EXCEL 95 - Doom](#)

[MS EXCEL 97 - Flight Simulator](#)

[MS Word \(любая версия\) - Небольшие приколы >](#)

[MS Word 6.0 - список авторов](#)

[Windows 95 - Кнопка "Пуск"](#)

[Windows 95 - Список разработчиков \[1\]](#)

[Windows 95 - Список разработчиков \[2\]](#)

[ScreenSaver - OpenGL 3Dtext](#)

[MS Internet Explorer 4.0 - Список разработчиков](#)

[MS OutLook \(не проверено\)](#)

[Norton Antivirus - фотки создателей](#)

[Apple QuickTime версии >= 2.1.1.57 - создатели](#)

[Модемы USRobotics - список создателей](#)

[mIRC - куча приколов>](#)

["Пасьянс" в Windows - фича](#)

["Сапер" \(Win 3.1, 3.11, NT 3.51\) - подсказка \(не проверено\)](#)

[PageMaker \(не проверено\)](#)

[Quattro Pro for Windows \(не проверено\)](#)

[Paradox for Windows \(не проверено\)](#)

[MS Excel 4.0 \(почему-то не сработало в версии 4.0a\)](#)

[MS Excel 5.0 \(не проверено\)](#)

Software Protection

Easter Egg Watermarks Examples

CorelDRAW! 3.0-5.0 (не проверено)

CorelDRAW! 6.0 - авторы, немного шоу

Norton Utilities for Windows 95 - фото авторов (не проверено)

Visual C++ 4.0 (не проверено)

Borland C++ 4.51 (не проверено)

Adobe Photoshop 4.0 - два прикола (не проверено)

Microsoft Exchange Server 4.0 - у кого есть дистрибутив (не проверено)

Microsoft Systems Management Server 1.1 & 1.2 (не проверено)

MS Visual Basic 4.0 - имена разработчиков (не проверено)

Corel Printshop (не проверено)

"Сапер" - время >33. Windows 95 - Font animation (не проверено)

MS Access 2.0 - puzzle (не проверено)

MS Access 95 - авторы, еще что-то (не проверено)

MS Excel неизвестной версии - число (не проверено)

Lotus Notes - список программеров (не проверено)

Netscape Navigator - "about:..."

Software Protection

Easter Egg Watermarks Examples

Netscape Navigator - "about:..."

Netscape Navigator - FishCam, Status Bar, URL (не проверено)

Netscape Navigator - 1994 (не проверено)

MS Office 95 Toolbar - экран лицензии (не проверено, попробуйте Of4.3)

MS PowerPoint (?) - разработчики (не проверено) >43. MS Windows 3.xx

Lotus Amipro 3.0 и 3.1 (не проверено)

AOL 3.0 - версия (не проверено)

HP JetPrint A.01.08 - разработчики, мультитик (не проверено)

Eudora Pro (не проверено)

Apple II GS (не проверено)

PowerBuilder v4.x (не проверено)

Oracle Book 2.0 и 2.1 (не проверено)

. QuarkXPress (не проверено)

MagnaRam 97 (не проверено)

Adobe Illustrator 4.0 (не проверено)

MS Word 7.0 - разработчики (не проверено)

MS Word 97 - прикол с меню >56. Microsoft Справка >

Microsoft Paint

Microsoft Access 97

Теория Информации

В.Н. Ярмолик

Лекция 25

2016

Software Protection

Obfuscation

Security through **obscurity** has long been viewed with disdain in the security and cryptography communities. There are, however, situations where higher levels of protection than that achievable through obscurity at the present time does not seem achievable. As far as we know, there do not exist any techniques for preventing attacks by **reverse engineering** stronger than what is afforded by **obscuring** the purpose of the code.

Given a set of obfuscating transformations $T = \{T_1, \dots, T_n\}$ and a program P consisting of source code objects (classes, methods, statements, etc.) $\{S_1, \dots, S_k\}$ find a new program $P^* = \{\dots, S_i^* = T_i(S_i), \dots\}$ such that:

P^* has the same observable behaviour as P , i.e., the transformations are **semantics-preserving**.

The **obscurity** of P^* maximized, i.e., understanding and reverse engineering P^* will be strictly more time-consuming than understanding and reverse engineering P .

The **resilience** of each transformation $T_i(S_j)$ is maximized, i.e., it will either be difficult to construct an automatic tool to undo the transformations or executing such a tool will be extremely time-consuming.

The **stealth** of each transformation $T_i(S_j)$ is maximized, i.e., the statistical properties of S_j^* are similar to those of S_j

The **cost** (the execution time/space penalty) of P^* is minimized.

Software Protection

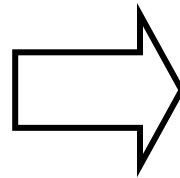
Obfuscation

Lexical Transformations

There are a number of Java **obfuscation** tools, most of which modify only the lexical structure of the program. Typically, they do nothing more than scramble identifiers. Such lexical transforms will surely be annoying to a reverse engineer and, therefore, will prevent some thievery of intellectual property in software.

Example

```
#ifndef ABSTRACT_CONTAINER_SEARCH_TYPE_  
#define ABSTRACT_CONTAINER_SEARCH_TYPE_  
enum search_type  
{  
    EQUAL = 1,  
    LESS = 2,  
    GREATER = 4,  
    LESS_EQUAL = EQUAL | LESS,  
    GREATER_EQUAL = EQUAL | GREATER  
};  
#endif
```



```
#ifndef O110010100111  
#define O110010100111  
enum l110010101000  
{  
    l110010101001 = 01 ,  
    O110010101010 = 02 ,  
    O110010101011 = 04 ,  
    O110010101100 = l110010101001 |  
    O110010101010 ,  
    O110010101101 = l110010101001 |  
    O110010101011  
};  
#endif
```

Software Protection

Obfuscation

```
for (iPos=0;iPos<nArrSize;iPos++)
{
nMinElemPos=iPos;
//find the least element
for (jPos=iPos;jPos<nArrSize;jPos++)
{
if (arr [jPos]<arr [nMinElemPos])
nMinElemPos=jPos;
}
//change places iPos and nMinElemPos
nTmpElem=arr[iPos];//temporary var
arr[iPos]=arr[nMinElemPos];
arr [nMinElemPos]=nTmpElem;
}
```



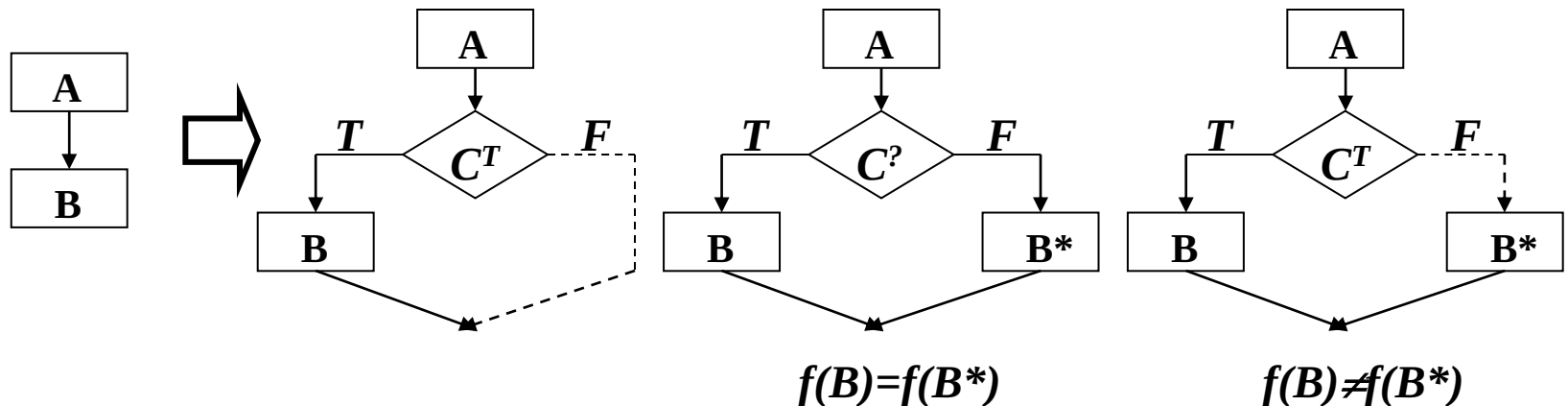
```
#define O110010100101 for
#define O110011100101 if
O110010100101(O110010100111=0;
O110010100111<l110010101000;
O110010100111++){O110010101011
=O110010100111;O110010100101
(O110010101010=O110010100111;
O110010101010<l110010101000;
O110010101010++){O110011100101
(l110010101001[O110010101010]
<l110010101001[O110010101011])
O110010101011=O110010101010;
}O110010101100=l110010101001
[O110010100111];l110010101001
[O110010100111]=l110010101001
[O110010101011];l110010101001
[O110010101011]=O110010101100;}
```

Software Protection

Obfuscation

Control Transformations

There are several control-altering transformations. These control transformations rely on the existence of **opaque predicates**. A predicate **C** is **opaque** if its outcome is known at **obfuscation** time, but is difficult for the deobfuscator to deduce. We write **CF (CT)** if **C** always evaluates to False (True) and **C?** if **C** may sometimes evaluate to True and sometimes to False. Given such opaque predicates, it is possible to construct obfuscating transformations that break up the flow-of-control of a procedure.



$\cos(x) > 1$, $-1 \leq \cos(x) \leq 1$, $x^2 < 0$, $x^8 \geq 0$, $x \times x \times (x+1) \times (x+1) \neq 0 \bmod 4$

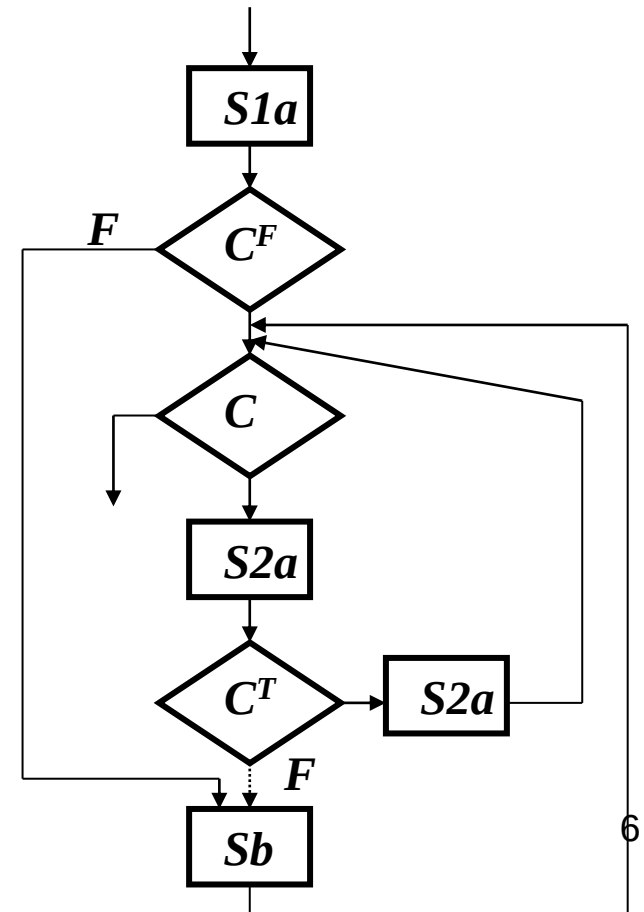
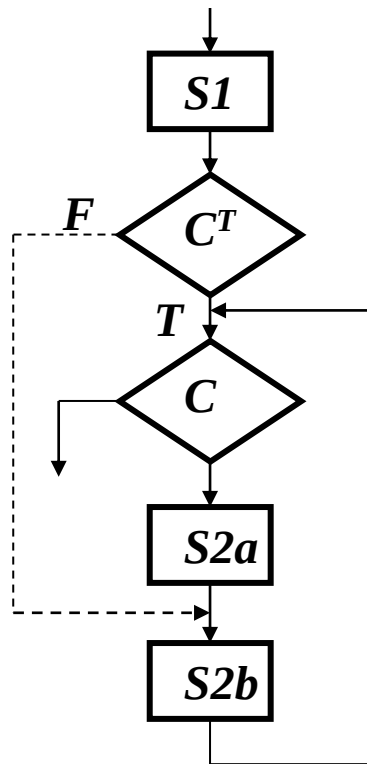
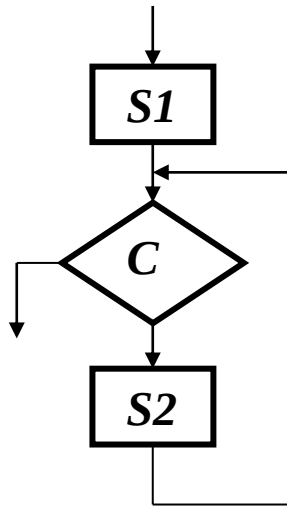
Software Protection

Obfuscation

Computation Transformations

Computation Transformations fall into three categories: hide the real control flow behind irrelevant statements that do not contribute to the actual computations, introduce code sequences at the object code level for which there exist no corresponding high-level language constructs, or remove real control flow abstractions or introduce spurious ones.

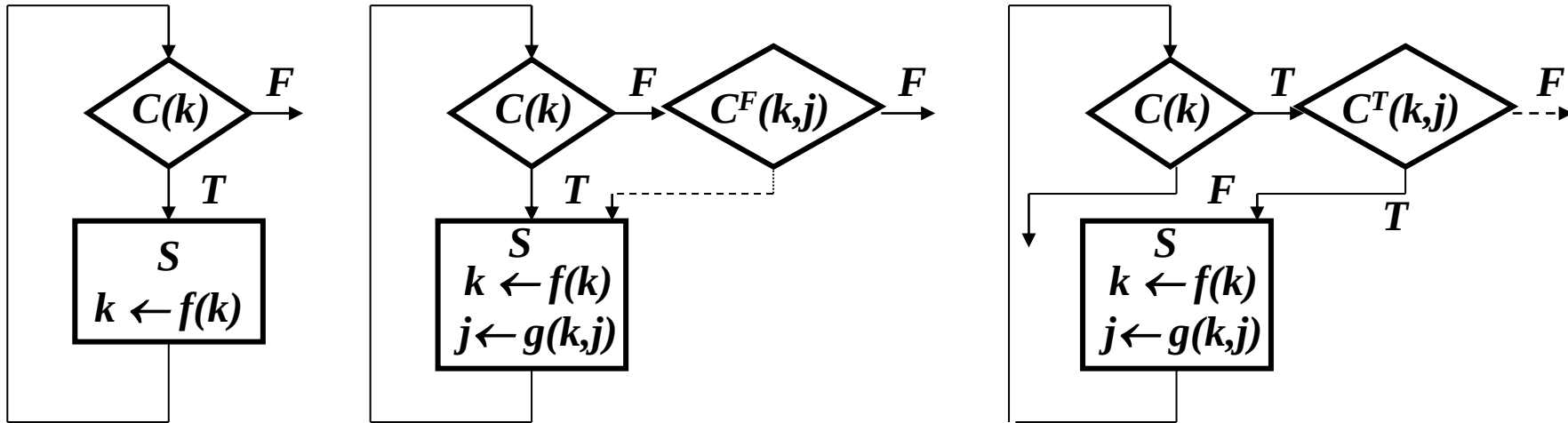
The Loop Condition Insertion transformation



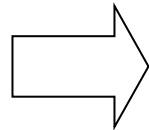
Software Protection

Obfuscation

The Loop Condition Insertion transformation



```
i=1;
while (i<100) {
    ...
    i++;
}
```



```
i=1; j=100
while ((i<100) && (j*j*(j+1)*(j+1)%4 == 4)) {
    ...
    i++;
    j=j*i+7;
}
```

Software Protection

Obfuscation

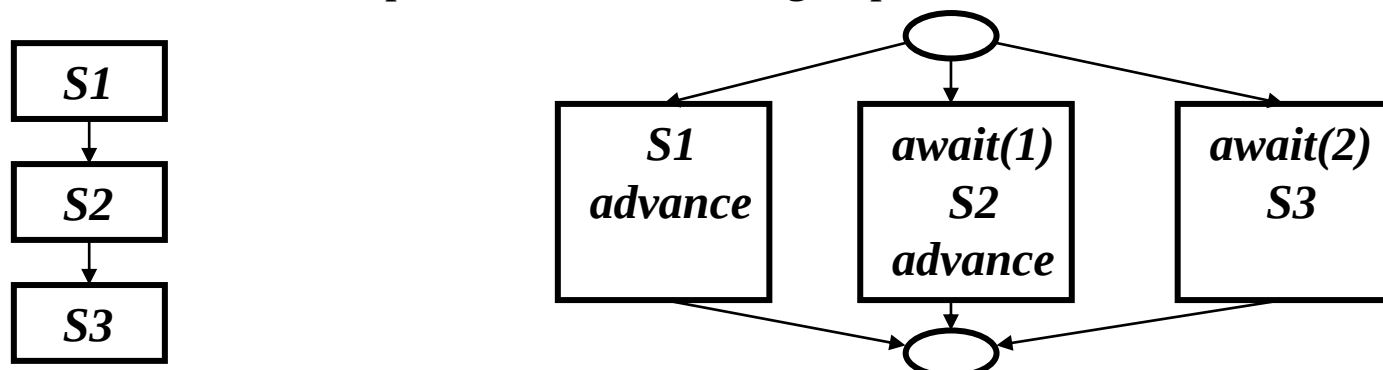
Add Redundant Operands

Once we have constructed some opaque variables we can use algebraic laws to add redundant operands to arithmetic expressions. In the obfuscated statement (1') below we make use of an opaque variable **P** whose value is 1. In statement (2') we construct an opaque sub expression **P/Q** whose values 2. Obviously, we can let **P** and **Q** take on different values during the execution of the program, as long as their quotient is 2 whenever statement (2') is reached.

(1) $X = X + V;$	(1') $X = X + V * P;$
(2) $Z = L + 1;$	(2') $Z = L + (P/Q)/2;$

Parallelize Code

Automatic parallelization is an important compiler optimisation used to increase the performance of applications running on multiprocessor machines. Our reasons for wanting to parallelize a program, of course, are different. We want to increase parallelism not to increase performance. but to obscure the actual flow of control. There are two possible operations available to us: 1) We can create dummy processes that perform no useful task. 2) We can split a sequential section of the application code into multiple sections executing in parallel.

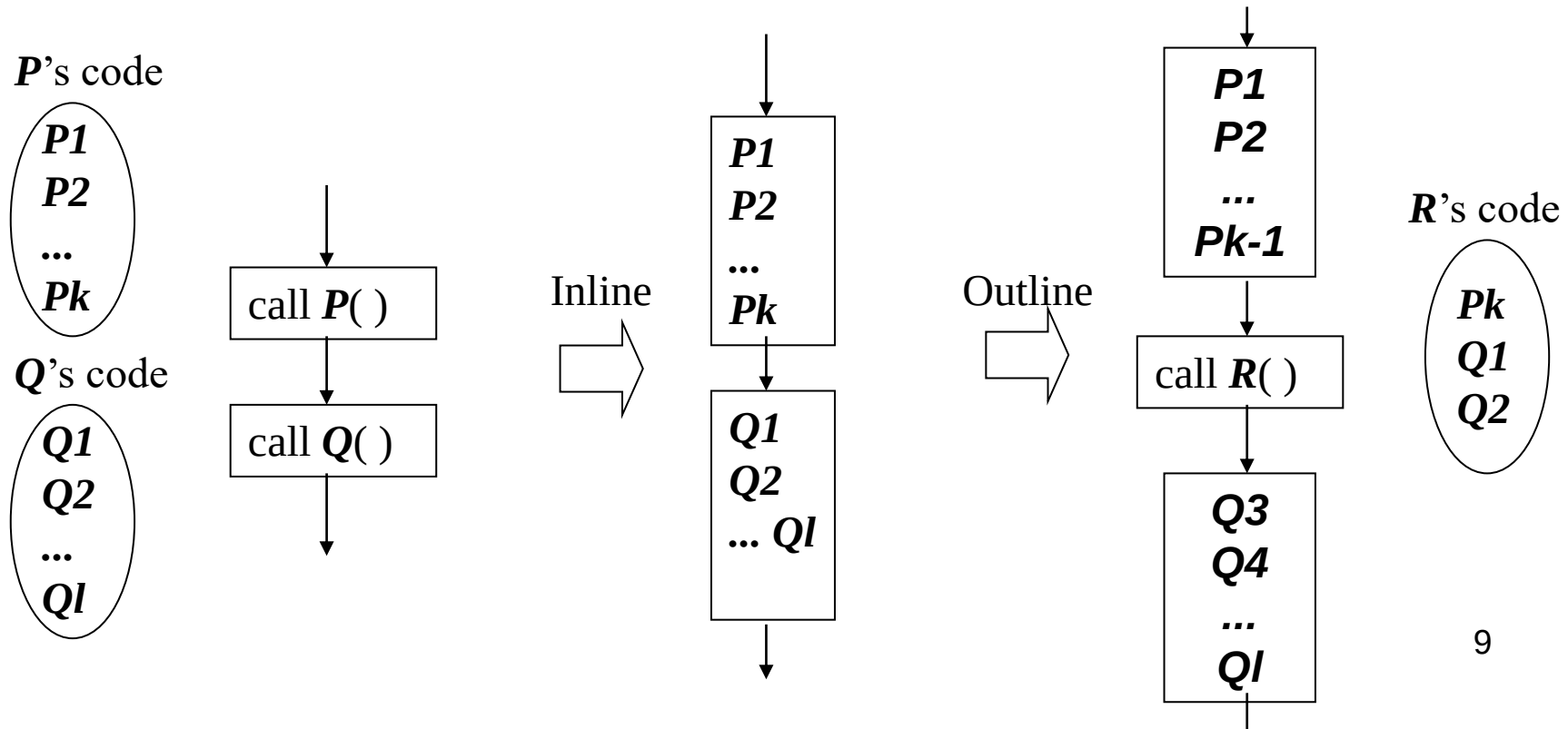


Software Protection

Obfuscation

Inline and Outline Methods

Inlining is, of course, an important compiler optimisation. It is also an extremely useful obfuscation transformation since it removes procedural abstractions from the program. Inlining is a highly resilient transformation, it is essentially one-way function, since once a procedure call has been replaced with the body of the called procedure and the procedure itself has been removed, there is no trace of the abstraction left in the code. Outlining (turning a sequence of statements into a subroutine) is a very useful companion transformation to inlining.



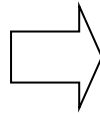
Software Protection

Obfuscation

Loop Transformations

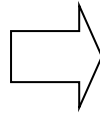
A large number of loop transformations have been designed with the intent to improve the performance of (in particular) numerical applications. Some of these transformations are useful to us since they also increase the obfuscation.

```
for (i=1, i<=n, i++)  
  for (j=1, j<=n, j++)  
    a[i,j]=b[i,j];
```



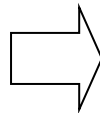
```
for (I=1, I<=n, I+=64)  
  for (J=1, J<=n, J+=64)  
    for (i=I, i<=min(I+63,n), i++)  
      for (j=J, j<=min(J+63,n), j++)  
        a[i,j]=b[i,j];
```

```
for (i=2, i<=(n-1), i++)  
  a[i] += a[i-1]*a[i+1];
```



```
for (i=2, i<=(n-2), i+=2) {  
  a[i] += a[i-1]*a[i+1];  
  a[i+1] += a[i]*a[i+2];  
}  
if (((n-2)%2) ==1)  
  a[n-1] += a[n-2]*a[n];
```

```
for (i=1, i<=n, i++)  
  a[i]+=c;  
x[i+1]=d+x[i+1]*a[i];
```



```
for (i=1, i<=n, i++)  
  a[i]+=c;  
for (i=1, i<=n, i++)  
  x[i+1]=d+x[i+1]*a[i];
```

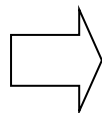
Software Protection

Obfuscation

Data Transformation. Change Encoding.

As a simple example of an encoding transformation we will replace an integer variable i by $i' = c1 * i + c2$ where $c1$ and $c2$ are constants. For efficiency, we could choose $c1$ to be a power of two. For the case $c1=8$ and $c2=3$ we have

```
int i=1;  
while (i<1000) {  
...a[i]... ;  
i++;  
}
```



```
int i=11;  
while (i<8003) {  
...a[(i-3)/8]... ;  
i+=8;  
}
```

Data Transformation. Split Variables

Boolean variables and other variables of restricted range can be split into two or more variables. We will write a variable V split into k variables $p1, \dots, pk$ as $V = [p1, \dots, pk]$. Typically, the potency of this transformation will grow with k . To allow a variable V of type T to be split into two variables p and q of type U requires us to provide three pieces of information.

- (1) A function $f(p, q)$ that maps the values of p and q into the corresponding value of V .
- (2) A function $g(V)$ that maps the value of V into the corresponding values of p and q .
- (3) New operations (corresponding to the primitive operations on values of type T) cast in terms of operations on p and q .

Software Protection

Obfuscation

Data Transformation. Split Variables

As an example we will assume that V is of type Boolean, and p and q are small integer variables.

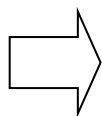
(

$g(V)$		$f(p,q)$	$2p+q$
p	q	V	
0	0	False	0
0	1	True	1
1	0	True	2
1	1	False	3

		p						
				A				
					0	1	2	3
$V(p,q)$		0	1					
				B				
		0	1	0	3	0	0	0
q		1	0	1	3	1	2	3
		0	1	2	0	1	2	3
		1	0	3	3	0	0	3
		$AND[A,B]$						

				A				
					0	1	2	3
$V(p,q)$		0	1					
				B				
		0	1	0	3	1	2	3
q		1	0	1	3	1	2	3
		0	1	2	0	1	2	3
		1	0	3	3	0	0	3
		$OR[A,B]$						

- (1) bool A,B,C;
- (2) A=True;
- (3) B=False;
- (4) C=False;
- (5) C=A & B;
- (6) C=A & B;
- (7) C=A | B;
- (8) if (A) ...;
- (9) if (B) ...;
- (10) if (C) ...;



- (1') short a1,a2,b1,b2,c1,c2;
- (2') a1=0; a2=1;
- (3') b1=0; b2=0;
- (4') c1=1; c2=1;
- (5') x=AND[2a1+a2,2b1+b2]; c1=x/2; c2=x%2;
- (6') c1=(a1 ^ a2) & (b1 ^ b2); c2=0;
- (7') x=OR[2a1+a2,2b1+b2]; c1=x/2; c2=x%2;
- (8') x=2a1+a2; if ((x==1) || (x==2))...;
- (9') if (b1 ^ b2)...;
- (10') if (V(c1,c2))...;

Software Protection

Obfuscation

Data Transformation. Merge Scalar Variables

Two or more scalar variables $V1, \dots, V_k$ can be merged into one variable VM provided the combined ranges of $V1, \dots, V_k$ will fit within the precision of VM . For example, two 32-bit integer variables could be merged into one 64-bit variable. Arithmetic on the individual variables would be transformed into arithmetic on VM . As a simple example, consider merging two 32-bit integer variables X and Y into a 64-bit variable Z . Using the merging formula

$$Z(X, Y) = 2^{32}Y + X,$$

we get the arithmetic identities:

$$Z(X+R, Y) = 2^{32}Y + (X+R) = Z(X, Y) + R;$$

$$Z(X, Y+R) = 2^{32}(Y+R) + X = Z(X, Y) + 2^{32}R;$$

$$Z(X*R, Y) = 2^{32}Y + (X*R) = Z(X, Y) + (R-1)X;$$

$$Z(X, Y*R) = 2^{32}(Y*R) + X = Z(X, Y) + (R-1)2^{32}Y$$

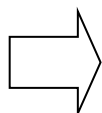
(1) `int X=45, Y=95;`

(2) `X += 5;`

(3) `Y += 11;`

(4) `X *= c;`

(5) `Y *= d;`



(1') `long Z=167759086119551045;`

(2') `Z += 5;`

(3') `Z += 47244640256;`

(4') `Z += (c-1)*(Z & 4294977295);`

(5') `Y += (d-1)*(Z & 18446744069414584320);`

Software Protection

Obfuscation

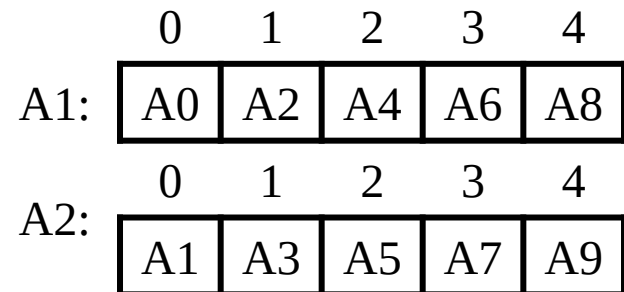
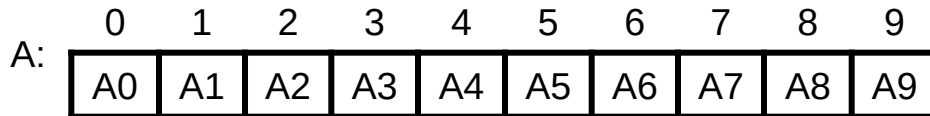
Data Transformation. Restructure Arrays

A number of transformations can be devised for obscuring operations performed on arrays, we can **split** an array into several sub-arrays, **merge** two or more arrays into one array, **fold** an array (increasing the number of dimensions), or **flatten** an array (decreasing the number of dimensions).

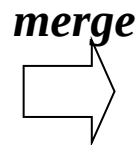
(1) *int* A[9];
(2) A[i]=...;
...



(1') *int* A1[4], A2[4];
(2') *if* ((i%2)==0) A1[i/2]=...
else A2[i/2] = ...;
...



(3) *int* B[9], C[19];
(4) B[i]=...;
(5) C[i]=...;
...



(3') *int* BC[29];
(4') BC[3*i]=...;
(5') BC[i/2*3+1+i%2]=...;
...

Software Protection

Obfuscation

Data Transformation. Restructure Arrays

	0	1	2	3	4	5	6	7	8	9
B:	B0	B1	B2	B3	B4	B5	B6	B7	B8	B9

	0	1	2	3	4	5	6	7		19
C:	C0	C1	C2	C3	C4	C5	C6	C7	...	C9

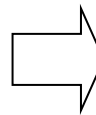
	0	1	2	3	4	5	6	...	29
BC:	B0	C0	C1	B1	C2	C3	B3	...	C19

```

(6) int D[9];
(7) for (i=0; i<=8; i++)
    D[i]=2*D[i+1];
...

```

fold



```

(6') int D1[1,4];
(7') for (j=0; j<=1; j++)
    for (k=0; k<=4; k++)
        if (k==4)
            D1[j,k]=2*D1[j+1,0];
        else
            D1[j,k]=2*D1[j,k+1];
...

```

Software Protection

Obfuscation

Data Transformation. Restructure Arrays

D:

	0	1	2	3	4	5	6	7	8	9
	D0	D1	D2	D3	D4	D5	D6	D7	D8	D9

D1:

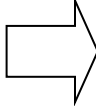
		0	1	2	3	4
0		D0	D1	D2	D3	D4
1		D5	D6	D7	D8	D9

```

(8) int E[2,2];
(9) for (i=0; i<=2; i++)
    for (j=0; j<=2; j++)
    swap (E[i,j], E1[j,i]);
...

```

flatten



```

(8') int E1[8];
(9) for (i=0; i<=8; i++)
    swap (E1[i],
    E1[3*(i%3)+i/3]);
...

```

E:

		0	1	2
0		E(0,0)	E(0,1)	E(0,2)
1		E(1,0)	E(1,1)	E(1,2)
2		E(2,0)	E(2,1)	E(2,2)

E1:

	0	1	2	3	4	5	6	7	8
	E(0,0)	E(0,1)	E(0,2)	E(1,0)	E(1,1)	E(1,2)	E(2,0)	E(2,1)	E(2,2)